

// AWKRS //

awkrs

v0.4.20

THE COMPANION

patterns · actions · the machine underneath

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTOPENSOURCE

Contents

Before You Begin	4
Chapter 1 — The Pitch	6
Chapter 2 — Hello, awkrs	9
Chapter 3 — The AWK Programming Model	12
Chapter 4 — Patterns	15
Chapter 5 — Fields and the Record Model	18
Chapter 6 — Expressions, Types, and Coercion	22
Chapter 7 — Control Flow and Statements	26
Chapter 8 — Arrays	30
Chapter 9 — User-Defined Functions	33
Chapter 10 — Regular Expressions	36
Chapter 11 — The Builtin Library	39
Chapter 12 — The gawk Extensions	42
Chapter 13 — printf and Numeric Formatting	46
Chapter 14 — The Command-Line Interface and Compatibility	50
Chapter 15 — Parallel Record Processing	54
Chapter 16 — The Machine Underneath	58
Chapter 17 — The fusevm Bridge, the JIT, and AOT	63
Chapter 18 — The Bytecode Cache	69
Chapter 19 — Bignum and Locale	72
Chapter 20 — Tooling: the Language Server and the Debugger	75
Chapter 21 — The Lexer and the Parser	78
Chapter 22 — The Virtual Machine and the Opcode Catalogue	81
Chapter 23 — The Record I/O Engine	84
Chapter 24 — Robustness: Limits, Errors, and the Sandbox	88

Chapter 25 — The Source Tree, the Build, and CI	91
Chapter 26 — A Tour of Real Programs	93
Chapter 27 — The String and Substitution Builtins, in Full	148
Chapter 28 — The Numeric, Bitwise, and Type Builtins, in Full	152
Chapter 29 — The I/O, Time, and gawk-Extension Builtins, in Full	156
Chapter 30 — The Special Variables, in Full	160
Chapter 31 — The Keyword and Statement Reference	163
Chapter 32 — The Compatibility Matrix, Walked	166
Chapter 33 — Formatting Internals	169
Chapter 34 — The Runtime and the Field Engine	171
Chapter 35 — The Namespace and Source-Expansion System	174
Chapter 36 — The Bignum Engine in Depth	177
Chapter 37 — Tests as Specification	180
Chapter 38 — The Benchmark Tables, Read in Full	183
Chapter 39 — Deployment and the Single Binary	185
Chapter 40 — The Idiom Catalogue	187
Chapter 41 — Field Recipes: One-Liners by Task	191
Chapter 42 — The Interactive REPL	194
Chapter 43 — The Linter, Diagnostic by Diagnostic	197
Chapter 44 — The Reformatter, the POT Generator, and the Profiler	201
Chapter 45 — The Debug Adapter, Internals	204
Chapter 46 — Reflection: the Banner, the Help HUD, and the Generated Reference	208
Chapter 47 — Lineage: awk and Its Implementations	211
Chapter 48 — The Introspection Arrays, Built and Refreshed	213
Chapter 49 — The Parallel-Safety Analyzer, Read Precisely	217
Chapter 50 — The fusevm-Native Migration: Host, Backend, and the Plan to Delete vm.rs	220
Chapter 51 — The Environment Surface	225
Chapter 52 — Errors, Flow, and Limits: Three Small Modules That Set the Rules	227
Chapter 53 — Locale, Message Catalogs, and the Limits of Both	229
Chapter 54 — Reading the Tree Critically	232

Chapter 55 — The Compiled Program: What Survives Compilation	235
Appendix A — Special Variables	238
Appendix B — Builtin Functions Summary	240
Appendix C — Command-Line Flags	242
Appendix D — Glossary	244
Appendix E — The Opcode Families	246
Appendix F — A Source Map	248
Appendix G — Migrating from gawk, mawk, and BSD awk	250
Appendix H — The Performance Field Guide	251
Appendix I — A Short Cookbook	252
Chapter 56 — Where the Eight Record Cutters Disagree	254
Chapter 57 — The Builtin Dispatch Arm, Read Exactly	261
Chapter 58 — rarray Is a Local Format, Not gawk's	268
Chapter 59 — What the Bytecode Cache Does Not Do	273

Before You Begin

This is not the reference manual. The reference enumerates every one of `awkrs`'s builtins, every special variable, every command-line flag, every special file — `length`, `substr`, `gensub`, `NR`, `FS`, `PROCINFO`, `/inet/tcp/...`, `-M`, and the rest — and you reach for it when you need to know exactly what `substr` does when the start index is less than one, or which keys `PROCINFO` carries after `BEGIN`. This book is the other thing: the one you read in a chair, front to back, to understand not just *what* `awkrs` does but *why* it is shaped the way it is.

`awkrs` is an AWK implementation written in Rust. That sentence is true and almost completely fails to describe it. A fuller description: `awkrs` is what you get when someone takes the forty-year-old AWK programming model — pattern, action, records, fields, the one-liner culture that has outlived nearly every language it shared a decade with — and rebuilds the machine underneath it from scratch in Rust, keeping the surface bit-for-bit faithful to POSIX `awk` and `gawk` while replacing the engine with a fused-superinstruction bytecode VM, a Cranelift JIT reached through a shared virtual machine called `fusevm`, a persistent on-disk bytecode cache, and a rayon-backed parallel record path that processes records across every core and still emits them in input order. It accepts the union of the POSIX, `gawk`, and `mawk` command-line vocabularies. It ships a Language Server and a Debug Adapter. And it does all of this from a single static binary that starts cold in single-digit milliseconds, because the thing AWK was always good at — being cheap enough to drop into a shell pipeline a hundred times an hour — is the thing `awkrs` refuses to give up.

The book is in six movements. The first — Chapters 1 through 2 — is the pitch and the smallest possible program, the language in miniature. The second — Chapters 3 through 13 — is the AWK programming model and the language itself: records and fields, patterns and actions, expressions and coercion, control flow, arrays, user functions, regular expressions, the builtin library, the `gawk` extensions, and the formatting machinery. The third — Chapters 14 and 15 — is the command-line surface, the compatibility matrix against three other `awks`, and the parallel record engine. The fourth — Chapters 16 through 25 — is the machine underneath: the bytecode VM and its fast paths, the `fusevm` bridge and its two JIT tiers, the AOT path, the bytecode cache, `bignum` and `locale`, the tooling, and then five chapters that open the engine all the way up — the lexer and the recursive-descent parser, the virtual machine and its full opcode catalogue, the record-I/O splitter that feeds it, the robustness layer of limits and errors and the sandbox, and the source tree and build that hold it all together. The fifth is a long tour (Chapter 26) through sixty-six real, complete `awk` programs from the `examples` directory, walked through line by line. The sixth — Chapters 27 through 47 — is the comprehensive reference catalogue and the deeper internals: every builtin function walked one at a time by family, with signature, semantics, edge cases, and a real example; every special variable; every keyword and statement; the full compatibility matrix against BSD `awk`, `mawk`, and `gawk` read row by row; then the formatting internals, the runtime and field engine, the namespace and `bignum` modules, the test suite as specification, the benchmark tables read in full, the deployment story, a distilled catalogue of `awk` idioms and a one-liner recipe book, and then a final run of chapters on the parts of the toolchain that sit beside the engine — the interactive REPL, the linter read diagnostic by diagnostic, the reformatter and POT generator and profiler, the debug adapter's concurrency architecture, and the reflection machinery behind the banner and the generated reference — closing with `awk`'s lineage and the family of implementations `awkrs` is measured against. Then come the quick-reference appendices — the special variables, the builtins, the flags, a glossary, the opcode families, and a source map — followed by three practical ones: a migration guide from `gawk`, `mawk`, and BSD `awk`, a performance field guide, and a short cookbook of complete programs.

Everything here is grounded in the actual repository. Every code snippet is either copied from a real file in the source tree — there are seventy-odd standalone example programs and more than two thousand parity cases to draw on — or is a minimal, valid `awk` program written to illustrate a behavior that the source actually implements. Where a number, a benchmark, or a subsystem is experimental or provisional, this book says so in those words rather than rounding up, because `awkrs`'s own documentation is scrupulous about that line and

this book holds to it. The JIT in particular is marked experimental in the crate's own description, and this book treats it that way. `awkrs` is the work of MenkeTechnologies, and this book is grounded in his code.

Let's start.

Chapter 1 – The Pitch

AWK is one of the few tools old enough to have a creation myth and useful enough that nobody has retired it. It was written at Bell Labs in 1977 by Aho, Weinberger, and Kernighan — the initials are the name — as a small language for the single most common task in all of computing that is not arithmetic: take a stream of text laid out in lines and columns, look at each line, and do something with the columns. Print the third field. Sum the first. Count the distinct values in the fifth. Match the lines that look like an error and tally them by hour. The surprising thing is not that AWK could do these things in 1977; it is that nearly fifty years later, with every general-purpose language in the world available on every machine, the shortest path from a question about a text file to an answer is still, very often, an `awk` one-liner. The activation energy of writing `awk '{print $1}'` has never been beaten. That is the niche `awkrs` is built for, and the bet it makes is that you can keep the niche exactly as it is — same language, same one-liners, same muscle memory — and replace everything behind it.

What `awkrs` is

`awkrs` is a pattern-action interpreter that runs AWK programs over input records, implemented in Rust as a bytecode virtual machine with a Cranelift JIT offload path and rayon-backed parallel record processing. Unpack that slowly, because each clause is load-bearing and several of them are unusual for an `awk`.

“Pattern-action interpreter that runs AWK programs over input records” is the part that is *not* unusual — it is the definition of `awk`, and `awkrs` honors it faithfully. A program is a sequence of `pattern { action }` rules. The input is read one record at a time, a record is by default a line, and for each record the interpreter tests every pattern and runs the action of each one that matches. `BEGIN` rules run before the first record and `END` rules run after the last. Fields within a record are addressed as `$1`, `$2`, and so on, with `$0` the whole record, split apart by the field separator `FS` and joined back together with the output field separator `OFS`. If you have written `awk` before, none of this will surprise you, and that is the point: `awkrs` accepts the POSIX `awk` language plus the `gawk` extensions that show up in real scripts, and it is verified byte-for-byte against `gawk`, `mawk`, and BSD `awk` by a parity suite of more than two thousand cases that runs in CI on every push.

“Implemented in Rust as a bytecode virtual machine” is where `awkrs` parts company with most of its peers. The original AWK — the one-true-`awk`, BWK `awk`, still maintained — is a tree-walking interpreter: it holds the parsed program as a tree in memory and re-walks the relevant subtree every time control flows through it. A loop that runs a million times walks the same nodes a million times, paying on each pass the cost of dispatching on node type and resolving variables by name. `awkrs` does not do this. It compiles your program once into a flat stream of bytecode instructions — ninety-six distinct opcodes — and runs that stream on a stack machine, so the per-iteration cost collapses to the cost of the instructions themselves. `gawk` and `mawk` also compile to bytecode, so this alone would not distinguish `awkrs`; what distinguishes it is the two layers built on top.

“With a Cranelift JIT offload path” is the first of those layers. Eligible chunks of purely numeric bytecode are handed to `fusevm` — a shared bytecode VM that also powers the Rust-rewritten shell `zshrs` and the language `strykelang` — which compiles them to native machine code through Cranelift, the same code generator that backs parts of the Rust and WebAssembly ecosystems. This path is on by default for the chunks it can handle, and it is honest about its limits: it engages for tight numeric loops and is inert for the string-and-field programs that make up most real `awk` workloads. The crate’s own one-line description calls the JIT *experimental*, and this book uses that word too. It is a real, measured win where it applies — a twenty-million-iteration `sin(i)` loop dropping from roughly 12.8 seconds on the interpreter to about 0.15 seconds on the JIT — and a no-op where it does not, and the chapters on the machine underneath are careful to say which is which.

“And rayon-backed parallel record processing” is the second layer, and it is the one feature on this list that no traditional `awk` has at all. When a program is *parallel-safe* — a property `awkrs` checks statically before it runs — records can be processed concurrently across a thread pool, with the output reassembled into input order so that a pipeline downstream of `awkrs` sees exactly the bytes it would have seen from a sequential run. This is off by default; you opt into it with `-j N`, and a program that is not parallel-safe falls back to sequential execution with a warning rather than silently producing wrong output. The whole of Chapter 15 is about what `awkrs` parallelizes and how it stays correct.

The single binary and the cold start

The reference implementation is one Rust binary. `cargo install awkrs` or `brew install awkrs` puts a single executable on your path; there is no separate runtime to install on each machine, no shared library to keep in sync, no package tree to resolve before a program runs. This is the same single-artifact discipline that makes `awk` itself so deployable, and `awkrs` keeps it. A container image needs nothing in it but the binary and your script.

The cold-start time is not an incidental detail; it is the whole reason `awk` has survived. A language you reach for in a shell pipeline pays its startup cost on every single invocation, and if that cost is large enough to feel, you stop reaching for the language. This is the tax that sinks Python and Julia in the one-liner role — Julia’s startup alone is tens of milliseconds of runtime warm-up paid every time — and it is the tax `awkrs` is built to avoid. The binary memory-maps at startup, the parallel pool is not spun up unless you ask for it with `-j`, and the bytecode cache (Chapter 18) means that the second time you run a script file, the lexer, parser, and compiler do not run at all — the compiled program is mmapped back from disk zero-copy. The benchmark tables in Chapter 14 show `awkrs` starting and finishing a million-line `{ print $1 }` in roughly sixteen milliseconds, against `gawk`’s hundred and `mawk`’s sixty-six, on the same input on the same machine.

The performance thesis, stated honestly

`awkrs`’s README opens with the phrase “world’s fastest `awk` bytecode engine,” and the benchmark tables are real, generated by hyperfine across five engines on the same input and copied verbatim into the repository. But the engineering report that accompanies the project is unusually careful about *why* `awkrs` is fast, and this book follows that care, because the honest version of the performance story is more interesting than the slogan.

`awkrs` is fast on the common `awk` workloads — field extraction, summation, regex filtering, string concatenation — for reasons that have nothing to do with the JIT. They come from the bytecode VM’s peephole-fused superinstructions, from a raw-byte field-extraction path that parses the *N*th whitespace field directly out of a memory-mapped file with no record copy and no UTF-8 validation, from direct-to-buffer output that never allocates a per-record string, and from inline fast paths that bypass the VM’s call context entirely for single-rule programs like `{ print $1 }`. These are the optimizations that move the benchmark numbers, and Chapter 16 walks through each of them against the source.

The JIT is a separate and narrower story. It is on by default, it compiles eligible numeric loop chunks to native code through `fusevm` and `Cranelift`, and on those chunks it is a large win — measured at fourteen to a hundred and ten times the bytecode interpreter on pure-arithmetic and builtin accumulator loops, and bit-for-bit identical to the interpreter on every one of them. But it does not engage for string-or-field-dominated programs, which is most of what `awk` is used for, so `awkrs` deliberately does not lead with it. The crate description marks it experimental; this book marks it experimental; and Chapter 17 explains exactly which chunks are eligible and why the others are not. The thing to carry out of this chapter is that `awkrs`’s everyday speed and `awkrs`’s JIT are two different claims, both true, with different scopes, and the book keeps them separate throughout.

What awkrs is not

It clarifies a pitch to say what it excludes, and awkrs is disciplined about its non-goals in a way worth stating up front, because each non-goal is a place a less careful project would have overclaimed. awkrs is *not* a new language — it is awk, the POSIX language plus the gawk extensions that show up in real scripts, and it deliberately does not add syntax of its own to the program surface; a program written for it is a program that runs on gawk, and that portability is the point. It is *not* a general-purpose programming environment trying to displace Python or Perl — its model is records-and-fields, sharp for the class of problems that shape fits and not pretending to be more. It is *not* claiming bit-identical behavior with any other awk in every corner; its compatibility matrix calls itself a feature matrix, not a proof of correctness, and is explicit that where POSIX leaves behavior unspecified — random sequences, hash iteration order, subtle rounding — differences are expected. It does *not* load arbitrary dynamic extensions, by design, drawing the line at its single-binary discipline. And its JIT is *not* the headline — it is experimental by the crate's own description, a large win on a narrow class of numeric loops and inert elsewhere. Naming these non-goals is not modesty for its own sake; it is the same honesty the rest of the book relies on, and it is what lets the claims awkrs *does* make — faithful to the standard, fast on the common case, modern in its tooling — be taken at face value, because they are not inflated by claims it has been careful not to make.

What this book assumes

This book assumes you can read awk — or are willing to learn it as you go, since the early chapters teach the model from the ground up — and that you are comfortable at a shell prompt. It does not assume you know Rust; the chapters on the machine underneath explain the architecture in terms of what it does and why, citing the source files by name so you can read them yourself, but you do not need to write Rust to understand them. Where the book quotes awkrs source, it is Rust; where it quotes awk programs, they are real programs from the `examples` and `parity` directories or minimal valid programs written to make a point. The distinction is always clear from context, and the awk is always labeled `awk` in the code fence.

Chapter 2 — Hello, awkrs

The first program in most language books prints a greeting and explains nothing. The first awk program traditionally does not even print a greeting — it prints a field — because awk's reason to exist is the input stream, and a program that ignores its input is a program that is not yet doing anything awk-shaped. So we will start with the greeting, because it teaches the `BEGIN` rule, and then immediately move to the field, because that is the language actually working.

```
BEGIN { print "hello, awkrs" }
```

That is a complete, runnable awkrs program. Save it to `hello.awk` and run it:

```
awkrs -f hello.awk </dev/null
```

or, more idiomatically for something this short, pass the program text directly as the first argument:

```
awkrs 'BEGIN { print "hello, awkrs" }' </dev/null
```

Both print `hello, awkrs` and a newline. There are several things in this small program worth reading closely, because each is a load-bearing piece of the model.

BEGIN, and why the input is redirected from `/dev/null`

The rule is `BEGIN { ... }`. `BEGIN` is one of awk's special patterns: its action runs exactly once, before the first record is read, before the input stream is even touched. Because this program has *only* a `BEGIN` rule and no record rules, it has no reason to read input at all — and awkrs, like every correct awk, knows this. A program whose only rules are `BEGIN` rules does not block waiting for input; it runs the `BEGIN` actions and exits. The `</dev/null` in the commands above is therefore belt-and-suspenders: it is there so that if you typed the program slightly wrong and it *did* have a record rule, the command would still terminate instead of hanging on your terminal waiting for you to type input. It is a good habit for one-liners, not a requirement of this particular program.

The print statement is `print "hello, awkrs"`. `print` writes its arguments followed by the output record separator `ORS`, which defaults to a newline. This is why you never write the trailing `"\n"` in an awk `print` — the record separator is part of the statement, not part of the argument. If you want to suppress it, you use `printf`, which prints exactly the bytes of its format string and nothing more, and Chapter 13 is entirely about `printf`. The distinction is the same one that has existed in awk since 1977, and awkrs keeps it exactly.

The program as the first argument

awkrs follows the universal awk command-line convention: the first non-option argument is the program text, and every argument after it is an input file (or a `var=value` assignment). So:

```
awkrs '{ print $1 }' data.txt
```

runs the program `{ print $1 }` over the file `data.txt`. If there are no file arguments, awk reads standard input, which is what makes awk a natural member of a shell pipeline:

```
ps aux | awkrs '{ print $1 }'
```

prints the first whitespace-separated field of every line `ps` produces — the user column. There is no `-e` required for the inline program form, exactly as in every other `awk`; you reach for explicit flags only when you have more than one program fragment to combine, or when you are mixing program text with library files, and Chapter 14 covers the full flag vocabulary. `awkrs` also installs under a second binary name, `aw`, a shorter spelling for the same engine, for the same reason `awk` users alias things: every keystroke in a one-liner is a keystroke you type a thousand times.

The language in miniature

Now the program that actually does `awk`'s job:

```
{ print $1 }
```

That is the entire program. There is no `BEGIN`, no `main`, no loop written by you. The rule has an empty pattern — nothing before the `{` — and an empty pattern matches every record. So the action `print $1` runs once per input line, and `$1` is the first field of the current line, which by default means the first run of non-whitespace characters. The loop over records is implicit; `awk` wrote it for you. This three-character pattern-action idea — match every record, print the first field — is the thing that has kept `awk` alive for half a century, and it is worth stating what it compresses. In a general-purpose language you would open the file, loop over its lines, strip the newline, split the line on whitespace, index the resulting array, print the element, and handle the end of input. `awk`'s model folds all of that into the structure of the language itself: the loop, the splitting, the field access, and the output separator are all implicit, and what is left on the page is only the part that is specific to your problem — `print $1`. `awkrs` implements this model exactly, and then makes the implicit machinery underneath it extraordinarily fast, which is the subject of the rest of the book.

Stack a pattern in front of the action and you filter:

```
$3 > 100 { print $1, $3 }
```

This runs the action only on records whose third field, interpreted as a number, exceeds one hundred, and prints the first and third fields separated by `OF5` (a space by default, because of the comma between the arguments). Put a regular expression where the pattern goes and you match against the whole record:

```
/error/ { count++ } END { print count }
```

This counts the records containing the string `error` and prints the total after the input is exhausted. `count` was never declared and never initialized; `awk` variables spring into existence as needed, and an uninitialized variable is both numeric zero and the empty string until you use it as one or the other. The `END` rule, the mirror of `BEGIN`, runs once after the last record. These three small programs — print a field, filter on a field, count matches — are `awk`'s entire reason for being, and you now have the shape of the language. The remaining chapters fill in the model rigorously, but the silhouette is already here: records flow past, patterns select, actions fire, and `BEGIN` and `END` bracket the stream.

Running a script file, and the cache that quietly speeds it up

For anything longer than a one-liner you put the program in a file and run it with `-f`:

```
awkrs -f report.awk access.log
```

This is the form to know about for a reason that is invisible until you measure it: the `-f script.awk` form is the one `awkrs` caches. The first time you run a script file, `awkrs` lexes it, parses it, compiles it to bytecode, and — because you invoked it as `-f` — memoizes the compiled program to an rkyv-archived cache at `~/.awkrs/scripts.rkyv`. The second time you run the same unchanged script, the lexer, parser, and compiler do not run at all; `awkrs` memory-maps the cached compiled program back in zero-copy and goes

straight to execution. The cache invalidates itself silently when the script's modification time changes or when the awkrs binary itself is newer than the cached entry, so you never get a stale result, and `AWKRS_CACHE=0` turns it off entirely. Chapter 18 is the full story; the thing to know now is that the ordinary, idiomatic way to run an awk script — `-f file` — is also the fast way, with no action required from you. Inline `-e` programs and the various library-loading forms skip the cache because they need the AST for other reasons, which Chapter 18 also explains.

The cold start, measured

There is one more thing worth doing before we leave the smallest programs, which is to take seriously the claim that awkrs starts fast, because it is the property the whole one-liner argument rests on. Run `time awkrs '{ print $1 }' big.log` and the wall-clock time you see is dominated, for a small input, not by the work but by the startup — and awkrs's startup is single-digit milliseconds, because the binary memory-maps at launch rather than loading and initializing a runtime, the parallel pool is not spun up unless `-j` asks for it, and on the second run of a `-f` script the compile is skipped entirely by the cache. The benchmark tables of Chapter 14 put a number on the combined startup-and-run: a million-line `{ print $1 }` finishing in roughly sixteen milliseconds, against gawk's hundred-odd and BSD awk's nearly two hundred. The reason this matters is the reason it has always mattered for awk: a tool you drop into a shell pipeline pays its startup on every invocation, and a build that runs an awk script once per file across ten thousand files pays it ten thousand times. A startup cost large enough to feel — Julia's tens of milliseconds of runtime warm-up, Python's interpreter bring-up — is a tax that quietly pushes you toward keeping the work in one big process, away from the compose-small-tools shell idiom that awk was built for. awkrs refuses that tax, and the refusal is what lets it occupy awk's niche rather than merely reimplement awk's language. The two-character alias `aw` is the same instinct one level up: when the startup is already cheap, the *typing* becomes the remaining friction, and `aw '{ print $1 }'` shaves that too.

With the silhouette of the language in hand, we can now go back to the beginning and build the model properly, starting with the thing everything else is built on: the record.

Chapter 3 – The AWK Programming Model

Everything awkrs does — every optimization, every extension, every benchmark number — is in service of a programming model that fits in a paragraph. A program is a list of rules. Each rule is a pattern and an action. The input is a sequence of records; the default record is a line. For each record, in order, awk tests every rule's pattern, and runs the action of each rule whose pattern is true. Before any record is read, the **BEGIN** rules run; after the last record, the **END** rules run. Records are split into fields; the fields are the columns. That is the whole model, and once it is in your hands the rest of the language is detail. This chapter makes each piece of that paragraph precise, because the precision is where awkrs's fidelity to POSIX and gawk lives.

Records, fields, and the implicit loop

A *record* is the unit awk processes one at a time. By default a record is a line of input — the bytes up to a newline — but this is configurable through the record separator **RS**, which Chapter 5 covers in full. The variable **NR** (number of records) counts records from the start of the input across all files; **FNR** (file number of records) counts within the current file and resets at each new file. The current record's text is the field variable **\$0**.

A record is divided into *fields*. By default the field separator **FS** is a single space, which awk treats specially: it means “split on runs of whitespace, ignoring leading and trailing whitespace.” So the line **alice 30 ny** has three fields — **alice**, **30**, **ny** — regardless of how many spaces surround and separate them. The fields are **\$1**, **\$2**, **\$3**; the count of fields is **NF**; and **\$NF** is therefore always the last field, a common idiom. **\$0** is the whole record. The relationship between **\$0**, the fields, **NF**, **FS**, and **OFS** is not a one-way split — it is a live, bidirectional model that Chapter 5 dissects, because awkrs implements the full POSIX rebuild semantics: assigning to a field rebuilds **\$0**, assigning to **\$0** re-splits the fields, and assigning to **NF** truncates or extends.

The loop over records is *implicit*. You never write it. A program of bare record rules is wrapped, conceptually, in “for each record: test the patterns, run the matching actions,” and awk supplies that loop. This is the single most important thing about the model and the reason awk programs are so short: the boilerplate of opening input, iterating lines, and splitting columns is the language's job, not yours.

Patterns and actions

A rule is written **pattern { action }**. Either part may be omitted, and the omission has a defined meaning:

- A rule with a pattern and no action — **\$3 > 100** — uses the default action, which is **{ print \$0 }**. So **\$3 > 100** on its own prints every record whose third field exceeds one hundred. This is awk's grep-with-arithmetic mode.
- A rule with an action and no pattern — **{ print \$1 }** — uses the default pattern, which matches every record. So the action runs on every line.
- A rule with both runs the action on the records the pattern selects.

awkrs follows gawk's rule about which patterns may omit the braces: the four special patterns (**BEGIN**, **END**, **BEGINFILE**, **ENDFILE**) *must* be written with an explicit action block, because an action-less **BEGIN** is meaningless; the default-action shorthand applies only to record-matching patterns. This is a small compatibility detail and awkrs honors it.

BEGIN and END

BEGIN and **END** are the bookends. **BEGIN** actions run once, in source order, before the first record is read; **END** actions run once, in source order, after the last record is consumed. They are where you do setup and teardown — set **FS** before the input is split, print a header, accumulate a running total during the record rules and print it at the end:

```
BEGIN { FS = ","; print "host,count" }
{ count[$1]++ }
END { for (h in count) print h "," count[h] }
```

This three-rule program sets the field separator to a comma, prints a header, tallies the first column of every record into an associative array, and prints the tally after the input ends. It is a complete, idiomatic awk program and it demonstrates the shape of nearly every real awk script: initialize in **BEGIN**, accumulate in the record rules, report in **END**.

awkrs's handling of **exit** interacts with **END** exactly as POSIX requires, and this is a place where naive implementations get it wrong. If **exit** is called from a **BEGIN** rule or a record action, the **END** rules *still run* before the program terminates — **exit** means “stop processing records,” not “stop the program.” Only an **exit** called from within an **END** rule actually ends the program immediately. awkrs implements this: an **exit** with a status code from anywhere but **END** jumps to the **END** rules, runs them, and then exits with the requested code. This lets the common pattern of “bail out early but still print the summary” work correctly.

BEGINFILE and ENDFILE

gawk added two more special patterns, and awkrs implements them: **BEGINFILE** runs before each input file is processed, and **ENDFILE** runs after each file is exhausted, with **FILENAME** set to the file in question. These are the per-file analogues of **BEGIN** and **END**, and they are useful for per-file headers, for resetting per-file state, and — **BEGINFILE** especially — for handling unreadable files gracefully, since gawk allows a **BEGINFILE** rule to inspect **ERRNO** and call **nextfile** to skip a file that could not be opened. awkrs follows gawk's constraint that **next** and **nextfile** are invalid inside a **BEGINFILE** rule in the way gawk specifies, a rule enforced in the VM.

The flow-control verbs

Within actions, several statements alter the implicit record loop directly, and they are part of the model rather than the expression language:

- **next** stops processing the current record and fetches the next one, restarting the rule tests from the top. It is how you say “I’m done with this line.”
- **nextfile** abandons the rest of the current input file and moves to the next one, as if the current file had ended.
- **exit [code]** stops record processing and proceeds to the **END** rules (then exits), as described above.
- **getline** reads a record explicitly, outside the implicit loop, in any of several forms — from the main input, from a file, from a command's output, or from a coprocess. Chapter 7 covers its forms and its return-value protocol in detail.

These verbs are the reason awk's implicit loop does not become a straitjacket: you can break out of it, skip ahead in it, and read around it when a problem needs more than one-record-at-a-time processing.

Why the model has lasted

It is worth pausing on why this particular model has outlived so many languages that were supposed to replace it. The answer is that it matches the shape of an enormous class of real problems exactly. Log files, CSV exports, `ps` and `df` and `netstat` output, `/etc/passwd`, tab-separated data dumps — all of it is records-and-fields, and all of it wants the same handful of operations: select some records, pull some fields, accumulate across records, report at the end. `awk`'s model is not general-purpose; it is specifically the model for this class, and within the class it is unbeaten on activation energy. `awkrs`'s contribution is to take this model — unchanged, down to the corner cases of `exit` and `ENDFILE` — and run it on an engine fast enough that you stop reaching for a faster tool, because there is no longer a faster tool to reach for.

The interleaving order, precisely

It is worth pinning the exact order in which the special and record rules fire across a whole run, because programs occasionally depend on it and a fuzzy mental model produces fuzzy bugs. The sequence is: first, all `BEGIN` rules run in source order, once. Then, for each input file in turn: the `BEGINFILE` rules run with `FILENAME` set, then each record of that file is read and every record rule whose pattern matches fires in source order, then after the file is exhausted the `ENDFILE` rules run. Finally, after all files, the `END` rules run in source order, once. Within a single record, the rules are tested top to bottom and the matching ones fire in that order, so a program can rely on an earlier rule's effect being visible to a later rule on the same record — which is how the `csv_pivot.awk` pattern of “skip the header with `NR == 1 { next }` then aggregate” works, the `next` short-circuiting the later rules for the header line only. The flow verbs cut across this order in defined ways: `next` ends the current record and restarts the rule tests on the next record, `nextfile` jumps to the next file's `BEGINFILE`, and `exit` from anywhere but `END` jumps to the `END` rules and then terminates, which is the “bail out early but still print the summary” behavior Chapter 3 opened with. Knowing this order is knowing exactly when your setup, your per-record work, your per-file work, and your teardown each run — and `awkrs` implements the sequence to POSIX, including the corner cases of `exit` and the per-file rules that naive implementations get wrong.

ARGV, assignments, and the input sequence

The implicit loop reads from “the input,” and it is worth being precise about what the input *is*, because `awk`'s argument handling is part of the model and `awkrs` implements it faithfully. The command-line arguments after the program become the array `ARGV`, with `ARGV[0]` the engine's own name and `ARGV[1]` through `ARGV[ARGC-1]` the operands, and the implicit loop walks them in order. Each operand is one of two things: a filename to read, or a `var=value` assignment. The assignment form is the subtle one — `awkrs '{print x, $0}' x=1 a.txt x=2 b.txt` sets `x` to `1` before reading `a.txt` and to `2` before reading `b.txt`, because the assignments take effect *at the point they are reached in the argument sequence*, not all at once at startup. This is how you thread per-file parameters into a run without a `BEGINFILE` rule, and it is a genuine part of the input model rather than a flag.

A program can also *edit* `ARGV` to change what it reads. Setting `ARGV[i]` to the empty string, or lowering `ARGC`, makes `awk` skip an operand — the gawk idiom for “I consumed that argument in `BEGIN`, do not also treat it as a file” — and `awkrs` honors this, re-reading `ARGC/ARGV` as it advances. A bare `-` operand means standard input. The takeaway is that the implicit loop is not hardwired to a fixed file list: it is driven by `ARGV` as the program left it after `BEGIN`, with assignments interleaved, which is what lets a single `awk` invocation process several files with changing parameters and still present them to your record rules as one stream.

Chapter 4 – Patterns

A pattern is the question awk asks of each record. Its answer — true or false — decides whether the action fires. awkrs supports the full POSIX pattern vocabulary plus the gawk additions, and the variety is worth knowing in full, because choosing the right pattern form is most of what makes an awk program read clearly.

The empty pattern

The simplest pattern is no pattern at all. A rule written `{ action }` matches every record. This is not a special case bolted on; it is the natural reading of “a rule with no condition runs unconditionally.” Most one-liners use it: `{ print $NF }` prints the last field of every line.

Regular-expression patterns

A pattern written as a regular-expression literal — `/error/` — matches a record when the regex matches anywhere in `$0`. So `/error/ { ... }` runs its action on every record containing the substring `error`, and `/^#/ { next }` skips comment lines that begin with a hash. The regex is a *literal* in pattern position, and awkrs’s regular-expression engine is the Rust `regex` crate, which is linear-time and does not backtrack — a property with real consequences that Chapter 10 explores, including the one feature it costs you (pattern-side backreferences) and the many it buys you (guaranteed linear matching, full POSIX character classes, Unicode).

There is a subtlety in how awk reads a bare `/regex/` that awkrs gets exactly right, and it is worth understanding because it explains why awk’s slash character is not ambiguous even though it is both “divide” and “start of regex.” At a *primary* position — where a value is expected — a `/` cannot begin a division, because division is a binary operator that needs a left operand. So the parser re-reads `/` at a primary position as the start of a regex literal. And in expression context, a bare `/re/` means `$0 ~ /re/` — a match against the current record — which is why `!/foo/` (records *not* matching `foo`), `if (/foo/)`, and `x = /foo/` all test the current record rather than manipulating a string literal. awkrs documents and implements this POSIX rule precisely, including the corner case that the right-hand side of `~` and `!~`, and the regex arguments to `gsub`, `sub`, `gensub`, `match`, `split`, and `patsplit`, still treat `/re/` as a pattern — so a stray slash inside a replacement string like `b/c` stays a division, not a regex.

Expression patterns

Any expression can be a pattern; the record matches when the expression is true. awk’s notion of truth is the C-derived one: a numeric value is true when nonzero, a string is true when nonempty, and an uninitialized variable is false. So:

```
NR % 2 == 0      { print }           # even-numbered records
$3 > 100         { print $1, $3 }      # third field over 100
$2 == "ERROR"    { errors++ }         # exact string match on field 2
length($0) > 80  { print FNR, $0 }    # long lines, with file line number
```

are all expression patterns. The comparison operators here carry awk’s characteristic number-or-string duality, which Chapter 6 makes rigorous: `$3 > 100` compares numerically because `100` is a number and `$3` is a *numeric string* (input that looks like a number), whereas `$2 == "ERROR"` compares as strings because

"**ERROR**" is a string constant. **awkrs** implements the POSIX comparison rules here faithfully, including the precise and frequently-misunderstood line between a numeric string from input and a string constant in the program, which is one of the corner cases its parity suite pins down.

A compound expression pattern combines a regex and an expression, and reads naturally:

```
/^GET/ 66 $9 == 200 { hits++ }
```

This matches records that begin with **GET** and whose ninth field is **200** — the leftmost **/^GET/** is the **\$0 ~ /^GET/** shorthand, combined with an ordinary comparison by **66**. **awkrs**'s parity tests cover exactly these compound regex-and-expression patterns.

Range patterns

A pattern written as two patterns separated by a comma — **pat1**, **pat2** — is a *range* pattern. It becomes true on the record where **pat1** matches and stays true through the record where **pat2** matches, then turns off; it is **awk**'s way of selecting a contiguous span of records. Both endpoints can be regexes or expressions:

```
/^BEGIN BLOCK/, /^END BLOCK/ { print } # everything between the markers  
NR == 10, NR == 20 { print } # records 10 through 20 inclusive
```

The range is inclusive on both ends, and the same record can both open and close a range if both endpoints match it. **awkrs** compiles range patterns to a small state machine with jump-patched offsets in the bytecode, which Chapter 16 describes. One consequence of how ranges carry hidden cross-record state is that a program using a range pattern is *not* parallel-safe — the range's on/off flag depends on the order records are seen — and **awkrs**'s static parallel-safety check specifically excludes range patterns, falling back to sequential execution. Chapter 15 returns to this.

The special patterns

Finally, the four special patterns — **BEGIN**, **END**, **BEGINFILE**, **ENDFILE** — covered in Chapter 3. They are not tested against records; they fire at fixed points in the input's lifetime. The thing to remember from a pattern-vocabulary standpoint is that they must carry explicit action blocks, and that they interleave with the record rules in a defined order: all **BEGIN** rules, then for each file the **BEGINFILE** rules, the record rules per record, and the **ENDFILE** rules, then all **END** rules.

Choosing a pattern

The art of **awk** is mostly in choosing the pattern. A filter is a regex or expression pattern; a per-section transform is a range pattern; setup and teardown are **BEGIN** and **END**; per-file work is **BEGINFILE** and **ENDFILE**. The same problem can often be written several ways, and the clearest version is usually the one where the pattern carries the condition and the action carries only the work — **/error/ { count++ }** rather than **{ if (\$0 ~ /error/) count++ }**. **awkrs** gives you the whole vocabulary and runs all of it at the same speed, so the choice is purely one of clarity.

A worked range pattern

The range pattern is the one pattern form that carries hidden state, and seeing it work makes both its usefulness and its cost concrete. Suppose a log file brackets each transaction between a **BEGIN TXN** marker and an **END TXN** marker, and you want only the lines inside transactions:

```
/^BEGIN TXN/, /^END TXN/ { print }
```

This rule's pattern becomes true on the record matching `/^BEGIN TXN/` and stays true through the record matching `/^END TXN/`, then turns off — so the action prints every line from the opening marker through the closing one, inclusive of both. The state that makes this work is a single on/off flag the range carries between records, which `awkrs` compiles to a small state machine with jump-patched offsets in the bytecode. That flag is precisely why a range pattern is *not* parallel-safe: whether the rule fires on a given record depends on whether a `BEGIN TXN` was seen on some *earlier* record, which is a cross-record dependency, and a worker processing records out of order could not know the flag's state. `awkrs`'s static safety check (Chapter 15) therefore excludes any program with a range pattern from the parallel path, falling back to sequential execution — the right call, because reordering the records would change which ones the range selects. The same record can both open and close a range if both endpoints match it, and the endpoints can be expressions as well as regexes, so `NR == 10, NR == 20` selects records ten through twenty. The range pattern is `awk`'s compact way to say “the span between two markers,” and it is worth knowing that its convenience and its sequential-only constraint are the same fact: the hidden flag that makes it expressive is the hidden flag that makes it order-dependent.

Chapter 5 — Fields and the Record Model

The field is awk's fundamental act of structure: it takes a flat line of bytes and imposes columns on it. Most of the time field access feels simple — `$1` is the first column — but underneath there is a precise, bidirectional model relating `$0`, the individual fields, `NF`, `FS`, and `OFS`, and awkrs implements that model in full, including the corner cases that separate a faithful awk from an approximate one. This chapter is that model.

The field separator FS

`FS` controls how `$0` is split into fields. Its value is interpreted by a small set of rules that awkrs follows exactly:

- The default value is a single space, which is *special*: it means split on runs of whitespace (spaces and tabs) and ignore leading and trailing whitespace. This is why `a b c` has three clean fields under the default `FS`. The engineering report notes that awkrs's `FS` defaults to the literal single space `" "` — matching POSIX and gawk — rather than empty, a detail its parity tests pin.
- A single character other than space — `FS = ","`, `FS = ":"` — splits on that exact character, with no whitespace trimming and with empty fields preserved. So `a,,c` under `FS=","` has three fields, the middle one empty. A single-character string `FS` is always a literal, independent of `IGNORECASE`.
- A multi-character `FS` is treated as a *regular expression*. `FS = "[,;]"` splits on either a comma or a semicolon; `FS = "+"` splits on runs of one or more spaces (different from the default space, which also trims). When `FS` is a regex, `IGNORECASE` applies to it.

You set `FS` either in a `BEGIN` rule (so it applies from the first record) or with the `-F` command-line flag, which is exactly equivalent: `awkrs -F, ...` is `awkrs 'BEGIN{FS=","} ...'`.

FIELDWIDTHS: fixed-width input

For fixed-width data — columns aligned by position rather than separated by a delimiter — gawk added `FIELDWIDTHS`, and awkrs implements it. Set `FIELDWIDTHS` to a space-separated list of column widths and awk ignores `FS` entirely, slicing each record into fields of exactly those byte widths:

```
BEGIN { FIELDWIDTHS = "5 3 8" }
{ print $2 } # the 3-character field starting at column 6
```

awkrs accepts gawk's full `FIELDWIDTHS` token vocabulary: a plain `width`, a `skip:width` form that discards `skip` bytes before taking `width`, and a `*` token. Following gawk, the last field is clamped to its declared width rather than auto-extended to the end of the line, so any trailing bytes past the last declared width are left unused — a parity detail awkrs's compatibility matrix calls out explicitly. `FIELDWIDTHS` is active whenever it is non-empty; clearing it returns to `FS`-based splitting.

FPAT: define fields by what they are

The inverse of `FS` is `FPAT`. Where `FS` describes the *separators between* fields, `FPAT` describes the *fields themselves* as a regular expression, and awk takes each match of the pattern as a field. This is the right tool

for data where the separator also appears inside fields — the classic case being CSV with quoted commas:

```
BEGIN { FPAT = "([^,]+)|(\"[^\"]+\")" }
```

This says a field is either a run of non-commas or a double-quoted run, which correctly keeps "Smith, John" as one field even though it contains a comma. awkrs implements FPAT with gawk's leftmost-longest semantics over the alternation — a parity fix its test suite records explicitly — so the matching agrees with gawk on the ambiguous cases.

CSV mode

Because comma-separated values are so common and so fiddly to parse correctly with FPAT by hand, awkrs ships a dedicated CSV mode, reached with `-k` or `--csv`, aligned with `gawk --csv`. It implements RFC-style quoting, including the doubled-quote escape ("" inside a quoted field means a literal quote), and it sets up field splitting through a purpose-built CSV parser rather than a regex. The compatibility matrix notes a parity detail worth knowing: a leading comma counts as an empty first field, so a record `,,,` has four fields, matching gawk. CSV mode is the right answer whenever the input is real CSV; FPAT is the fallback when you need something CSV-shaped but not quite CSV.

split and patsplit

The same splitting machinery is available as a function. `split(string, array [, fieldsep])` breaks `string` into `array[1]`, `array[2]`, ... using the given separator (or `FS` if omitted), and returns the number of fields. The third argument accepts a regex exactly as `FS` does. awkrs implements the gawk extension where `split(string, array, fieldsep, seps)` populates a fourth array `seps` with the actual separator text that appeared between consecutive fields — useful when you need to reconstruct the original with its exact delimiters. A defined corner case awkrs honors: `split("", arr, fs)` returns `0` and produces no empty pseudo-field.

`patsplit(string, array [, fieldpat [, seps]])` is the FPAT analogue: it splits by *matching* the field pattern rather than by separators, and its four-argument form populates `seps` with the gaps between matches. awkrs accepts two to four arguments for `patsplit` and rejects other arities at compile load time with a clear error.

The bidirectional record model

Here is the part that separates a real awk from a toy: `$0`, the fields, and `NF` are not a one-time split but a live, mutually-consistent model, and awkrs implements the full POSIX semantics in all directions.

- **Assigning to a field rebuilds `$0`.** If you write `$2 = "X"`, awk does not just remember a new value for the second field — it reconstructs `$0` by joining all the fields with `OF5`. So under default `OF5` (a space), starting from a tab-separated record, `$2 = $2` (assigning a field to itself) has the visible effect of normalizing the separators to single spaces, a classic awk idiom for reformatting whitespace.
- **Assigning to `$0` re-splits the fields.** If you write `$0 = "a b c"`, awk re-runs field splitting on the new value, updating `$1`, `$2`, `$3`, and `NF`.
- **Assigning to `NF` truncates or extends.** Setting `NF` to a smaller number drops the trailing fields and rebuilds `$0` with the survivors; setting it larger pads with empty fields. Either way `$0` is rebuilt with `OF5`.
- **Referencing a field past `NF`** yields the empty string without changing the record, but *assigning* to a field past `NF` extends the record, creating any intervening empty fields and bumping `NF`.

awkrs implements all of these, and its compatibility matrix records that an invalid `NF` assignment or a reference to a negative field number is a fatal error, matching gawk, and tested. This bidirectional model is what makes

`awk` a *record editor* and not merely a record reader: you can take a line apart, change a column, and the line reassembles itself correctly.

RS and RT: choosing the record

`RS`, the record separator, decides what a record *is*, and `awkrs` implements its full range:

- The default `RS` is a newline: records are lines.
- A single character `RS` makes that character the record terminator.
- `RS = ""` is *paragraph mode*: records are separated by blank lines, so a record is a paragraph, and within paragraph mode newline is always a field separator in addition to whatever `FS` is.
- A multi-character `RS` is a *regular expression* (a `gawk` extension), so records can be separated by any pattern.

Whenever `RS` matches something other than a fixed newline, `gawk` exposes the *actual* text that terminated each record in the variable `RT`, and `awkrs` sets `RT` to match. Its compatibility matrix documents the precise paragraph-mode behavior — in `RS=""` mode, `RT` captures the full run of trailing newlines plus the blank lines that separated the records, and the final record captures any EOF-trailing newlines — a corner case `awkrs` pins with a parity test. `awkrs` supports streaming multi-character and regex `RS` over both `stdin` and files, with trailing-newline trimming that matches `gawk`.

Why the field model is also a performance story

The field model is not only a correctness story; it is where a large part of `awkrs`'s speed comes from, and the two are connected. Because the common case is *reading* a field and never modifying it, `awkrs` stores fields as (`offset`, `length`) byte ranges into the record string rather than as owned copies — a field is just a slice until you assign to it. For the most common program of all, `{ print $1 }` over a whitespace-delimited file, `awkrs` goes further still: on a memory-mapped regular file it scans raw bytes to find the Nth whitespace field and writes it straight to the output buffer, with no record copy and no UTF-8 validation. The full record model — rebuild on field assignment, re-split on `$0` assignment, all of POSIX's bidirectional consistency — is there when you need it, and the moment your program only reads fields, `awkrs` collapses the whole edifice to a byte-range scan. Chapter 16 returns to these paths in detail; the point here is that the rich model and the fast path are the same model, with the expensive machinery elided exactly when it is provably unused.

Trailing newlines and the carriage-return rule

One field-model detail is easy to overlook until it bites: what is in `$0` at the edges of a record. When `RS` is the default newline, the terminating newline is *not* part of the record — `$0` is the line without its `\n`, which is why `length($0)` counts the visible characters and why `print $0` re-adds the separator through `ORS` rather than doubling it. The corner case is CRLF input, the Windows line ending, read on a Unix system. The POSIX text-mode behavior `gawk` follows, and that `awkrs` now matches, is to strip *only* the trailing `\n` and to leave any `\r` before it in place — so a CRLF-terminated line read on Unix carries a visible `\r` at the end of `$0`, where `length` counts it and a pattern can see it. An earlier `awkrs` stripped the `\r` too, which silently dropped carriage returns; the current behavior keeps them, on the principle that the reader should not quietly alter your bytes. If you want the `\r` gone — and for clean cross-platform processing you often do — you remove it yourself, with `sub(/\r$/, "", $0)` or by setting `RS = "\r\n"`, which makes the two-character sequence the record separator and leaves neither byte in `$0`. Chapter 23 covers the splitter that implements this; from the field model's side, the rule to remember is that `awkrs` strips the newline and preserves everything else, so what you see in `$0` is exactly what was between the separators.

FS, FPAT, FIELDWIDTHS, and CSV mode, chosen by data shape

awkrs offers four ways to turn a record into fields, and the right one is determined by the *shape* of the data, not by preference. The default is `FS`, which describes the *separators between* fields — a single space for whitespace-with-trimming, a single character for a literal delimiter, a regex for anything more complex. Reach for `FS` whenever the fields are cleanly delimited and the delimiter never appears inside a field: tab-separated data, colon-separated `/etc/passwd`, simple comma data. When the delimiter *can* appear inside a field — the classic case being CSV with quoted commas — `FS` cannot help, because it has no notion of quoting, and you reach instead for either CSV mode or `FPAT`. CSV mode, the `-k/--csv` flag aligned with `gawk --csv`, is a purpose-built RFC-style parser that handles quoting and the doubled-quote escape correctly, and it is the right answer whenever the input is genuinely CSV. `FPAT`, which describes the *fields themselves* as a regex rather than the separators, is the fallback for data that is CSV-shaped but not quite CSV, or for any case where it is easier to say what a field looks like than what divides them — `FPAT = "[0-9]+"` to pull numbers out of noise. And `FIELDWIDTHS` is for the fourth shape, fixed-width columns aligned by position rather than separated at all, where you give the byte widths and awk slices the record without any delimiter. The decision tree is mechanical: delimited and clean, use `FS`; real CSV, use `-k`; fields easier to describe than separators, use `FPAT`; aligned by column position, use `FIELDWIDTHS`. awkrs implements all four to gawk's semantics, and choosing correctly among them is most of what makes field-splitting on messy real-world data come out right.

Chapter 6 – Expressions, Types, and Coercion

`awk` has exactly two scalar types — number and string — and a third thing that is neither and both: the *numeric string*, the value that came from input and looks like a number. Almost every subtle behavior in `awk` traces back to how these three interact, and `awkrs` implements the POSIX rules for them precisely, down to the corner cases its parity suite pins against `gawk`. This chapter is the type system, such as it is, and the coercion rules that animate it.

Numbers, strings, and the uninitialized value

A scalar in `awk` is a number or a string. Numbers are IEEE-754 double-precision floats; there is no separate integer type in the language surface, though `awkrs` prints integer-valued floats without a decimal point and, under `-M`, switches to arbitrary-precision MPFR numbers (Chapter 19). A string is a sequence of bytes. An *uninitialized* variable — one you reference before assigning — is simultaneously the number zero and the empty string, and `awk` picks whichever the context demands: `x + 0` is `0`, `x ""` is `""`, and `if (x)` is false. `awkrs` notes one place where it follows the simpler numeric reading: uninitialized variables compare like numeric `0` in contexts where POSIX permits a dual `0/""` interpretation.

Values flow between the two types automatically. A string used in arithmetic is converted to a number by reading the longest numeric prefix: `"3.14abc" + 0` is `3.14`, and `"abc" + 0` is `0`. A number used as a string is formatted according to `CONVFMT` (for general conversion) or `OFMT` (specifically for `print`), both of which default to `"%.6g"`. This is why `print 0.1 + 0.2` shows `0.3` and not the full binary expansion — `OFMT`'s six significant digits round it — even though, as we will see, the *comparison* `0.1 + 0.2 == 0.3` is now false in `awkrs` because comparison is bit-exact.

The numeric string

The numeric string is the type that makes `awk`'s comparison rules subtle, and understanding it is the key to predicting `awk`'s behavior. When `awk` reads a field or a line of input, or pulls a value from `ENVIRON` or `ARGV` or a `getline`, and that value *looks* like a number, `awk` tags it as a numeric string — a value that is textually a string but carries the knowledge that it could be a number. The tag matters only for one thing: comparison.

When `awk` compares two values, it compares them numerically if both are numbers or numeric strings, and as strings otherwise. So if `$1` is the text `10` and `$2` is the text `9`, then `$1 > $2` is true (numeric `10 > 9`) because both fields are numeric strings. But `"10" > "9"` written as string *constants* in the program is false, because string constants are not numeric strings — they compare lexically, and `"1"` sorts before `"9"`. This is the single most common source of `awk` surprise, and `awkrs` implements the rule exactly: program string literals are *not* numeric strings for the ordering and equality operators, while input-derived values can be. Its compatibility matrix records the precise behavior, including that `typeof` of noisy numeric text like `"42abc"` reports `"string"` (a numeric prefix is not enough to make a `strnum`), while pure-numeric input text like `"42"` reports `"strnum"` — `gawk`'s vocabulary for the numeric string.

`awkrs` also fixes a place where an earlier version of itself diverged: numeric equality on doubles is now bit-exact, as POSIX requires. A previous build used a fuzzy `f64::EPSILON` tolerance, which made `0.1 + 0.2 == 0.3` return true even though the two sides differ by about `5.55e-17`; `awkrs` now returns `0` there, matching `gawk`.

And when a number is compared against a string literal and the comparison falls back to string form, awkrs stringifies the number via `CONVFMT` rather than a hardcoded `%.6g`, so `CONVFMT="%.2f"; print 3.14159 == "3.14"` prints `1`. These are the kinds of corner cases that separate a faithful awk from an approximate one, and they are exactly the cases awkrs's test-then-fix loop has closed.

Operators

awkrs implements awk's full operator set with the standard precedence and associativity:

- **Arithmetic:** `+` `-` `*` `/` `%` `^` (and `**` as a synonym for `^`). Exponentiation is right-associative, so `2 ^ 3 ^ 2` is `2 ^ (3 ^ 2) = 512`. The unary operators `+`, `-`, and `!` bind *looser* than `^`, so `-2 ^ 2` parses as `-(2 ^ 2) = -4`, not `(-2) ^ 2 = 4`. The `calc_id.awk` example in Chapter 26 implements exactly this precedence by hand, and its input file includes `-2 ^ 2` and `(-2) ^ 2` as test cases precisely to demonstrate the rule.
- **Comparison:** `<` `<=` `==` `!=` `>=` `>`, with the number-or-string semantics above.
- **String concatenation:** juxtaposition. Two expressions side by side with no operator between them concatenate their string forms: `$1 $2` is the first and second fields glued together, and `$1 "," $2` joins them with a comma. There is no operator character; adjacency *is* the operator, which is why awk concatenation is so visually light.
- **Logical:** `&&` `||` `!`, short-circuiting. `a && b` does not evaluate `b` if `a` is false; `a || b` does not evaluate `b` if `a` is true. awkrs compiles these to jump-patched short-circuit branches in the bytecode, not function calls.
- **Ternary:** `cond ? a : b`, the C conditional. awkrs accepts an assignment in the else-branch — `1 ? x = 1 : x = 2` — parsing it as an assignment expression per the gawk grammar, a case an earlier build rejected.
- **Membership:** `k in arr` tests whether `k` is a key of `arr` without creating it; the multidimensional form `(i, j) in arr` tests a `SUBSEP`-joined key.
- **Match:** `~` and `!~` test a string against a regex — `$0 ~ /error/` is true when the record matches. The right-hand side is a regex pattern.
- **Increment and decrement:** `++` and `--`, prefix and postfix, on variables, fields (`$n`), and array elements (`a[k]`).
- **Assignment:** `=` and the compound forms `+=` `-=` `*=` `/=` `%=` `^=` (and `**=`). awkrs added `^=` and `**=` as a parity fix; both compound through the same arithmetic.

Division by zero is fatal

One operator behavior deserves its own note, because it is a deliberate divergence from the C-style “produce infinity” reflex and a deliberate match to gawk. Division by zero — whether by `/`, by the compound `/=`, by `%`, or by `%=` — is a *fatal error* in awkrs, not a silent infinity or NaN. The program aborts with the gawk-style message `division by zero attempted` (or the `%` variant for modulo). This is the right behavior for a data-processing tool: a zero divisor is almost always a bug in the program or a surprise in the data, and producing `inf` and continuing tends to corrupt a whole run silently. awkrs makes it loud. Remarkably, this guarantee survives even into JIT-compiled code: when a hot division loop is offloaded to fusevm and compiled to native machine code, the compiled code carries a guarded zero-divisor early-exit that raises the same POSIX fatal rather than producing `inf` or hanging — Chapter 17 details exactly how. The fatal-on-zero rule is not a property of the interpreter that the JIT quietly drops; it is a property of the *language* that both execution paths honor.

Number literals follow gawk's source rules

How awk reads a number *in your program source* also follows gawk's rules, which awkrs implements: `0x` or `0X` introduces hexadecimal; a leading `0` introduces octal *only* when all the digits are `0` through `7` (so `01238` is decimal `1238`, not an error, because the `8` disqualifies the octal reading); and a number with a `.` uses a

decimal integer part (`077.5` is `77.5`, not an octal `77`). These are small rules, but they are exactly gawk's small rules, and awkrs matches them so that programs ported from gawk read their numeric literals the same way.

Whole arrays are not scalars

A final type rule that awkrs enforces strictly: an array name used in a scalar context — `print a` where `a` is an array, or `a ""`, or `a ~ /x/` — is a *fatal runtime error*, matching gawk, not a silent empty string. The converse is also fatal: using a scalar as an array, by writing `x[k]`, `k in x`, or `for (k in x)` when `x` holds a scalar, aborts with `attempt to use scalar 'x' as an array`. awkrs's compatibility matrix records that earlier builds silently auto-promoted or returned empty here, and that the current build fatals across read, `in`, `for-in`, and assignment — bringing it in line with gawk. The discipline this enforces is worth the strictness: a program that confuses a scalar and an array has a real bug, and awk surfacing it immediately beats awk papering over it.

A coercion decision table

Because coercion is where awk surprises people most, it helps to have the whole set of rules in one place, as a decision table. awkrs implements each of these, and the parity suite pins them:

- **String used in arithmetic** — converted to a number by reading the longest numeric prefix; non-numeric text reads as `0`. `"3.14abc" + 0` is `3.14`, `"abc" + 0` is `0`, `" 10" + 0` is `10` (leading whitespace is skipped).
- **Number used as a string** — formatted by `CONVFMT` for general use and `OFMT` for the implicit `print` of a number, both `%.6g` by default — *except* that an integer-valued number prints exactly, bypassing the six-significant-digit rounding, so `print 100000000` is `100000000` and not `1e+08`.
- **Uninitialized variable** — simultaneously numeric `0` and the empty string; the context picks. `x + 0` is `0`, `x ""` is `""`, `if (x)` is false.
- **Comparison of two values** — numeric if *both* are numbers or numeric strings, string otherwise. Two input fields `10` and `9` compare numerically (`10 > 9`); the program constants `"10"` and `"9"` compare lexically (`"10" < "9"`).
- **Number against a string constant** — falls back to string comparison, and the number is stringified through `CONVFMT`, so `CONVFMT="%.2f"; 3.14159 == "3.14"` is true.
- **Equality of two numbers** — bit-exact, as POSIX requires, so `0.1 + 0.2 == 0.3` is false (the two sides differ by about `5.55e-17`); awkrs removed an earlier fuzzy tolerance here to match gawk.

The single rule under all of these is that awk decides number-or-string by *origin* and *context*: a value's history (did it come from input? is it a program constant?) and the operator applied to it together determine which type it presents, and the numeric-string tag is the bookkeeping that makes the comparison cases come out right.

Operator precedence at a glance

Because precedence bugs are quiet — the program parses and runs, it just computes the wrong thing — it is worth having awk's precedence ladder in one compact view, from tightest-binding to loosest. Field reference `$` binds tightest of all, so `$NF` is `$(NF)` and `$i++` increments `i` then takes the field, which surprises people. Then come the postfix and prefix `++/--`; then exponentiation `^/**`, which is *right*-associative; then the unary operators `+`, `-`, `!`, which bind *looser* than `^` (so `-2 ^ 2` is `-(2^2)` = `-4`); then `*` `/` `%`; then `+` `-`; then string *concatenation* (juxtaposition, no operator), which sits below the arithmetic operators but above the comparisons; then the relational and equality comparisons `<` `<=` `==` `!=` `>=` `>` and the regex-match `~/!~`; then array membership `in`; then logical `&&`; then logical `||`; then the ternary `?:`; and loosest of all, assignment `=` and its compound forms, which are right-associative. Two rungs cause most real bugs: that concatenation binds tighter than comparison, so `"a" < "b" "c"` compares `"a"` against `"bc"`; and that `^` binds tighter than unary minus while associating right. The other classic trap is not precedence but parsing — `getline var <`

`file` needs parentheses against a following comparison, the `(getline var < file) > 0` idiom — covered in Chapters 7 and 21. `awkrs` implements this ladder exactly, the same one POSIX and `gawk` use, so a program ported between them parses its expressions identically; when in doubt, parenthesize, which costs nothing and removes the ambiguity.

Chapter 7 — Control Flow and State-ments

Inside an action, `awk` is a small imperative language with C-shaped control flow. This chapter covers the statement vocabulary — the loops, the conditionals, the `switch`, the loop-control verbs, and `getline`, the one statement complex enough to have its own return-value protocol.

Conditionals and loops

The conditional is C's: `if (cond) stmt` with an optional `else stmt`. The loops are C's too:

```
if (x > 0) print "positive"; else print "non-positive"

while (i < n) { sum += a[i]; i++ }

do { line = process(line) } while (line != "")

for (i = 1; i <= NF; i++) print i, $i
```

`awkrs` implements all four — `if/else`, `while`, `do/while`, and the C-style three-clause `for` — compiling each to jump-patched bytecode with the loop condition fused where possible into a single comparison-and-branch opcode (the VM has, for example, a `JumpIfSlotGeNum` superinstruction that fuses a slot read, a numeric comparison, and a conditional jump into one instruction, which is exactly the shape of a counted `for` loop's test). Chapter 16 covers the fusion; from the language's side, these are the ordinary C loops and they behave as you expect.

The second `for` form is `awk`'s signature iteration construct: `for (k in arr)` walks the keys of an associative array:

```
for (host in count) print host, count[host]
```

The order of iteration is, by default, unspecified — it follows the hash table's internal order — which is the POSIX-conformant behavior. `gawk` added a way to control it through `PROCINFO["sorted_in"]`, and `awkrs` implements that too; Chapter 8 covers the sorting modes in full.

The loop-control verbs

Within loops, `break` exits the innermost loop and `continue` skips to its next iteration, both exactly as in C. At the level of the record loop — outside any explicit loop — the verbs from Chapter 3 apply: `next` fetches the next record, `nextfile` skips to the next file, and `exit` proceeds to `END`. These compose: a record action can run an explicit `while` loop with `break` and `continue` inside it, and still `next` out of the whole record when it is done.

switch

`gawk` added a C-style `switch`, and `awkrs` implements it in `gawk`'s dialect:

```

switch ($1) {
case "GET":
case "HEAD":
    reads++
    break
case /^POST|PUT$/:
    writes++
    break
default:
    other++
}

```

Two things distinguish gawk's switch, and awkrs matches both. First, a `case` can be a *regular expression* — `case /^POST|PUT$/` matches when the switch value matches the pattern — in addition to a constant value. Second, and contrary to C, gawk's switch does *not* fall through implicitly; each `case` body runs only its own statements. The `break` statements above are therefore not strictly required to prevent fall-through the way they are in C, though they remain the idiomatic way to close a case. awkrs's parity suite includes switch among the gawk-only cases it verifies.

getline and its return protocol

`getline` is awk's explicit read. Where the implicit loop reads one record per iteration automatically, `getline` reads a record *on demand*, from any of several sources, and it is the tool for any problem that needs to look ahead, read a second file, or pull input from a command. It comes in several forms, and awkrs implements all of them:

- `getline` — read the next record from the main input into `$0`, updating `NF`, `NR`, and `FNR`.
- `getline var` — read the next main-input record into `var`, updating `NR` and `FNR` but not `$0` or the fields.
- `getline < "file"` — read the next record from a named file into `$0`.
- `getline var < "file"` — read from a named file into `var`.
- `cmd | getline` and `cmd | getline var` — run `cmd` as a subprocess and read its output, a record at a time.
- `cmd |& getline var` — read from a *coprocess*, the two-way pipe gawk added, paired with a `print ... |& cmd` that writes to it.

Every form returns a value, and the return value is a small protocol you should always check: `1` means a record was successfully read, `0` means end of input, and `-1` means an error occurred (the file could not be opened, the command failed). awkrs adds gawk's fourth value: `-2` for a retryable I/O condition, when `PROCINFO[input, "RETRY"]` is set and a timed read can be retried. The idiomatic loop is therefore:

```

while ((getline line < "config") > 0) {
    # process line
}
close("config")

```

The parentheses around `getline line < "config"` matter — without them, the `<` would bind to the comparison rather than the redirection, a precedence trap awk programmers learn early. awkrs's I/O model keeps the main record loop and an unredirected `getline` sharing one buffered reader, so that line order matches POSIX when you interleave the implicit loop with explicit `getline` calls — a detail that is easy to get wrong and that awkrs gets right. One consequence for parallelism: a program that uses *primary* `getline` (reading from the main input, including in `BEGIN`) runs sequentially even under `-j`, because the shared input position cannot be split across workers; Chapter 15 returns to this.

close and fflush

Two statements manage the I/O handles `getline` and redirected `print` open. `close(name)` closes a file or command by the exact string used to open it, returning `-1` for a name that was never opened, and for a clean close the command's exit code or `0`. `fflush([name])` flushes buffered output — all of it with no argument, or a specific handle by name. `awkrs` implements both with `gawk` parity, including the detail that `system()` flushes buffered stdout, pipes, and files before invoking its subprocess, so that output ordering across the boundary is correct. The coprocess and two-way forms (`|&`, `<&`) run under `sh -c`, and mixing `|` and `|&` on the same command is an error.

A worked getline: joining against a lookup file

The most common real use of `getline` is not look-ahead but the *side file* — a second input, read in full before the main loop, that the main loop consults. The idiom reads the lookup file with a redirected `getline` in `BEGIN`, building an array, and then indexes that array per record:

```
BEGIN {
    while ((getline line < "users.tsv") > 0) {
        split(line, f, "\t")
        name[f[1]] = f[2]
    }
    close("users.tsv")
}
{ print $1, name[$1] }
```

Several things in this small program are the points the chapter has been making. The loop condition (`getline line < "users.tsv" > 0`) has its mandatory parentheses, so the `<` is the file redirection and the `> 0` is the return-protocol check — the loop runs while records are successfully read and stops cleanly at end of input (`0`) or on error (`-1`). Each line is split on a tab into a temporary array `f`, and the lookup table `name` is built mapping the first column to the second. The `close("users.tsv")` releases the handle by the exact string used to open it, so a later reopen starts from the top. Then the main loop — the *implicit* one over the real input — does the join: for each record it prints the first field and the looked-up name. This two-input pattern is how `awk` does a relational join without a database, and it works because a file-redirectioned `getline` has its own independent stream position, separate from the main loop's — which is also, as Chapter 15 noted, why a file-redirectioned `getline` does not block parallelism the way a primary `getline` does. The lookup-file idiom is worth committing to muscle memory: it turns `awk` from a single-stream filter into a tool that can enrich one stream with data from another.

switch versus the if-chain, and portability

Two ways to write a multi-way branch deserve a direct comparison, because the choice has a portability consequence the tour's programs make consistently. `gawk`'s `switch`, which `awkrs` implements, is the explicit form — `switch (expr) { case ...: ... }` with regex cases and no fall-through — and it reads cleanly when there are many cases on one value. But `switch` is a *gawk extension*: a program using it will not run on BSD `awk` or `mawk`. The portable alternative is the chain of independent `if` statements or `else if`, and it is exactly what the tour's command-dispatch examples use — `rpn.awk`, `trie.awk`, `sql_like.awk`'s `row_passes`, `base_conv.awk` all dispatch on a value with a cascade of `if (t == "+") ...; if (t == "-") ...`, each branch `continue`-ing or `return`-ing so the conditions stay mutually exclusive in effect. That the example programs choose the `if`-chain over `switch` is deliberate: they are verified against `gawk` and meant to read as portable `awk`, and the `if`-chain runs everywhere. So the guidance is: reach for `switch` when you are targeting `gawk` and the readability of many cases on one value is worth the `gawk` dependency; reach for the `if`-chain — or its even terser cousin, the pattern-action dispatch where each command verb gets its own top-level rule, as `trie.awk` and `rle.awk` do — when portability matters. `awkrs` runs both at the same speed,

so the choice is purely about which awks the program must run on.

Chapter 8 – Arrays

awk has one data structure, and it is enough: the associative array. Every collection in awk — a set, a map, a counter, a multidimensional table, a stack, a graph, a trie — is built from associative arrays, and the examples in Chapter 26 build exactly those structures out of them. This chapter is the array model: how keys work, how the multidimensional emulation works, how iteration order is controlled, and the sorting builtins awkrs provides.

Associative by nature

An awk array is a map from string keys to scalar values. There is no array of a fixed size, no contiguous integer-indexed vector with a length; every array is a hash table, and even when you use integer indices — `a[1]`, `a[2]`, `a[3]` — the indices are really the strings "1", "2", "3". You never declare an array; it springs into existence the first time you subscript it. You add a key by assigning to it, test for a key with `in`, remove a key with `delete`, and count the keys with `length(arr)`:

```
{ count[$1]++ }           # tally first field
END {
    print length(count), "distinct keys"
    if ("root" in count) print "saw root", count["root"]
    delete count["nobody"]
}
```

`count[$1]++` is the canonical awk idiom and it shows the array's two conveniences at once: the key `$1` is created on first reference, and the value, being uninitialized, reads as `0` so that `++` makes it `1`. A histogram in awk is one line.

awkrs is strict about the array/scalar boundary, as Chapter 6 noted: using a scalar as an array or an array as a scalar is a fatal error, matching gawk. It is also careful about a POSIX subtlety its test suite pins: *reading* `a[k]` for a key that does not exist *creates* that key with an empty value, because awk cannot read a nonexistent element without materializing it. This is why `if (a[k])` has the side effect of creating `a[k]`, and why you test membership with `k in a`, which does not. awkrs fixed this auto-create-on-read behavior to match gawk exactly.

delete

`delete a[k]` removes a single key. `delete a` (with no subscript) removes every key, emptying the array but leaving it an array. awkrs implements both, and it follows gawk on the error cases: `delete x` or `delete x[k]` on a *scalar* `x` is the fatal *attempt to use scalar 'x' as an array*, while `delete` on an unassigned name is a silent no-op as POSIX allows. Several Chapter 26 examples lean on `delete arr` to reset a working table between records — `levenshtein.awk` does exactly this, clearing its DP table with `delete d` after each pair of words.

Multidimensional arrays and SUBSEP

awk has no true multidimensional arrays, but it emulates them with a convention that awkrs implements faithfully. When you write `a[i, j]`, awk joins the subscripts with the value of `SUBSEP` — a separator character

that defaults to a non-printing byte (`0x1c`, the ASCII file separator) precisely so it is unlikely to occur in real keys — producing a single string key `i SUBSEP j`. So `a[2, 3]` is really `a["2\0343"]`, and the two-dimensional access is sugar over a one-dimensional map with composite keys. The membership test has a matching form: `(i, j) in a` tests for the composite key without creating it.

This convention is more powerful than it looks, because it generalizes to any number of dimensions and any mix of types, and the Chapter 26 examples use it constantly: `levenshtein.awk` holds its edit-distance DP table in `d[i, j]`, `trie.awk` keys its trie edges as `edge[node, char]`, and `a_star.awk` keys a priority queue on coordinate tuples. The idiom even appears in `print: (i, j) in arr` may stand alone as a `print` argument to emit several fields, because the parenthesized comma list is gawk-legal in that position.

Iteration and sorted_in

`for (k in arr)` walks the keys. The default order is the hash table's order, which is unspecified by POSIX and differs across awk implementations — a fact that matters because it means a program that relies on iteration order is not portable. gawk gave you control over the order through the special element `PROCINFO["sorted_in"]`, and awkrs implements it fully:

```
PROCINFO["sorted_in"] = "@ind_str_asc"
for (k in arr) print k, arr[k]    # keys in ascending string order
```

The built-in modes follow gawk's vocabulary — `@ind_str_asc`, `@ind_num_asc`, `@val_str_asc`, `@val_num_desc`, and the rest — sorting by index or by value, as string or as number, ascending or descending. awkrs goes further and supports a *user comparator function*: set `PROCINFO["sorted_in"]` to the name of a function, and awk calls it to order the keys. A two-argument comparator receives indices and does an index sort; a four-argument comparator (`i1, v1, i2, v2`) receives both indices and values and does a value sort; either returns a negative, zero, or positive number like a C `qsort` comparator. The `csv_pivot.awk` example in Chapter 26 sets `PROCINFO["sorted_in"] = "@ind_str_asc"` precisely so that its per-host output is sorted by host name for deterministic, byte-stable output — which is also what lets it pass the parity suite against gawk.

asort and asorti

When you need a sorted *copy* rather than a sorted iteration, awkrs provides gawk's `asort` and `asorti`. `asort(source [, dest [, how]])` sorts the *values* of `source` and stores them under integer keys `1, 2, 3, ...` in `dest` (or back into `source` if `dest` is omitted), returning the count. `asorti` does the same for the *indices*. Both accept the same comparison-mode strings as `sorted_in`. awkrs's compatibility matrix records a careful corner case: `asort` or `asorti` on an *unassigned* name treats it as an empty array and returns `0` rather than erroring, while a *scalar* argument still raises the “first argument is not an array” fatal — and the compiler tracks these argument positions so the name is promoted to an array slot correctly. One parallelism note: `asort` and `asorti` are among the operations that make a program *not* parallel-safe, because they impose a global ordering across the whole array, so a program using them runs sequentially under `-j`.

Arrays as the universal structure

The payoff of the associative array is that it is the *only* structure you need to learn, and everything else is a pattern of using it. A set is an array whose values you ignore (`seen[$0] = 1`). A counter is an array you increment (`count[$1]++`). A graph is an array of adjacency keyed by (`from, to`). A stack is an array plus an integer top-of-stack index. A tree or trie is an array of node records keyed by (`node, field`). The Chapter 26 tour is, in large part, a demonstration of how far this one structure reaches: BSTs, heaps, tries, union-find, suffix arrays, segment trees, and Held-Karp bitmask DP tables are all built from nothing but associative arrays and the `SUBSEP` convention. awkrs makes the hash table underneath fast, but the model — one structure,

used a hundred ways — is `awk`'s, unchanged since 1977.

Counting keys and the cost of membership

Three operations on an array deserve a precise note because programs lean on them constantly and a wrong mental model produces subtle bugs. `length(arr)` returns the number of keys — `awk` implements the `gawk` extension where `length` of an array is its element count, distinct from `length` of a string — so a histogram's distinct-key count is one call. The membership test `k in arr` answers whether a key exists *without creating it*, and this is the one to reach for in a condition, because the alternative — reading `arr[k]` in an `if` — has the side effect Chapter 6 and this chapter both stress: *reading* a nonexistent element materializes it with an empty value, because `awk` cannot return a value for `arr[k]` without the element existing. So `if (arr[k])` silently grows the array, while `if (k in arr)` does not, and the difference matters in a loop that tests many keys, both for correctness (a later `for-in` would visit the phantom keys) and for memory. `delete arr[k]` removes one key and `delete arr` empties the whole array while leaving it an array, the reset the DP examples of Chapter 26 use between iterations. The cost model underneath is the hash table's: insertion, lookup, membership, and deletion are all expected constant-time, and `length` is tracked rather than counted, so none of these is a hidden linear scan — the one linear cost is `for-in` itself, which must visit every key, and `asort/asorti`, which must order them.

Multidimensional keys of any arity

The `SUBSEP` convention is not limited to two dimensions, and the tour's programs lean on that generality. `a[i, j]` joins two subscripts with `SUBSEP`; `a[i, j, k]` joins three; and the membership form `(i, j, k) in a` tests the composite key without creating it. Because the mechanism is simply “join the subscripts with a separator byte,” it generalizes to any number of dimensions and any mix of value types — `tsp_dp.awk` keys its DP table on `(bitmask, last_city)`, `aho_corasick.awk` keys its trie edges on `(node, char)`, `sudoku.awk` keys three witness sets on `(unit, value)`. The one caveat is the one POSIX names: `SUBSEP` defaults to the non-printing byte `0x1c` precisely so it is unlikely to occur inside a real subscript, but if your *data* can contain that byte, a composite key could collide, which is why `SUBSEP` is settable — you change it to a byte your keys cannot contain. The round trip completes with `split(key, parts, SUBSEP)`, which recovers the original subscripts from a key obtained during `for-in` iteration, as `markov.awk` and `floyd_warshall.awk` do. So the “multidimensional array” is, precisely, a one-dimensional map whose keys are `SUBSEP`-joined tuples, and the dimensionality is a property of how you build and split the key, not of the array — which is why a single mechanism serves two, three, or any number of dimensions with no special cases.

Chapter 9 – User-Defined Functions

awk’s one-liners get the attention, but awk is a real programming language, and the proof is that you can write recursive-descent parsers, graph algorithms, and arbitrary-precision arithmetic in it — the Chapter 26 examples do all three. The construct that makes this possible is the user-defined function, and awkrs implements it with the full semantics that the examples depend on: parameter locals, recursion, and — the subtle one — call-by-reference for array arguments.

Defining and calling

A function is declared with `function`, takes a parenthesized parameter list, and has a brace-delimited body:

```
function max(a, b) {  
    return (a > b) ? a : b  
}
```

You call it like any builtin: `m = max($1, $2)`. Functions can be mutually recursive and can be defined in any order — awk reads the whole program before running it, so a function may call another defined later in the file. A function that does not execute a `return`, or executes a bare `return` with no value, returns the empty string / numeric zero.

Parameters are locals; extra parameters are your locals

Here is awk’s one genuinely unusual convention, and it is a convention every awk programmer internalizes: awk has no separate syntax for local variables. A function’s *parameters* are its locals, and to get additional locals you simply declare more parameters than the caller will pass. The universal style — visible in every Chapter 26 example — is to list the real parameters, then some extra whitespace, then the “local” parameters that callers never supply:

```
function lev(a, b,    la, lb, i, j, cost, mn) {  
    la = length(a); lb = length(b)  
    ...  
}
```

In this real signature from `levenshtein.awk`, `a` and `b` are the arguments the caller passes, and `la`, `lb`, `i`, `j`, `cost`, and `mn` — separated by the conventional run of spaces — are locals. They are uninitialized on entry, private to each invocation, and invisible to the caller. The extra spacing is purely a human convention to mark the boundary; awk itself does not distinguish “real” parameters from “local” ones, it simply binds as many as the caller supplies and leaves the rest uninitialized. This is awk’s idiom for lexical scope, and once you have read a few function signatures it is perfectly legible: everything after the visual gap is scratch space.

Recursion, and the cap that keeps it safe

awk functions recurse, and the examples lean on it heavily — `calc_rd.awk`’s expression grammar is a set of mutually recursive descent functions, `bst.awk`’s traversals recurse over the tree, `scc_tarjan.awk` and

`sudoku.awk` recurse deeply. `awkrs` supports all of this, and it adds a safety net the language standard does not require: user-defined recursion is *capped*. In non-test builds the limit is 150 nested calls (lower in unit tests), so a pathological self-call — a function that recurses without a base case — fails with a clear error instead of overflowing the host process's stack and crashing. This is a deliberate robustness choice: an `awk` script is often run on untrusted or malformed input, and a stack-overflow segfault is a worse failure than a controlled error message. The cap is high enough that no reasonable algorithm hits it and low enough that runaway recursion is caught.

Arrays pass by reference

Scalars pass to functions *by value* — a function that modifies a scalar parameter does not affect the caller's variable. But arrays pass *by reference*, and this is the property that makes `awk` functions actually useful for building data structures, because it means a function can populate or mutate a caller's array. `awkrs` implements array call-by-reference correctly, including the harder cases: a function can read an array parameter, write to it, iterate it with `for-in`, and `delete` from it, and all of those operations act on the caller's array. The engineering report lists “array call-by-reference for user functions (incl. delete)” and the “frame-aware `array_elem_get` / `array_elem_set` / `for_in_keys` / `delete` for by-ref array params” among its fixed divergences, because getting every one of those operations to act through the reference — not just plain reads and writes but iteration and deletion too — is the part naive implementations miss. The Chapter 26 examples depend on this constantly: any example whose helper functions build up a shared table — `trie.awk`'s `tr_add` mutating the global trie arrays, the many DP examples filling tables — relies on arrays being shared, not copied.

The asymmetry — scalars by value, arrays by reference — is exactly POSIX `awk`'s, and it has a clean justification: a scalar is small and copying it is cheap and usually what you want, while an array can be enormous and copying it on every call would be ruinous, so `awk` shares it. The practical consequence to internalize is that a function returns *one* scalar through `return`, but can produce *many* results by filling an array passed in by the caller — which is why so many `awk` library functions take an array as an “out parameter,” the way `split`, `match` with a third argument, and `asort` do.

Indirect calls

`gawk` added the ability to call a function *by name computed at runtime*, and `awkrs` implements it. If a variable holds the string name of a function, `@name(...)` calls it, and `@(expr)(...)` calls the function whose name `expr` evaluates to:

```
function add(a, b) { return a + b }
function mul(a, b) { return a * b }
BEGIN {
    op = "add"
    print @op(2, 3)      # 5 - calls add
    op = "mul"
    print @op(2, 3)      # 6 - calls mul
}
```

This is `awk`'s reflection-flavored feature, useful for dispatch tables and plugin-style architectures where the operation is chosen by data. `awkrs` supports it, with the caveat its compatibility matrix records that some edge cases versus `gawk` are not guaranteed. One parallelism consequence: indirect calls make a program not parallel-safe — `awkrs` cannot statically prove what an indirect call does, so it conservatively forces sequential execution under `-j`.

Functions and the namespace system

A function name can be namespaced with gawk's `@namespace` mechanism (Chapter 12), so a function defined under `@namespace "graph"` and named `bfs` becomes `graph::bfs` to code outside the namespace. The namespace pass renames the function definition and its qualified call sites, while leaving the function's own parameters unprefixed — they are locals and stay local. This is gawk's solution to the library-collision problem, and awkrs implements the prefixing rules in `namespace.rs`, which Chapter 12 covers in detail.

Returning many values: the out-parameter discipline

A function returns exactly one scalar through `return`, and that single fact shapes how awk libraries are written, because real functions often need to produce more than one result. The asymmetry from the previous section is the lever: scalars pass by value but arrays pass by reference, so a function that must yield several values takes an *array* parameter from the caller and fills it. The builtins model this — `split` fills its array, three-argument `match` fills its capture array, `asort` fills its destination — and user functions follow suit. The Chapter 26 examples use the pattern constantly: `tokenize` fills a global token array rather than returning the tokens, the DP solvers fill a shared table, and where the result is a small fixed *pair* rather than a list, the convention shifts to agreed global variables — `dijkstra.awk`'s heap pop writes its two results to `popd` and `popk` because a heap entry is a distance and a node and a function cannot return both. The discipline to internalize is that a function's `return` value is its *one* answer, and everything else it produces it produces through arrays passed in or globals agreed on — which is not a limitation so much as awk's particular spelling of multiple return values, and once you read it that way the library functions and the example functions are using the same idiom.

Variable scope: the global default and the parameter-local exception

awk's scoping model is worth stating bluntly because it is the opposite of most languages' default and a frequent source of subtle bugs. Every variable in an awk program is *global* unless it is a function parameter. There is no block scope, no file scope, no `local` keyword — a variable mentioned anywhere, in any rule or function, is the same variable everywhere. The *only* locals are a function's parameters, which is why awk's universal convention, visible in every tour example, is to declare extra parameters past the ones the caller supplies and use them as scratch locals, set off by the conventional run of spaces — `function lev(a, b, la, lb, i, j, cost, mn)` where `a` and `b` are arguments and the rest are locals. The practical consequence is a real hazard: a loop variable `i` used inside a function, if not declared as a parameter-local, is the *same* `i` as in the calling rule, so a function that loops on `i` and is called from inside another loop on `i` will corrupt the caller's loop. The discipline that prevents it is universal: every variable a function uses for its own work, including every loop index, goes in the parameter list as a local. The tour's programs follow this without exception, which is why their helper functions list so many trailing parameters — `manacher`, `huffman`, `scc_tarjan` each declare a dozen locals — and reading those signatures, everything after the visual gap is the function's private scratch space. awk's global-by-default model is a holdover from its small-program origins, and the parameter-local idiom is the discipline that makes larger programs safe within it.

Chapter 10 – Regular Expressions

Regular expressions are not a feature bolted onto `awk`; they are part of its grammar. A `/pattern/` is a first-class literal, `~` and `!~` are operators, and half a dozen builtins take a regex argument. `awkrs`'s regex story has one large architectural decision at its center — it uses the Rust `regex` crate rather than a hand-rolled or GNU-derived engine — and that decision determines both what `awkrs` can do that traditional `awks` cannot and the one thing it deliberately cannot do that they can.

The engine: Rust regex, linear-time

`awkrs`'s regular expressions are compiled and matched by the Rust `regex` crate, a finite-automaton engine that guarantees *linear-time* matching in the length of the input. This is a significant property and a deliberate trade. The classic backtracking regex engines used by Perl, PCRE, and many `awks` can, on certain pathological patterns (the famous `(a+)+` against a long non-matching string), take time *exponential* in the input length — the “catastrophic backtracking” that has taken down production systems. The Rust engine cannot do this: it is built on Thompson-NFA and DFA techniques that match every pattern in time linear in the subject, no matter how the pattern is written. For a tool that routinely runs over millions of lines of possibly-adversarial input, this is the right default, and it shows up directly in the benchmarks — `awkrs`'s regex-filter benchmark runs a million lines in about eleven milliseconds while `gawk` takes three hundred and fifty.

The trade is that a linear-time engine cannot support *backreferences in the pattern* — constructs like `(.)\1` that match a character followed by a copy of itself — because backreferences are precisely what make matching potentially super-linear. `awkrs` is explicit about this in its compatibility matrix: pattern-side backreferences are **not** supported, because the Rust regex is linear-time and does not implement them. This is the one regex capability you give up, and in practice it is rarely missed in `awk` work — the overwhelming majority of `awk` patterns are anchors, classes, alternations, and quantifiers, all of which the engine handles. And backreferences in *replacement* text — `\1` through `\9` and `&` in `gensub` — are fully supported, because those are a different mechanism (referring to captured groups when *building* the output) that does not affect matching complexity.

`awkrs` configures the engine to match `awk`'s expectations. All compiled regexes use `dot_matches_new_line(true)`, so `.` matches a newline, following the `gawk` ERE convention. Interval quantifiers `{m,n}` are always enabled, which is why the `-r/--re-interval` flag that `gawk` uses to turn them on is a harmless no-op in `awkrs` — they are never off. POSIX character classes like `[[:digit:]]` and `[[:alpha:]]` are supported. And `awkrs` follows `gawk` on the corner cases its parity work pinned down: `\d` and `\D` are treated as the literal letters `d` and `D` (`gawk` ERE does not give `\d` the Perl “digit” meaning), and a `\1` through `\9` in a POSIX ERE no longer hard-errors.

Regexes everywhere they appear

A regex literal appears in several roles, and `awkrs` handles the lexical disambiguation — division-slash versus regex-slash — exactly as Chapter 4 described. In pattern position, `/re/` is `$0 ~ /re/`. As the right-hand side of `~` and `!~`, it is the pattern to match. As an argument to the regex builtins — `match`, `sub`, `gsub`, `gensub`, `split`, `patsplit` — it is the pattern. And a regex can also be *dynamic*: any string can be used where a regex is expected, and `awk` compiles it on the fly. `$0 ~ pat` where `pat` is a variable holding a pattern string works, which is what lets `calc_id.awk` write `c ~ /[0-9.]/` with a literal but also lets programs build patterns at runtime. `awkrs` caches compiled `Regex` objects in a hash table keyed by the pattern string, so a dynamic regex

used in a loop is compiled once, not once per iteration — an important optimization for the common case of a pattern that is “dynamic” in syntax but constant in value.

Typed regex values: `@/pat/`

gawk added a genuine *regexp* type — a value that is a compiled pattern, not a string — written `@/pattern/`, and awkrs implements it. The expression `@/foo/` evaluates to a regexp value, `typeof` reports it as `"regexp"`, and it can be stored in a variable and used with `~`:

```
BEGIN {
    pat = @/^[0-9]+$/
    if ("123" ~ pat) print "numeric"
}
```

The distinction between a regexp value and a string that happens to contain regex metacharacters matters in a few corner cases gawk defined, and awkrs carries the type through so `typeof` and matching behave as gawk specifies. The DAP debugger’s value formatter even renders a regexp value as `@/re/` so you can see the type in the debugger, a small sign that the type is real throughout the implementation and not just at the surface.

IGNORECASE

Setting `IGNORECASE` to a true value makes regex matching case-insensitive, and awkrs implements gawk’s exact scope for it: `IGNORECASE` applies to multi-character regex `FS`, to `match`, `sub`, `gsub`, `split`, and `gensub`, and to the `~` and `!~` operators. Crucially — and matching gawk — it does *not* apply to a single-character string `FS` or a single-character `split` separator, which stay literal regardless of `IGNORECASE`. On Unix, awkrs also extends case-insensitive and locale-aware behavior to `==`, `!=`, and ordering comparisons via `strcoll` when `IGNORECASE` is set, so case-insensitive equality of fields works as gawk intends. This precise scoping — on for regex contexts and multi-char separators, off for single-char literal separators — is a gawk parity detail awkrs’s compatibility matrix spells out.

The replacement builtins

Three builtins rewrite text through regexes, and they differ in important ways:

- `sub(re, repl [, target])` replaces the *first* match of `re` in `target` (or `$0` if omitted) with `repl`, returning the number of substitutions (0 or 1). In `repl`, an `&` stands for the matched text and `\&` is a literal ampersand.
- `gsub(re, repl [, target])` is the *global* version, replacing every non-overlapping match, returning the count. awkrs implements gawk’s zero-width-match behavior: `gsub(/, ...)` matches at every position.
- `gensub(re, repl, how [, target])` is gawk’s more powerful substitution: it does not modify the target in place but *returns* the new string, `how` selects which match to replace (a number, or `"g"` for all), and `repl` may contain `\1` through `\9` to refer to captured groups as well as `&` for the whole match. awkrs implements the `\N` backreference expansion, a parity fix its report records.

awkrs optimizes these heavily for the common cases — when the target is `$0`, the substitution and the record rebuild happen in one step; literal needles reuse a cached substring finder; and constant operands avoid per-call allocation by passing through a copy-on-write reference. The memory-mapped fast path even recognizes `gsub("lit", "repl"); print` with a literal pattern and, when the needle is absent from a line, copies the line straight through without running the VM at all — which is exactly the no-match `gsub` benchmark in Chapter 14, where awkrs runs a million lines in fourteen milliseconds against mawk’s seventy-four and gawk’s four hundred and thirty-six.

match and its capture array

`match(string, re)` finds the first match of `re` in `string`, returns its 1-based position (or 0 for no match), and sets `RSTART` to that position and `RLENGTH` to the match length (or -1 for no match). gawk added a three-argument form, `match(string, re, arr)`, that captures the submatches into `arr`, and awkrs implements it with gawk's exact subscripting: for each successful submatch `i`, it writes `arr[i]` (the matched text), `arr[i, "start"]` (the 1-based character index), and `arr[i, "length"]`, and — matching gawk — an *unmatched* optional group produces *no* entries rather than empty ones. This precise behavior is in awkrs's compatibility matrix, and it makes the three-argument `match` a complete sub-pattern extractor: one call gives you every group's text and position.

The compiled-regex cache, and dynamic patterns in loops

A regex literal is compiled once, at program load, into a Rust `Regex` — but a *dynamic* regex, where a string variable stands in for a pattern, raises a cost question: is the pattern recompiled on every use? For a pattern used inside a loop over millions of records, recompiling per iteration would dominate the runtime. awkrs avoids this with a compiled-regex cache: it keeps a hash table mapping a pattern string to its compiled `Regex`, so the first use of a given dynamic pattern compiles it and every subsequent use — even across records — reuses the compiled automaton. This is what makes `$0 ~ pat` cheap when `pat` is “dynamic” in syntax but constant in value, the overwhelmingly common case, where a variable holds a fixed pattern that simply was not written as a literal. The cache keys on the pattern text, so a pattern that genuinely changes every iteration still pays per distinct value, but a pattern that is the same string each time is compiled exactly once. The typed regexp value `@/.../` of the previous section is the other half of this story: when you want to be sure a pattern is a compiled regex carried as a value rather than a string reinterpreted on each match, `@/pat/` makes that explicit, and the engine threads the compiled form through directly. Between the load-time compilation of literals, the string-keyed cache for dynamic patterns, and the typed regexp value, awkrs ensures the linear-time engine's one real cost — compiling the automaton — is paid as few times as the program's structure allows.

A regex feature checklist

It helps to have, in one place, what the Rust-engine regexes do and do not support, since the boundary is sharp and a program that strays across it fails clearly rather than silently. Supported: the anchors `^` and `$`; the dot `.`, which matches newline under awkrs's `dot_matches_new_line` setting (gawk ERE convention); character classes `[...]` including ranges and negation `[^...]`; the POSIX named classes `[[:digit:]]`, `[[:alpha:]]`, `[[:space:]]`, and the rest; alternation `|`; grouping `(...)` with capture, used by `gensub` and three-argument `match`; the quantifiers `*`, `+`, `?`, and the interval forms `{m}`, `{m,}`, `{m,n}`, which are always enabled so the `-r` flag is a no-op; and Unicode-aware matching. Not supported: pattern-side *backreferences* like `(.)\1`, the one capability the linear-time engine cannot provide and the matrix's only `No` cell — backrefs in *replacement* text (`gensub`'s `\1`, `\9` and `&`) are a different mechanism and are fully supported. Two gawk-parity details awkrs pins: `\d` and `\D` are treated as the literal letters `d` and `D` rather than the Perl digit class, because gawk ERE does not give them that meaning, and a `\1`-through-`\9` in a POSIX ERE no longer hard-errors. The guarantee this buys is the important one: because the engine is finite-automaton-based, every supported pattern matches in time linear in the subject, with no catastrophic-backtracking cliff — which for a tool that runs over millions of lines of possibly-adversarial input is the right trade for giving up pattern-side backreferences.

Chapter 11 — The Builtin Library

awk’s builtin library is small by the standards of a modern language — a few dozen functions — and that smallness is a feature: you can hold the whole library in your head, and there is no import step, no namespace to qualify, no documentation to search before you can call `length`. awkrs implements the full POSIX core plus gawk’s additions, sixty-three builtin names in all, and this chapter is a tour of them by category. The reference appendix at the back of the book gives the exact signatures; this chapter is the guided version, with the behaviors and corner cases that matter.

String functions

The string builtins are awk’s bread and butter, since awk is a text tool:

- `length([s])` returns the length of `s`, or of `$0` if called with no argument (the bare `length` with no parentheses is the idiomatic “length of the current record”). With `-b`, length is counted in bytes; otherwise in characters.
- `substr(s, start [, len])` extracts a substring. awkrs follows the *gawk* rule rather than the strict POSIX one for an out-of-range start: if `start < 1`, awk clamps it to `1` and leaves `len` unchanged, where strict POSIX would shorten the result by `1 - start`. This is a documented, deliberate gawk-parity choice.
- `index(s, t)` returns the 1-based position of the first occurrence of `t` in `s`, or 0 if absent. An empty needle returns `1`, matching gawk.
- `split` and `pat_split` — covered in Chapter 5 — break a string into an array.
- `sub`, `gsub`, `gensub`, `match` — covered in Chapter 10 — do regex rewriting and matching.
- `sprintf(fmt, ...)` formats like `printf` but returns the string instead of printing it; Chapter 13 is the full formatting story.
- `tolower(s)` and `toupper(s)` change case.

Arithmetic functions

The math library is the POSIX standard set, and awkrs implements it with a careful eye on the edge cases:

- `sin`, `cos`, `atan2`, `exp`, `log`, `sqrt` — the transcendentals. awkrs follows gawk on the domain errors: `log` and `sqrt` of a negative argument emit a warning on stderr (`awkrs: warning:`) and return NaN, rather than crashing or silently producing garbage, and the warning is gated so it fires when `LINT` is active. The NaN sign is normalized to `+nan` at the math-function source, so awkrs and gawk agree on the spelling.
- `int(x)` truncates toward zero.
- `rand()` returns a pseudo-random number in `[0, 1)`, and `srand([seed])` seeds the generator (seeding from the current time if no seed is given, and returning the previous seed). The exact random *sequence* is, like every awk’s, not guaranteed to match any other engine — random sequences are explicitly unspecified by POSIX — but a given seed produces a repeatable sequence within awkrs.

awkrs additionally exposes `intdiv(a, b, arr)` (gawk’s integer division producing quotient and remainder) and the always-zero-safe `intdiv0`, which returns 0 rather than erroring on a zero divisor.

I/O and general functions

- `print` and `printf` — the output statements, with redirection to files (`>`, `>>`), pipes (`|`), and coprocesses (`|&`).
- `getline` — the explicit read, Chapter 7.
- `close(name)` and `fflush([name])` — handle management, Chapter 7.
- `system(cmd)` runs `cmd` through the shell and returns its exit status, flushing buffered output first so ordering is correct across the boundary.

Time functions

gawk's time builtins are implemented:

- `systime()` returns the current Unix timestamp in seconds.
- `strftime([format [, timestamp [, utc]])` formats a timestamp using a `strftime`-style format; with no arguments it uses the current time and a default format. awkrs's default format is gawk's `date(1)`-equivalent `"%a %b %e %H:%M:%S %Z %Y"` (exposed as `PROCINFO["strftime"]`), not `"%c"` — a parity detail its matrix records.
- `mktime(spec [, utc])` converts a `"YYYY MM DD HH MM SS [DST]"` specification to a timestamp; awkrs implements the optional second argument that forces UTC interpretation when truthy, with the one-argument form using local time.

Bitwise functions

gawk's bit operations work on values treated as unsigned 64-bit integers, and awkrs implements all six: `and`, `or`, `xor` (each variadic, taking two or more arguments), `compl` (bitwise complement), `lshift`, and `rshift`. awkrs follows gawk on the error cases: `lshift`, `rshift`, and `compl` of a *negative* argument are fatal (**negative values are not allowed**), because a negative shift count or complement is a value-dependent trap. This distinction matters for the JIT, as Chapter 17 explains: the trap-free `and`, `or`, and `xor` can be compiled to native code, while the trapping `lshift/rshift/compl` stay on the interpreter because their fatal-on-negative behavior is value-dependent.

Type and array functions

- `typeof(x)` reports the type of `x` as a string — `"number"`, `"string"`, `"strnum"`, `"array"`, `"regexp"`, `"untyped"` for an unset variable, or `"unassigned"` — using gawk 5.x vocabulary. awkrs fixed `typeof` to return `"untyped"` for unset variables to match gawk.
- `isarray(x)` returns 1 if `x` is an array, 0 otherwise.
- `mkbool(expr)` produces a typed boolean value (gawk's boolean type).
- `asort` and `asorti` — the array sorters, Chapter 8.

strtonum and the non-decimal builtins

`strtonum(s)` converts a string to a number honoring `0x` hex and `0` octal prefixes — the conversion that the `-n/--non-decimal-data` flag turns on for *input* generally. So `strtonum("0x1F")` is 31 and `strtonum("010")` is 8. This is gawk's function for reading non-decimal numeric text explicitly, and awkrs implements it.

The arity discipline

A cross-cutting property worth calling out: awkrs enforces builtin *arity* uniformly and safely. Calling a builtin with the wrong number of arguments produces a uniform error — `"N is invalid as number of arguments for X"` — across the whole library, rather than panicking on some functions, silently ignoring extra arguments on others, and using inconsistent wording on the math functions, all of which earlier builds did. The engineering report lists this as a post-v0.4.2 parity batch: arity panics became clean runtime errors across `close`, `rand`, `srand`, `asort`, `asorti`, `system`, `printf`, `delete`, the bitwise and time builtins, and `typeof`. The static checker also rejects obviously wrong arities at compile load time — `gsub/sub` need at least two arguments, `split/match` at least two, `patsplit` two to four, `gensub` three or four — so a misused builtin fails before the program runs rather than midway through. This is the kind of robustness that does not show up in a feature list but makes the difference between a tool you trust on bad input and one you don't.

Why the library stays small, and why that is a feature

It is worth stating plainly why awkrs's builtin library is sixty-three names and not six hundred, because in an age of batteries-included standard libraries the smallness can read as a limitation rather than a choice. It is a choice, and it is awk's oldest one. The whole library fits in your head: a working awk programmer holds the string functions, the math functions, the I/O statements, and the gawk additions in memory and reaches for them without documentation, which is exactly the property that makes the one-liner culture possible — you cannot improvise an `awk '{...}'` at a shell prompt if you have to look up which of two hundred functions does what. The smallness is also why there is no import step, no namespace to qualify before calling `length`, no version skew between the language and its library. awkrs preserves this deliberately: it adds gawk's *capabilities* — the filesystem access, the time functions, the bit operations, the sorting — but it adds them as a few dozen more barewords, not as a package ecosystem, and it draws the line at dynamic extension loading (Chapter 12) precisely to keep the library a closed, knowable set inside one binary. The contrast with a language like the sibling strykelang, which makes the opposite bet with ten thousand builtins, is instructive: strykelang's wager is that an encyclopedic library is worth learning; awk's wager, which awkrs keeps, is that a *small* library you have fully internalized beats a large one you have to search, for the specific workload of improvising text processing at a prompt. Both bets can be right for their workload. awkrs's is right for awk's.

A map to the full catalogue

This chapter and the two that follow it gave the *guided* tour of the builtin library — the behaviors and corner cases that matter, grouped by what they are for. For the exhaustive, one-function-at-a-time reference — every builtin with its exact signature, semantics, edge cases, and a worked example — the catalogue is Chapters 27 through 29, organized so a reader looking something up finds it fast: Chapter 27 is the string and substitution builtins (`length`, `substr`, `index`, `split`, `patsplit`, `sub`, `gsub`, `gensub`, `match`, `sprintf`, `tolower`, `toupper`); Chapter 28 is the numeric, bitwise, and type builtins (the math library, `rand`/`srand`, `strtonum`, `intdiv`, the six bit operators, `typeof`, `isarray`, `mkbool`, `asort`/`asorti`); and Chapter 29 is the I/O, time, and gawk-extension builtins (`print`/`printf`/`getline`/`close`/`fflush`/`system`, the time functions, and the native filesystem and gettext extensions). The signatures there are the ones awkrs's Language Server reports on hover, from the same single doc corpus that generates `reference.html`, so the catalogue, the editor, and the published reference all agree. Read this chapter and Chapters 11 and 13 to learn the library; reach for Chapters 27 through 29 when you need the precise behavior of one function; and consult Appendix B for the one-line summary of all sixty-three names at once.

Chapter 12 – The gawk Extensions

POSIX awk is the common denominator, but the awk that people actually write in 2026 is, very often, gawk — and gawk has accreted a substantial set of extensions over thirty years that show up in real scripts. awkrs's positioning, stated in its own README, is to implement “the gawk extensions that show up in real scripts,” and this chapter is the catalog of what made the cut: the introspection variables, the source directives, the namespace system, the network and coprocess I/O, the gettext internationalization builtins, and the filesystem extensions. Each is grounded in the source.

PROCINFO, SYMTAB, and FUNCTAB: introspection

gawk exposes the running interpreter's own state through three special arrays, and awkrs implements all three, with the precise key sets its compatibility matrix documents.

PROCINFO is the richest. It is an array of facts about the process and the runtime, refreshed both before and after **BEGIN** so that values you set persist and values the runtime maintains stay current. awkrs populates it with gawk-style keys: **platform** (mapped to gawk's **posix/mingw/vms** vocabulary rather than Rust's **macos/linux**), **version**, the process and group ids, **errno**, the awk API major/minor numbers, **argv**, **identifiers** (the program's variable names), **FS** (the active split mode), **strftime** (the default time format), **pgripid**, the supplementary group ids as **groupN**, and **mb_cur_max**. It also carries the **sorted_in** control element from Chapter 8 and, in bignum mode, the MPFR configuration keys **prec**, **roundmode**, **prec_min**, **prec_max**, **gmp_version**, and **mpfr_version**. The per-input read-timeout and retry machinery lives here too, through composite keys like **PROCINFO[input, "READ_TIMEOUT"]** and **PROCINFO[input, "RETRY"]**, with a documented fallback chain down to the global **PROCINFO["READ_TIMEOUT"]** and the **GAWK_READ_TIMEOUT** environment variable; on Unix the primary record read uses **poll** when a timeout applies. The thing to take away is that **PROCINFO** is a real window into the runtime, not a stub — its key set is broad and its values are live.

SYMTAB is the symbol table as an array: you can read and assign global variables through it (**SYMTAB["x"]** aliases the global **x**), iterate it with **for-in**, and take **length(SYMTAB)**. awkrs implements gawk's global-introspection flavor of **SYMTAB** — assignment, iteration, and length — rather than GNU's variable-object-reference flavor, a scope its compatibility matrix is explicit about.

FUNCTAB is the analogous table of function names, supporting membership and iteration so a program can discover the functions defined in it — which pairs naturally with the indirect-call mechanism of Chapter 9 to build dispatch over the program's own functions.

@include, @load, @namespace: source directives

gawk added three **@**-prefixed source directives, processed before the program runs, and awkrs implements them with one important boundary.

@include "file" textually includes another awk source file, expanded before parsing — the awk equivalent of a C **#include**, used to pull in a library of functions. awkrs expands these in a pre-parse pass (**source_expand.rs**).

@load "name" in gawk loads a dynamic extension. Here awkrs draws a firm line, and it is the right one for a single-binary tool: awkrs does **not** load arbitrary **.so/gawkapi** shared objects. Instead, **@load** of one of gawk's *bundled* extension names — **filefuncs**, **readdir**, **time**, and the rest — is accepted as a no-op, because

the functionality those extensions provide is already built into awkrs as native Rust builtins (the filesystem and time functions below). An `@load` of a `.awk` path is inlined like an include. But an `@load` of an arbitrary `.so` errors at parse time rather than pretending to load foreign native code. This is a deliberate divergence its compatibility matrix lists under “known intentional divergences”: gawk’s dynamic-extension loading has no equivalent in awkrs, by design.

`@namespace "ns"` opens a namespace, and awkrs implements gawk’s prefixing rules in `namespace.rs`. Inside a namespace, unqualified identifiers are prefixed with `ns::` — so a variable `x` becomes `ns::x` and a function `f` becomes `ns::f` — *except* for names that are already qualified (containing `::`), function parameters and locals, the special global variables, and the builtin function names, all of which stay unprefixed. The builtin name table that governs this has sixty-three entries, and the special-global table twenty-eight, and the prefixing pass walks the whole AST applying the rule. A qualified name like `awk::x` (the default namespace gawk uses) stays as written. The effect is gawk’s solution to the library-collision problem: a function library can use short internal names without fear of clashing with the calling program, because the namespace prefix keeps them distinct. The `email_extract.awk` and other Chapter 26 examples are single-namespace and so do not exercise this, but it is there for the multi-file library case.

/inet: network sockets

gawk turned the special-file mechanism into a network client: opening `/inet/tcp/0/host/port` or `/inet/udp/0/host/port` as a file connects a socket, and you read and write it with `getline` and `print` redirection. awkrs implements these client sockets via the `socket2` crate, so an awk program can be a TCP or UDP client — fetch a URL, talk to a line-oriented service, send a UDP datagram — without leaving the language. This is a genuine extension of awk’s reach into territory the original language never imagined, and awkrs carries it, with the compatibility matrix noting that coprocess and socket parity with gawk is not guaranteed in every edge case.

Coprocesses: two-way pipes

The coprocess, written `|&`, is gawk’s two-way pipe: `print x |& "cmd"` writes to a subprocess’s standard input and `"cmd" |& getline y` reads from its standard output, so an awk program can carry on a conversation with a filter. awkrs implements coprocesses — the runtime has the coprocess handle types and the parser accepts `cmd |& getline` and `close(cmd, "to"|"from")` — running them under `sh -c`, with the matrix’s caveat that full parity with gawk is not guaranteed. The constraint from Chapter 7 applies: mixing `|` and `|&` on the same command is an error.

gettext: internationalization

gawk integrates with GNU gettext so that awk programs can be localized, and awkrs implements the gettext builtins — `bindtextdomain`, `dcgettext`, and `dcngettext` — backed by `.mo` catalogs through the Rust `gettext` crate, with `TEXTDOMAIN` selecting the active domain. The `-g/--gen-pot` flag generates a `.pot` translation template from the marked strings in a program and exits before execution. This is a niche feature, but it is a real one, and awkrs carries enough of it that localized awk programs work.

Filesystem and OS extensions

gawk’s bundled extensions provide filesystem and OS access, and awkrs implements them as native builtins (which is why `@load "filefuncs"` and friends are no-ops — the functions are already there). The set, from `gawk_extensions.rs`:

- `stat(path, arr)` fills `arr` with a file's metadata — type, size, and on Unix the full `stat` fields (device, inode, mode, link count, uid, gid, rdev, block size, blocks, and the access/modify/change times) — returning 0 or -1.
- `statvfs(path, arr)` fills `arr` with filesystem statistics (block sizes, free and available blocks and inodes, flags, max name length) on Unix.
- `readdir(path, arr)` reads a directory into `arr`, each entry as "`filename/filetype`" where the type is `d`, `l`, `f`, or `u`.
- `fts(root, arr)` walks a directory tree recursively and fills `arr` with the sorted paths.
- `chdir(path)`, `rename(old, new)` — directory and file operations returning 0/-1 and setting `ERRNO`.
- `readfile(path)` returns a whole file's contents as a string.
- `writeta(path, arr)` and `reada(path, arr)` persist an array to disk and read it back, in awkrs's own tab-separated "rvarray" format with a magic header (`awkrs-rvarray-v1`) — so an awk program can checkpoint a table between runs.
- `ord(s)` and `chr(n)` convert between a character and its codepoint.
- `gettimeofday(arr)` and `getlocaltime(arr [, ts])` fill arrays with high- resolution and broken-down time.
- `sleep(seconds)` sleeps for a fractional number of seconds.
- `inplace_tmpfile(path)` and `inplace_commit(tmp, dest)` provide the primitives for safe in-place file editing — write to a uniquely-named temporary in the same directory, then atomically rename it over the original.
- `revoutput` and `revtwoway` are gawk demo extensions that reverse a string, kept for completeness.

All of these I/O-touching builtins route through awkrs's sandbox check first, so that `-S/--sandbox` can block them, and they set `ERRNO` on failure in the gawk manner. The collection turns awk from a pure stream processor into something that can also inspect and manipulate the filesystem — still within the one binary, with no dynamic loading, which is exactly the trade awkrs makes throughout: gawk's *capabilities*, implemented natively, without gawk's dynamic-extension *mechanism*.

Introspection put to work

The three introspection arrays and the indirect-call mechanism are more than curiosities when you combine them, because together they let an awk program reason about and dispatch over its own structure. `FUNCTAB` enumerates the program's functions; the indirect call `@name(...)` invokes a function whose name is a string; and the two compose into a dispatch table built from the program itself. A command interpreter can map each input verb to a same-named handler function and call it indirectly, so adding a command is adding a function with no change to the dispatch loop — the loop reads the verb, checks it against `FUNCTAB`, and `@`-calls it. `SYMTAB` does the same for variables, letting a program read or set a global whose name it computed, which is how a configuration loader can assign to program variables named by the keys in a config file. `PROCINFO` rounds this out with facts about the runtime — the platform, the process ids, the version, the active field-split mode — and the `sorted_in` control that steers iteration order. These features are gawk's, and awkrs implements them with the key sets its compatibility matrix documents, marking `PROCINFO` and the `SYMTAB/FUNCTAB` reflection `Part` where coverage is a subset rather than the whole. The reason to know they exist is that they turn awk from a language you write *in* into one a program can, to a degree, inspect about *itself* — modest reflection, but real, and exactly enough to build clean dispatch tables and self-configuring scripts without leaving the language.

The line awkrs draws at native code

It is worth stating once, plainly, the boundary that runs through this chapter's extensions and the JIT chapter alike, because it is awkrs's single most consistent design decision. awkrs implements gawk's *capabilities* but not gawk's *mechanism* for extending them. gawk lets you load arbitrary `.so` shared objects through `@load`, running foreign native code inside the awk process; awkrs does not, and the refusal is deliberate. Instead, the functionality gawk's bundled extensions provide — the filesystem access of `filefuncs`, the directory reading of `readdir`, the time functions, the array persistence of `rvarray` — is built into awkrs as native Rust builtins,

already present, so `@load "filefuncs"` is a no-op because the functions are already there. A `@load` of a `.awk` path is inlined like an include; a `@load` of an arbitrary `.so` errors at parse time. The reason is the single-binary discipline of Chapters 1 and 39: `awkrs` ships as one self-contained, fast-starting executable with no external runtime and no dependency tree, and loading foreign native code at run time would break exactly that property — it would reintroduce the “which version of which `.so` is on this machine” problem the single binary exists to eliminate. So the trade is consistent everywhere: you get `gawk`’s reach — the sockets, the filesystem, the coprocesses, the `bignum`, the JIT — but implemented natively inside the one binary, never by loading something from outside it. It is the same line the JIT draws (reuse `fusevm`’s engine rather than load a code generator) and the same line the cache draws (a self-contained on-disk format, not a dependency), and naming it once makes the rest of `awkrs`’s choices read as instances of one principle rather than separate decisions.

Chapter 13 — printf and Numeric Formatting

`printf` is where awk meets C, and it is the part of the language with the most exacting specification, because every flag, width, precision, and conversion has a defined meaning that programs depend on for byte-exact output. awkrs implements the full POSIX `printf` plus gawk's additions in `format.rs`, and because byte-exact output is exactly what its parity suite checks, the formatting is unusually well-pinned. This chapter is the formatting model, with the corner cases that distinguish a faithful implementation.

The format string

`printf(fmt, ...)` writes `fmt` with each `%` conversion replaced by the corresponding argument; `sprintf` does the same but returns the string. (awkrs accepts the function-call form `printf("fmt", a, b)` as equivalent to the statement form `printf "fmt", a, b`, a parity fix — though a mixed form like `printf(a, b), c` is still rejected.) The conversions awkrs supports are the full set: `%s` (string), `%d` and `%i` (signed decimal), `%u` (unsigned decimal), `%o` (octal), `%x` and `%X` (hex), `%f` and `%F` (fixed-point), `%e` and `%E` (scientific), `%g` and `%G` (general), `%c` (character), `%a` and `%A` (hex float), and `%%` (a literal percent).

Each conversion can carry flags (`-` left-justify, `+` always-sign, space, `#` alternate form, `'` thousands-grouping, `0` zero-pad), a field width, and a precision, and both width and precision can be `*` to take the value from an argument — including the positional forms `%m$` and `*n$` that select arguments by number. awkrs parses the `h`, `l`, and `L` length modifiers and ignores them, as awk should (awk has one numeric type at the language level). The width is capped at 100,000 to bound output, and all the digit parsing uses saturating arithmetic, a fuzz-hardening detail that means a malformed format string cannot trigger an overflow.

`%g` and the rounded-exponent rule

`%g` and `%G` are the trickiest conversions, and awkrs gets the subtle parts right. The precision of `%g` is the number of *significant digits* (the C99/POSIX meaning), not digits after the decimal point, and a precision of 0 is treated as 1. The choice between fixed and scientific form is made from the *rounded* exponent, which is the detail naive implementations miss: `%1g` of `9.5` is `1e+01`, not `10`, because rounding `9.5` to one significant digit gives `10`, whose exponent is `1`, which crosses the threshold into scientific form. awkrs's compatibility matrix calls this out specifically. Exponents are always signed and always at least two magnitude digits — `e+03`, not `e+3` — matching C.

`%u` and the two's-complement and 2^{64} rules

`%u` formats a value as unsigned, and the corner cases are where awks diverge. awkrs follows gawk: a negative value wraps via two's complement through an `i64`-to-`u64` reinterpretation, so `%u` of `-5` is `18446744073709551611`, not a clamp to zero. Values in the representable `u64` range cast directly. The exact boundary 2^{64} renders as `u64:MAX` (a saturating-cast parity point), and a value *strictly greater* than 2^{64} falls back to `%g`-style formatting, so `2^65` prints as `3.68935e+19`. An earlier awkrs saturated everything over `u64` at `u64:MAX`; the current behavior matches gawk's, and the matrix documents the exact transitions.

The thousands-grouping flag

The `'` flag groups the integer digits in threes using the locale's thousands separator, and `awkrs` applies it to `%d`, `%i`, `%u`, and the integer portion of `%f`, `%e`, and `%g`. The separator comes from the C library's `localeconv()`, and the behavior depends on the locale: in `LC_ALL=C` the separator is empty and the flag is a harmless no-op (no comma is synthesized), while in a locale like `en_US.UTF-8` it is a comma and `printf "%'d", 1000000` yields `1,000,000`. Two subtleties `awkrs` handles: the `'` flag consults `localeconv()` regardless of whether `-N/--use-lc-numeric` was passed (matching `gawk`), and there is a platform wrinkle the source notes — Apple's C library reports a comma even in the C locale while `glibc` reports empty — so the exact no-op-versus-comma behavior follows the host library. Chapter 19 covers the `-N` interaction in full.

Hex floats: %a and %A

`%a` and `%A` format a floating-point number in C99 hexadecimal-float form — `[-]0xh.hhhhp±d` — and `awkrs` implements the full specification in `format_hex_float`: it normalizes subnormals by their leading one, truncates the hex digits to an explicit precision with round-half-to-even, shows all significant hex digits trimming trailing zeros in the default form, uses `0X` and `P` for the uppercase variant, and honors the locale radix character. This is a rarely-used conversion, but it is the exact, round-trippable representation of a float's bits, and `awkrs`'s compatibility matrix confirms `gawk` parity on it.

Non-finite values, consistently spelled

Floating-point infinities and NaNs have to be spelled *somehow* in output, and the spelling has to be consistent between `print` and `printf` or programs that mix them get inconsistent output. `awkrs` uses `gawk`'s spelling — `+inf`, `-inf`, `+nan`, `-nan` in lowercase contexts and `INF`, `NAN` for the uppercase `%E/%G` variants — and, critically, it uses the *same* spelling in both `format.rs` (for `printf/sprintf`) and `runtime.rs`'s `format_number` (for `print` and plain numeric-to-string conversion). So `print x` and `printf "%s", x` agree on how an infinity looks, which is the kind of consistency that only happens when someone deliberately routes both paths through the same convention. The NaN *sign* is normalized at the math-function source — `sin`, `cos`, `exp`, `atan2`, and the `inf - inf` arithmetic case all canonicalize a NaN to `+nan` — so the sign is predictable too.

CONVFMT, OFMT, and the string boundary

When a number is used as a string — concatenated, printed without an explicit format, or compared against a string — it is formatted by `CONVFMT` (general conversion) or `OFMT` (specifically the implicit `print` of a number), both defaulting to `"%.6g"`. `awkrs` threads `CONVFMT` through the places POSIX requires and that earlier builds missed: it is honored in string concatenation (the `ConcatPoolStr` peephole optimization was fixed not to bypass it), in array subscripts, and in the string-compare fallback when a number is compared against a string literal. One important exception `awkrs` implements for correctness: *integer-valued* numbers bypass `CONVFMT` and print exactly, so a large integer does not get mangled by `%.6g`'s six-significant-digit rounding — `print 1000000000` shows `1000000000`, not `1e+08`. The engineering report lists “print of large integers bypasses OFMT past i64::MAX” and “integer-valued floats print exact via %.0f regardless of magnitude” among its fixes, because an `awk` that rounds your integers in output is an `awk` you cannot trust with identifiers, timestamps, or any large exact number — and those are everywhere in real data.

The traditional-mode quirk

One last detail shows the depth of the parity work. POSIX says the `0` (zero-pad) flag applies only to numeric conversions, so `awkrs` pads `%s` and `%c` with spaces even when `0` is given — *except* when `--traditional`

is set, which flips a process-wide flag to reproduce the older Bell-Labs/BSD `awk` behavior of zero-padding strings too. That `awkrs` implements *both* behaviors, selected by a compatibility flag, rather than picking one and calling it correct, is characteristic of how seriously it takes being a drop-in replacement for whichever `awk` you are coming from.

Dynamic width, precision, and positional arguments, worked

The flags and conversions are easier to hold once you have seen the harder formatting features actually used. Width and precision can each be a literal, or a `*` that takes its value from an argument, and the two compose:

```
BEGIN {
    w = 10
    printf "%*s|\n", w, "right"      # width from the argument w
    printf "%-*s|\n", w, "left"     # left-justified, dynamic width
    printf "%.*f\n", 3, 3.14159     # precision from the argument
    printf "%*.*f\n", 8, 2, 3.14159 # both dynamic: width 8, precision 2
}
```

The first line right-justifies `right` in a ten-character field whose width came from `w`, not from the format string — useful when the column width is computed at runtime, say from the longest value in a column. The `%.*f` form takes the precision from an argument, so a program can format to a number of decimal places it decided on the fly. And `%*.*f` consumes *two* arguments before the value, the width then the precision, which is the fully dynamic form. `awkrs` also implements the positional forms `%m$` and `*n$`, which select which argument supplies a value or a width by number — `printf "%2$s %1$s\n", "a", "b"` prints `b a` — which matters for internationalization, where a translated format string may need to reorder the arguments without the program changing. These are the corners of `printf` that turn it from a fixed template into a programmable formatter, and `awkrs` parses and applies all of them, with the saturating-arithmetic and width-cap hardening of Chapter 24 guarding the dynamic cases against a hostile width.

sprintf, and building strings you do not print

`printf` writes formatted output; `sprintf` does the identical formatting but *returns* the string instead of emitting it, and that one difference makes it the workhorse for building formatted text you intend to manipulate rather than print. Every formatting feature this chapter covered — the conversions, the flags, the dynamic width and precision, the locale grouping — is available through `sprintf`, so it is the tool for assembling a fixed-width key, a zero-padded identifier, a hex-encoded byte, or any string whose shape a format string captures. The Chapter 26 examples lean on it constantly and for exactly this reason: `hexdump.awk` builds each two-digit hex pair with `sprintf("%02x", _ord[c])` and accumulates them into a column string, `huffman.awk` builds a zero-padded composite sort key with `sprintf("%012d %s %d", ...)` so that string comparison orders by frequency then character then id, and `base_conv.awk`'s relatives use it to format fixed-width report rows. The pattern in all of these is the same: a value needs a precise textual shape — padded, aligned, based, rounded — and `sprintf` produces that shape as a string the program then stores, concatenates, compares, or uses as an array key. Because it shares its entire engine with `printf`, anything you can print you can also capture, which means the formatting machinery is not only an output facility but a general string-construction facility, and the difference between sending bytes to a file and building a value in memory is the single letter that distinguishes the two builtins.

A gallery of conversions

It helps to see the conversions side by side on concrete values, because the differences between them are easiest to learn by example. The following table reads as “format applied to value yields output”:

```

printf "%d\n", 3.9      # 3      - truncates toward zero
printf "%d\n", -3.9     # -3     - truncates toward zero, not floor
printf "%5d\n", 42      # "   42" - right-justified in width 5
printf "%-5d\n", 42     # "42   " - left-justified
printf "%05d\n", 42     # "00042" - zero-padded
printf "%+d\n", 42      # "+42"  - always-sign
printf "%x %X\n", 255, 255 # "ff FF" - hex, lower and upper
printf "%o\n", 8         # "10"   - octal
printf "%e\n", 12345.6   # "1.234560e+04" - scientific
printf "%g\n", 12345.6   # "12345.6" - general, shortest
printf "%.2f\n", 3.14159 # "3.14" - fixed, two decimals
printf "%10.2f\n", 3.14159 # "      3.14" - width 10, two decimals
printf "%c\n", 65        # "A"    - numeric code to character
printf "%c\n", "hello"   # "h"    - first character of a string
printf "%s\n", 3.14159   # via CONVFM (default "3.14159")
printf "%%\n"            # "%"    - a literal percent

```

Several behaviors worth fixing in memory are visible here. `%d` *truncates* toward zero, so `-3.9` becomes `-3` and not `-4` — it is `int`, not floor. The `%c` conversion is dual: given a number it emits the character with that code, given a string it emits the string's first character — the inverse operations `ord` and a substring, folded into one conversion. `%g` chooses the shorter of fixed and scientific and trims trailing zeros, which is why it is the default for `OFMT/CONVFMT`. And `%%` is the only way to put a literal percent in a format string. Every one of these works identically through `sprintf` for building strings rather than printing them, and `awkrs` implements each to the byte-exact behavior its parity suite checks, including the corner cases — the rounded-exponent `%g`, the two's-complement `%u`, the space-padded `%0s` — that Chapter 32's formatting section catalogues.

Chapter 14 – The Command-Line Interface and Compatibility

awk lives at the command line, and the command-line interface is therefore part of the language in a way it is not for most tools. A program that ports between awks has to find its flags supported, with the same meanings, or it does not port. awkrs's stance, stated in its README, is to accept "the union of POSIX, gawk, and mawk options" — to be the awk that runs whatever invocation you throw at it, whichever awk you learned. This chapter is the flag vocabulary, the compatibility matrix against the three reference awks, and the benchmark numbers that justify the "fastest bytecode engine" claim.

The core POSIX flags

The flags every awk has, and awkrs implements as `Match`:

- `-f progfile` reads the program from a file. Repeatable — multiple `-f` flags concatenate their files into one program — and, as Chapter 2 and Chapter 18 note, the single `--f` form is the one the bytecode cache accelerates.
- `-F fs` sets the field separator, equivalent to `BEGIN { FS = fs }`.
- `-v var=value` sets a variable before `BEGIN` runs. Repeatable.
- The first non-flag argument is the program text (if no `-f` was given), and the rest are input files or `var=value` assignments interleaved with them.

The GNU and gawk flags

awkrs implements gawk's command-line surface, with each flag's exact fidelity documented:

- `-e/--source` supplies a program fragment inline, and `-i/--include` includes a library file; both are repeatable and can be mixed, the standard way to combine a library with an inline program.
- `-E/--exec` runs a program file with the rest of the arguments treated as data, a security-oriented form.
- `-b/--characters-as-bytes` makes `length`, `substr`, and `index` count bytes rather than characters.
- `-c/--traditional` selects traditional-compatibility behaviors (like the string zero-padding of Chapter 13).
- `-C/--copyright` prints the license and exits.
- `-d/--dump-variables [file]` dumps the global variables after the run.
- `-D/--debug [file]` produces a static rule-and-function listing — *not* gawk's interactive debugger (that role is filled by the DAP server of Chapter 20).
- `-g/--gen-pot` generates a gettext `.pot` template and exits.
- `-k/--csv` enables CSV mode (Chapter 5).
- `-l/--load` loads a library by name from `AWKPATH`.
- `-L/--lint` runs the static linter, with `fatal`, `invalid`, and `no-ext` sub-modes; `-t/--lint-old` is the older-style lint.
- `-M/--bignum` enables MPFR arbitrary precision (Chapter 19).
- `-N/--use-lc-numeric` applies the locale's decimal radix and grouping to formatting (Chapter 19).
- `-n/--non-decimal-data` makes input recognize `0x` hex and `0` octal.
- `-o/--pretty-print [file]` produces an AST pretty-print — not gawk's canonical reformatter, an awkrs-specific listing.

- `-O/--optimize` accepts the optimize request (the JIT is on unless `-s`).
- `-p/--profile [file]` produces a wall-clock-and-hit-count summary (single-thread only) — not gawk’s per-line profile format.
- `-P/--posix` selects stricter POSIX behavior in a few places, like skipping gawk-style `for-in` ordering.
- `-r/--re-interval` is a no-op because interval quantifiers are always on.
- `-s/--no-optimize` disables peephole and JIT optimization, forcing the plain bytecode interpreter — the same as `AWKRS_JIT=0`.
- `-S/--sandbox` blocks `system()`, redirects, pipes, coprocesses, and network I/O.

Where awkrs’s behavior differs from gawk’s — the `-D`, `-p`, and `-o` flags do something awkrs-specific rather than gawk-identical — the README and the compatibility matrix document the gap honestly rather than claiming a parity that is not there. This is the right posture for a compatibility tool: be a no-op or a documented-different behavior where you cannot be identical, never a silent wrong answer.

The mawk and awkrs-specific flags

For mawk compatibility, awkrs accepts `-W opt` (repeatable, comma-splittable): `help/usage`, `version`, and `dump` are honored, `exec=path` sets the program file, and other mawk `-W` tokens (`posix_space`, `interactive`, `random`, `sprintf=`) are silently accepted so that a mawk invocation does not error.

Then the flags that are awkrs’s own, marked `Ext` in the matrix because no other awk has them:

- `-j/--threads N` sets the parallel worker count (default 1) — Chapter 15.
- `--read-ahead N` sets the stdin chunk size for parallel mode (default 1024).
- `--lsp` starts the Language Server over stdio (Chapter 20).
- `--dap [host:port]` starts the Debug Adapter, over stdio with no argument or over TCP with a `host:port` (Chapter 20).
- `--aot out` ahead-of-time-compiles a `BEGIN`-only program to a native executable (Chapter 17).

The mode-dispatch order in awkrs’s entry point is worth knowing because it explains a few precedences: `--help`, `--version`, the mawk `-W` early-exit actions, and `--copyright` are handled first; then `--lsp` takes over the process as a language server; then `--dap` takes over as a debug adapter; and only after those does awkrs fall through to `--aot`, the cache, and ordinary execution. The two server modes commandeer the process because they speak a protocol on stdio and cannot share it with normal output.

The compatibility matrix

awkrs ships a full compatibility matrix, `docs/COMPATIBILITY.md`, comparing itself against BSD awk, mawk, and gawk across the CLI, source directives, language constructs, special variables, builtins, formatting, and regular expressions. It is careful to frame itself as a *feature matrix*, *not a proof of correctness*, and it is explicit that awkrs does not claim bit-identical behavior, zero defects, or complete coverage — and that where POSIX leaves behavior unspecified (random sequences, hash iteration order, subtle rounding), differences are expected and acceptable. This honesty is the matrix’s most important feature. Its legend has five cells: **Match** (intended to follow the reference, covered by tests or design), **Part** (a subset or different edge cases), **Ext** (an extension that POSIX awk may lack), **No** (unsupported), and **—** (not applicable).

The high-level shape of the matrix: against gawk, awkrs has the highest overlap, implementing most of gawk’s builtins and globals directly. Against BSD awk and mawk, the gawk-only features awkrs implements are marked as extensions those awks lack. The places awkrs marks itself **Part** rather than **Match** are honest about their limits — `PROCINFO` carries many but not every gawk key; `SYMTAB/FUNCTAB` reflection is best-effort; coprocess and socket parity is not guaranteed in every edge; MPFR precision and rounding are configurable but not bit-audited against gawk. And the matrix lists the intentional, unavoidable divergences in its own section: the JIT must match the interpreter (a JIT mismatch is a bug, not “gawk is wrong”); parallel mode can diverge for order-dependent programs; dynamic `.so` extensions have no equivalent; `PROCINFO["platform"]` uses a `posix/mingw` style rather than every host’s gawk string; and `for-in` order differs across engines when sorting

is off. Reading the matrix is the fastest way to know, for any specific feature, exactly how faithful awkrs intends to be — and the answer is backed by the 2,116-case parity suite that diffs awkrs's output against all three reference awks on every CI push.

The benchmark numbers, in context

awkrs's README leads with “world's fastest awk bytecode engine,” and the benchmark tables back it up — but Chapter 1's caution applies, and the numbers should be read with the understanding that awkrs's everyday speed comes from the bytecode VM and its fast paths, not the JIT. The tables were generated by hyperfine across five engines — BSD awk, gawk 5.4.0, mawk 1.3.4, and awkrs in two configurations — on the same one-million-line input on an Apple M-series machine, and copied verbatim into the repository. A representative sample:

- **{ print \$1 } over a million lines:** BSD awk 195 ms, gawk 101 ms, mawk 66 ms, awkrs about 16 ms — roughly four times faster than mawk, the fastest of the traditional awks, and twelve times faster than BSD awk.
- **{ s += \$1 } END { print s }:** gawk 63 ms, mawk 38 ms, awkrs 13 ms. This is the raw-byte field path — awkrs parses the first field directly from the mmaped buffer.
- **Regex filter /alpha/ { c += 1 } with no matches:** gawk 351 ms, mawk 19 ms, awkrs 11 ms. The linear-time Rust regex shines here, and gawk's backtracking engine is the slowest of the group.
- **No-match gsub("alpha", "ALPHA"); print:** gawk 436 ms, mawk 74 ms, awkrs 14 ms — the literal-needle slurp-inline path copying lines verbatim when the needle is absent.
- **Associative array a[\$5] += 1:** gawk 342 ms, mawk 610 ms, awkrs 340 ms. Here awkrs is at parity with gawk rather than ahead — the hash-table work dominates and there is no fast path to exploit, an honest data point the README includes rather than hiding.

The two awkrs rows in each table — default and `AWKRS_JIT=0` — are nearly identical on these workloads, which is exactly the point Chapter 1 made: these are string-and-field programs, the JIT does not engage for them, and so turning it off changes nothing. The README is scrupulous about this, carrying a dated caveat that the “JIT” rows were generated against a former in-tree Cranelift module since removed, that JIT now means the fusevm offload, and that the offload does not engage for these programs anyway. The honest summary is the one Chapter 1 gave: awkrs is meaningfully faster than the traditional awks on the common workloads because of its VM and its fast paths, it is at parity where the work is irreducible hash-table churn, and its JIT is a separate large win on the narrow class of numeric loops Chapter 17 describes.

The environment variables that steer awkrs

Beyond the flags, a handful of environment variables change awkrs's behavior, and they are worth gathering because they are the knobs you reach for when something is behaving unexpectedly or when you want to isolate a subsystem. `AWKRS_CACHE=0` disables the bytecode cache of Chapter 18, which is the first thing to try if you suspect a stale compile — though the cache invalidates itself on source-mtime and binary-version changes, so a genuine stale hit should be impossible. `AWKRS_JIT=0` disables peephole fusion and the JIT together, forcing the plain unfused interpreter, which is how the benchmark tables' “bytecode” rows are produced and how you would isolate a suspected fusion bug; it is equivalent to the `-s/--no-optimize` flag. `AWKRS_FUSEVM=0` is the narrower switch that forces every chunk back onto awkrs's own bytecode interpreter without touching peephole fusion — the way to confirm that the fusevm offload of Chapter 17 is producing bit-identical results to the interpreter, since the two should agree exactly. The machine-code cache the JIT writes is steered by `FUSEVM_JIT_CACHE_DIR` and `FUSEVM_JIT_CACHE_MAX_BYTES`. `AWKRS_DAP_LOG` turns on debug-adapter protocol tracing to stderr (Chapter 20), and the conventional `NO_COLOR` forces plain text in the help display. None of these is needed for normal use — awkrs's defaults are the fast, safe, cached path — but each exists so that when you need to pin down which layer of the machine is responsible for a behavior, you can turn that layer off and see.

The flags grouped by task

The flag vocabulary is long, but it sorts cleanly by what you are trying to do, and grouping it that way makes it memorable. To *supply the program*: `-f file` reads it from a file (and is the cached form), `-e code` supplies an inline fragment, `-i file` includes a library, and `-E file` runs a program file with the rest as data. To *configure input*: `-F fs` sets the field separator, `-k` turns on CSV mode, `-n` recognizes non-decimal input, and `-v var=value` presets a variable. To *change number behavior*: `-M` switches to MPFR bignum, `-N` localizes numeric formatting, and `-b` counts bytes rather than characters. To *get diagnostics*: `-L` lints, `-o` pretty-prints, `-D` lists rules, `-d` dumps variables, `-p` profiles, and `-g` generates a gettext template. To *control the engine*: `-O` requests optimization (on by default), `-s` disables the JIT and fusions, `-S` sandboxes, and `-P` selects stricter POSIX behavior. To *use awkr's own extensions*: `-j N` sets the parallel thread count, `--read-ahead N` sizes the stdin chunk, `--lsp` and `--dap` start the tooling servers, and `--aot` compiles a `BEGIN`-only program to a native binary. And `-r` is a harmless no-op because interval quantifiers are always on. Grouped this way — supply, input, numbers, diagnostics, engine, extensions — the surface is small enough to hold in mind, and each group maps to a chapter: input shaping to Chapter 5, numbers to Chapter 19, the engine to Chapters 16 and 17, the extensions to Chapters 15 and 20. The full table with each flag's exact fidelity is the compatibility matrix of Chapter 32; this grouping is the map for finding the one you want.

Chapter 15 – Parallel Record Processing

Here is the feature no traditional awk has at all. Every other awk processes records strictly sequentially, one at a time, in input order, because that is the language's defined model and because cross-record state — a running sum, a range pattern's on/off flag, an accumulating array — depends on that order. awkrs keeps the sequential model as the default and the guarantee, and then adds an opt-in parallel path that processes records concurrently across a thread pool *when it can prove doing so is safe*, reassembling the output into input order so that nothing downstream can tell the difference. This chapter is what awkrs parallelizes, how it decides it is safe, and how it stays correct.

The opt-in and the default

Parallelism is off by default: `-j/--threads` defaults to 1, and a program runs on the sequential path — the same bytecode VM, the same fast paths from Chapter 16 — exactly as a traditional awk would. You opt into parallelism by passing a higher thread count, `-j 4` or `-j 8`, and you do so when you have a large input and a program that is *parallel-safe*. If you ask for parallelism on a program that is not parallel-safe, awkrs does not silently produce wrong output: it prints a warning to stderr and runs the program sequentially. The default is safe, the opt-in is explicit, and the fallback is correct — which is the only responsible way to add concurrency to a language whose semantics are sequential.

The parallel-safety check

The heart of the feature is a *static* analysis, `record_rules_parallel_safe`, that walks the program's AST once (at parse time, or read back from the cached `parallel_safe` flag on a cache hit) and decides whether the record rules can run in any order without changing the result. A program is parallel-safe only if its record rules contain *none* of the following, each of which would make a record's behavior depend on the records before it:

- **Range patterns** — their on/off state is inherently order-dependent.
- **exit or nextfile** — these alter the processing of *subsequent* records.
- **Primary getline** (reading the main input into `$0` or a variable) and **pipe/coprocess getline** — these consume from a shared input position that cannot be split across workers. (A file-redirected `getline x < "file"` is allowed, because it has its own independent position.)
- **Any assignment to a global** — a field assignment, an index assignment, an increment or decrement, or a plain variable assignment — because a value written by one record and read by another is a cross-record dependency. This is the big one: it is what excludes `{ s += $1 }`, the running-sum program, from the parallel path.
- **delete** — mutates shared array state.
- **asort/asorti in a record rule** — impose a global ordering.
- **Indirect or dynamic calls** (`@f()`) — awkrs cannot prove what they do.
- **print/printf with redirection** — concurrent writes to the same file or pipe would interleave unpredictably.

The check is conservative by design: when it cannot prove safety, it assumes unsafety. The helper functions `stmt_blocks_parallel` and `expr_blocks_parallel` recurse the whole AST looking for any of these

constructs, and the presence of a single one drops the program to sequential. What is parallel-safe is a large and useful class: stateless per-record transforms and filters — programs that read fields, compute, and print, without writing any global state. `{ print $1, $3 }, /error/ { print }, $3 > 100 { print $1 }, { print toupper($2) }` — the filter-and-project programs that make up an enormous fraction of real awk usage — are all parallel-safe, and they are exactly the programs where parallelism helps, because the work is per-record and embarrassingly parallel.

How the parallel path runs

When the program is parallel-safe and `-j N` with `N > 1` is given, awkrs takes one of two paths depending on the input source.

For **regular files**, awkrs memory-maps the file read-only (no `read()` copy of the whole thing into memory) and splits it into records using the same `RS`-aware splitter as the sequential path — newline-split, paragraph-split, regex-split, or literal-delimiter-split via `memchr`'s substring finder — producing owned record strings. It then dispatches the records across a rayon thread pool with `into_par_iter`, each record processed by a per-worker `Runtime` that shares the compiled program through an `Arc` (one compile, a cheap refcount bump per worker) and carries its own per-record state: `NR` and `FNR` set to the record's position, and a per-record RNG seed derived deterministically from a base seed and the record index, so that `rand()` is reproducible even in parallel.

For **stdin**, awkrs cannot memory-map, so it buffers up to `--read-ahead` lines (default 1024) into a batch, processes the batch in parallel, emits the results in order, and refills — a chunked pipeline that keeps memory bounded while still parallelizing. The stdin parallel path requires the default newline `RS`, because the line-buffering assumes lines.

Staying correct: output reordering

The correctness guarantee rests on one mechanism: **output reordering**. Each worker does not write to standard output directly; it buffers its record's print output into a per-record chunk tagged with the record's original index. After a batch completes, awkrs sorts the chunks by index and writes them out in input order. So even though record 5 may have finished before record 2, record 2's output appears first, and a consumer downstream of awkrs sees exactly the byte stream a sequential run would have produced. The first record flagged as having hit an `exit` sets the exit code (though `exit` in a record rule makes the program not parallel-safe in the first place, so this is a belt-and-suspenders path). `RT` in parallel mode is set approximately, since per-record terminators are not tracked individually across workers.

This is the crux of why awkrs can offer parallelism without breaking the language: the *execution* is concurrent, but the *output* is sequential, and for the parallel-safe class of programs — where records do not influence each other — concurrent execution producing sequentially-ordered output is indistinguishable from sequential execution. The static check guarantees the “records do not influence each other” precondition; the reordering guarantees the “output is sequential” postcondition; and together they make `-j 8` a transparent speedup for the programs it applies to.

What END sees, and the honest limits

There is one limit worth stating plainly, because it follows from the design and the README states it: in parallel mode, `END` rules see only the post-`BEGIN` global state, *not* the accumulated effect of the record rules — because the record rules, being parallel-safe, made no global mutations to accumulate. This is not a bug; it is a consequence of the safety check. A program that wants to accumulate across records into a global — the `{ s += $1 } END { print s }` pattern — is *not* parallel-safe (it assigns to a global), so it never takes the parallel path; it runs sequentially, and its `END` sees the full accumulation. The two facts are consistent: parallel programs do not accumulate, accumulating programs do not parallelize, and you never get a program

that both takes the parallel path and expects `END` to see record-rule mutations. `awkrs`'s compatibility matrix is explicit that parallel mode “can diverge from any sequential reference” for order-dependent or side-effecting programs — which is exactly why the static check excludes them.

The practical guidance is simple. Reach for `-j N` when you have a large input and a stateless per-record program — a filter, a projection, a per-line transformation. Leave it at the default for anything that accumulates, ranges, or reads ahead. And trust the warning: if `awkrs` tells you it fell back to sequential, it is because running your program in parallel would have changed its answer, and `awkrs` chose the right answer over the fast one.

Choosing a thread count

When a program *is* parallel-safe and the input *is* large, the remaining question is what number to pass to `-j`, and the honest answer is that the useful range is bounded on both ends by things that are not the core count. On the low end, parallelism has a fixed cost — splitting the input into records, dispatching them across the rayon pool, buffering each worker's output, and sorting the buffers back into input order — and that cost only pays off when the per-record work is large enough or the record count high enough to dwarf it. A short program over a small file is faster sequential, which is exactly why `-j` defaults to 1: the parallel path is opt-in because it is not always a win. On the high end, the work for the parallel-safe class is per-record and embarrassingly parallel, so throughput scales with workers until it hits the real ceiling, which for a filter-and-project program is usually memory bandwidth or the downstream consumer's ability to absorb output, not CPU. A sensible default is to set `-j` to the core count for a genuinely large input and a stateless program, and to measure rather than assume, because the speedup depends on the ratio of per-record compute to I/O. The two input sources behave differently, too: a regular file is memory-mapped and split with no copy, so its parallel path is leanest, while stdin is buffered in `--read-ahead` chunks (default 1024 lines), and tuning that chunk size trades memory for batch efficiency. The guidance stays the same as the previous section's — parallelism is for large inputs and stateless programs — with the refinement that the right `-j` is found by measuring the specific workload, not read off the core count.

A worked parallel run, end to end

Trace a concrete parallel-safe program to see the whole mechanism. Take `{ print toupper($2) }` — a stateless per-record transform — over a large file, run as `awkrs -j 8 prog.awk big.log`. At compile time, the static safety check `record_rules_parallel_safe` walks the AST and finds no range pattern, no `exit/nextfile`, no primary `getline`, no global assignment, no `delete`, no redirected output — so the program is flagged parallel-safe and that flag is serialized into the compiled program. At run time, because the input is a regular file, `awkrs` memory-maps it read-only and splits it into records with the same `RS`-aware splitter the sequential path uses (Chapter 23), producing owned record strings. It dispatches those records across a rayon thread pool with `into_par_iter`, each worker holding its own `Runtime` that shares the compiled program through an `Arc` — one compile, a cheap refcount bump per worker — and carrying per-record state: `NR` and `FNR` set to the record's position, and a per-record RNG seed derived deterministically from a base seed and the record index so `rand()` would be reproducible. Each worker does *not* write to stdout directly; it buffers its record's output into a chunk tagged with the record's original index. When the batch completes, `awkrs` sorts the chunks by index and writes them out in input order. So even though record 5 may have finished before record 2, record 2's output appears first, and a consumer downstream sees exactly the bytes a sequential run would have produced. The execution was concurrent; the output is sequential; and because the program writes no global state, the two are indistinguishable from a sequential run — which is the whole correctness argument, made concrete on one program.

Refactoring toward the parallel-safe class

Because the safety check is all-or-nothing — a single global assignment drops the whole program to the sequential path — the practical skill is not memorizing the seven-item exclusion list but learning to *reshape* a

program so the expensive part lands inside the parallel-safe class and only the cheap part stays sequential. The instinct that fails is to try to make an accumulator parallel; `awkrs` will not let you, and it is right not to, because a running sum written by one record and read by the next is exactly the cross-record dependency the check forbids. The instinct that works is to split the computation along the classic map-reduce seam: do the heavy per-record work — the regex, the field parsing, the formatting, the arithmetic on each line — in a stateless transform that *is* parallel-safe, and let a second, cheap pass do the accumulation. In a shell that means two `awkrs` invocations in a pipe: `awkrs -j 8 'MAP' big.log | awkrs 'REDUCE'`, where the first stage is a filter-and-project that scales across cores and the second is a trivial `{ s += $1 } END { print s }` that runs sequentially over the already-reduced stream. The heavy work parallelizes; the light work does not need to.

The refactor is worth doing only when the map stage is genuinely the expensive one, which is the common case for the programs people reach for `-j` on: a regex-heavy filter over a multi-gigabyte log, a per-line reformatting of a wide CSV, a computation that touches every field of every record. For those, moving the compute into the parallel stage and leaving a one-line reduce on the tail is a real speedup with no change in output, because the pipe preserves order and the reduce sees the records in sequence. For a program whose per-record work is already trivial — `{ s += $1 }` over plain numbers — there is nothing to move; the accumulation *is* the whole program, it is inherently sequential, and `-j` was never going to help it, which is why `awkrs` falls back rather than pretending otherwise. Reading a program against the exclusion list, then, is a diagnosis, not a verdict: the construct that tripped the check tells you where the cross-record dependency lives, and the map-reduce split tells you whether it can be moved off the hot path or whether it is the hot path.

Chapter 16 — The Machine Underneath

Everything to this point has been the language — what `awkrs` does. This chapter and the four that follow are the machine — how it does it, fast. The architecture is a single-pass pipeline, classic in shape and unremarkable in outline: source text becomes tokens, tokens become a syntax tree, the tree becomes bytecode, and a stack virtual machine runs the bytecode. What makes `awkrs` fast is not the outline but the work done at each stage and the fast paths that bypass stages entirely when they can be proven unnecessary. The engineering report gives the source layout — roughly fifty-one thousand lines of Rust across forty modules, with the parser, runtime, compiler, lexer, and VM the largest — and this chapter walks the pipeline against it.

The pipeline in five stages

The path from a string of `awk` source to a running program is:

1. `lexer.rs` turns the source text into a stream of tokens. It handles the POSIX and `gawk` token set, the string escapes (`\a`, `\b`, `\f`, `\v`, `\xHH` hex, `\NNN` octal, and the rest), and scientific notation as single tokens (`1e7` is one number token, a parity fix the report records). It is also where the division-versus-regex disambiguation begins, tracking whether a `/` at the current position can start a regex.
2. `parser.rs` is a recursive-descent parser — the largest module in the project — that turns tokens into an abstract syntax tree. It implements the full grammar including the `gawk` extensions, the compound assignment operators `^=` and `**=`, the `cmd |& getline` coprocess form, and the precedence rules from Chapter 6 (right-associative `^`, loose unary operators).
3. `ast.rs` defines the tree itself — the expression, statement, pattern, and rule node types — and a companion `ast_fmt.rs` knows how to pretty-print it, which is what the `-o/--pretty-print` flag uses.
4. `compiler.rs` walks the AST and emits bytecode `Chunks` — the second- largest module — doing slot allocation (assigning fast `u16` indices to user scalars), the peephole fusion that this chapter covers below, and the static checks that reject malformed programs at load time. It also tracks which names are arrays for `PROCINFO["identifiers"]` and the parallel-safety flag.
5. `vm.rs` is the stack machine that runs the bytecode — its main loop, opcode dispatch, the fused fast paths, and array call-by-reference. `runtime.rs` holds the central mutable state the VM operates on: the fields, `$0`, the variables, the slots, the separators, and (under `-M`) the MPFR integration.

This is a *single-pass* pipeline in the sense that the program is compiled once, up front, before any input is read, and then the compiled form runs — which is exactly what distinguishes it from a tree-walker that re-interprets the AST on every record. The compile happens once; the run happens a million times; and the cost that a tree-walker would pay on every one of those million records — node dispatch, name resolution — `awkrs` pays once, at compile time.

The bytecode

The compiled program is a `CompiledProgram` — a struct holding the `BEGIN`, `END`, `BEGINFILE`, and `ENDFILE` chunks, the record rules (each a pattern and a body chunk), the user functions, the interned string pool, the slot count and names, the array-name set, and the `parallel_safe` flag. Each chunk is a flat `Vec<Op>` — a vector of opcodes — and the `Op` enum has ninety-six variants, each kept small (a test asserts every `Op` fits in 32 bytes) so the bytecode is compact and cache-friendly. Jump targets are absolute instruction indices, patched in at compile time, so control flow at runtime is a single index assignment to the program counter rather than any kind of search.

The opcodes come in two families that reflect awkrs's central performance idea — *not every variable is equal*. User scalar variables get `u16 slot` indices and are accessed through `GetSlot/SetSlot` opcodes that index a flat array directly, which is as fast as a local-variable access in a compiled language. The special variables (`NR`, `F5`, `OF5`, and friends) and the array names stay on a `HashMap` path, accessed through `GetVar/SetVar` by a `u32` string-pool index, because they are fewer and need the map's flexibility. So a tight loop over a counter `i` and an accumulator `s` — both user scalars — runs entirely on the flat slot array with no hashing, while the occasional touch of `NR` or `F5` takes the map path. This split is the foundation everything else builds on.

Peephole fusion: superinstructions

The single most important optimization in the bytecode VM is *peephole fusion*: the compiler recognizes common short sequences of opcodes and replaces them with a single fused *superinstruction* that does the whole job without the intermediate stack traffic. This is where a large part of awkrs's speed advantage over the traditional awks comes from, and the fused opcodes map directly onto the idioms awk programs are made of:

- `print $N` fuses into `PrintFieldStdout(N)` — a direct write of the Nth field to the output buffer, replacing the three-opcode sequence of push-field-number, get-field, print with one instruction.
- `s += $N` fuses into `AddFieldToSlot{field, slot}` — an in-place numeric parse of the field added straight into the accumulator slot, replacing four opcodes with one.
- `i = i + 1`, `i++`, and `++i` all fuse into `IncrSlot(slot)` — one numeric add with no stack traffic at all, replacing up to five opcodes with one.
- `s += i` between two slots fuses into `AddSlotToSlot{src, dst}`.
- a literal concatenation like `$1 " , " $2` fuses into `ConcatPoolStr` — appending a pooled string constant without a separate push.
- `NR++` on the `HashMap` path fuses into `IncrVar`.

These six are the headline fusions, but the set is larger. The bytecode module also defines `PushFieldNum` (push a field already parsed as a number), `GetNR/GetFNR/GetNF` (read these specials without a `HashMap` lookup), `JumpIfSlotGeNum{slot, limit, target}` (the fused comparison-and-branch that is the shape of a counted `for` loop's test, collapsing four opcodes to one), `AddMulFieldsToSlot` (the `sum += $1 * $2` idiom in one instruction), `ArrayFieldAddConst`, and three multi-field print variants (`PrintFieldSepField`, `PrintThreeFieldsStdout`). Each one corresponds to a real awk idiom, and each one was added because that idiom showed up in the benchmark programs and the profiler pointed at the dispatch overhead of the unfused form. The engineering report even notes that seven *dormant* peephole fusions were activated by a normalization pass (`normalize_field_indices`) that rewrites a small-integer-then-get-field sequence into a form the fusion patterns can match — a reminder that the fusions only fire when the compiler emits exactly the shape they expect, and that getting the most out of them is its own piece of work.

The `-s/--no-optimize` flag (equivalently `AWKRS_JIT=0`) disables peephole fusion, forcing the plain unfused interpreter — which is how the benchmark tables' "bytecode" rows are generated, and how you would isolate a suspected fusion bug.

Inline fast paths: bypassing the VM

Below the fused interpreter is a layer that bypasses the VM's call machinery entirely. For a *single-rule* program whose body is one fused opcode — `{ print $1 }, { s += $1 }` — awkrs's record loop does not even set up the VM's execution context (`VmCtx`); it recognizes the shape and runs a specialized loop that writes fields directly from the memory-mapped input. The README calls these the inline fast paths, and they are why the `{ print $1 }` benchmark is as fast as it is: for that program, on a regular file, there is essentially no interpreter in the loop at all, just a byte scan and a buffer write.

The raw-byte field extraction underneath this is worth describing because it is so aggressive. For `print $N` with the default whitespace `F5` on a memory-mapped regular file, awkrs scans the raw bytes of the mapped buffer to find the Nth whitespace-delimited field, writes those bytes straight to the output buffer, and appends

the cached `ORS` bytes — with no record copy, no field array construction, and no UTF-8 validation. The record never becomes an owned Rust `String`; the field never becomes a separate allocation; the whole operation is a scan over bytes that are already in memory because the file is mapped. The `gsub("lit", "repl"); print` literal-pattern path is similar: when the literal needle is absent from a line, `awkrs` copies the line verbatim from the mapped buffer and skips the VM. These are the optimizations that move the benchmark numbers, and they all share one idea — recognize a common program shape and run a hand-written loop for it instead of the general interpreter.

Direct-to-buffer output and cached separators

The output path is engineered with the same care. `print` does not build a per-record `String`, call `format!()`, or lock stdout per record. Instead, output accumulates in a 64-kilobyte `Vec<u8>` that is flushed at file boundaries, so a million `print` statements become a handful of large `write` calls rather than a million small ones. The `OF5` and `ORS` separators are cached as byte slices on the runtime and updated only when the program assigns to them, so the common case — unchanged separators — pays nothing to look them up. The field representation is zero-copy: a field is an `(offset, length)` byte range into the record string, and only becomes an owned `String` when you assign to it. Numbers fast-path through a plain-decimal-integer parser before falling back to the general float parse. Newline scanning uses `memchr`. None of these is individually dramatic; together, they are why `awkrs`'s constant factor is small enough that the language's expressiveness costs almost nothing at runtime.

Why the architecture matters

The lesson of this chapter is that `awkrs`'s speed is not one trick but a stack of them, each targeting a specific cost: peephole fusion attacks per-opcode dispatch, slot indexing attacks variable-lookup cost, the inline fast paths attack the interpreter's setup cost for simple programs, the raw-byte field path attacks record-copy and validation cost, and direct-to-buffer output attacks per-record syscall and allocation cost. The JIT in the next chapter attacks a sixth cost — the interpreter's per-operation overhead in tight numeric loops — and it is the one that is experimental and narrow. The five costs this chapter covered are the ones `awkrs` has already beaten across the board, on the ordinary string-and-field programs that are `awk`'s daily work, which is why `awkrs` is fast where it matters even with the JIT entirely turned off.

A worked trace: { s += \$1 } end to end

It pulls the chapter together to trace the canonical accumulator program, `{ s += $1 } END { print s }`, through the whole machine. At compile time, the compiler recognizes the body as the fusable shape `s += $N` and emits the single superinstruction `AddFieldToSlot { field: 1, slot: s }`, which replaces the four-opcode sequence of push-field-number, get-field, compound-add, pop. It assigns `s` a `u16` slot, so the accumulator lives in the flat slot array rather than the `HashMap`, and it notes that the program assigns to a global (`s`), which marks it *not* parallel-safe — so this program runs sequentially even under `-j`, exactly as Chapter 15 said. At run time, because the program is a single record rule whose body is one fused op, `awkrs` takes the inline fast path: it does not set up the VM's `VmCtx` at all. On a memory-mapped regular file it scans the raw bytes of each line to find the first whitespace field, parses it as a number through the fast plain-integer path before any float fallback, and adds it directly into the accumulator slot — no record copy, no field array, no UTF-8 validation, no interpreter dispatch in the loop. The `END` rule then prints the slot. This is why the `{ s += $1 }` benchmark runs a million lines in about thirteen milliseconds: for the body of the loop there is essentially no interpreter, just a byte scan and an add, and the result lands in a slot read directly at the end. Every layer this chapter described shows up in that one trace — the peephole fusion, the slot allocation, the inline fast path, the raw-byte field scan, the direct numeric parse — which is the concrete shape of the claim that `awkrs`'s speed is a stack of specific optimizations, each removing a specific cost, rather than any single trick.

The normalize-field-indices pass and dormant fusions

There is a subtlety in how the peephole fusions actually fire that is worth surfacing, because it shows that an optimization is only as good as its ability to *match* the code the compiler emits. A fusion like `PrintFieldStdout(N)` only replaces the unfused sequence if the compiler emitted exactly the shape the fusion pattern looks for — a small-integer field index immediately followed by a field-get. But the compiler does not always emit fields in that canonical shape; depending on how an expression was parsed, a `$1` might be lowered as a push of the constant `1` followed by a `GetField` in a form the fusion pattern did not recognize, leaving the fusion *dormant* — present in the optimizer but never triggered. `awkrs` addresses this with a normalization pass, `normalize_field_indices`, that rewrites these small-integer-then-get-field sequences into the exact canonical form the fusion patterns expect, and the engineering report records that adding this pass *activated seven previously dormant peephole fusions* — optimizations that were written but were silently never firing because the code never quite matched. The lesson generalizes: a superinstruction optimizer is two halves, the fused opcodes and the pattern-matcher that recognizes when to use them, and a fusion that the matcher cannot reach is dead weight. Getting the full benefit means normalizing the bytecode into the shapes the patterns match, which is its own piece of work and exactly the kind of detail that separates a peephole optimizer that helps a little from one that helps a lot.

Reading the fast-path decision for your own program

The chapter has walked the fast paths one at a time; the practitioner’s question is the inverse — given *this* program, which of them fire? The honest answer is that you can reason it out from the program’s shape without reading the compiler, because each fast path keys off a structural property you can see on the page. Four bear on everyday performance, and they compose in a rough order.

The first is the **inline single-rule bypass**. A program that is one unconditional rule with a simple body — the canonical `{ print $1 }`, or `{ print $1, $3 }`, or `/pat/ { print }` — never enters the VM’s general call context at all; the driver recognizes the shape and runs a hand-written loop that pulls fields and writes output directly. The tell is structural: one rule, no user functions on the hot path, a body the driver knows how to special-case. Add a second rule, a `BEGIN` that reconfigures state in a way the fast loop cannot assume, or a body the special-case does not cover, and the program falls back to the general VM — still fast, but no longer bypassing it. This is why the very shortest programs are disproportionately the fastest ones: they hit the path with the least overhead per record.

The second is the **raw-byte field path**. Whenever the program reads whitespace-separated fields under the default `F5` from a memory-mapped regular file, `awkrs` extracts the *N*th field directly out of the mapped bytes — no record copy, no UTF-8 validation, no allocation — for the fields you actually reference. The tell is again structural: default `F5`, a real file argument (not a pipe or stdin, which cannot be mapped), and field access by index. Switch `F5` to a multi-character regex, force a `$0` rebuild by assigning to a field, or feed the program from a pipe, and you leave this path for the general splitter. The everyday `{ print $1 }` hits *both* the inline bypass and the raw-byte path at once, which is exactly why it is the benchmark’s best case.

The third is **superinstruction fusion**. Common opcode pairs and triples — the load-add-store of an accumulator, the field-fetch-and-coerce of `$1 + 0`, the compare-and-branch of a loop condition — are recognized by the peephole matcher and emitted as single fused opcodes, collapsing several dispatches into one. This one you cannot see as directly, because it is a property of the emitted bytecode rather than the source, but the shapes that fuse are the shapes that recur: tight accumulation (`s += $1`), tight iteration (`for (i = 1; i <= NF; i++)`), and the coercions of Chapter 6. The `normalize-field-indices` pass discussed just above is what keeps these fusions reachable, and its absence is what let some of them lie dormant before the fix.

The fourth is the **JIT**, and it is the narrowest: it fires only for the eligible numeric chunks of Chapter 17 — tight loops of pure arithmetic and numeric builtins with no string or field work in the hot part — and is inert for everything else. The tell here is unambiguous: if the loop touches a field, a string, or an array in a way the eligibility pass rejects, it stays on the interpreter, and the interpreter is what most `awk` programs run on start to finish. The thing to carry away is that the first three paths are what make `awkrs` fast on the programs you

actually write, and they engage for string-and-field work, not against it; the JIT is a bonus on top for the rare numeric-loop program, not the source of the everyday speed. When you want a program to be fast, then, the levers are the structural ones — keep the rule simple, keep [F5](#) default, read from a file when you can, and let the fusions fire — not the JIT, which either applies to your loop or does not and is not something you tune toward.

Chapter 17 — The fusevm Bridge, the JIT, and AOT

This is the experimental chapter, and the book flags it as such because awkrs flags it as such: the crate’s own one-line description calls the JIT experimental, the engineering report marks awkrs’s JIT support with a half-filled circle where the others are full checks or empty, and the benchmark tables carry a dated caveat about which “JIT” the numbers were generated against. What follows is the honest account of a real, working, but deliberately narrow capability — a path that compiles certain numeric loops to native machine code through a shared virtual machine called fusevm and the Cranelift code generator, with a large measured speedup where it applies and no effect at all where it does not.

fusevm: the shared VM

awkrs does not have its own JIT. Instead it has a *bridge* to fusevm — a separate bytecode VM, published as its own crate, that is also used by the Rust shell zshrs, the language strykelang, and the editor ports vimlr and elisprs. fusevm has what awkrs’s own bytecode VM does not: Cranelift-backed JIT tiers that compile its bytecode to native code, and a persistent on-disk cache of that native code. The architectural decision awkrs makes is to *reuse* this machinery rather than build a second copy — when a chunk of awkrs bytecode is purely numeric and eligible, awkrs translates it into a `fusevm::Chunk` and runs it on `fusevm::VM`, which JITs it. The bridge lives in three files: `fusevm_bridge.rs` (the translation and the per-record caching), `fusevm_compile.rs` (the lowering of awkrs ops to fusevm ops), and `fusevm_host.rs` (the host-side helpers the JIT calls). This path is on by default; `AWKRS_FUSEVM=0` forces every chunk back onto awkrs’s own bytecode interpreter.

What is eligible, and why the rest is not

The JIT path engages only for *purely numeric* bytecode chunks, and the eligibility check (`is_fusevm_eligible`) is an allowlist. A chunk is eligible when its whole body is: numeric constants; arithmetic and comparisons; `++/--`; the compound assignments; division and modulo; jumps and fused loop tests; scalar slot reads and writes whose values stay numeric; and a specific set of builtins that have native fusevm opcodes — `int(x)`, the transcendentals `sin`, `cos`, `exp`, and `atan2`, and the bitwise `and`, `or`, and `xor`. A chunk that touches strings, fields, arrays, regexes, `getline`, `print`, user calls, or any builtin outside that set is *ineligible* and runs on the interpreter.

The boundary is drawn precisely, and the reasons are instructive. `int(x)` lowers to a native `fusevm::Op::AwkInt` (a Cranelift truncation) — the README notes it as the first AWK builtin reachable end-to-end from awkrs into fusevm’s native AWK-op set. The transcendentals lower to native ops (`AwkSin`, `AwkCos`, `AwkExp`, `AwkAtan2`) implemented as Cranelift libcalls to small Rust helpers that normalize a NaN result to `+nan`, matching the sign convention from Chapter 13. The bitwise `and/or/xor` lower to native `band/bor/bxor` over a *saturating* float-to- int conversion that matches awkrs’s own conversion of huge or NaN operands. But `sqrt` and `log` stay on the interpreter — because they emit a host stderr warning on a negative argument, which a pure native op cannot reproduce — and `lshift/rshift/compl` stay on the interpreter because they raise a *fatal* on a negative argument, a value-dependent trap that the trap-free `and/or/xor` do not have. The eligibility boundary, in other words, is drawn exactly at the line between operations whose behavior is a pure numeric function of their inputs and operations that have *side effects* (a warning) or *value-dependent traps*.

(a fatal) that only the interpreter can faithfully produce. This is the discipline that lets awkrs claim the JIT is bit-for-bit identical to the interpreter: it only JITs the operations for which “identical” is achievable.

Division by zero survives into native code

The most impressive piece of fidelity in the JIT is what happens to division. Division and modulo *are* eligible and *are* compiled — they lower to native trapping opcodes `AwkDivJit` and `AwkModJit` — but the fatal-on-zero rule from Chapter 6 must survive into the native code, or the JIT would diverge from the interpreter on the most important corner case. It does survive: the block JIT compiles these ops with a *guarded zero-divisor early-exit* — a `fcmp eq divisor, 0.0` that, on a true result, calls a trap libcall raising the POSIX fatal `division by zero attempted`, and otherwise does the native `fdiv/fmod`. So a hot division loop runs as native code — a twenty-million-iteration division loop dropping from about 8.0 seconds on the interpreter to about 0.12 seconds, roughly sixty-eight times — while a zero divisor reached *inside* the compiled loop still raises the same fatal the interpreter would, rather than producing `inf` or hanging. The trap libcall is not a registered host helper, so these chunks JIT in-process only and skip the on-disk cache, a detail with no user-visible effect beyond cache scope.

The two JIT tiers and eager compilation

fusevm has two JIT tiers, and both persist their compiled native code to `~/.cache/fusevm-jit` (overridable with `FUSEVM_JIT_CACHE_DIR`, capped with `FUSEVM_JIT_CACHE_MAX_BYTES`, the same cache zshrs uses). The *block JIT* compiles a fully-eligible per-record numeric chunk to native code, writing a `.blk.fjit` artifact; the *tracing JIT* compiles hot in-chunk loop traces, writing a `.trc.fjit` artifact. This cache is the *machine-code* cache, entirely distinct from awkrs’s own *bytecode* cache of Chapter 18 — one caches fusevm-emitted native code keyed by chunk op-hash, the other caches awkrs bytecode keyed by source.

There is a clever wrinkle in how a loop in `BEGIN` or `END` gets compiled. fusevm’s normal block JIT compiles a chunk only after it has been invoked some threshold number of times — warm-up before compilation, the standard JIT tactic. But a `BEGIN/END` loop offloaded as a region calls `fusevm::VM::run()` exactly *once* (the loop is inside that single run), so under the normal threshold it would *never* compile and the whole loop would run on fusevm’s slower interpreter. awkrs detects this: because the region selector only picks a region that contains a backward jump — a genuine hot loop — the bridge forces the block-JIT threshold to zero around that single `run()`, so the loop compiles to native code on the first and only invocation. This *eager* compilation is what makes the headline numbers real: a twenty-million-iteration `s += sin(i)` loop dropping from about 12.8 seconds to about 0.15 seconds, and an `x = and(x, i) + 1` loop from about 14.0 to about 0.13 — roughly eighty-five to a hundred and ten times, both bit-for-bit identical to `AWKRS_FUSEVM=0`. The block JIT round-trips awk’s `f64` values through fusevm’s bit-pattern slot model (a slot is an `i64` holding the raw bits of an `f64`), so an accumulator like `x = int(x + c)` compiles natively and caches its artifact for reuse on the next run.

The per-record caches

Even with native code cached on disk, there is per-record work the disk cache cannot eliminate — building the `fusevm::Chunk` from the awkrs chunk, looking it up, setting up the VM — and the bridge attacks this with a set of in-process caches on the runtime. The built `fusevm::Chunk` is cached per awkrs chunk so the eligibility check and the two-pass translation run once per chunk rather than once per record; a single-slot side-table hoists even the hash-map lookup out of the common single-rule case; a VM pool recycles `fusevm::VM` instances across records, preserving their allocations through `reset`; the cache records the precomputed set of write-slots so per-record writeback walks only the modified slots; and slot seeding goes directly from the runtime’s slots into the VM with no intermediate allocation. The measured cumulative win from these caches on the JIT path is on the order of ten percent on a count-over-ten-million-records program. The README is candid that on tight one-op-body micro-benchmarks the JIT path is still *net slower* than awkrs’s own interpreter,

because fusevm's per-call VM setup outweighs the tiny body — the JIT wins on *loops*, where the body runs millions of times per `run()`, not on per-record one-liners.

AOT: ahead-of-time compilation

The `--aot` flag is the JIT's static sibling. For a `BEGIN`-only program — one with no record rules, a pure computation — awkrs can ahead-of-time compile it to a native executable, linking fusevm's AOT-compiled objects against the awk runtime (which is why `Cargo.toml` builds awkrs as a `staticlib` producing `libawkrs.a`). The `aot.rs` module drives this. It is the narrowest of the execution paths — it applies only to input-free `BEGIN` programs — and it is best understood as the logical endpoint of the JIT story: if a numeric computation can be compiled to native code at run time, a self-contained numeric computation can be compiled to a standalone binary ahead of time. Like the JIT, it is an advanced, optional path, not the way most awk programs run.

The honest summary

The JIT and AOT are real, they are measured, and on the narrow class of numeric-loop programs they apply to they are a large, faithful speedup — fourteen to a hundred and ten times the interpreter, bit-for-bit identical to it, with even division-by-zero fatals preserved into native code. They are also experimental, by the project's own description, and inert for the string-and-field programs that are most of awk. The right mental model is the one awkrs's own documentation projects: the JIT is a bonus on top of an already-fast interpreter, on by default because it never hurts (ineligible chunks simply run on the interpreter as before), worth knowing about for numeric-heavy work, and explicitly *not* the headline — the headline is the bytecode VM and its fast paths from Chapter 16, which is why awkrs leads its benchmark tables even with `AWKRS_FUSEVM=0`.

When to reach for the JIT, and when not

Because the JIT is on by default and inert where it does not apply, most users never think about it — which is the design intent — but it is worth knowing the shape of the programs where it earns its keep, so you recognize them. The JIT wins on *tight numeric loops*: a `BEGIN` or `END` block, or a hot region, that runs an arithmetic recurrence millions of times over scalar variables, calling only the eligible operations — arithmetic, comparisons, `++/--`, division and modulo, the transcendentals `sin/cos/exp/atan2`, `int`, and the bitwise `and/or/xor`. The headline measurements are exactly these: a twenty-million-iteration `s += sin(i)` loop dropping from about 12.8 seconds to about 0.15, an `and`-accumulator loop from about 14.0 to about 0.13 — fourteen to a hundred and ten times, bit-for-bit identical to the interpreter. Where the JIT does *nothing* is the entire rest of awk: any program that touches strings, fields, arrays, regexes, `getline`, `print`, or a user function is ineligible and runs on the interpreter, which is most real awk work. And on a tight *per-record* one-liner — a body of a single fused op run once per record — the JIT path is actually net slower than awkrs's own interpreter, because fusevm's per-call VM setup outweighs the tiny body; the JIT wins on loops, where the body runs millions of times inside one `run()`, not on per-record one-liners. So the practical rule is: if your program is a numeric kernel — a simulation, a numeric integration, a hot arithmetic loop in `BEGIN/END` — the JIT is silently making it dramatically faster, and you can confirm that by comparing against `AWKRS_FUSEVM=0`. If your program is field extraction, filtering, or text rewriting, the JIT is not engaged and the speed you are getting is the bytecode VM and its fast paths, exactly as Chapter 16 described. Either way you do nothing: the engine routes each chunk to the path that runs it fastest, and the bit-for-bit identity guarantee means the routing can never change your answer.

The two caches, kept straight

It is worth fixing the distinction between awkrs's two caches, because they are easy to conflate and they accelerate entirely different stages. The *bytecode* cache of Chapter 18 lives at `~/.awkrs/scripts.rkyv` and caches the *compiled program* — the output of the lexer, parser, and compiler — keyed by the source script,

so the second run of `awkrs -f script.awk` skips compilation. The *machine-code* cache of the JIT lives at `~/.cache/fusevm-jit` (overridable with `FUSEVM_JIT_CACHE_DIR`, capped with `FUSEVM_JIT_CACHE_MAX_BYTES`) and caches fusevm-emitted *native code* — the output of Cranelift — keyed by a chunk's op-hash, so a numeric loop that was JIT-compiled on one run reloads its native code on the next rather than recompiling it. They differ in what they store (awkrs bytecode versus fusevm native code), where they live (two directories), what they key on (source versus chunk op-hash), and what they invalidate on (source mtime and binary version versus chunk content). The same fusevm machine-code cache is shared with the sibling tools of Chapter 47, since it is fusevm's, not awkrs's. One caches the *compile*; the other caches the *JIT*. A cold first run pays both the compile and any JIT; a warm run reloads the compiled program from the first cache and the native code from the second, which is why a repeatedly-invoked numeric script gets faster across its first few runs and then stays fast. Neither cache can ever return a stale or wrong result — both fail safe toward regenerating — so both are on by default and invisible, exactly as a cache should be.

What `-aot` actually does, step by step

The AOT path is short enough to walk completely, and walking it explains both why the `BEGIN`-only restriction exists and what the resulting binary actually contains.

`build_native` parses the program text with the ordinary parser — the same `parse_program` everything else uses — and then checks the restriction: every rule's pattern must be `BEGIN`. Anything else is rejected immediately with a message stating the reason, that per-record rules and `END` need the record-loop driver.

The check is on the *pattern*, not on the presence of input, and that is the precise statement of the scope. `awk 'BEGIN { ... }'` qualifies. A program with a single bare- pattern rule does not, even if you intend to give it no input, because the compiled form would need the record loop that reads, splits, and dispatches.

Passing the check, the program's `BEGIN` actions lower to a *single* fusevm chunk through `compile_begin_only`, with user functions appended as sub-chunks. An empty chunk — a program that compiled to nothing — is rejected rather than producing a binary that does nothing.

`fusevm::aot::compile_object` lowers that chunk to a relocatable object file next to the requested output path. Then awkrs writes a two-line C entry stub:

```
extern long fusevm_aot_run_embedded(void);
int main(void){ return (int)fusevm_aot_run_embedded(); }
```

and invokes `cc` on the stub, the object, and the awkrs runtime static library, adding `-framework CoreFoundation` on macOS, producing the output executable. The stub and the object are removed afterward whether or not the link succeeded, and a nonzero `cc` exit becomes a link-failed error carrying the compiler's exit code.

Two things about this shape are worth noticing. First, the produced binary is not a wrapper around awkrs — it is the compiled chunk plus the awkrs runtime, statically linked, with a `main` that calls straight into the embedded run. Second, the toolchain dependency is `cc` at *build* time only. The output binary needs nothing but its own linked contents to run.

Why the crate is a staticlib

Chapter 25 noted that `Cargo.toml` builds awkrs as a `staticlib` producing `libawkrs.a`; this is the reason. The AOT-produced executable calls into the awk runtime — the field engine, the builtins, the print buffer, the value model — and that runtime has to be linkable into a C-linked executable. A Rust `rlib` is not; a `staticlib` is.

The library is located at build time by `runtime_staticlib`, which honors an explicit `AWKRS_AOT_RUNTIME_LIB` environment variable and otherwise looks for `libawkrs.a` beside the running executable. If neither yields a path, the error names the variable to set. The beside-the-executable default is the one that works for a

build tree without configuration; the environment override is what makes an installed or relocated layout work without patching anything.

The registration hook, and the leaked runtime

The most interesting part of the module is what happens when the AOT binary starts, because the compiled chunk cannot run without the awk runtime standing behind it.

`fusevm`'s AOT runtime calls a hook named `fusevm_aot_register_builtins`, and `awkrs` implements it. The hook constructs a `Runtime`, seeds `ARGV` and `ARGC` from the process arguments — `BEGIN` can read them, and a `BEGIN`-only program reads no records so nothing else consumes them — installs `awkrs`'s host and regex host and builtins onto the run VM, and registers an exit flush.

The runtime is created with `Box::leak`, producing a `&'static mut`, and the guard that installs it as the thread's current runtime is deliberately `std::mem::forget`-ten so the installation never unwinds. Both are unusual things to write and both are correct here for the same reason: this is a one-shot process whose entire lifetime is one run of one chunk. There is no second program, no teardown phase, no reuse of the slot. Leaking the runtime for the process lifetime is not a leak in any meaningful sense — the memory is reclaimed by process exit, which is the next thing that happens — and it removes the lifetime plumbing that would otherwise be needed to hand a borrowed runtime across an FFI boundary into generated code.

The comments in the source say exactly this, and it is worth reading them as a general lesson about scope-appropriate engineering. The same construction inside the normal interpreter, which runs many programs in a process under `--lsp` or the REPL, would be a genuine bug. In a single-entry AOT binary it is the simplest thing that is correct.

The atexit flush, and why it is necessary

awk buffers `print` output in the runtime's print buffer (Chapter 34) and flushes it at the end of the record loop. The AOT binary has no record loop, so it has no such end — `main` calls the embedded run and returns.

Without something else, a `BEGIN`-only program compiled to native code would compute its answer, leave it in the buffer, and exit silently. The fix is an `atexit` handler registered during the startup hook, which reads the leaked runtime through an `AtomicPtr`, writes the print buffer to stdout, and flushes.

The `AtomicPtr` is how the handler — a plain `extern "C" fn` with no arguments, because that is what `atexit` takes — reaches the runtime the hook created. It is stored with `Release` ordering at registration and loaded with `Acquire` in the handler, and the handler returns immediately on a null pointer, so an exit before registration is harmless.

This is a small piece of plumbing but it is the kind that is easy to omit and produces a mystifying symptom — a program that works interpreted and produces no output compiled. It exists because the AOT path deleted the stage that used to do the flushing, and deleting a stage means auditing what that stage was responsible for.

The honest scope, restated

The AOT path is the narrowest execution path in `awkrs` and the module says so in its own documentation: the scope is `BEGIN`-only programs, the `awk 'BEGIN{...}'` calculator case, which it identifies as awk's compute-bound sweet spot.

That framing is right. A `BEGIN`-only awk program is a pure computation with no input, no field splitting, no record loop — which is to say, it is the one shape of awk program where awk's defining machinery is entirely absent and only the expression evaluator is doing work. That is exactly the shape a static compiler can handle well, and exactly the shape where the interpreter's remaining overhead is a meaningful fraction of the runtime. It is also, obviously, not what most awk programs are.

Read together with the JIT, the two paths bracket the same insight from opposite ends: a numeric loop in an awk program can be compiled to native code at run time, and a program that is *nothing but* numeric computation can be compiled to a standalone binary ahead of time. Neither is the reason awkr is fast on ordinary text-processing work — the bytecode VM and its fast paths are — and the project's own documentation is careful to keep them positioned as optional advanced paths rather than as the headline.

Chapter 18 — The Bytecode Cache

Of all of awkrs's engineering, the bytecode cache is the one the project claims as a genuine first. Based on a survey of the major public awk implementations — BWK awk, gawk, mawk, goawk, fawk, and zawk — awkrs appears to be the first awk to pair a bytecode VM with a *persistent on-disk bytecode cache*: a cache that remembers the *compiled program* across separate process invocations, so the second time you run a script the lexer, parser, and compiler do not run at all. This chapter is how that cache works, why it is safe, and what it does and does not accelerate.

The problem it solves

awk's startup cost has two parts: bringing up the runtime, and compiling the program. The runtime part is small and fixed. The compile part — lex, parse, build the AST, emit bytecode, run the peephole fusion — scales with the size of the program, and for a substantial script it is real work that, without a cache, is repeated identically on every single invocation. For a one-liner this is negligible, but awk scripts are run *constantly* — in loops, in pipelines, once per file in a `find -exec`, thousands of times in a build — and a script that is recompiled from scratch on every run is paying its compile cost over and over for no reason, because the source has not changed. The cache makes you pay it once.

The design: rkyv outer, bincode inner

The cache is a single file at `~/.awkrs/scripts.rkyv`. Its outer container is an rkyv archive — a `ScriptShard` holding a header and a hash map from a script key to a `ScriptEntry` — and each entry's compiled-program blob is a bincode-serialized `CompiledProgram`. This rkyv-outer/bincode-inner split is the same architecture zshrs and strykelang use for their caches, and it is chosen deliberately: rkyv's archived format can be memory-mapped and read *zero-copy*, so the read path does not deserialize the whole shard — it mmaps the file, validates the archived root once with rkyv's `check_archived_root`, looks up the matching entry in the archived hash map directly in the mapped bytes, and only bincode-decodes that one entry's blob. A shard with a thousand cached scripts costs nothing to open; only the script you are running is decoded.

The shard header carries a magic number (`"AWKR"`), a format version, the awkrs version string, the pointer width, and a build timestamp, and the read path rejects the whole cache if any of these do not match — so a cache written by a different awkrs build, or on a different architecture, is simply ignored rather than misread. Writes are serialized with an exclusive `flock` on a lock file and made durable with the write-temp-then-atomic-rename pattern, so concurrent awkrs processes cannot corrupt the shard and a reader always sees either the complete old cache or the complete new one, never a torn write.

Invalidation: never a stale answer

A cache that could return a stale compiled program would be worse than no cache, so awkrs's invalidation is conservative and automatic. A cached entry is a hit only when *both* conditions hold: the source file's modification time (seconds and nanoseconds) exactly matches what was recorded when the entry was cached, *and* the awkrs binary that wrote the entry is not older than the current binary. The first condition means editing the script invalidates its cache entry — you can never run a stale compile of a script you just changed. The second means upgrading or rebuilding awkrs silently invalidates the *entire* cache — because a new awkrs

might compile the same source to different bytecode, and reusing the old binary's output would be wrong. Both checks are conservative: when in doubt, recompile. `AWKRS_CACHE=0` disables the cache entirely, and the cache also self-maintains, with an eviction pass that drops entries whose source files have vanished or changed.

What engages it, and what does not

The cache engages only for the *exact* invocation shape `awkrs -f script.awk` — one program file, no inline program, no library mixing, and none of the AST-dependent diagnostic flags. Specifically, `cacheable_script_path` returns a cacheable path only when there is exactly one `-f` proffile and *none* of `-i/--include`, `-l/--load`, `-e/--source`, `-E/--exec`, `-D/--debug`, `-L/--lint`, `-t/--lint-old`, `-o/--pretty-print`, or `-g/--gen-pot`. The reason for the exclusions is uniform: every excluded flag needs the AST, not just the compiled bytecode — the linter and pretty-printer walk the tree, the include and load forms expand source before parsing, the debug listing reflects the rules — and the cache stores only the compiled `CompiledProgram`, not the AST. So the cache is precisely scoped to the one form where the compiled program is all that is needed: a plain script run on data. This is also the most common form by far, which is why the cache helps in practice despite its narrow scope.

A subtle but important detail: the `CompiledProgram` that round-trips through the cache carries the `parallel_safe` flag and the rule count with it. So a cache hit does not just skip compilation — it also skips the *re-analysis* of whether the program is parallel-safe (Chapter 15) and the sizing of the range-pattern state, because those decisions were made at compile time and serialized into the cached program. The cache preserves not only the bytecode but the compile-time *decisions about* the bytecode, so a cached run takes the same fast-path branches a fresh compile would have.

The competitive claim, stated carefully

awkrs's claim to be the first awk with a persistent bytecode cache is made carefully, and it is worth repeating the care. `fawk` is the closest prior art on JIT — it has a VM and a Cranelift/LLVM JIT — but it recompiles on every invocation; its documentation contains no mention of disk-persisted compiled artifacts. `gawk`'s `pm-gawk` persists script-defined *variables and functions* across runs — a different feature, a persistent heap, not a compiled-bytecode cache. So the specific claim — a bytecode VM paired with a persistent on-disk cache of compiled *bytecode* — appears, on the survey awkrs conducted, to be awkrs's alone. And the project is equally careful to keep this separate from the fusevm machine-code cache of Chapter 17: one caches awkrs bytecode keyed by source, the other caches fusevm-emitted native code keyed by chunk op-hash; they live in different directories, invalidate on different conditions, and accelerate different stages. The bytecode cache is the one that is the claimed first, and it is the one that makes the ordinary `awkrs -f script.awk` invocation skip compilation the second time you run it — quietly, safely, with no action required from you.

A worked cache lifecycle

It helps to trace a single script through the cache to see where the time goes. The first time you run `awkrs -f report.awk big.log`, the invocation is the cacheable shape — one `-f`, no AST-dependent flags — so awkrs lexes `report.awk`, parses it, compiles it to bytecode, runs the peephole fusion and the parallel-safety analysis, and then, because nothing has gone wrong and the shape is cacheable, serializes the resulting `CompiledProgram` into the entry for `report.awk` in `~/awkrs/scripts.rkyv`, recording the script's modification time alongside it. The write takes an exclusive lock and lands atomically, so a second awkrs running the same script concurrently cannot corrupt the shard. The program then executes over `big.log`. The *second* time you run the same command, awkrs memory-maps the shard, validates its archived root once, finds the `report.awk` entry directly in the mapped bytes, checks that the recorded modification time still matches the file's and that the awkrs binary is not newer than the entry, and — both holding — bincode-decodes just that one entry's compiled-program blob and goes straight to execution. The lexer, the parser, the compiler, the

fusion pass, and the parallel-safety analysis all do not run, because their entire output was serialized and is being read back. Now edit `report.awk` and run a third time: the modification time no longer matches, the entry is a miss, and awkrs recompiles and rewrites it. Upgrade the awkrs binary and run a fourth time: the binary is now newer than every entry, so the whole shard is treated as invalid and the script recompiles. At no point does a stale compile run, because both invalidation conditions fail safe toward recompiling. The visible effect across a build that runs the same script a thousand times is that the compile cost is paid once and the other nine hundred ninety-nine runs skip it.

Chapter 19 – Bignum and Locale

Two of awkrs's gawk-compatible features change how numbers themselves behave, and they sit close enough together that this chapter covers both: the `-M` bignum mode, which replaces awk's double-precision floats with arbitrary-precision MPFR numbers, and the `-N` locale mode, which makes numeric *formatting* honor the locale's conventions. They are different mechanisms with one thing in common — each changes the relationship between a number's value and its textual form — and each has a precisely-bounded scope that awkrs documents carefully.

-M: arbitrary-precision arithmetic

awk's numbers are IEEE-754 doubles, which means about fifteen to seventeen significant decimal digits and a hard ceiling on integer exactness around 2^{53} . For most data processing this is plenty, but for the cases where it is not — cryptographic arithmetic, exact large-integer work, high-precision numerics — gawk added the `-M/--bignum` flag, which switches the engine to GNU MPFR (arbitrary-precision floats) and GMP (arbitrary-precision integers). awkrs implements this through the Rust `rug` crate, which wraps MPFR and GMP, and the bignum value type is an MPFR `Float`.

The default working precision is **256 bits** — far beyond a double's 53 — and it is configurable at runtime through `PROCINFO["prec"]`, which awkrs reads (only in `-M` mode) as a bit count clamped to the range 53 to 1,000,000; an out-of-range or absent value falls back to 256. The rounding mode is `PROCINFO["roundmode"]`, whose first character selects the MPFR rounding direction — `N` for nearest (the default), `Z` toward zero, `U` up, `D` down, `A` away from zero. In bignum mode `PROCINFO` also exposes `prec_min`, `prec_max`, `gmp_version`, and `mpfr_version`, so a program can discover the precision bounds and library versions it is running against.

When `-M` is active, the arithmetic operators `+` `-` `*` `/` `%` `^` route through MPFR rather than f64, as do `int`, `intdiv`, `strtonum`, `++/--`, the bitwise operations, the transcendental math functions, and `srand` (seeding from the low 32 bits of the previous seed). String-to-number parsing bypasses f64 entirely: a hex `0x` prefix parses through a GMP integer, gawk-style octal (a leading `0` followed only by digits 0–7 with no `8`, `9`, `.`, or `e`) parses as octal, and otherwise the longest numeric prefix is parsed at full MPFR precision. The formatting follows: `sprintf/printf` integer conversions print the exact arbitrary-precision integer with no i64 or f64 clamp, so a 200-digit number prints all 200 digits, and `%s` of an MPFR float prints exact integers as plain decimal digit strings and trims trailing zeros on non-integers. A nice consequence the README highlights: the default number-to-string conversion uses each value's *own* precision for the MPFR formatting path, so raising `PROCINFO["prec"]` actually widens the displayed precision rather than being undercut by a hardcoded display width. And `compl(0)` in `-M` mode yields the full unsigned 64-bit value `18446744073709551615`, the bignum-correct complement.

The one hard interaction to remember: **`-M` disables the JIT entirely**. The fusevm offload path of Chapter 17 represents awk values as `f64` slots, and an arbitrary-precision MPFR number cannot be squeezed into a 64-bit float without losing exactly the precision `-M` exists to provide. So when you ask for bignum, you get the bytecode interpreter for everything, and the numeric-loop JIT is off. This is the correct trade — precision over the narrow numeric-loop speedup — and it is unconditional. The `bigint_mul.awk` and `modexp.awk` examples in Chapter 26 exercise large-integer arithmetic; under `-M` they compute at full precision on the interpreter.

-N: locale-aware numeric formatting

The `-N/--use-lc-numeric` flag addresses a much narrower question: how a number *looks* in different locales. In much of the world the decimal separator is a comma and the thousands separator a period — `1.234,56` where an American writes `1,234.56` — and gawk's `-N` makes awk's numeric *output* honor the active `LC_NUMERIC` locale. `awkrs` implements this in `locale_numeric.rs`, calling `setlocale(LC_NUMERIC, "")` exactly once (guarded against the multi-thread `setlocale` race), and from then on using the locale's decimal point and thousands separator for the formatting paths: `sprintf`, `printf`, `print`, and the `CONVFMT/OFMT` conversions, including the `%'` grouping flag of Chapter 13.

The scope is bounded with a sharpness worth stating, because it is the boundary that most often surprises people. `-N` affects *formatting only* — how numbers are *written*. It does **not** affect *parsing* — how input text is read as a number. The decimal radix for interpreting `$1` as a number, for `strtonum`, for the implicit string-to-number coercion of a field, stays the period regardless of `-N` or the locale. So a program reading European-formatted input with comma decimals is *not* helped by `-N`; locale-aware numeric *input* is explicitly not implemented, and the README and the source comment both say so plainly. This is, as it happens, the same split gawk draws, and `awkrs` matches it: `-N` is an output convenience, not a bidirectional locale mode. There is also a platform wrinkle from Chapter 13 worth recalling — the thousands separator comes from the C library's `localeconv`, which in the C/POSIX locale is empty on glibc (making `%'` a no-op) but reported as a comma on Apple's libc — so the exact grouping behavior in the default locale follows the host.

The two flags together round out `awkrs`'s numeric story. By default, awk numbers are fast doubles formatted in the C locale. `-M` trades the doubles for exact arbitrary precision at the cost of the JIT. `-N` keeps the doubles but localizes their appearance, on output only. Each is a single flag with a single, well-defined effect, and each is faithful to gawk's version of the same feature, down to the scope boundaries that are the easiest part to get wrong.

A worked -M example: exact large integers

The clearest way to feel what `-M` changes is to watch a computation that overflows a double become exact under bignum. A factorial is the classic case, because it grows past `2^53` — the largest integer a double represents exactly — almost immediately:

```
BEGIN {
    f = 1
    for (i = 1; i <= 30; i++) f *= i
    print f
}
```

Run without `-M`, this computes in IEEE-754 doubles, and by the time `i` passes about twenty the running product has exceeded `2^53`, so the low-order digits are no longer exact — the printed result is a correctly-rounded double, accurate to about fifteen significant figures but wrong in its trailing digits, and printed in `%g` form once it grows large. Run the same program with `awkrs -M`, and every multiplication routes through MPFR at the default 256-bit working precision, so `30!` — a thirty-three-digit number — prints in full, every digit exact, because the integer conversion path under `-M` emits the arbitrary-precision integer with no `i64` or `f64` clamp. Raise `PROCINFO["prec"]` and even larger computations stay exact, up to the million-bit ceiling. The cost, as the chapter stressed, is that `-M` turns the JIT off entirely — an MPFR number cannot live in fusevm's `f64` slot — so a bignum factorial loop runs on the bytecode interpreter. That is the right trade: the whole point of `-M` is exactness, and exactness is incompatible with squeezing the value into a 64-bit float, so `awkrs` gives you the interpreter and the exact answer rather than a fast wrong one. The `bigint_mul.awk` and `modexp.awk` examples exist precisely to exercise this path.

Reading and tuning precision and rounding

`-M` mode exposes two dials through `PROCINFO`, and knowing them is knowing how to trade precision for speed and how to control rounding for reproducibility. `PROCINFO["prec"]` sets the working precision in bits — `awkrs` reads it (only in `-M` mode) clamped to the range 53 to 1,000,000, with an out-of-range or absent value defaulting to 256 — so a program needing more than the default's roughly seventy-seven decimal digits raises it, and one needing less can lower it to go faster, since MPFR arithmetic cost scales with precision. `PROCINFO["roundmode"]` selects the MPFR rounding direction by its first character: `N` for round-to-nearest (the default), `Z` toward zero, `U` up, `D` down, `A` away from zero — the control you reach for when a computation must round a specific way for an external specification or a reproducible result. The mode also exposes `prec_min`, `prec_max`, `gmp_version`, and `mpfr_version` so a program can discover the bounds and library versions it is running against. A nice consequence the README highlights: the default number-to-string conversion uses each value's *own* precision, so raising `PROCINFO["prec"]` actually widens the displayed digits rather than being capped by a hardcoded width. The two dials together let a bignum program tune exactly how much precision it pays for and how it rounds — the knobs `gawk`'s `-M` offers, implemented through the same `PROCINFO` keys so a program written for one runs on the other.

Chapter 20 – Tooling: the Language Server and the Debugger

A language is more than its runtime; it is also the experience of writing in it, and `awk`'s traditional tooling has been thin — syntax highlighting, maybe, and `print`-statement debugging. `awkrs` ships two pieces of modern tooling built into the same binary: a Language Server, reached with `--lsp`, that gives any LSP-aware editor completion, hover documentation, diagnostics, and navigation for `awk`; and a Debug Adapter, reached with `--dap`, that gives any DAP-aware editor real breakpoint-and-step debugging of an `awk` program. Both are protocols spoken over a pipe, so they work in VS Code, Neovim, and any other editor that speaks the standard, with no `awkrs`-specific plugin logic beyond the wiring.

The Language Server

`awkrs --lsp` starts a Language Server that speaks the Language Server Protocol over stdin and stdout — pure JSON-RPC, no terminal output, the way every LSP server is launched by its editor. It identifies itself as `awkrs` with the crate version, and it advertises a substantial capability set: full-document text sync, completion, hover, go-to-definition, find-references, document highlight, document symbols, folding ranges, and signature help (triggered on `(` and `,`).

The features are exactly what you want when editing `awk`:

- **Diagnostics** are computed by actually running `awkrs`'s parser on the buffer. A parse error becomes a diagnostic on the offending line, tagged with source `awkrs`, so you see syntax errors as you type — including errors from `@include` source expansion, reported at the top of the buffer. This is real parsing, not a heuristic linter: the same `parse_program` the runtime uses.
- **Completion** offers the `awk` keywords, the sixty-three builtins (each with its signature as the completion detail), the special variables, and the user-defined function names scraped from the open buffer — so completing a call to a function you defined three screens up just works.
- **Hover** shows documentation. Over a builtin it renders the function's signature in an `awk` code block plus a description; over a special variable or a keyword it shows the relevant markdown. The documentation corpus is hand-authored — signatures and docs for around fifty builtins, the special variables, and every keyword — and it is the *same* corpus that the offline `gen-docs` binary renders into the static `reference.html`, so the editor hover and the published reference can never drift apart: they have one source.
- **Signature help** scans backward from the cursor, counting parentheses and commas to find the enclosing call and which argument you are on, and shows the signature with the active parameter highlighted — the “you are typing the third argument of `substr`” affordance.
- **Document symbols** lists the program's functions (as function symbols) and its `BEGIN/END/BEGINFILE/ENDFILE` rules (as event symbols), so the editor's outline view and symbol-jump work. **Folding** pairs braces while correctly skipping braces inside strings and comments. **Definition, references, and highlight** work by whole-word scans.

The significance of the shared documentation corpus is worth dwelling on, because it is the same single-source-of-truth discipline that runs through `awkrs`'s architecture. The builtin list comes from the same `BUILTIN_NAMES` table the namespace system uses; the special-variable list from the same `SPECIAL_GLOBAL_NAMES` table; the signatures and docs from hand-authored tables consumed by *both* the LSP and the reference generator. There is exactly one place each fact is written, and the editor experience, the published docs, and the runtime's

own name tables all read from it — so a builtin added in one place shows up everywhere, and none of the three can describe `awk` differently from the others.

The Debugger

`awkr` `--dap` starts a Debug Adapter — the protocol VS Code and an increasing number of editors use to drive a debugger. The architecture is a clean two-layer split. `debugger.rs` is the front-end-agnostic state machine — a `Debugger` struct holding the breakpoints (by line and by function name), the stepping mode, the call-depth tracking for step-over and step-out, and a same-line guard so that two statements on one physical line stop only once while a loop body still re-fires its breakpoint each iteration. `dap.rs` is the protocol server that wraps the state machine and speaks DAP to the editor. The VM drives the whole thing: the compiler, under `--dap`, emits a zero-cost `DebugLine` marker opcode before each source line, and when the VM executes one it asks the debugger whether to stop, and if so captures a snapshot of the program state and blocks until the editor says to resume. When `--dap` is not in effect, the `DebugLine` markers are not emitted at all, so debugging support costs nothing in a normal run.

The debugger supports the operations you expect from a real source-level debugger:

- **Breakpoints** on lines and on function names, set through the editor.
- **Stepping** — step over (run the current line, including any calls, and stop at the next line at this call depth or shallower), step into (stop at the next line in any frame, including inside a call), step out (run until the current function returns), and continue. The call-depth bookkeeping is what makes step-over and step-out distinguish “the next line here” from “a line inside a function I called.”
- **Stop on entry** — the debugger starts in stepping mode so the program halts on its first line, the standard launch behavior.
- **Stack frames and scopes** — the editor’s call stack shows the `awk` function frames, and each frame exposes a Locals scope and a Globals scope.
- **Variable inspection** — scalars show their values (strings quoted, regexp values rendered as `@/re/`), and *arrays expand* — the editor can drill into an associative array element by element, with the `SUBSEP` byte rendered as a comma so multidimensional keys read naturally, capped at a few thousand elements so a huge array does not flood the editor.
- **Evaluate** — the editor’s watch and hover-evaluate boxes can look up a variable by name in the current snapshot.

There are two transports, and the distinction is practical. With a bare `--dap`, the adapter speaks DAP on stdin and stdout — simple, but the program’s own `print` output interleaves with the protocol on the same streams. With `--dap host:port`, the adapter connects a TCP socket for the protocol and leaves stdout free for the program’s output — which is the mode IDE plugins use, because it keeps the debug conversation and the program’s output cleanly separated. An environment variable, `AWKRS_DAP_LOG`, turns on protocol tracing to stderr for debugging the debugger, off by default precisely so it cannot corrupt the stdio protocol stream.

What the debugger deliberately does *not* have is a terminal REPL — the strykelang-style `perl -d` interactive prompt was intentionally left out, and DAP is the only front-end. This is a defensible choice: rather than maintain a bespoke terminal debugger UI, `awkr` implements the one protocol that every modern editor already knows how to drive, and lets the editor be the UI. The result is that debugging an `awk` program in `awkr` is the same experience as debugging any other language in your editor — set a breakpoint in the gutter, hit a line, inspect the variables, step through — which is something `awk` has essentially never had. Together with the Language Server, it makes `awkr` not just a faster place to *run* `awk` but a genuinely modern place to *write* it.

The single-source-of-truth discipline, restated

The tooling chapter is the right place to gather a thread that has run through the whole book, because the tooling is where it pays off most visibly: `awkr` is built so that each fact about the language is written in exactly one place and consumed everywhere. The builtin names live in one table, `BUILTIN_NAMES`, and the namespace system’s

prefixing pass, the LSP's completion list, and the static arity checker all read from it — so a builtin cannot be known to one part of the engine and unknown to another. The special-variable names live in one table, `SPECIAL_GLOBAL_NAMES` (mirrored as `SPECIAL_VARS`), consulted by the compiler's slot-versus-map decision and by the namespace pass that must leave specials unprefixes. The builtin documentation and signatures live in one hand-authored corpus, consumed by *both* the LSP's hover and the offline `gen-docs` binary that renders `reference.html`, so the tooltip in your editor and the published reference page are the same words and cannot drift. And the parser is one implementation, run by the runtime, by the LSP's diagnostics, and by the `@include` expander alike, so the syntax error your editor underlines is the runtime's own verdict. This discipline is not glamorous and does not appear in a feature list, but it is why the experience of writing awkrs is coherent: there is no second implementation of the builtin list to fall out of sync, no separately-maintained lint grammar to disagree with the real one, no documentation that describes a behavior the engine does not have. One source per fact, read everywhere — the same instinct that makes awkrs reuse fusevm's JIT rather than build a second one, applied to the names, the docs, and the grammar.

A debugger session, start to finish

To make the debugger concrete, trace a session over a small script. You launch the adapter pointed at the program — an IDE plugin runs `awkrs --dap host:port` so the protocol takes the socket and the program's own `print` output stays on stdout — and because the debugger starts in stepping mode with stop-on-entry, the program halts on its first line before any record is read. The editor's gutter shows the line, the call stack shows a single frame, and the Locals and Globals scopes are populated from a snapshot of the program state. You set a breakpoint on a line inside a record action by clicking the gutter, which sends a set-breakpoints request the adapter records by line; you set a breakpoint on a function by name the same way. You hit continue, and the VM runs — executing the zero-cost `DebugLine` markers the compiler emitted under `--dap`, each one asking the debugger whether to stop — until a record reaches your breakpoint, at which point the VM captures a snapshot and blocks. Now you inspect: a scalar shows its value (strings quoted, a regexp value rendered as `@/re/`), and an *array* expands, so you drill into an associative array element by element with the `SUBSEP` byte shown as a comma so a multidimensional key reads naturally. You step over a line — the call-depth bookkeeping running the line, including any function it calls, and stopping at the next line at this depth — or step into a function to follow a call, or step out to run until the current function returns. A watch expression evaluates a variable by name against the current snapshot. The loop body re-fires its breakpoint on each iteration, because the same-line guard only suppresses *two statements on one physical line* from stopping twice, not a line revisited by a loop. When you are done you continue to the end. The whole experience is the one every other language in your editor already gives you, and it is the thing awk has essentially never had — delivered not by a bespoke terminal UI but by speaking the one protocol the editor already knows how to drive.

The other diagnostic flags

Beyond the LSP and the debugger, awkrs carries a set of static-analysis flags that report on a program without running it normally, and they round out the tooling story. `-l/--lint` runs the linter, with `fatal`, `invalid`, and `no-ext` sub-modes that escalate from warnings to hard errors and can flag uses of non-POSIX extensions — the static counterpart to the runtime `LINT` variable. `-o/--pretty-print` emits an AST pretty-print of the program, awkrs's reformatted listing (not gawk's canonical reformatter text, a documented difference). `-D/--debug` produces a static rule-and-function listing — again not gawk's interactive debugger, that role belonging to the DAP server. `-d/--dump-variables` dumps the global variables after a run. `-p/--profile` produces a wall-clock-and-hit-count summary, single-threaded, an awkrs-specific format rather than gawk's per-line profile. And `-g/--gen-pot` generates a gettext `.pot` translation template from the marked strings and exits. The throughline is that each of these flags needs the AST rather than just the compiled bytecode — the linter and pretty-printer walk the tree, the debug listing reflects the rules — which is exactly why, as Chapter 18 noted, every one of them excludes the program from the bytecode cache: the cache stores the compiled form, and these flags need the tree the compiler consumed. They are the static-analysis half of awkrs's tooling, complementing the live LSP and DAP servers with offline reports a build or an editor command can invoke.

Chapter 21 – The Lexer and the Parser

Chapter 16 walked the pipeline end to end and named its five stages. The next five chapters open each part of the machine all the way up, and they begin where the machine begins: at the front, where a flat string of awk source becomes the tokens and the syntax tree that everything downstream consumes. Two modules own this front end, and they are the two largest source files in the project — `lexer.rs` at roughly thirty-six hundred lines and `parser.rs` at roughly fifty-one hundred. That size is not accident or sprawl; it is the cost of implementing the *whole* of awk's surface — every POSIX token, every gawk extension, every lexical corner case that distinguishes a faithful awk from an approximate one — and it is the same care that the rest of the book has been tracing, applied to the part of the engine a user never sees directly but feels in every error message and every correctly-parsed program.

The lexer's job, and awk's two lexical hazards

The lexer turns the source text into a stream of tokens: keywords (`if`, `while`, `function`, `BEGIN`), identifiers, builtin names, numeric constants, string constants, regex literals, operators, and punctuation. For most of a program this is mechanical — scan a run of letters, decide whether it is a keyword or an identifier, emit a token — and the bulk of `lexer.rs` is exactly that mechanical work done thoroughly. The interesting part, the part that makes a lexer for awk harder than a lexer for most languages, is two specific hazards that awk's grammar builds in: the meaning of a backslash inside a string, and the meaning of a slash in the program text. Both are resolved at lex time, and both are places awk's fidelity is tested.

String escapes, resolved at lex time

When the lexer reads a double-quoted string constant, it interprets the C-style escape sequences in it then and there, so that by the time the parser sees a string token its bytes are already the bytes the program intended. awk implements the full escape vocabulary: the single-character escapes `\a` (alert/bell), `\b` (backspace), `\f` (form feed), `\n` (newline), `\r` (carriage return), `\t` (tab), `\v` (vertical tab), `\\` (a literal backslash), `\"` (a literal quote), and `\/` (a literal slash); the hexadecimal escape `\xHH`, which reads one or more hex digits and emits the corresponding byte; and the octal escape `\NNN`, which reads up to three octal digits. An unknown escape — a backslash before a character with no special meaning — follows awk's lenient rule of keeping the backslash-and-character roughly as written rather than erroring, which is what most awk programs in the wild expect. Doing this work in the lexer rather than the runtime means a string constant is a settled sequence of bytes the moment it is tokenized, and the same escape handling is reused for regex literals where it interacts with the regex engine's own metacharacters.

Numbers as single tokens

A number is lexed as one token, including its exponent. This sounds trivial and is a place an earlier awk got wrong: `1e7` must be a *single* numeric token worth ten million, not the identifier-ish `1`, `e`, `7` salad a naive scanner might produce, and the report records pinning scientific notation as a single token as a parity fix. The lexer also implements gawk's source-level number rules from Chapter 6 — a `0x/0X` prefix introduces hexadecimal, a leading `0` introduces octal only when every following digit is `0` through `7` (so `01238` lexes as decimal because the `8` disqualifies the octal reading), and a number containing a `.` is decimal with a fractional part rather than an octal integer. These rules live in the lexer because they are decisions about how a run of digit-characters

becomes a single numeric value, and getting them here means a program ported from gawk reads its numeric literals identically.

The slash: division or the start of a regex

awk's single most famous lexical ambiguity is the slash. The character `/` is both the division operator and the delimiter of a regex literal, and which one a given `/` means depends entirely on what came before it. The lexer resolves this by tracking, as it scans, whether a `/` at the current position *could* begin a regex — and the answer is a function of the previous token. After a value-yielding token — a number, an identifier, a closing parenthesis, a `$`-field, a string constant — a `/` is division, because division is a binary operator and a value is sitting to its left waiting to be its left operand. At a *primary* position, where a value is expected and none has yet appeared — at the start of an expression, after `(`, after `,`, after `~`, after a keyword like `return` — a `/` cannot be division (there is no left operand), so the lexer re-reads it as the opening delimiter of a regex literal and scans to the closing slash. This is the mechanical heart of the rule Chapter 4 described from the language side: it is why `a / b` divides, why `/re/` in pattern position matches `$0`, and why `x = /re/` tests the current record rather than dividing by a regex. The decision is made once, in the lexer, by a single piece of state that remembers whether a regex is currently legal — and because it is made at lex time, the parser never has to agonize over an ambiguous slash.

The parser: recursive descent over the whole grammar

`parser.rs` is a hand-written recursive-descent parser, and it is the largest module in the project because it implements the entire awk grammar plus the gawk extensions in one place. A program parses as a sequence of *items*: rules (`pattern { action }` in all the forms Chapter 4 enumerated) and function definitions (`function name(params) { body }`). A rule's pattern may be a special pattern, a regex, an expression, or a range; its action is a brace-delimited list of statements; either part may be absent with the defined meanings from Chapter 3. The statement grammar is the C-shaped vocabulary of Chapter 7 — `if/else`, `while`, `do/while`, the three-clause `for` and the `for (k in arr)` form, `switch`, `break`, `continue`, `next`, `nextfile`, `exit`, `return`, `delete`, and the I/O statements — each parsed by its own function that recurses into the expression grammar for its sub-parts.

The precedence ladder

The expression grammar is the part where a recursive-descent parser earns its keep, because awk's operators have a specific precedence and associativity that the parser encodes as a ladder of functions, each handling one precedence level and calling the next-tighter level for its operands. From loosest-binding to tightest-binding, the ladder awkrs implements is: assignment (`=` and the compound forms, right-associative); the ternary `?:`; logical-or `||`; logical-and `&&`; array membership `in`; the match operators `~` and `!~`; the relational and equality comparisons `<` `<=` `==` `!=` `>=` `>`; *string concatenation* (juxtaposition, no operator character); additive `+` `-`; multiplicative `*` `/` `%`; unary `+` `-` `!`; exponentiation `^`/`**` (right-associative); the postfix and prefix `++`/`--`; and finally the primaries — literals, parenthesized expressions, field references `$n`, array subscripts, function calls, and `getline`. Two rungs of this ladder are the ones that surprise people and that awkrs gets exactly right. Concatenation sits *above* the comparison operators but *below* the arithmetic ones, so `1 " " 2` concatenates while `"a" < "b" "c"` compares `"a"` against the concatenation `"bc"`. And exponentiation binds *tighter* than unary minus and associates to the *right*, so `-2 ^ 2` parses as `-(2 ^ 2)` and `2 ^ 3 ^ 2` as `2 ^ (3 ^ 2)` — the precise behavior the `calc_id.awk` example in Chapter 26 reimplements by hand and tests with those very inputs.

The getline grammar, and the parenthesis trap

`getline` is the one expression complex enough to bend the grammar around itself, and the parser handles each of its forms from Chapter 7 — bare `getline`, `getline var`, `getline < file, cmd | getline`, and the coprocess `cmd |& getline var`. The form that creates a famous trap is the file-redirected read combined with a comparison: `getline line < "config" > 0` does *not* mean “read into `line`, then compare the file against zero” the way a left-to-right reading suggests, because the `<` here is a redirection bound tightly to `getline` while the `>` is a comparison, and the two `</>` look symmetric but are not. The idiom that disambiguates it is the parenthesization Chapter 7 insisted on — `(getline line < "config") > 0` — and the parser’s job is to bind the redirection to the `getline` so that the parenthesized form means what the programmer wants. The coprocess `|&` is parsed as its own redirection kind, feeding the `Coproc` variant the compiler emits, and mixing `|` and `|&` on one command is rejected.

Errors with a line, and one parser for everything

When the parser cannot proceed, it produces an `Error::Parse { line, msg }` — a diagnostic carrying the one-based source line and a human-readable message in the “expected X, found Y” idiom — and the lexer-parser pipeline reports the first fault it hits with that line context rather than guessing past it. The important architectural fact about this error path is that there is exactly *one* parser in `awkrs`, and everything that needs to understand `awk` source runs it. The runtime parses your program with `parse_program`; the Language Server of Chapter 20 computes its diagnostics by running *the same* `parse_program` on your editor buffer, which is why the squiggle under a syntax error in your editor is the runtime’s own opinion and not a separate heuristic linter that might disagree with it; and the `@include` expansion of Chapter 12 feeds its assembled source through the same parser, so an error in an included file surfaces with the same machinery. One grammar, one implementation, consulted everywhere `awk` source is read.

`ast.rs`, `ast_fmt.rs`, and the boundary the cache cannot cross

The parser’s output is the abstract syntax tree defined in `ast.rs` — the expression, statement, pattern, and rule node types — and a companion module `ast_fmt.rs` knows how to render that tree back into readable `awk`, which is what the `-o/--pretty-print` flag uses to emit a reformatted listing. The AST is also the quiet protagonist of Chapter 18’s cache story. The bytecode cache stores the compiled `CompiledProgram`, *not* the AST, because the AST is large and the compiled form is what execution needs — and that single fact is why the cache is scoped exactly as it is. Every invocation form that needs the tree rather than just the bytecode — the linter that walks it, the pretty-printer that re-renders it, the `@include/@load` forms that expand source before parsing, the debug listing that reflects the rules — is excluded from the cache, because the thing those features consume is precisely the thing the cache throws away after compilation. The front end produces the AST; the compiler consumes it into bytecode; and the line between “needs the AST” and “needs only the bytecode” is the line the cache draws.

Chapter 22 – The Virtual Machine and the Opcode Catalogue

The compiled program is a stream of bytecode, and the bytecode is run by a stack machine. Chapter 16 introduced the shape of this — ninety-six opcodes, each kept small, jump targets as absolute indices — and showed the headline fused superinstructions. This chapter is the catalogue: the full opcode set organized by family, the compiled structures the VM operates on, and the two mechanisms that are subtle enough to deserve their own treatment, array call-by-reference and the `for-in` iteration protocol. It is the most reference-shaped chapter in the book, because the opcode set *is* the engine's instruction-set architecture, and knowing its families is knowing what the machine can do in one step.

The compiled structures

The unit of compiled code is a `Chunk` — a flat `Vec<Op>`, a vector of opcodes. Rule bodies, function bodies, and each of the `BEGIN/END/BEGINFILE/ENDFILE` blocks compile to chunks, and a whole program is a `CompiledProgram` gathering them: the begin, end, beginfile, and endfile chunk lists; the record rules, each a pattern and a body chunk; the user functions in a name-keyed map; the shared `StringPool`; and a block of metadata. The `StringPool` is the interning table — every string constant and every variable name is stored once and referred to everywhere by a cheap `u32` index, so an opcode that mentions a variable carries a small integer, not a string. The metadata is where the compiler's decisions are preserved for the VM and for the cache: `slot_count` and `slot_names` size and name the flat slot array, `slot_map` is the reverse name-to-slot lookup the cold path uses, `array_var_names` records which names are used as arrays for `PROCINFO` introspection, and the two cached-analysis fields `parallel_safe` and `prog_rules_len` carry the Chapter 15 safety verdict and the rule count so that a cache hit (Chapter 18) does not have to re-derive them. `init_slots` builds the runtime's initial slot vector from this, seeding each slot from any matching pre-set variable.

The opcode families

The `Op` enum's ninety-six variants fall into a dozen families, and walking them is the fastest way to see the machine's whole vocabulary.

- **Constants.** `PushNum(f64)` pushes a float literal; `PushNumDecimalStr` pushes a decimal-integer literal by its digit-string pool index so that under `-M` it can be parsed exactly at full precision; `PushStr` pushes an interned string; `PushRegexp` pushes a gawk `@/.../` typed-regexp value.
- **Variable access.** This is the two-path split from Chapter 16 made concrete. `GetSlot(u16)/SetSlot(u16)` are the fast flat-array path for user scalars; `GetVar(u32)/SetVar(u32)` are the HashMap path for the special variables; `GetField/SetField` read and write `$n` with the index on the stack; `GetArrayElem/SetArrayElem` index an associative array; and `SymtabKeyCount` serves `length(SYMTAB)`. The dedicated `GetNR/GetFNR/GetNF` opcodes read those three hot specials without touching the map at all.
- **Compound assignment and increment.** `CompoundAssignSlot/CompoundAssignVar/CompoundAssignField/CompoundAssignIndex` each carry a `BinOp` and apply `lvalue op= rhs` in place for the four lvalue kinds; the `IncDec` family does `++/--` on each lvalue kind, with statement-context variants (`IncrVar`, `DecrVar`) that discard the result.

- **Arithmetic, comparison, and unary.** `Add Sub Mul Div Mod Pow` pop two and push one; `Pow` is the right-associative exponentiation. The six comparisons `CmpEq ... CmpGe` push a numeric 0/1 using the POSIX number-or- string rules of Chapter 6. `Neg Pos Not` are the unary operators, and `ToBool` normalizes a value to 0.0/1.0.
- **String and regex.** `Concat` joins two values' string forms; `RegexMatch` and `RegexNotMatch` implement `~` and `!~`; `MatchRegexp(u32)` tests a regex constant against `$0` for a bare-regex pattern.
- **Control flow.** `Jump(usize)` is an unconditional jump to an absolute index; `JumpIfFalsePop` and `JumpIfTruePop` pop the top and branch on its truth. Because targets are absolute indices patched at compile time, a branch at runtime is a single assignment to the program counter.
- **Output.** `Print` and `Printf` each carry an argument count and a `RedirKind` — `Stdout`, `Overwrite` (`>`), `Append` (`>>`), `Pipe` (`|`), or `Coproc` (`|&`) — popping their arguments and, for the non-stdout kinds, the destination path.
- **Flow signals.** `Next`, `NextFile`, `ExitWithCode`, `ExitDefault`, `ReturnVal`, and `ReturnEmpty` cause the VM to return control to the record loop or the caller, carrying the Chapter 7 verbs into the engine. These map to the `Flow` enum the record loop reads.
- **Function calls.** `CallBuiltin` and `CallUser` invoke a builtin or a user function by name index with an argument count; `CallIndirect` resolves a callee whose name is computed at runtime (the `@name(...)` form of Chapter 9); and `CallUserBindArrays` is the array-by-reference call described below.
- **Type introspection.** The `Typeof` family — `TypeofVar`, `TypeofSlot`, `TypeofArrayElem`, `TypeofField`, `TypeofValue` — implements `typeof` for each kind of operand, with the lvalue forms able to report "untyped" that the value form cannot.
- **Arrays.** `InArray` tests membership without creating a key; `DeleteArray` empties an array; `DeleteElem` removes one key; and `JoinArrayKey(n)` pops `n` values and joins them with `SUBSEP` into the composite multidimensional key of Chapter 8.
- **The complex builtins.** `GetLine` carries its optional target variable, its `GetlineSource` (`Primary`, `File`, `Coproc`, `Pipe`), and whether it pushes a result; `SubFn/GsubFn` carry a `SubTarget` describing the lvalue they rewrite (`Record`, `Var`, `SlotVar`, `Field`, or `Index`); `Split/Patsplit` carry the destination array, whether a separator was supplied, and an optional `seps` array; `MatchBuiltin` carries an optional capture array; and `Asort/Asorti` carry source and optional destination array names.
- **Iteration and stack.** `ForInStart/ForInNext/ForInEnd` run the `for-in` protocol; `Pop` and `Dup` are the stack primitives, `Dup` used by `switch` to test one value against several cases.

A test asserts that every `Op` variant fits in thirty-two bytes, which keeps a chunk compact and cache-friendly — a long rule body is a tight array of small instructions, exactly the layout a fast dispatch loop wants.

Array call-by-reference, in the bytecode

Chapter 9 promised that arrays pass to functions by reference, including the hard operations — a callee can read, write, iterate, and `delete` through an array parameter and have it act on the caller's array. The opcode that makes this real is `CallUserBindArrays`. An ordinary `CallUser` pops the argument values and calls; `CallUserBindArrays` additionally pops, in a second group, the *names* of the arguments, and after the function returns it writes each `Value::Array` parameter back to the caller's named variable when the name is non-empty. The stack is laid out as two consecutive groups — the names above the values — and the opcode walks them in lockstep. This is what turns awk's "arrays are shared" from a slogan into a mechanism: the function body operates on its parameter, and the binding opcode ensures that whatever the body did to an array parameter is reflected in the caller's array, which is exactly what the trie-building and DP-filling examples of Chapter 26 depend on. The frame-aware array element get/set, the `for-in` over a by-reference parameter, and the `delete` through it are all routed so they act through the binding, which the report lists among the divergences it deliberately fixed.

The for-in protocol

Iterating an associative array is a three-opcode dance because awk's `for (k in arr)` must walk a snapshot of the keys safely even as the body reads the array. `ForInStart(arr)` collects the array's keys (in `PROCINFO["sorted_in"]` order if that control is set, else hash order) onto an iterator stack. `ForInNext { var, end_jump }` pops the next key into the loop variable and, when the keys are exhausted, jumps to the loop's exit. `ForInEnd` tears the iterator down. Splitting iteration into start/next/end opcodes rather than a single magic loop is what lets `break`, `continue`, and `next` interact correctly with a `for-in` — `continue` jumps back to `ForInNext`, `break` jumps past `ForInEnd` — and it is the same shape the sorted-iteration control of Chapter 8 plugs into, because the ordering is decided once, at `ForInStart`, when the key snapshot is taken.

The dispatch loop and the fast paths above it

The VM proper — `vm.rs`, near three thousand lines — is the loop that reads the current opcode, executes it, advances the program counter, and repeats, with a `VmCtx` holding the execution context for a running chunk and the central mutable state living in `runtime.rs`. The fused superinstructions of Chapter 16 are ordinary opcodes to this loop; they simply do more per dispatch. Above the loop sit the inline fast paths Chapter 16 described, which for a single-rule program whose body is one fused opcode skip the `VmCtx` setup entirely and run a hand-written loop over the memory-mapped input. The catalogue in this chapter is the full set of moves the general machine can make; the fast paths are the handful of common shapes the engine recognizes and runs without the general machine at all. Both are the same compiled program — the difference is only how much interpreter stands between the bytecode and the bytes.

A worked compile: from source to opcodes

To make the opcode catalogue concrete, trace a small program through compilation. `{ print $1, $3 }` compiles, after slot allocation and peephole fusion, to essentially a single fused instruction: the compiler recognizes the shape “print two fields with `OF5` between” and emits a multi-field print fusion (the `PrintFieldSepField` family), rather than the unfused sequence of push-field-1, get-field, push-field-3, get-field, print-with-two-args. A counted loop like `for (i = 1; i <= n; i++) s += a[i]` compiles to: an `IncrSlot` or assignment to initialize `i`, a `JumpIfSlotGeNum { slot: i, limit: n, target: end }` that fuses the loop-condition read, comparison, and conditional branch into one instruction, the body (a `GetArrayElem` and a compound-add), an `IncrSlot(i)` to advance, and a `Jump` back to the test. The user scalars `i` and `s` get `u16` slots and are touched through `GetSlot/SetSlot` and the increment fusions with no hashing, while `n` and the array `a` — if `n` is a plain scalar it too gets a slot, and `a` stays on the `HashMap` path because it is an array. A function call `f(x, arr)` where `arr` is an array compiles to `CallUserBindArrays`, which carries the argument names so the array parameter is written back to the caller after the call. Every construct in the catalogue corresponds to a shape the compiler emits, and the fusions correspond to the shapes the peephole pass recognizes — so reading a program's compiled form is reading which idioms it is made of, with the common ones collapsed to single fused instructions exactly where the profiler said the dispatch overhead mattered. The `-s/--no-optimize` flag turns the fusions off, emitting the unfused sequences, which is how you would inspect the difference or isolate a suspected fusion bug.

Chapter 23 – The Record I/O Engine

Before the VM can run a record rule, something has to decide where one record ends and the next begins, and hand the engine a record to run on. That something is the record-I/O layer in `record_io.rs`, and it is more subtle than “split on newlines” because awk’s `RS` is not a newline — it is a newline *by default*, and a single character, an empty string, or a regular expression when the program asks for one. This chapter is how awkrs turns a byte stream into records faithfully across all of those modes, how it keeps `RT` correct, the CRLF rule that an earlier version got wrong, and how the same splitter feeds both the sequential loop and the parallel path of Chapter 15.

One entry point, four modes

The reading is funneled through a single function, `read_next_record`, which takes the current `RS`, the shared input reader, and a small set of scratch buffers, and returns one record. It dispatches on the shape of `RS` into four modes, which are exactly the four `RS` behaviors Chapter 5 described from the language side:

- When `RS` is the default newline, it reads until the next `\n`.
- When `RS` is the empty string, it reads a *paragraph* — a maximal run of text bounded by blank lines.
- When `RS` is longer than one character, it is a *regex* (the gawk extension), and the reader scans for the next match of that pattern.
- Otherwise `RS` is a single literal character (or multi-byte literal), and the reader scans for that exact byte substring.

Each mode fills the record into an output buffer and, crucially, records the *actual terminator* it found into a separate buffer that becomes `RT` — so a program that reads `RT` after each record sees the precise bytes that ended it, which for a regex or multi-character `RS` is the one piece of information a fixed separator cannot give you back.

The leftover buffer and streaming

The detail that makes the non-newline modes correct is the `leftover` buffer. When the reader scans ahead for a separator — a regex match, a multi-byte literal, a blank-line boundary — it may read past the end of the current record into the start of the next one, and those extra bytes must not be lost. awkrs threads a `leftover Vec` through consecutive calls on a single stream: bytes read past a record’s terminator are stashed there and consumed first on the next call. This is what lets awkrs stream a regex or multi-character `RS` over a pipe or a file without buffering the whole input — it reads in chunks, splits what it has, and carries the unsplit tail forward. The newline fast path uses `memchr` to find the `\n`, and the literal path uses a byte-substring search, so the common cases stay cheap while the general cases stay correct.

The CRLF rule, and a fixed bug

When a record is delimited by a newline, awk strips the trailing newline from the record’s content — `$0` does not include the `\n`. The question an earlier awkrs answered wrongly is what to do with a *carriage return* before that newline, on CRLF-terminated input. The POSIX text-mode behavior on Unix, which gawk follows, is to strip *only* the `\n` and to keep the `\r` as part of the record — so a Windows-terminated line read on Unix carries its `\r` into `$0` and into `length`, visible and removable rather than silently gone. The `trim_end_record_bytes`

function implements exactly this: it removes trailing `\n` bytes and leaves `\r` alone. The source comment is explicit that older `awkrs` versions also stripped `\r`, which silently dropped carriage returns on Unix, and that the fix restores the `gawk` behavior — if you want the `\r` gone you ask for it, with `BINMODE` or an explicit `gsub(/\r$/, "", ...)`, rather than having the reader decide for you. It is a small rule, and it is the kind of small rule that turns a subtle data-corruption bug into a visible, controllable byte.

Paragraph mode and regex `RS`, precisely

Paragraph mode — `RS = ""` — is the mode with the most defined corner behavior, and Chapter 5 already noted that `awkrs` sets `RT` in it to capture the full run of trailing newlines and the blank lines that separated records, with the final record capturing any EOF-trailing newlines. The reader implements this by treating a run of one-or-more blank lines as the separator, emitting the paragraph as the record and the blank run as `RT`, and — the POSIX rule paragraph mode adds — making newline a field separator within the paragraph in addition to whatever `FS` is, so a multi-line paragraph splits into fields across its lines. The regex `RS` mode runs a `regex::bytes` matcher over the streamed bytes, so the separator can be any pattern, and trailing-newline trimming at end of input matches `gawk`. These are the modes that turn `awk` from a line processor into a record processor over arbitrarily-structured input, and `awkrs` implements each against the `gawk` behavior its parity tests pin.

The shared reader, and why primary `getline` is sequential

The input is held behind a `SharedInputReader` — a reader protected by a lock so that the main record loop and an unredirected `getline` share *one* position in the stream. This shared position is what makes interleaving the implicit loop with a primary `getline` produce POSIX-correct line order, as Chapter 7 noted, because both draw from the same cursor. It is also the reason, from Chapter 15, that a program using primary `getline` cannot be parallelized: a single shared input position cannot be split across workers without changing which worker reads which record. The parallel path therefore takes a different route to records entirely — for a regular file it memory-maps the bytes and splits them with the *same* `RS`-aware logic this chapter describes, producing owned record strings it can hand to independent workers, and for `stdin` it buffers the `--read-ahead` line chunks the parallel path consumes. The splitting rules are one implementation; the sequential loop reads through the shared reader, and the parallel path applies the same rules to a mapped or buffered span. Records are records, however they are produced, and `record_io.rs` is the single place that decides what a record is.

`RT` and the value of knowing the terminator

One small feature of the record engine deserves a closing note because it is easy to overlook and occasionally indispensable: `RT`, the variable holding the actual text that matched `RS` for the current record. With the default newline `RS`, `RT` is just `"\n"` and rarely interesting. But the moment `RS` is a regex or a multi-character separator, `RT` becomes the only way to recover what *actually* ended each record, which a fixed separator cannot tell you. If `RS = "[; ,]"` splits on either a semicolon or a comma, `RT` tells you which one terminated each record — information that is otherwise lost the instant the record is split. In paragraph mode (`RS = ""`), `RT` captures the full run of blank lines and trailing newlines that separated the records, so a program can reconstruct the exact spacing of the original. This is `gawk`'s design, and `awkrs` sets `RT` to match in every mode, with the precise paragraph-mode behavior pinned by a parity test. The practical use is reconstruction: a program that splits a record stream on a variable separator and needs to *rejoin* it — preserving the original terminators rather than normalizing them — reads `RT` to know what to put back. It is a small variable, set as a side effect of every record read, and it is the difference between a splitter that loses information and one that does not. The one limit, from Chapter 15: under parallel mode `RT` is set only approximately, since per-record terminators are not tracked individually across workers — another reason order-sensitive work stays sequential.

The leftover buffer, and why it is the module's central invariant

Everything in `record_io.rs` is organized around one shared piece of state: a `leftover` byte vector holding input that was read past the previous record's terminator. Its contract is stated in the reader's own documentation — the same `Vec` must be passed across consecutive calls on a single stream — and violating it loses input.

The reason it exists is that not every reader can stop reading exactly at a record boundary. The default newline path can: `BufRead::read_until` consumes precisely up to and including the delimiter, so nothing is over-read and `leftover` stays empty. But the regex path and the multi-byte literal path cannot, because they have to see bytes before they can know whether a boundary is in them. A regex `RS` may match a variable number of bytes; a two-byte delimiter may straddle a read boundary. Those readers pull fixed 4 KiB chunks, scan the accumulated buffer, and when they find a match, emit the prefix as the record, record the matched bytes as `RT`, and drain through the match — leaving whatever followed in `leftover` for the next call.

The consequence is that every reader has to check `leftover` first, even the ones that never write to it, because `RS` can change mid-stream. Assign `RS = "\n"` after reading records with a regex `RS` and the newline reader inherits a non-empty `leftover`; it therefore begins by looking for a newline in the leftover bytes, serving the record from there and draining, and only falls through to `read_until` when the leftover holds no terminator. The same structure appears in the single-byte literal path and inside paragraph mode's line loop. This is what makes mid-stream `RS` reassignment work rather than silently dropping the buffered tail, and it is the reason all four readers share one signature.

Four readers, one dispatch

`read_next_record` clears the output and terminator buffers, takes the input reader's lock, and dispatches on `RS`:

`RS == "\n"` — the POSIX default — goes to `read_until_lf`, which is the fast path and the only one that leans entirely on `BufRead::read_until`. It sets `RT` to a newline when the record was actually newline-terminated, which is how a final record with no trailing newline reports an empty `RT`.

`RS == ""` is paragraph mode, covered below.

A regex `RS` — which `awk` uses when `RS` is longer than one character, matching `gawk` — goes to the chunked scanner, which sets `RT` to the *actual matched bytes*. This is the case where `RT` earns its existence: with a variable-length separator, the only way to know what separated two records is to be told, and `RT` is the telling.

Anything else is a literal delimiter. The single-byte case gets a `memchr` scan of the leftover and then `read_until`; the multi-byte case gets the chunked scan with a sliding window compare.

Paragraph mode's RT contract

Paragraph mode is the most intricate reader in the module, and almost all of the intricacy is in getting `RT` right rather than in finding the record.

Finding the record is straightforward: read lines, skip leading blank ones, accumulate content lines, and stop at the first blank line after content. A line counts as blank when every byte is a space, tab, carriage return, or newline — so a line of trailing whitespace separates paragraphs, which is what “blank line” means to a human editing the file.

The `RT` contract is where the work is, and it follows `gawk`: `RT` in paragraph mode is the *full* run of trailing newlines from the last content line *plus* the blank lines separating the records. For input where a paragraph's last content line is `b` followed by a blank line, `RT` is two newlines, not one. Producing that requires three steps in order. First, the newlines that were accumulated onto the end of the record are trimmed off the record

and appended to `RT` — the record content itself must not carry them. Second, the blank line that ended the paragraph is appended. Third, the reader *peeks forward*, consuming any further consecutive blank lines into `RT`, and on reaching a non-blank line pushes it back to the front of `leftover` for the next record.

That push-back is the subtle part, and it is why paragraph mode needs `leftover` even though it reads line-at-a-time. The reader cannot know a run of blank lines has ended without reading the line that ends it, and that line belongs to the *next* record. It is prepended — the peeked line first, then the existing leftover — preserving stream order.

The EOF paths do the same trimming. Reaching end of input with content accumulated strips the trailing newlines into `RT` and returns the record; reaching it with nothing accumulated returns false. Only the record's content ends up in `$0`, and everything between records ends up in `RT`, which is the invariant a program reading stanzas depends on.

Carriage returns, and a parity bug worth naming

`trim_end_record_bytes` strips only `\n` from the end of a record — never `\r`. The function's own documentation records why: this is gawk parity for POSIX text mode on Unix, and stripping `\r` means a CRLF input silently loses its CR bytes from `$0` and from `length`.

The documentation also records that older awkrs versions *did* strip `\r`. That is a useful thing to have written down in the source, because it is exactly the kind of change that looks like a fix from one direction and a regression from the other. Stripping CR makes CRLF files “just work” for the common case of a Windows-authored CSV; not stripping it makes `length($0)` agree with gawk on the same file and keeps a program that explicitly handles `\r` from double-handling it. awkrs takes the parity side and points at the two explicit tools — `BINMODE`, or a `gsub(/\r$/, "")` in the program — for the programs that want the bytes gone. The general rule the choice illustrates: when convenience and reference-parity conflict on input handling, awkrs takes parity and documents the escape hatch, because a program written against gawk that silently gets different field contents under awkrs is a worse outcome than one that needs an explicit `gsub`.

The mmap splitters

The streaming readers above serve the sequential path. The parallel path and the slurp path need something different: they have the whole input as one byte slice and need it cut into records all at once, with no copying. That is `split_input_into_records`, and it returns *borrowed* slices — views into the mapped bytes — not owned strings, which is what makes the parallel path's per-record cost start at zero.

There are four splitters mirroring the four readers. The newline splitter walks the data producing line slices. The paragraph splitter groups on blank-line boundaries. The regex splitter uses the byte regex's match positions as cut points. And the literal-delimiter splitter uses a substring finder to locate each occurrence.

The relationship between the two families is the one worth holding onto: the same four `RS` interpretations, implemented twice against two different input models — a streaming reader that must not over-read, and a complete byte slice that can be cut anywhere. The sequential path takes the first, the memory-mapped parallel path (Chapter 15) takes the second, and the record boundaries they produce are identical, which is the precondition for the parallel path being transparent.

Chapter 24 — Robustness: Limits, Errors, and the Sandbox

A tool that runs over other people’s data on other people’s machines has to fail well, because it *will* be fed malformed input, pathological programs, and hostile files, and the difference between a tool you trust in a pipeline and one you do not is mostly what happens when something goes wrong. `awkrs` treats this as a first-class concern, and the evidence is spread across several small modules and a consistent discipline that runs through the large ones: a hard recursion cap, a typed error taxonomy, saturating arithmetic on every attacker-influenced size, a sandbox flag, and the loud-failure rules — fatal division by zero, fatal scalar/array confusion — that earlier chapters met one at a time. This chapter gathers the robustness story into one place.

The recursion cap

`awk` functions recurse, and Chapter 9 noted that `awkrs` caps user-defined recursion so that a runaway self-call fails with a clear error instead of overflowing the host process’s stack and crashing. The cap lives in `limits.rs` as `MAX_USER_CALL_DEPTH`, and its value is chosen against a real constraint the source comment spells out: each level of `awk` recursion consumes native stack in the VM, and the default eight-megabyte Rust thread stack starts to overflow somewhere around two hundred frames in unoptimized builds, so the production cap is set to **150** — comfortably below the danger line — and the test builds use **32** so the cap’s behavior can be exercised without deep recursion. The point of the cap is not to limit legitimate algorithms; a depth of 150 is far past anything a reasonable `awk` program reaches, and the Chapter 26 examples that recurse deeply — the recursive-descent calculator, the tree traversals, the Tarjan SCC and Sudoku solvers — all run well within it. The point is that the *failure mode* of pathological recursion is a controlled `awk` error with a message, not a segmentation fault, because a segfault on bad input is a strictly worse outcome than a diagnostic, especially for a tool invoked unattended in a pipeline.

The error taxonomy

`awkrs`’s errors are a small typed enum in `error.rs`, and the types map to the distinct ways a run can fail. `Parse { line, msg }` is a syntax error with its source line, the front-end fault of Chapter 21. `Runtime(String)` is an execution-time fault — the fatal division by zero, the scalar-used-as-array, the bad builtin arity — carrying a gawk-style message. `Io` wraps an underlying operating-system I/O error. `ProgramFile` and `InputFile` distinguish the two file-access failures a run can hit, and `InputFile`’s message is phrased like gawk’s “cannot open file ... for reading” precisely so that scripts and harnesses keying on the wording see consistent text across implementations. And `Exit(i32)` is not really an error at all — it is how an `exit` propagates its status code up through functions and expressions to the top level, where it interacts with the `END` rules as Chapter 3 described. The companion `flow.rs` defines the `Flow` enum the record loop reads — `Normal`, `Next`, `NextFile`, `ExitPending` — which is how the VM’s flow-signal opcodes tell the loop what to do next. Keeping these as typed values rather than ad-hoc strings or panics is what lets the top level distinguish “the program asked to exit 3” from “the program has a bug” from “the file was unreadable,” and report each correctly.

Fuzz-hardening: saturating arithmetic and bounded output

A format string is attacker-influenced data — it can come from input, from a variable, from anywhere — and a naive `printf` implementation is a classic place for an integer overflow or an unbounded allocation to hide. `awkrs` hardens the formatting path against exactly this. As Chapter 13 noted, the field width is capped at one hundred thousand to bound how much a single conversion can emit, and the digit parsing throughout the format machinery uses *saturating* arithmetic, so a width or precision specified as a gigantic number saturates at the cap rather than wrapping around or overflowing. This means a malformed or malicious format string — `%2147483648d` or a width built from untrusted input — produces bounded, defined output instead of a panic or a runaway allocation. It is a small, unglamorous category of fix, and it is exactly the category that separates a parser-and-formatter that has been fuzzed from one that has not.

The arity discipline, gathered

Chapter 11 introduced `awkrs`'s uniform builtin-arity checking as a property of the builtin library; seen from the robustness angle it is one more instance of the same principle. Calling a builtin with the wrong number of arguments produces a uniform runtime error — the `"N is invalid as number of arguments for X"` message — across the whole library, where earlier builds variously panicked, silently ignored extra arguments, or used inconsistent wording. The static checker rejects the obviously wrong arities at compile load time so a misused builtin fails before the program runs rather than midway through a long input. The throughline from the recursion cap to the saturating widths to the arity checks is one stance: an `awk` program's bug, or its bad input, should produce a clean diagnostic at a defined point, never a crash and never a silent wrong answer. That stance is what “robust” means for a tool in this role.

The sandbox

The `-S/--sandbox` flag is `awkrs`'s blunt instrument for running an *untrusted* program safely. With it set, the operations that reach outside the process — running a command through `system()`, redirecting `print` to a file, opening a pipe or a coprocess, connecting a `/inet` socket — are blocked rather than performed. The filesystem-and-OS builtins of Chapter 12 route through the sandbox check first, so `stat`, `readdir`, `chdir`, `rename`, `readfile`, and the rest are gated too. The use case is concrete: when you run an `awk` program you did not write over data you do care about — a program from a colleague, from a code-generator, from the internet — the sandbox lets you get its computation without granting it the ability to touch the world around it. It is the natural companion to the `-E/--exec` form, and it is the kind of capability a single-binary text tool can offer cheaply precisely because every outside-touching operation in `awkrs` already routes through one place that the flag can close.

Loud failures as a feature

The chapters before this one met `awkrs`'s loud-failure rules individually: division by zero is a fatal error rather than an infinity (Chapter 6), preserved even into JIT-compiled native code (Chapter 17); using a scalar as an array or an array as a scalar is fatal rather than a silent empty string (Chapter 6); `log` and `sqr` of a negative argument warn rather than return garbage (Chapter 11); the bitwise shifts and complement of a negative argument are fatal (Chapter 11). Seen together, these are not separate quirks but one coherent choice about what an `awk` should do when a program meets a value it cannot sensibly handle. The alternative — produce `inf`, auto-promote the scalar, return an empty string, keep going — tends to corrupt a whole run quietly, so that the bug surfaces far downstream as a wrong number with no indication of where it came from. `awkrs` makes the failure happen at the point of the mistake, with a message that names it. For a data-processing

tool, a run that stops with “division by zero attempted” on the line that divided by zero is strictly more useful than a run that completes and hands you a file full of `inf`, and `awkrs` chooses the former every time.

A field guide to the fatal messages

Because `awkrs` fails loudly, it is worth knowing the fatal messages by sight, since each names a specific class of mistake. `division by zero attempted` (and the `%` variant for modulo) means a zero divisor was reached — almost always a surprise in the data or a missing guard, and the message appears even from inside JIT-compiled code. `attempt to use scalar 'x' as an array` means a name holding a scalar was subscripted, iterated with `for-in`, tested with `in`, or `deleted` as an array — a confusion between the two kinds of value, fatal across all those operations to match `gawk`. The converse, using an array name where a scalar is expected (`print a, a "", a ~ /x/`), is likewise fatal. `negative values are not allowed` comes from `compl`, `lshift`, or `rshift` on a negative argument, the value-dependent bitwise trap. `N is invalid as number of arguments for X` is the uniform arity error, raised when a builtin is called with the wrong number of arguments. And the recursion cap surfaces as a clear error rather than a crash when user recursion exceeds 150 frames. Two non-fatal warnings round out the set: `awkrs: warning:` precedes the NaN-returning `log/sqrt` of a negative argument, gated on lint being active. Each message corresponds to exactly one kind of program error, so reading the message tells you not just *that* something went wrong but *which* mistake to look for — which is the entire value of failing loudly with a specific message rather than silently producing a wrong number.

Exit codes and the shell contract

A message on `stderr` is only half of failing loudly; the other half is the exit status, because a program embedded in a shell pipeline communicates success or failure to everything downstream not through its output but through the number it returns to the shell. `awkrs` honors the `awk` convention here, which is worth stating precisely, because the whole point of a loud failure is lost if the process still exits zero. A program that runs to completion without an explicit `exit` returns status 0 — success — the moment `END` finishes. An explicit `exit N` sets the status to `N`, and this is the mechanism a program uses to report a *domain* result to the shell: a linter that finds a problem exits non-zero, a filter that matched nothing exits non-zero, a validator that rejects its input exits with a code the caller can branch on. The status set by `exit` in a record rule or in `BEGIN` is remembered and used when `END` completes, unless an `END` rule runs its own `exit` with a different value, which then wins — the last `exit` executed is the one the shell sees. A fatal error — any of the messages the previous section catalogued, a syntax error at load time, a division by zero, an arity violation — terminates the process with a non-zero status distinct from the clean-run 0, so the difference between “ran and chose to report failure” and “could not run” is visible to the caller, and neither is confusable with success.

This matters most under `set -e` and in pipelines, where a silent non-zero-should-have-been is the class of bug that corrupts a data workflow without anyone noticing. A shell script that does `awkrs 'PROG' data | consumer` and runs under `set -e -o pipefail` will abort the whole script if `awkrs` dies on a malformed record, rather than feeding the consumer a truncated stream and pressing on — which is the behavior you want, and which depends entirely on `awkrs` having exited non-zero. The design of Chapter 24, then, reaches all the way out to the shell: a saturating-arithmetic guard that kept a fuzzer from crashing the process is an internal robustness measure, but the fatal error that *does* fire on a genuine mistake is an external contract, and the exit status is how that contract is signed. The practical corollary for anyone scripting around `awkrs` is the standard one for `awk`: check the status, use `exit N` deliberately to report results the caller cares about, and do not assume that absence of output means success — absence of output with status 0 means the program ran and had nothing to say, while absence of output with a non-zero status means it could not finish, and the two call for opposite responses.

Chapter 25 – The Source Tree, the Build, and CI

The last chapter of the machine section steps back from any single subsystem to the shape of the whole — the modules and how they divide the work, the build that turns them into the binaries you run, and the continuous-integration discipline that keeps the whole thing honest against three reference awks on every push. This is the map you would want if you were going to read the source yourself, which the book has encouraged throughout by citing files by name, and it is also the clearest single view of how much of the project is the language and how much is the engineering around it.

Fifty thousand lines across forty modules

`awkrs` is roughly fifty thousand lines of Rust across about forty modules, and the size distribution tells you where the difficulty lives. The two largest modules are the front end of Chapter 21 — `parser.rs` at around fifty-one hundred lines and `lexer.rs` at around thirty-six hundred — because implementing the entire awk surface faithfully is genuinely the largest job. Next come the runtime and the machine: `runtime.rs` near five thousand lines holds the central mutable state the VM operates on; `compiler.rs` near thirty-nine hundred lowers the AST to bytecode and runs the peephole fusion; `vm.rs` near three thousand is the dispatch loop. The fusevm bridge of Chapter 17 is three substantial files — `fusevm_bridge.rs`, `fusevm_compile.rs`, and `fusevm_host.rs`, together over four thousand lines — which is a reminder that the JIT path, though experimental and narrow in what it accepts, is not a small piece of code. The formatting machinery of Chapter 13 is its own twenty-three-hundred-line `format.rs`. And then a long tail of focused modules: the record-I/O of Chapter 23 in `record_io.rs`, the namespace system in `namespace.rs`, the bignum integration in `bignum.rs`, the locale handling in `locale_numeric.rs`, the cache in `script_cache.rs`, the tooling in `lsp.rs` and `dap.rs` and `debugger.rs`, and the small but load-bearing `limits.rs`, `error.rs`, and `flow.rs` of Chapter 24. The shape is the shape of a real language implementation: a large front end, a large runtime, a large compiler, and a constellation of subsystems each owning one concern.

The two binaries and the staticlib

The crate builds more than one artifact, and the `Cargo.toml` is explicit about why. There are two binary targets that are the same engine under two names: `awkrs`, the full spelling, and `aw`, the short one — because, as Chapter 2 noted, every keystroke in a one-liner is a keystroke you type a thousand times, and an awk that makes you type four characters before any program is an awk with a small daily tax. A third binary, `gen-docs`, is the offline documentation generator that renders the LSP doc corpus into the static `reference.html`, which is the mechanism behind the single-source-of-truth claim of Chapter 20 — the editor hover and the published reference are the same corpus because one binary turns it into the page. The library target is built as both a normal `lib` and a `staticlib`, which produces `libawkrs.a`, and that static library exists for exactly one reason: the `--aot/fusevm:aot` path of Chapter 17 links its ahead-of-time-compiled objects against the awk runtime into a standalone executable, which needs the runtime as a linkable archive. The packaging is disciplined too — the `Cargo.toml` excludes the `parity/`, `tests/`, and `docs/` directories from the crates.io tarball, so the published crate is the engine, not the multi-megabyte test corpus, while the git repository keeps all of it for local and CI testing.

The parity suite as the source of truth

The claim that runs through this entire book — that `awkrs` is faithful to POSIX `awk` and `gawk`, down to the corner cases — is not an assertion, it is a test suite. `awkrs` ships a parity harness in `parity/` that runs `awk` programs through `awkrs` and through the reference `awks` — `gawk`, `mawk`, and BSD `awk` — and diffs the output byte-for-byte, across more than two thousand cases organized into general, `gawk`-specific, and portable suites. This is the machine that turns every “matching `gawk`” sentence in this book into a checkable fact: when the book says `awkrs` clamps `substr`’s start index the `gawk` way, or that `%1g` of `9.5` is `1e+01`, or that paragraph-mode `RT` captures the trailing newlines, there is a parity case that runs both engines and confirms the bytes agree. The suite runs in CI on every push, alongside the seventy-odd example programs of Chapter 26 — themselves verified against `gawk` — so a change that broke a corner case would fail the build rather than ship. The compatibility matrix of Chapter 14 is careful to call itself a feature matrix and not a proof of correctness, and the parity suite is why the matrix can be honest about the distinction: the cases that are checked are checked hard, and the places marked `Part` are the places the author has chosen not to claim more than the tests support.

What the build discipline buys

The throughline of this chapter is that `awkrs` is engineered as much as it is programmed. The single-binary deployment of Chapter 1, the two-name convenience of Chapter 2, the AOT static-library trick of Chapter 17, the shared documentation corpus of Chapter 20, and the parity-on-every-push discipline here are all the same kind of decision: spend effort in the build and the test harness so that the running tool is small, fast, faithful, and trustworthy. It is the unglamorous half of why a forty-year-old programming model on a new engine is a tool you would actually reach for — not because the engine is clever, though it is, but because the engineering around it means the clever engine ships as one honest binary that does what the reference `awks` do and tells you when it cannot. With the machine fully open, the book turns in its last chapter to the thing the machine exists to run: real `awk` programs, read line by line.

The documentation is tested too

A last detail closes the build story, and it is characteristic of the whole project: even the *documentation* is held to mechanical checks. The `tests/` directory carries shell-based test scripts that lint the very files this book sits among — `docs-final-newline.sh`, `docs-has-h1.sh`, `docs-has-body-tag.sh`, `docs-no-deprecated-tags.sh`, `docs-no-http-links.sh`, `docs-no-inline-handlers.sh`, `docs-no-placeholder-href.sh`, `docs-target-blank-rel-noopener.sh`, and their README and man-page siblings — so the published HTML docs cannot regress into a missing heading, a dangling placeholder link, an insecure `target="_blank"`, or a plain-HTTP URL without failing CI. The `reference.html` page is itself generated from the LSP doc corpus by the `gen-docs` binary (Chapter 25), so it cannot drift from the editor’s hover text, and these doc-lint scripts then check the generated output’s shape. The pattern is the same single-source-of-truth-plus-verification discipline that governs the engine: write each fact once, generate the published form from it, and let CI verify the result mechanically. It means the reference page a reader consults, the tooltip an editor shows, and the name tables the runtime uses are provably the same corpus, push after push — and it is the reason this book could ground every “matching `gawk`” claim in a specific source file with confidence that the source and its documentation had not silently diverged. The discipline that makes the engine trustworthy is the same one that makes its documentation trustworthy, which is a fitting note on which to leave the machine and turn to the programs.

Chapter 26 – A Tour of Real Programs

The best way to understand what a language is capable of is to read real programs written in it, and `awkrs` ships more than seventy of them in its `examples` directory — substantial, standalone `awk` programs that implement classic algorithms and data structures, each paired with an input file and each verified byte-for-byte against `gawk` by the parity job in CI. They are not toys written to flatter the implementation; they are real implementations of real algorithms — recursive-descent parsers, edit-distance DP, tries, JSON pretty-printers — that happen to be written in `awk`, and reading them is the fastest way to see how far `awk`'s small model reaches. This chapter walks through sixty-six of them line by line. Every program here is copied verbatim from the repository; nothing is invented.

How to read the tour

The sixty-six programs ahead are grouped loosely by what they teach, and a few orienting notes will make them read faster. Each walkthrough quotes the load-bearing part of the program — usually one or two functions or the key rule — rather than the whole file, because the point is the *technique*, and the surrounding boilerplate (reading input, dispatching commands) is the same across many of them. The programs fall into rough families: the field-and-aggregate jobs `awk` was born for (`csv_pivot`, `sql_like`, `email_extract`, `markdown_basic`); the data structures (`trie`, `bst`, `avl_tree`, `lru_cache`, the heaps, `kruskal`'s union-find); the dynamic-programming solvers (`levenshtein`, `knapsack`, `coin_change`, `diff_lcs`, `tsp_dp`); the graph algorithms (`dijkstra`, `bellman_ford`, `floyd_warshall`, `topo_sort`, `scc_tarjan`, the BFS pair); the string algorithms (`kmp`, `boyer_moore`, `z_function`, `manacher`, `aho_corasick`, `rabin_karp`, `suffix_array`); the parsers and interpreters (`calc_rd`, `shunting_yard`, `json_pretty`, `brainfuck`, `regex_engine`); the number theory (`gcd_extended`, `modexp`, `pollard_rho`, `bigint_mul`); the codecs and ciphers (`base64`, `lzw`, `huffman`, `rle`, `vigenere`); and the simulations and games (`conway`, `rule30`, `mandelbrot`, `n_queens`, `sudoku`, `tictactoe`). The same idioms — catalogued in Chapter 40 — recur across all of them, so the programs are best read not as isolated feats but as arrangements of a shared vocabulary. Every one is verified byte-for-byte against `gawk` in CI, which is why so many take care to be deterministic; that determinism is a feature of the examples, not an accident. Read a few from a family you know and the rest of that family reads itself.

`prime_sieve.awk`: the Sieve of Eratosthenes

We begin with one of the simplest, because it shows the whole shape of an `awk` program that does real work in a single record rule. The task is the Sieve of Eratosthenes: given a number `N`, print every prime up to `N`, ten per row, then a count.

```

NR == 1 {
    N = $1 + 0
    if (N < 2) { print "COUNT: 0"; exit 0 }
    for (i = 2; i <= N; i++) is[i] = 1
    for (i = 2; i * i <= N; i++) {
        if (!(i in is)) continue
        if (is[i] == 0) continue
        for (j = i * i; j <= N; j += i) is[j] = 0
    }
    col = 0
    for (i = 2; i <= N; i++) {
        if (is[i] == 1) {
            printf "%6d", i
            col++
            if (col % 10 == 0) print ""
            cnt++
        }
    }
    if (col % 10 != 0) print ""
    printf "COUNT: %d\n", cnt
    exit 0
}

```

The pattern is `NR == 1` — this rule runs only on the first input record, which is the line containing `N`. The first line of the action, `N = $1 + 0`, is the awk idiom for “force this field to a number”: adding zero coerces the numeric-string field `$1` to a number, so `N` is numeric for the comparisons that follow. The guard `if (N < 2) { print "COUNT: 0"; exit 0 }` handles the degenerate case and exits — and note that even this `exit` would run any `END` rules first, though this program has none.

The sieve itself is three loops over a single associative array `is`. The first loop marks every number from 2 to `N` as prime (`is[i] = 1`). The second loop is the sieve proper: for each `i` from 2 up to the square root of `N` (the condition `i * i <= N` avoids a separate square-root call), if `i` is still marked prime, it crosses off every multiple of `i` starting from `i * i` by setting `is[j] = 0`. The two `continue` guards — `if (!(i in is)) continue` and `if (is[i] == 0) continue` — skip numbers already crossed off, the `in` test demonstrating membership-without-creation from Chapter 8. The third loop prints: it walks 2 to `N` in index order, prints each still-prime number right-justified in a six-column field with `printf "%6d"`, and after every tenth prime emits a newline with `print ""`, tracking the column count in `col` and the total in `cnt`. The trailing `if (col % 10 != 0) print ""` closes a partial final row, and `printf "COUNT: %d\n", cnt` reports the count.

What is worth noticing is how *ordinary* this is — it is the same program you would write in C, with the same three loops and the same array, except that the array is associative and springs into existence as needed, the field coercion is a `+ 0`, and the whole thing is a single pattern-action rule with no `main` and no input loop. The comment in the source even notes the design choice that makes it work: “iteration order doesn’t matter — we walk 2..N in index order,” because the final loop indexes `is[i]` explicitly rather than relying on `for-in` order. That is the kind of awareness of awk’s unspecified iteration order that Chapter 8 stressed, applied correctly.

trie.awk: a tree in associative arrays

The trie example is the clearest demonstration of Chapter 8’s central claim — that the associative array, with the `SUBSEP` multidimensional convention, is enough to build any data structure. A trie is a tree where each edge is labeled with a character and each path from the root spells a prefix; it is the natural structure for prefix queries. Here it is built entirely from flat arrays.

```

function tr_add(s, i, p, c, k) {
    p = 0
    subcnt[p]++
    for (i = 1; i <= length(s); i++) {
        c = substr(s, i, 1)
        k = p SUBSEP c
        if (!(k in edge)) { edge[k] = ++tn; subcnt[tn] = 0 }
        p = edge[k]
        subcnt[p]++
    }
    term[p] = 1
}

```

The trie's entire state is three global arrays, described in the program's header comment: `edge[node, char]` maps a node and a character to the child node's id; `term[node]` marks a node as the end of an inserted word; and `subcnt[node]` counts how many inserted words pass through a node's subtree. Node ids are integers, the root is 0, and `tn` is the running maximum node id.

`tr_add` inserts a word `s`. Read its signature first: `s` is the argument, and `i, p, c, k` after the gap are the function's locals, exactly the Chapter 9 idiom. The function walks the characters of `s`, maintaining the current node `p` (starting at the root, 0). For each character `c`, it forms the composite key `k = p SUBSEP c` — the `SUBSEP` concatenation that makes `edge` a two-dimensional array keyed by `(node, char)`. If that edge does not exist (`!(k in edge)`), it creates a new node by incrementing `tn` and recording the edge; either way it advances `p` to the child and bumps that node's subtree count. After the loop, `term[p] = 1` marks the final node as a word ending. The crucial detail is that `edge`, `term`, and `subcnt` are *global* arrays mutated by the function — which works precisely because Chapter 9's array call-by-reference (here, global access from within a function) lets `tr_add` build up shared state across calls.

```

function tr_find(s, i, p, c, k) {
    p = 0
    for (i = 1; i <= length(s); i++) {
        c = substr(s, i, 1); k = p SUBSEP c
        if (!(k in edge)) return 0
        p = edge[k]
    }
    return (p in term) ? 1 : 0
}

```

`tr_find` walks the same edges; if it ever falls off the trie (`!(k in edge)`) the word is absent and it returns 0, and if it reaches the end it returns whether the final node is marked terminal. `tr_pfxcount` (not shown but identical in shape) walks to the end of a prefix and returns `subcnt[p]`, the count of words in that subtree — an $O(\text{prefix-length})$ prefix query, which is exactly what a trie is for.

The dispatch is three record rules, one per command verb:

```

$1 == "ADD" { tr_add($2); next }
$1 == "FIND" { printf "%s -> %s\n", $2, (tr_find($2) ? "HIT" : "miss"); next }
$1 == "PFX" { printf "%s -> COUNT %d\n", $2, tr_pfxcount($2); next }

```

Each rule matches on the first field — the command — runs the corresponding function on the second field, and `nexts` to the following record. This is `awk`'s pattern-action model used as a tiny command interpreter: the input is a stream of commands, the patterns dispatch on the verb, and the actions call into a small library of functions over shared arrays. It is a remarkably clean architecture for fifty lines of `awk`, and it is built from nothing but associative arrays, `SUBSEP`, and user functions.

levenshtein.awk: dynamic programming in a SUBSEP table

Edit distance — the minimum number of single-character insertions, deletions, and substitutions to turn one string into another — is the textbook dynamic-programming problem, and its awk implementation shows the **SUBSEP** two-dimensional array used as a DP table.

```
function lev(a, b, la, lb, i, j, cost, mn) {
    la = length(a); lb = length(b)
    for (i = 0; i <= la; i++) d[i, 0] = i
    for (j = 0; j <= lb; j++) d[0, j] = j
    for (i = 1; i <= la; i++) {
        for (j = 1; j <= lb; j++) {
            cost = (substr(a, i, 1) == substr(b, j, 1)) ? 0 : 1
            mn = d[i - 1, j] + 1
            if (d[i, j - 1] + 1 < mn) mn = d[i, j - 1] + 1
            if (d[i - 1, j - 1] + cost < mn) mn = d[i - 1, j - 1] + cost
            d[i, j] = mn
        }
    }
    return d[la, lb]
}
```

The DP table is the global array **d**, indexed **d[i, j]** — which, by the **SUBSEP** convention, is really **d["i\034j"]**. The function fills the standard edit-distance recurrence: the first row and column are initialized to the index (the cost of deleting or inserting that many characters), and each interior cell **d[i, j]** is the minimum of three options — delete (**d[i-1, j] + 1**), insert (**d[i, j-1] + 1**), or substitute (**d[i-1, j-1] + cost**, where **cost** is 0 if the characters match and 1 otherwise). The answer is **d[la, lb]**, the bottom-right corner. This is the algorithm exactly as it appears in any algorithms textbook, transcribed into awk with the two-dimensional table expressed through **SUBSEP** subscripting.

The record rule is two lines:

```
{
    printf "%s %s %d\n", $1, $2, lev($1, $2)
    delete d
}
```

For each input line — a pair of words — it prints the two words and their edit distance, then **delete d** clears the entire table so the next pair starts fresh. This **delete arr** is the whole-array delete from Chapter 8, and it is essential: without it, the second pair's table would inherit stale cells from the first. The example is small, but it makes a real point — awk is a perfectly reasonable language for dynamic programming, the multidimensional array is genuinely multidimensional in use even though it is one-dimensional in mechanism, and **delete** between iterations is the idiom for reusing a working table.

calc_rd.awk: a recursive-descent expression evaluator

The recursive-descent calculator is the example that proves awk is a real programming language, because a recursive-descent parser is the canonical “real program” — a tokenizer feeding a set of mutually recursive functions that implement a grammar with precedence. The grammar here is arithmetic: **+** **-** ***** **/** **%**, right-associative **^**, unary minus, and parentheses, exactly the precedence rules from Chapter 6.

```

function tokenize(s, i, n, c, num) {
    ntk = 0
    delete tk
    n = length(s)
    i = 1
    while (i <= n) {
        c = substr(s, i, 1)
        if (c == " " || c == "\t") { i++; continue }
        if (c ~ /[0-9.]/) {
            num = ""
            while (i <= n && substr(s, i, 1) ~ /[0-9.]/) { num = num substr(s, i, 1); i++ }
            tk[++ntk] = num
            continue
        }
        if (c ~ /[+ \- * \^ \% ( )]/) { tk[++ntk] = c; i++; continue }
        printf "tokenize: unexpected '%s' at %d\n", c, i
        return 0
    }
    tk[++ntk] = "$"
    return 1
}

```

`tokenize` is a hand-written lexer. It walks the input string character by character, skipping whitespace, accumulating runs of digits-and-dots into number tokens, and emitting single-character operator and parenthesis tokens, storing them in the global array `tk` indexed `1..ntk`. The `c ~ /[0-9.]/` test uses the match operator against a character class — a regex used at the character level — and the inner `while` greedily consumes the whole number. An unexpected character produces an error message and returns 0. The final token is a `"$"` sentinel marking end-of-input, the classic parser trick to avoid bounds checks. Note `delete tk` at the top resets the token array between lines.

The parser is a set of mutually recursive functions, one per precedence level, each calling the next-tighter level:

```

function p_atom( v ) {
    if (peek() == "(") { eat("("); v = p_expr(); eat(")"); return v }
    v = tk[tp] + 0; tp++; return v
}

function p_power( a, b ) {
    a = p_atom()
    if (peek() == "^") { eat("^"); b = p_unary(); return a ^ b }
    return a
}

function p_unary( v ) {
    if (peek() == "-") { eat("-"); return -p_unary() }
    if (peek() == "+") { eat("+"); return p_unary() }
    return p_power()
}

```

This is textbook recursive descent. `peek()` returns the current token and `eat(t)` consumes an expected token (erroring if it does not match), both maintaining the global token index `tp`. `p_atom` handles a parenthesized subexpression or a number literal (the `tk[tp] + 0` coercing the number token to a numeric value). `p_power` handles exponentiation, and notice how it achieves right-associativity: after reading the base with `p_atom`, if it sees `^` it recurses into `p_unary` for the exponent, so `2 ^ 3 ^ 2` nests to the right. `p_unary` handles leading `+/-` by recursing into itself, so `--5` works and unary minus binds as Chapter 6 described — looser than `^`, which is why `p_unary` sits above `p_power` in the call chain and `-2 ^ 2` evaluates as `-(2 ^ 2)`. The `p_muldiv` and `p_expr` functions (not shown) handle the left-associative `*` `/` `%` and `+` `-` levels with `while` loops.

The driver ties it together:

```

NF == 0 { next }
{
    if (!tokenize($0)) next
    tp = 1
    v = p_expr()
    if (peek() != "$") { printf "%s = parse error (trailing token %s)\n", $0, peek(); next }
    if (v == int(v)) printf "%s = %d\n", $0, v
    else printf "%s = %g\n", $0, v
}

```

Skip blank lines (`NF == 0`), tokenize the line and bail on a lex error, reset the token pointer, parse a full expression, and check that the sentinel `"$"` is the only thing left — a leftover token means trailing garbage. Finally print the result as an integer if it is integer-valued (`v == int(v)`) and as a general float otherwise. The input file deliberately includes `2 ^ 3 ^ 2`, `-2 ^ 2`, and `(-2) ^ 2` as test cases, which are exactly the precedence and associativity cases Chapter 6 used — so this example is also a live demonstration that `awk`'s `^` is right-associative and its unary minus binds loosely, matching the hand-written grammar's intent. A recursive-descent parser with correct operator precedence, written in `awk`, in about sixty lines: that is the example's quiet argument that `awk` is more language than its one-liner reputation suggests.

csv_pivot.awk: aggregation, the everyday awk job

The previous three examples were algorithms; this one is the *actual job* `awk` is reached for most often — aggregating tabular data. The task is a pivot: group rows by one column, and for each group compute a count, a sum, a min, a max, and a mean of another column. This is the report every data analyst writes ten times a week, and in `awk` it is fifteen lines.

```

BEGIN { FS = "," }

NR == 1 { next }

{
    h = $2; v = $3 + 0; mem = $4 + 0
    cnt[h]++
    sum_cpu[h] += v
    sum_mem[h] += mem
    if (!(h in min_cpu) || v < min_cpu[h]) min_cpu[h] = v
    if (!(h in max_cpu) || v > max_cpu[h]) max_cpu[h] = v
}

```

The `BEGIN` rule sets the field separator to a comma — this program assumes simple CSV with no embedded commas, and its comment notes that for real CSV you would use `awk`'s `-k/--csv` mode from Chapter 5. The `NR == 1 { next }` rule skips the header row. The main rule is the aggregation, and it shows the *parallel arrays* idiom: a separate associative array for each statistic, all keyed by the same group key `h` (the host, column 2). `cnt[h]++` counts rows per host; `sum_cpu[h] += v` and `sum_mem[h] += mem` accumulate sums; and the two `if` statements maintain running min and max, using the `!(h in min_cpu)` membership test to detect the first value for a host (Chapter 8's "is this key new?" idiom) so the min and max initialize correctly rather than comparing against an uninitialized zero. The `$3 + 0` and `$4 + 0` coercions force the numeric-string fields to numbers, as in `prime_sieve`.

The report comes in `END`:

```

END {
    printf "%-10s %5s %8s %8s %8s %10s\n", "host", "n", "min_cpu", "max_cpu", "avg_cpu",
        "tot_mem"
    PROCINFO["sorted_in"] = "@ind_str_asc"
    for (h in cnt) {
        printf "%-10s %5d %8.2f %8.2f %8.2f %10d\n", \
            h, cnt[h], min_cpu[h], max_cpu[h], sum_cpu[h] / cnt[h], sum_mem[h]
    }
}

```

First a header row with `printf` and a fixed-width format string — `%-10s` left-justifies the host name in ten columns, the numeric columns are right-justified. Then the key line for our purposes: `PROCINFO["sorted_in"] = "@ind_str_asc"`, the `gawk` sorted-iteration control from Chapter 8, which makes the following `for (h in cnt)` walk the hosts in ascending string order rather than hash order. This is not cosmetic — the program's header comment says the output is “sorted by host name ascending for deterministic byte parity,” and that determinism is exactly what lets this example pass the parity suite against `gawk`, which would otherwise differ in iteration order. The loop body prints each host's statistics, computing the mean inline as `sum_cpu[h] / cnt[h]` and formatting the floats with `%8.2f` for two decimal places. The line continuation `\` lets the long `printf` span two source lines.

This example is the one to internalize if you take only one from this chapter, because it is the shape of real `awk` work: set the separator, skip the header, accumulate parallel arrays keyed by a group, then iterate in a controlled order and report with formatted `printf`. Counting, summing, averaging, and finding extremes, grouped by a key, with clean columnar output — that is the daily bread of data processing, and `awk` does it in a single screen with no libraries, no imports, and no ceremony. `awkrs` runs it fast, sorts it deterministically, and matches `gawk` byte-for-byte.

hexdump.awk: bytes, formatting, and an ordinal table

The `hexdump` example shows `awk` doing low-level byte work — an `xxd`-style dump with an address column, a hex column, and a printable-ASCII column — and it demonstrates two techniques worth knowing: building a character-to-ordinal lookup table, and buffering the whole input to handle it as one byte string.

```

BEGIN {
    for (i = 0; i < 256; i++) _ord[sprintf("%c", i)] = i
    buf = ""
}

{ buf = buf $0 "\n" }

```

The `BEGIN` rule builds `_ord`, an associative array mapping each of the 256 single-byte characters to its ordinal value, by walking 0 to 255 and using `sprintf("%c", i)` to produce the character for each code — the inverse of the ordinal lookup, precomputed once so the body can look up a character's value with a simple array index. (`awkrs` also provides the `ord` builtin from Chapter 12 for this, but the example builds its own table, which is portable to any `awk`.) The record rule accumulates every line, with its stripped newline reinserted, into a single buffer `buf` — because a hex dump needs to treat the input as one contiguous byte stream with correct offsets, not as separate lines.

The work happens in `END`, calling a helper `emit_row` for each sixteen-byte chunk:

```

function emit_row(off, bytes, i, n, c, hex, asc, glyph, gap) {
    n = length(bytes)
    hex = ""; asc = ""
    for (i = 1; i <= 16; i++) {
        if (i <= n) {
            c = substr(bytes, i, 1)
            hex = hex sprintf("%02x", _ord[c])
            glyph = (_ord[c] >= 32 && _ord[c] < 127) ? c : "."
            asc = asc glyph
        } else {
            hex = hex " "
            asc = asc " "
        }
        gap = (i == 8) ? " " : ""
        hex = hex gap
    }
    printf "%08x  %s |%s|\n", off, hex, asc
}

```

`emit_row` formats one sixteen-byte row. For each of the sixteen positions, if there is a byte (`i <= n`), it formats the byte's hex with `sprintf("%02x", _ord[c])` — two zero-padded hex digits, the ordinal looked up from the table — and builds the printable column, substituting a `.` for any byte outside the printable ASCII range 32 to 126. Positions past the end of a short final row pad with spaces. The `gap` logic inserts an extra space after the eighth byte, the `xxd` convention of splitting the hex column into two groups of eight. The final `printf "%08x %s |%s|\n"` emits the row: an eight-digit zero-padded hex offset, the hex column, and the ASCII column bracketed by pipes. The `END` driver (not shown) trims the trailing newline so byte offsets match the original input exactly, then walks the buffer in sixteen-byte strides calling `emit_row`, and finishes by printing the total byte count.

The lesson here is that `awk`'s string functions — `substr` for byte extraction, `sprintf` for fixed-width hex formatting, the ordinal table for byte values — are enough to do genuinely byte-level work, and that the formatting precision `awk` gives you through `printf` (zero-padding, field widths, hex conversions, all from Chapter 13) is exactly what a structured dump format needs. This is `xxd` in forty lines of `awk`, and every formatting detail in it is a `printf` conversion `awk` implements to spec.

roman.awk: a bidirectional converter with table-driven logic

The Roman numeral converter shows a clean table-driven design and the kind of input-classification a record rule does naturally. It converts integers to Roman numerals and Roman numerals back to integers, in the same program, choosing the direction by inspecting the input.

```

BEGIN {
    n = 0
    n++; vals[n] = 1000; syms[n] = "M"
    n++; vals[n] = 900;  syms[n] = "CM"
    n++; vals[n] = 500;  syms[n] = "D"
    ...
    n++; vals[n] = 1;     syms[n] = "I"
    nsyms = n

    rv["I"] = 1;   rv["V"] = 5;   rv["X"] = 10;   rv["L"] = 50
    rv["C"] = 100; rv["D"] = 500; rv["M"] = 1000
}

```

The `BEGIN` rule builds two parallel tables: `vals` and `syms`, holding the decimal values and their Roman symbols in descending order, *including* the subtractive forms (`CM` for 900, `CD` for 400, `XC` for 90, and so on) interleaved

at their correct magnitudes. This is the trick that makes integer-to-Roman a simple greedy loop — by putting `900 = "CM"` directly in the table between `1000 = "M"` and `500 = "D"`, the greedy algorithm produces the subtractive form without special-casing it. The second table, `rv`, maps single Roman letters to values for parsing the other direction.

```
function to_roman(x, i, out) {
    if (x < 1 || x > 3999) return "ERR(out of range 1..3999)"
    out = ""
    for (i = 1; i <= nsyms; i++) {
        while (x >= vals[i]) { out = out syms[i]; x -= vals[i] }
    }
    return out
}
```

`to_roman` is the greedy algorithm: for each table entry from largest to smallest, append its symbol and subtract its value as many times as it fits. Because the subtractive forms are in the table, `4` produces `IV` and `900` produces `CM` naturally. `to_int` (not shown) walks the Roman string left to right, looking each letter up in `rv`, and applies the subtractive rule — if a letter's value is less than the next letter's, subtract it instead of adding (so the `IV` in `IV` sees `I < V` and subtracts).

The dispatch classifies the input:

```
NF == 0 { next }
{
    t = $1
    if (t ~ /^[0-9]+$/) printf "%s -> %s\n", t, to_roman(t + 0)
    else printf "%s -> %s\n", t, to_int(t)
}
```

Skip blank lines, then for each token decide the direction with a regex test: `t ~ /^[0-9]+$/` asks “is this all digits?” — if so, it is an integer to convert to Roman (with `t + 0` coercing it to a number); otherwise it is a Roman string to convert to an integer. This is `awk`'s pattern-matching used for type classification — the same `~` operator from Chapter 10, anchored with `^` and `$` to match the whole field — and it is a clean way to write a bidirectional tool: one regex decides which way to go. The table-driven greedy conversion and the single-regex dispatch together make a complete, correct Roman numeral converter in about fifty readable lines.

json_pretty.awk: a tokenizer and a re-emitter

The final example is the most involved: a JSON pretty-printer that tokenizes arbitrary JSON and re-emits it with two-space structural indentation. It shows `awk` handling a nested, structured format — which `awk` is not “supposed” to be good at — through the same tokenize-then-process architecture as the calculator.

```

function tokenize(s, n, i, c, t, j, depth) {
  n = length(s)
  i = 1
  while (i <= n) {
    c = substr(s, i, 1)
    if (c == " " || c == "\t" || c == "\n" || c == "\r") { i++; continue }
    if (c == "{" || c == "}" || c == "[" || c == "]" || c == ":" || c == ",") {
      tk[++ntk] = c; i++; continue
    }
    if (c == "\"") {
      t = "\""; j = i + 1
      while (j <= n) {
        cc = substr(s, j, 1)
        if (cc == "\\") { t = t cc substr(s, j + 1, 1); j += 2; continue }
        if (cc == "\"") { t = t cc; j++; break }
        t = t cc; j++
      }
      tk[++ntk] = t; i = j; continue
    }
    ...
  }
}

```

The tokenizer walks the input, skipping whitespace, emitting each structural character (`{ }` `[ ]` `:` `,`) as its own token, and — the interesting part — scanning a double-quoted string as a single token, carefully handling backslash escapes so that a `\` inside a string does not end it. The escape handling (`if (cc == "\\") { t = t cc substr(s, j+1, 1); j += 2 }`) copies both the backslash and the following character verbatim and skips past them, so the string token preserves its original bytes exactly — the program is a pretty-printer, not a validator or a re-encoder, and it keeps strings byte-for-byte. Numbers and the literals `true/false/null` are scanned greedily up to the next structural character.

The re-emitter walks the token array, maintaining an indentation `depth`:

```

for (i = 1; i <= ntk; i++) {
  t = tk[i]; nx = (i < ntk) ? tk[i + 1] : ""
  if (t == "{" || t == "[") {
    out = out t
    if (nx == "}" || nx == "]") {
      # empty container - keep on one line
    } else {
      depth++
      out = out "\n" pad(depth)
    }
    continue
  }
  if (t == "}" || t == "]") {
    pv = tk[i - 1]
    if (!(t == "}" && pv == "{" || (t == "]" && pv == "["))) {
      depth--
      out = out "\n" pad(depth)
    }
    out = out t
    continue
  }
  if (t == ",") { out = out ",\n" pad(depth); continue }
  if (t == ":") { out = out ":"; continue }
  out = out t
}
print out

```

Each token is handled by its kind. An opening brace or bracket increases the depth and emits a newline plus indentation — *unless* the very next token closes it, in which case the empty container `{ }` or `[ ]` is kept on one line, a real-world formatting nicety. A closing brace or bracket decreases the depth and emits a newline — again unless it directly follows its matching open (the empty case). A comma emits itself, a newline, and the current indentation, so each element starts on its own line. A colon emits itself with a trailing space (`" : "`). Anything else — a string token, a number, a literal — emits verbatim. The `pad(depth)` helper (not shown) returns `depth` levels of two-space indentation. The whole formatted document is accumulated in `out` and printed once.

This is a genuinely non-trivial program — a two-pass tokenize-and-reformat over a recursive data format, with lookahead (`nx`), lookbehind (`pv`), the empty-container special case, and careful string-escape handling — and it is maybe seventy lines of `awk`. It is the strongest argument in the example set for the claim this book has made throughout: `awk`'s small model, taken seriously, reaches much further than its reputation. The associative array, the string builtins, user functions, and the pattern-action loop are enough to write tokenizers, parsers, DP algorithms, tries, and structured-format processors — and `awkrs` runs all of it at the speeds Chapter 14 measured, debugs it with the tooling of Chapter 20, and verifies every one of these examples byte-for-byte against `gawk` in CI. The examples are not demonstrations of a toy; they are the proof that `awkrs` is a faithful, fast, complete `awk`, and that the language underneath it was always more capable than the one-liners let on.

`rpn.awk`: a stack machine in an associative array

Where `calc_rd.awk` parsed infix arithmetic with a recursive-descent grammar, the reverse-Polish calculator does the same arithmetic the other way — postfix, with an explicit operand stack — and it is the cleanest small example of using an associative array plus an integer index as a stack, the data-structure pattern Chapter 8 listed. The input is one space-separated postfix expression per line; numbers push, operators pop two and push the result, and a handful of stack verbs (`dup`, `swap`, `drop`, `neg`) manipulate the stack directly.

```

function err(msg, line) { printf "%s = ERR %s\n", line, msg }

{
    sp = 0
    delete st
    ok = 1
    for (i = 1; i <= NF; i++) {
        t = $i
        if (t ~ /^-?[0-9]+(\.[0-9]+)?$/) { st[++sp] = t + 0; continue }
        if (t == "+" || t == "-" || t == "*" || t == "/" || t == "%" || t == "^") {
            if (sp < 2) { err("underflow at "t, $0); ok = 0; break }
            b = st[sp--]; a = st[sp--]
            if (t == "+") st[++sp] = a + b
            if (t == "-") st[++sp] = a - b
            if (t == "*") st[++sp] = a * b
            if (t == "/") { if (b == 0) { err("div0", $0); ok = 0; break } st[++sp] = a / b
                ~ }
            if (t == "%") { if (b == 0) { err("mod0", $0); ok = 0; break } st[++sp] = a % b
                ~ }
            if (t == "^") st[++sp] = a ^ b
            continue
        }
        ...
    }
    if (!ok) next
    if (sp != 1) { err("stack depth "sp" at end", $0); next }
    v = st[1]
    if (v == int(v)) printf "%s = %d\n", $0, v
    if (v != int(v)) printf "%s = %g\n", $0, v
}

```

The whole machine is the array `st` and the integer top-of-stack index `sp`. Each record resets them — `sp = 0`; `delete st` — so a malformed line cannot leak state into the next, and `delete st` is again the whole-array clear of Chapter 8. The loop walks the fields with `for (i = 1; i <= NF; i++)`, using `NF` as the token count directly, which is the natural `awk` idiom for a space-separated line: the fields *are* the tokens, already split for you, so this calculator needs no tokenizer at all where the infix one needed a hand-written lexer. A number — recognized by the regex `/^-?[0-9]+(\.[0-9]+)?$/`, anchored to match the whole token — pushes with `st[++sp] = t + 0`, the pre-increment growing the stack and the `+ 0` coercing the numeric- string field to a number. A binary operator pops `b` then `a` with the post- decrement idiom `st[sp--]` (note the order: the second-popped value `a` is the left operand, because it was pushed first and sits deeper), computes, and pushes. The two division operators guard their zero divisor explicitly and emit an error rather than triggering `awk`'s fatal division-by-zero — a deliberate choice to keep the calculator's own error reporting rather than aborting the whole run on one bad line. Notice the cascade of independent `if` statements rather than `else if`: because each condition is mutually exclusive and each branch `continues` or falls through harmlessly, the flat form reads as a dispatch table. After the loop, a well-formed expression has left exactly one value on the stack (`sp == 1`); any other depth is an error, and the lone result prints as an integer if integer-valued and as `%g` otherwise — the same `v == int(v)` test the infix calculator used. It is a complete stack-machine evaluator in forty lines, built from one array and one index.

base_conv.awk: number bases without strtonum

The base converter turns a number from any base between 2 and 36 into any other, in either direction, and it does so *without* leaning on `strtonum` or `printf`'s `%x` — it does the digit arithmetic by hand, which makes it a clean illustration of two ideas: a precomputed character-to-value table built in `BEGIN`, and the classic divide-and-collect-remainders loop for emitting a number in a base.

```

BEGIN {
    digs = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    for (i = 1; i <= length(digs); i++) {
        ch = substr(digs, i, 1)
        val[ch] = i - 1
        val[tolower(ch)] = i - 1
    }
}

function to_int(s, base, neg, n, i, c, total) {
    neg = 0
    if (substr(s, 1, 1) == "-") { neg = 1; s = substr(s, 2) }
    total = 0
    n = length(s)
    for (i = 1; i <= n; i++) {
        c = substr(s, i, 1)
        if (!(c in val) || val[c] >= base) return "ERR(bad digit " c ")"
        total = total * base + val[c]
    }
    return neg ? -total : total
}

function from_int(x, base, out, neg, r) {
    if (x == 0) return "0"
    neg = (x < 0)
    if (neg) x = -x
    out = ""
    while (x > 0) {
        r = x % base
        out = substr(digs, r + 1, 1) out
        x = int(x / base)
    }
    if (neg) out = "-" out
    return out
}

```

The `BEGIN` rule builds `val`, a map from every digit character to its numeric value, walking the canonical digit string `0-9A-Z` and recording both the uppercase and the `tolower` form so parsing is case-insensitive. This is the same precompute-a-lookup-table technique `hexdump.awk` used for its ordinal table, and it is the natural `awk` way to turn a character into a number: an array indexed by the character. `to_int` parses a string in a given base by Horner's method — `total = total * base + digit`, accumulated left to right — with two guards that make it robust: a digit not in the table (`!(c in val)`, the membership test that does not create the key) or a digit too large for the base (`val[c] >= base`) returns an error string rather than a wrong number. `from_int` is the inverse, the textbook emit-in-base loop: repeatedly take the remainder `x % base`, prepend the corresponding digit character with `substr(digs, r + 1, 1)`, and integer-divide `x` by the base with `int(x / base)` until nothing is left. Prepending (`out = digit out`) rather than appending is what puts the digits in the right order without a reversal step. The driver coerces the two base fields with `+ 0`, range-checks them, and threads the value through `to_int` then `from_int`, detecting `to_int`'s error string with the regex `n ~ /^ERR/`. The program is a reminder that `awk`'s arithmetic and string builtins are enough to implement number-base conversion from first principles, no library and no format-conversion trickery required.

ini_parser.awk: pattern dispatch as a grammar

The INI parser is the example that shows `awk`'s pattern-action model used as a small *grammar* — each line type gets its own pattern, and the patterns together recognize the INI format without a single explicit conditional in the main body. It is the clearest demonstration in the example set that, in `awk`, the patterns are the parser.

```

function trim(s) { sub(/^[\t]+/, "", s); sub(/[ \t]+$/, "", s); return s }

BEGIN { sec = "(global)"; secs[sec] = 1 }

/^[[:space:]]*([#;].*)?$/ { next }

/^[[:space:]]*\.[*]\.[[:space:]]*$/ {
    line = $0
    sub(/^[[:space:]]*\[/, "", line)
    sub(/\)[[:space:]]*$/, "", line)
    sec = trim(line)
    secs[sec] = 1
    next
}

/=/ {
    eq = index($0, "=")
    k = trim(substr($0, 1, eq - 1))
    v = trim(substr($0, eq + 1))
    store[sec, k] = v
    keys_in[sec] = (sec in keys_in) ? keys_in[sec] " " k : k
    nkeys++
    next
}

```

Read the four rules as a grammar. The **BEGIN** rule establishes the current section as a **(global)** pseudo-section, so keys before any **[section]** header have a home. The first record rule, **/^[[:space:]]*([#;].*)?\$/ { next }**, matches a blank line or a comment (one beginning with **#** or **;** after optional whitespace) and skips it — the POSIX character class **[[:space:]]** from Chapter 10, and an optional group that makes the whole comment part skippable. The second rule matches a section header — **[name]** possibly surrounded by whitespace — and extracts the name by **sub**-ing away the brackets and surrounding space, then **trim**s it and makes it the current section. The third rule matches any line containing an **=** and splits it at the *first* **=** (found with **index**, then **substr** on either side) into a trimmed key and value, storing the value in the two-dimensional **store[sec, k]** keyed by section-and-key via **SUBSEP**. Each rule ends in **next**, so the rules are mutually exclusive in practice: a line is a comment, or a header, or a key-value pair, and the first matching rule consumes it. The **trim** helper is a two-**sub** idiom — strip leading whitespace, strip trailing whitespace — that every awk programmer writes eventually, and it shows **sub** modifying its target string in place (here the parameter **s**, which is a local copy, returned explicitly).

```

END {
    PROCINFO["sorted_in"] = "@ind_str_asc"
    nsec = 0
    for (s in secs) {
        nsec++
        if (!(s in keys_in)) continue
        split(keys_in[s], arr, " ")
        n = asort(arr)
        for (i = 1; i <= n; i++) printf "[%s] %s = %s\n", s, arr[i], store[s, arr[i]]
    }
    printf "SECTIONS: %d KEYS: %d\n", nsec, nkeys + 0
}

```

The **END** rule reports, and it combines three Chapter 8 mechanisms at once. It sets **PROCINFO["sorted_in"] = "@ind_str_asc"** so the outer **for (s in secs)** walks the sections in sorted order; it **splits** each section's accumulated key list back into an array; and it calls **asort(arr)** to sort the keys within a section before printing. The keys were accumulated as a space-joined string **keys_in[sec]** during parsing — a common awk way to keep an ordered list in a scalar — and split apart for sorting at the end. The result is deterministic, sorted, sectioned output, which is both what a human wants from an INI dump and what lets the

example pass the parity suite. Four patterns recognize the format; the `END` rule sorts and emits it; and the whole INI grammar fits on one screen with no explicit parser state machine, because the patterns *are* the state machine.

url_parser.awk: match, RSTART, RLENGTH, and index surgery

The URL parser decomposes a URL into scheme, user, password, host, port, path, query, and fragment, and it is the example that most thoroughly exercises awk's string-surgery builtins — `match` with its `RSTART`/`RLENGTH` side effects, and `index`/`substr` used to peel a string apart from both ends.

```
{
    scheme = ""; user = ""; pass = ""; host = ""
    port = ""; path = ""; query = ""; fragment = ""

    url = $0
    rest = url

    # scheme
    if (match(rest, /^[a-zA-Z][a-zA-Z0-9+\-.]*:\/\/)) {
        scheme = substr(rest, 1, RLENGTH - 3)
        rest = substr(rest, RLENGTH + 1)
    }

    # fragment (trailing)
    if ((p = index(rest, "#")) > 0) {
        fragment = substr(rest, p + 1)
        rest = substr(rest, 1, p - 1)
    }

    # query
    if ((p = index(rest, "?")) > 0) {
        query = substr(rest, p + 1)
        rest = substr(rest, 1, p - 1)
    }
    ...
}
```

The strategy is to maintain a shrinking `rest` string and bite components off it in the order the grammar makes unambiguous. The scheme comes first, matched with a regex against the front of `rest`: `match` returns the position and, as Chapter 10 described, sets `RLENGTH` to the match length. Because the matched text includes the trailing `://`, the scheme name is `substr(rest, 1, RLENGTH - 3)` — the match minus those three characters — and `rest` advances past the whole match with `substr(rest, RLENGTH + 1)`. The fragment and the query are peeled from the *end*, using `index` to find the `#` and `?` and `substr` to split there, each assignment-in-a-condition (`p = index(rest, "#") > 0`) doing the find and the test in one expression — exactly the idiom Chapter 7 noted needs its parentheses. Peeling the fragment before the query matters: a `?` can appear inside a fragment but a `#` cannot appear before the real fragment, so removing the trailing fragment first keeps the query search from matching a `?` that belongs to the fragment. The authority section — the optional `user:pass@host:port` — is then split on `@` and `:` by the same `index/substr` surgery. The result is a parser for a genuinely irregular real-world format written entirely with `match`, `index`, and `substr`, no regex captures and no `split` — a demonstration that awk's oldest string builtins, used carefully, reach further than they look like they should.

rle.awk: a codec with two directions and a number scanner

The final added example is a run-length codec — it encodes a string by collapsing runs of a repeated character into a count-and-character, and decodes the reverse — and it shows two things worth seeing once: building output by character-at-a-time concatenation, and hand-scanning a multi-digit number out of a string, which is the same micro-lexer pattern the calculators used, in miniature.

```
function rle_encode(s, n, i, cur, run, out) {
    n = length(s)
    if (n == 0) return ""
    cur = substr(s, 1, 1); run = 1
    out = ""
    for (i = 2; i <= n; i++) {
        c = substr(s, i, 1)
        if (c == cur) { run++; continue }
        out = out (run > 1 ? run : "") cur
        cur = c; run = 1
    }
    out = out (run > 1 ? run : "") cur
    return out
}
```

`rle_encode` walks the string once, carrying the current run's character `cur` and its length `run`. Each new character either extends the run (`c == cur`) or closes it: closing emits `(run > 1 ? run : "") cur` — the count *only when it exceeds one*, so a single character emits as itself and `aaa` emits as `3a` — and starts a fresh run. The final run is flushed after the loop, the classic off-by-one care that a run-accumulating loop always needs. The ternary inside a concatenation is a compact awk idiom: the conditional chooses the count string or the empty string, and juxtaposition glues it to the character.

```
function rle_decode(s, n, i, c, num, out, j) {
    n = length(s)
    out = ""
    i = 1
    while (i <= n) {
        c = substr(s, i, 1)
        if (c ~ /[0-9]/) {
            num = ""
            while (i <= n && substr(s, i, 1) ~ /[0-9]/) { num = num substr(s, i, 1); i++ }
            if (i > n) return out
            c = substr(s, i, 1); i++
            for (j = 0; j < (num + 0); j++) out = out c
        } else {
            out = out c
            i++
        }
    }
    return out
}
```

`rle_decode` is the inverse, and its inner `while` is the number scanner: when it sees a digit, it greedily consumes the whole run of digits into the string `num`, exactly the way `calc_id.awk`'s tokenizer accumulated a number, then reads the character that follows and appends it `num + 0` times — the `+ 0` coercing the accumulated digit string to a count for the `for` loop bound. A bare character with no preceding count appends once. The two record rules dispatch on the first field — `$1 == "ENC"` and `$1 == "DEC"` — and operate on the rest of the line via `substr($0, 5)` to preserve internal whitespace exactly (using `$0` rather than re-joining fields, because the text may contain spaces that are part of the data). It is a complete, reversible codec in about forty lines, and it closes the tour the way the tour began: with an ordinary program doing ordinary work,

where `awk`'s smallness is not a limitation but the reason the whole thing fits in the head at once.

anagrams.awk: a canonical key and a hand-written sort

Grouping anagrams is the textbook use of a *canonical key* — turn each word into a signature that is identical for all its anagrams, and group by the signature — and the `awk` version shows both that idea and a hand-written insertion sort, because `awk` has no builtin that sorts the characters of a string. It is a compact lesson in making your own ordering when the library does not hand you one.

```
function sig(w, n, i, a, sorted, key, j) {
    n = length(w)
    for (i = 1; i <= n; i++) a[i] = substr(w, i, 1)
    for (i = 2; i <= n; i++) {
        key = a[i]; j = i - 1
        while (j >= 1 && a[j] > key) { a[j + 1] = a[j]; j-- }
        a[j + 1] = key
    }
    sorted = ""
    for (i = 1; i <= n; i++) sorted = sorted a[i]
    return sorted
}
```

`sig` computes a word's signature: it explodes the word into an array of single characters, sorts that array, and reglues it into a string. The sort is insertion sort written out by hand — the inner `while` shifts larger characters right to open a slot for `key`, the classic insertion-sort inner loop — and it compares characters with `>`, which here compares them as strings (both operands are string-typed locals holding one character each), so the result is lexicographic by byte. Two words are anagrams exactly when their sorted-character signatures are equal, so the signature is the key. `awk` does ship `asort`, and the example *could* have sorted the character array with it; writing the sort by hand keeps the program portable to any `awk` and makes the canonical-key idea fully visible. The record rule lowercases each word with `tolower`, computes its signature, and appends the word to that signature's group — `group[s] = group[s] ((s in seen) ? " " : "") w`, the membership test deciding whether to prepend a separating space so the group is a clean space-joined list with no leading space. The `END` rule walks the groups in sorted signature order (`PROCINFO["sorted_in"]`) and prints each. The whole program is the canonical-key pattern — normalize to a key, accumulate by key, report by key — which is the same shape as `csv_pivot.awk`'s aggregation, with a hand-rolled sort standing in for a builtin `awk` does not have.

dijkstra.awk: a binary heap in parallel arrays

Dijkstra's shortest-path algorithm needs a priority queue, and `awk` has no such structure — so this example *builds* one, a binary min-heap, out of two parallel associative arrays and an integer size. It is the most ambitious data structure in the tour, and it shows that even a heap, with its index arithmetic and its sift-up and sift-down, is within reach of nothing but arrays and integer math.

```

function hpush(d, n, i, td, tn, p) {
    hn++
    hd[hn] = d; hk[hn] = n
    i = hn
    while (i > 1) {
        p = int(i / 2)
        if (hd[p] > hd[i] || (hd[p] == hd[i] && hk[p] > hk[i])) {
            td = hd[p]; tn = hk[p]
            hd[p] = hd[i]; hk[p] = hk[i]
            hd[i] = td; hk[i] = tn
            i = p
        } else { break }
    }
}

```

The heap is two arrays — `hd` holding distances (the sort key) and `hk` holding the node names (the payload) — indexed in lockstep by a one-based position, with `hn` the current size. This is the array-of-records idiom done with *parallel* arrays: rather than pack a distance and a node into one composite value, the program keeps two arrays sharing an index, which is the awk-idiomatic way to store a struct. `hpush` appends at `hn` and sifts up: the parent of position `i` is `int(i / 2)`, the integer division that is heap indexing's heart, and the element swaps upward while it is smaller than its parent, with ties broken by node name (`hk[p] > hk[i]`) so the heap order — and therefore the algorithm's output — is deterministic when several frontier nodes share a distance. `hpop` (the sift-down mirror) removes the root, moves the last element to the top, and sifts it down toward the smaller child, returning its result through the globals `popd/popk` — awk functions return one scalar, so a function that must yield two values writes them to agreed globals, the same out- parameter pattern Chapter 9 described, here used for a pair rather than an array.

```

while (hn > 0) {
    hpop()
    u = popk; curd = popd
    if (curd > dist[u]) continue # stale entry
    for (k = 1; k <= outdeg[u]; k++) {
        v = adj_to[u, k]; w = adj_w[u, k]
        nd = curd + w
        if (!(v in dist) || nd < dist[v]) {
            dist[v] = nd; parent[v] = u
            hpush(nd, v)
        }
    }
}

```

The main loop is Dijkstra exactly as the textbook gives it, with one practical refinement worth naming: *lazy deletion*. A standard binary heap cannot cheaply decrease a key already inside it, so instead of finding and updating an old entry, the program just pushes a new, better entry for the same node and lets the stale one sit in the heap — when a stale entry later pops, the guard `if (curd > dist[u]) continue` recognizes that this node already has a shorter settled distance and skips it. The adjacency list is itself a pair of `SUBSEP` arrays, `adj_to[u, k]` and `adj_w[u, k]`, indexed by source node and edge number, with `outdeg[u]` counting a node's edges — a graph stored entirely in associative arrays. After the search, the `END` rule reconstructs each path by walking the `parent` chain back to the source, prepending as it goes, and prints distances and paths in sorted node order. A priority queue, an adjacency list, a shortest-path search, and path reconstruction — all of classical graph algorithmics — built from associative arrays and `int(i / 2)`.

conway.awk: a 2-D grid and a double buffer

The last example is Conway's Game of Life, and it is the clearest demonstration of the `SUBSEP` two-dimensional array used as a genuine grid, plus the *double-buffer* discipline that any cellular-automaton step needs: you must compute the whole next generation from the current one before overwriting it, or cells will see their

neighbors half-updated.

```
function get(r, c) {
    if (r < 1 || r > H || c < 1 || c > W) return 0
    return (cell[r, c] == 1) ? 1 : 0
}

function neighbors(r, c) {
    return get(r-1,c-1)+get(r-1,c)+get(r-1,c+1)+ \
        get(r,c-1)+           get(r,c+1)+ \
        get(r+1,c-1)+get(r+1,c)+get(r+1,c+1)
}
```

The board is the array `cell`, indexed `cell[r, c]` — a two-dimensional grid that is, by `SUBSEP`, a one-dimensional map with composite keys. `get` reads a cell with edge clamping: an out-of-bounds coordinate returns dead (0) rather than creating a key, which both implements the finite-board boundary and avoids polluting the array with phantom cells — note it does *not* use `cell[r, c]` until after the bounds check, because a read would otherwise auto-create the key (the Chapter 8 subtlety). The `neighbors` function sums the eight surrounding cells with the line-continuation `\` spanning the expression across three source lines for readability, the empty middle slot conspicuously leaving out the cell itself.

```
function step() {
    delete nxt
    for (r = 1; r <= H; r++) {
        for (c = 1; c <= W; c++) {
            n = neighbors(r, c)
            alive = (cell[r, c] == 1)
            if (alive && (n == 2 || n == 3)) nxt[r, c] = 1
            else if (!alive && n == 3)      nxt[r, c] = 1
            else                             delete nxt[r, c]
        }
    }
    delete cell
    for (k in nxt) cell[k] = nxt[k]
}
```

`step` is the double buffer made explicit. It computes the entire next generation into a *separate* array `nxt` — applying Conway's rules (a live cell with two or three neighbors survives, a dead cell with exactly three is born, everything else is dead) — and only after the whole grid is computed does it `delete cell` and copy `nxt` back. Computing into a second buffer is not optional decoration; it is the correctness requirement of the algorithm, because the rules for cell `(r, c)` depend on neighbors that this same pass would otherwise have already changed. The copy-back loop `for (k in nxt) cell[k] = nxt[k]` walks the next buffer's keys — and here iteration order genuinely does not matter, because it is a straight copy, so the unspecified `for-in` order of Chapter 8 is harmless. The driver reads the grid dimensions and initial cells from input, then `shows` generation zero and steps `N` times, printing each generation. A finite-grid cellular automaton with correct double-buffered evolution, in about fifty lines, on nothing but a `SUBSEP` grid and a second array — the final proof of Chapter 8's thesis that the associative array, taken seriously, is every data structure you need.

kmp.awk: the Knuth-Morris-Pratt failure function

String search is awk's home turf, but the *interesting* string-search algorithms — the ones that beat the naive double loop — need an auxiliary table, and KMP is the canonical example. This program builds the KMP failure function and uses it to find every occurrence of a pattern in a text in linear time, and it is the example that shows awk implementing a genuinely non-obvious algorithm character by character.

```

function build_fail(p, m, i, k) {
    m = length(p)
    delete fail
    fail[1] = 0
    k = 0
    for (i = 2; i <= m; i++) {
        while (k > 0 && substr(p, k + 1, 1) != substr(p, i, 1)) k = fail[k]
        if (substr(p, k + 1, 1) == substr(p, i, 1)) k++
        fail[i] = k
    }
}

```

`build_fail` computes the failure function — `fail[i]` is the length of the longest proper prefix of the pattern's first `i` characters that is also a suffix of them — into the global array `fail`, rebuilt with `delete fail` each time the pattern changes. The construction is the classic one: a single index `k` tracks the current prefix-match length, the inner `while` falls back through earlier failure values when the next character does not extend the match (`k = fail[k]`, the self-referential recurrence that is KMP's whole trick), and `k` advances when it does. Every character access is a `substr(p, ..., 1)`, awk's way of indexing a string, and the algorithm is transcribed essentially verbatim from the textbook. `kmp_search` then scans the text with the same fall-back-on-mismatch structure, and each time the match length `q` reaches the pattern length `m` it records a one-based hit position (`i - m + 1`) and falls back via `q = fail[q]` to keep finding overlapping matches. The two record rules turn this into a tiny REPL — `PAT` sets and rebuilds the pattern, `TXT` searches a line — using `substr($0, 5)` to take the argument after the verb. The program is the strongest single answer to anyone who thinks awk is only for column extraction: it is a correct linear-time string matcher with a precomputed automaton, written in awk, in forty lines.

knapsack.awk: 0/1 dynamic programming with trace-back

`levenshtein.awk` showed a DP table; the knapsack solver shows the *other* half of dynamic programming that examples often skip — the traceback that recovers *which* choices produced the optimal value, not just the value itself. It solves the 0/1 knapsack and then walks the table backward to report the chosen items.

```

END {
    for (w = 0; w <= CAP; w++) dp[0, w] = 0
    for (i = 1; i <= n; i++) {
        for (w = 0; w <= CAP; w++) {
            dp[i, w] = dp[i - 1, w]
            if (wt[i] <= w) {
                cand = dp[i - 1, w - wt[i]] + vl[i]
                if (cand > dp[i, w]) dp[i, w] = cand
            }
        }
    }

    # Traceback.
    w = CAP
    for (i = n; i >= 1; i--) {
        if (dp[i, w] != dp[i - 1, w]) {
            taken[i] = 1
            w -= wt[i]
        }
    }
    ...
}

```

The whole solve runs in `END`, after the record rules have read the capacity and loaded the items into the parallel arrays `nm`, `wt`, `vl` (name, weight, value). The forward pass fills `dp[i, w]`, a `SUBSEP` two-dimensional table where row `i` considers the first `i` items and column `w` a capacity of `w`: each cell starts as the value without item `i` (`dp[i-1, w]`) and is improved, when the item fits (`wt[i] <= w`), by the value of taking it (`dp[i-1, w - wt[i]] + vl[i]`). This is the standard 0/1-knapsack recurrence. The traceback is the part worth studying: starting from the full capacity and the last item, it asks at each row whether the optimal value *changed* when item `i` became available — `dp[i, w] != dp[i-1, w]` — and if so concludes that item `i` was taken, marks it, and reduces the remaining capacity by its weight. Walking the decision table backward to reconstruct the choices is a technique that applies to every DP problem, and seeing it in `awk` — `taken[i] = 1` in one array while the table sits in another — makes concrete that `awk`'s arrays are enough not just to *compute* a DP optimum but to *explain* it.

markov.awk: bigram counts and splitting a SUBSEP key

The final example is a bigram analyzer — it reads a text corpus, counts which words follow which, and reports the most common continuations — and it shows two techniques the earlier examples did not: building a frequency table keyed by a word *pair*, and the inverse of the `SUBSEP` join, *splitting* a composite key back into its parts with `split(key, parts, SUBSEP)`.

```
function bigrams_for_line(line, n, i) {
    gsub(/^[^a-zA-Z]+/, " ", line)
    line = tolower(line)
    n = split(line, w, " ")
    for (i = 1; i < n; i++) {
        if (w[i] == "" || w[i+1] == "") continue
        bg[w[i], w[i+1]]++
        starts[w[i]]++
    }
}
```

Tokenizing is two builtins: `gsub(/^[^a-zA-Z]+/, " ", line)` replaces every run of non-letters with a single space — normalizing punctuation and digits away in one call — and `tolower` folds case, after which `split(line, w, " ")` breaks the line into a word array. The bigram count lives in `bg[w[i], w[i+1]]`, a two-dimensional array keyed by *consecutive word pairs*, incremented as the loop slides a window of two across the words; `starts[w[i]]` separately counts how often each word begins a bigram. This is the same counting idiom as `csv_pivot.awk`, lifted to a composite key.

```
function top_next(prev, k, a, b, ...) {
    ...
    for (k in bg) {
        split(k, parts, SUBSEP)
        if (parts[1] != prev) continue
        a = parts[2]; b = bg[k]
        ...
    }
    ...
}
```

The reporting walks `bg`'s keys, and here is the inverse of the multidimensional convention: a key `k` from `for (k in bg)` is the *joined* string `prev SUBSEP curr`, and `split(k, parts, SUBSEP)` cracks it back into `parts[1]` and `parts[2]` — the two words. This is the round trip Chapter 8 implied: `bg[a, b]` joins with `SUBSEP` on the way in, and `split(k, parts, SUBSEP)` separates on the way out, so a program that iterates a multidimensional array can recover the original subscripts. The function keeps a running top-three by count (descending) with a lexicographic tie-break, so the output is fully deterministic without any call to `rand()` — which is the program's deliberate design, noted in its header, so that its output is byte-stable for the parity suite. The `walk` function does a greedy deterministic chain — at each step take the single highest-count continuation, ties broken by name — to generate a twelve-word sequence. A bigram language model, top-k

continuation ranking, and a greedy text walk, built from two associative arrays and the [SUBSEP](#) round trip: it is the natural-language-processing example the tour needed to round it out, and it is once again nothing but arrays, [split](#), and [gsub](#).

topo_sort.awk: Kahn's algorithm and cycle detection

A topological sort orders the nodes of a directed graph so every edge points forward, and it fails exactly when the graph has a cycle — so the algorithm is also a cycle detector. This program implements Kahn's algorithm, the in-degree-based version, and it is the cleanest small example of the graph-as-arrays representation the [dijkstra](#) example introduced, used for a different traversal.

```
function add_edge(u, v, k) {
    if (!(u in nodes_seen)) { nodes_seen[u] = 1; indeg[u] = 0; outdeg[u] = 0 }
    if (!(v in nodes_seen)) { nodes_seen[v] = 1; indeg[v] = 0; outdeg[v] = 0 }
    k = ++outdeg[u]
    adj[u, k] = v
    indeg[v]++
}
```

The graph lives in four arrays: `nodes_seen` is the set of all nodes, `indeg[node]` counts incoming edges, `adj[u, k]` is the `k`-th successor of `u`, and `outdeg[u]` counts successors. `add_edge` registers both endpoints on first sight — the `!(u in nodes_seen)` guard initializing a node's counters exactly once — then records the edge and bumps the destination's in-degree. Kahn's algorithm then repeatedly emits a node whose in-degree has fallen to zero (it has no unsatisfied prerequisites) and decrements its successors' in-degrees, as if removing it from the graph:

```
while (1) {
    pick = ""
    for (n in nodes_seen) {
        if (!(n in done) && indeg[n] == 0) { pick = n; break }
    }
    if (pick == "") break
    out = out sep pick; sep = " "
    done[pick] = 1
    emitted++
    for (k = 1; k <= outdeg[pick]; k++) {
        v = adj[pick, k]
        indeg[v]--
    }
}

if (emitted == total) print out
else print "CYCLE"
```

Each round scans for the first not-yet-emitted node with zero in-degree, under `PROCINFO["sorted_in"] = "@ind_str_asc"` so the scan visits nodes in sorted order and ties break deterministically — the program's header notes this is “clarity over speed,” re-scanning the ready set each round rather than maintaining a queue, which is the right call for an example meant to be read. When no zero-in-degree node remains, the loop stops; if it has emitted every node the order is valid, and if some nodes remain unemitted they must lie on a cycle, which is exactly the `emitted == total` test that turns a topological sort into a cycle detector. The whole graph algorithm — representation, traversal, and the cycle-detection corollary — is again nothing but associative arrays and the `(node, index)` adjacency convention.

huffman.awk: a priority queue, a tree, and a round trip

The richest example in the tour is Huffman coding — build an optimal prefix code from character frequencies, emit the code table, encode the text, and decode it back — and it brings together nearly every technique the tour has shown: a frequency table, a priority queue built by hand, a binary tree in parallel arrays, recursion to assign codes, and a tree walk to decode. It is the capstone, and it earns the place.

```
function extract_min( best, key, i) {
    best = 0; key = ""
    for (i in active) {
        k = sprintf("%012d %s %d", fr[i], lo[i], i)
        if (best == 0 || k < key) { best = i; key = k }
    }
    delete active[best]
    return best
}
```

The tree is five parallel arrays indexed by node id: `ch[i]` the leaf character (empty for internal nodes), `fr[i]` the frequency, `lo[i]` the lexicographically smallest character in the subtree (the tie-breaker), and `lc[i]/rc[i]` the left and right children. `extract_min` is the priority-queue pop, and it uses a clever trick worth pausing on: rather than maintain a heap, it linearly scans the `active` set and compares nodes by a *composite string key* — `sprintf("%012d %s %d", fr[i], lo[i], i)` — which zero-pads the frequency to twelve digits so that string comparison of the key orders first by frequency, then by the subtree's minimum character, then by node id. Encoding the multi-field sort order into one zero-padded sortable string is a classic awk idiom for “sort by several keys” when you only have string comparison, and here it makes the priority queue's order fully deterministic, which is what makes the output byte-stable. The build loop extracts the two minimum nodes, creates an internal parent whose frequency is their sum and whose `lo` is the smaller of theirs, and marks it active, until one node — the root — remains.

```
function assign_codes(node, prefix) {
    if (lc[node] == 0 && rc[node] == 0) {
        code[ch[node]] = (prefix == "" ? "0" : prefix) # single-char text edge case
        return
    }
    if (lc[node] != 0) assign_codes(lc[node], prefix "0")
    if (rc[node] != 0) assign_codes(rc[node], prefix "1")
}
```

`assign_codes` recurses the tree, accumulating a bit-string `prefix` — `0` for a left branch, `1` for a right — and recording each leaf's code, with a guard for the degenerate single-character input where the root is itself a leaf and the natural prefix would be empty. This is the recursion-with-an-accumulator pattern, the prefix threaded down as a parameter rather than held in a global, so each path to a leaf spells that leaf's code. Encoding is then a lookup-and-concatenate over the text, and decoding walks the tree bit by bit from the root, emitting a character and resetting to the root each time it reaches a leaf — the decoder is the encoder's mirror, and the round trip (`ENC` then `DEC` recovering the original) is the program's own proof of correctness. The final `COMPRESSION` line reports the original size in bits against the encoded size. A complete, optimal, self-verifying compressor — priority queue, tree construction, code assignment, and a decode round trip — in about seventy lines of awk, on nothing but associative arrays, `sprintf` for the composite sort key, and recursion. If any single example justifies the book's recurring claim that awk's small model reaches far past its one-liner reputation, it is this one.

fenwick.awk: bit tricks and the binary indexed tree

The Fenwick tree, or binary indexed tree, supports point updates and prefix-sum queries in logarithmic time, and it earns its place in the tour for a different reason than the others: it is the example where awk's *bitwise* builtins do real work, computing the low-bit of an index that drives the whole structure.

```

function lowbit(x) { return x - and(x, x - 1) } # gawk rejects negative bit-op args

function bit_update(i, v) {
    while (i <= N) { bit[i] += v; i += lowbit(i) }
}

function bit_prefix(i, s) {
    s = 0
    while (i > 0) { s += bit[i]; i -= lowbit(i) }
    return s
}

```

The whole tree is one array, `bit[1..N]`, and the magic is `lowbit(x)` — the value of the lowest set bit of `x`, computed as `x - and(x, x - 1)`. The identity is worth seeing: `x - 1` clears the lowest set bit and sets everything below it, so `and(x, x - 1)` is `x` with its lowest set bit removed, and subtracting that from `x` leaves exactly the lowest set bit. This is the bit trick at the heart of the Fenwick tree, and it uses the `and` builtin from Chapter 11, which the program’s own comment flags as the reason the indices must stay positive — gawk, and awkrs matching it, makes the bitwise operators fatal on negative arguments. `bit_update` walks up the tree adding `lowbit(i)` to the index each step, and `bit_prefix` walks down subtracting it, visiting only the logarithmically-many nodes that cover a prefix. A range sum is the difference of two prefix sums, computed inline in the `RSUM` rule. The Fenwick tree is the example that shows awk is not confined to decimal arithmetic and string work: the bitwise builtins are first-class, they implement the index arithmetic exactly, and a logarithmic-time prefix-sum structure drops out of one array and one bit trick.

z_function.awk: linear-time string matching

The Z-function computes, for every position in a string, the length of the longest substring starting there that matches a prefix of the whole string — and from it falls out linear-time pattern matching. It is the most algorithmically dense string example in the tour, and it shows awk sustaining a subtle linear-time invariant across a loop.

```

function z_array(s, n, i, l, r, k) {
    n = length(s)
    delete z
    z[1] = n
    l = 0; r = 0
    for (i = 2; i <= n; i++) {
        if (i <= r) z[i] = (r - i + 1 < z[i - l + 1]) ? r - i + 1 : z[i - l + 1]
        else z[i] = 0
        while (i + z[i] <= n && substr(s, z[i] + 1, 1) == substr(s, i + z[i], 1)) z[i]++
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1 }
    }
    return n
}

```

The `z` array holds the result, and `[l, r]` track the rightmost Z-box found so far — the interval that the algorithm reuses to avoid redundant comparisons. For each position `i`, if it falls inside the current box (`i <= r`) the algorithm seeds `z[i]` from the already-computed value at the mirror position `z[i - l + 1]`, clamped to the box’s remaining width; otherwise it starts from zero. The inner `while` then extends the match character by character with `substr` comparisons, and the box is advanced when the new match reaches further right. The linear-time guarantee comes from the box reuse: the inner loop only ever extends past `r`, and `r` only moves forward, so the total work is bounded. `find_matches` applies this to pattern matching by the standard trick of forming `pattern "#" text` with a separator the pattern cannot contain, and reporting every position whose Z-value reaches the pattern length. The driver even parses an `X in Y` syntax with `index` and `sub` to split pattern from text. It is a genuinely advanced string algorithm — the kind found in competitive-programming libraries — transcribed faithfully into awk, and it works because awk’s `substr`-based character access and its

arrays are enough to maintain the Z-box invariant exactly.

gcd_extended.awk: number theory and the multiple-return idiom

The extended Euclidean algorithm computes not just the greatest common divisor of two numbers but the Bézout coefficients x and y such that $a*x + b*y = \text{gcd}(a, b)$, and from those a modular inverse. It is the cleanest example of the multiple-return idiom the book has mentioned — a function that must yield three values returns one and writes the other two to globals — done iteratively to avoid recursion entirely.

```
function ext_gcd(a, b,    old_r, r, old_s, s, old_t, t, q, tmp) {
    old_r = a; r = b
    old_s = 1; s = 0
    old_t = 0; t = 1
    while (r != 0) {
        q = int(old_r / r)
        tmp = r; r = old_r - q * r; old_r = tmp
        tmp = s; s = old_s - q * s; old_s = tmp
        tmp = t; t = old_t - q * t; old_t = tmp
    }
    gx = old_s; gy = old_t
    return old_r
}
```

The function maintains three parallel pairs of coefficients and advances them by the same quotient q each iteration — the standard iterative extended-Euclid recurrence, with tmp holding the old value across the simultaneous update that awk, lacking tuple assignment, must spell out one variable at a time. When the remainder reaches zero, old_r is the gcd and old_s/old_t are the coefficients, which the function returns as old_r and stashes in the globals gx and gy — the same out-parameter pattern *dijkstra.awk* used for its heap pop, here yielding a triple. *mod_inv* builds on it: the modular inverse of $a \bmod m$ exists exactly when their gcd is one, and is then the Bézout coefficient gx reduced into $[0, m)$ with the positive-modulo idiom $(inv \% m + m) \% m$ that corrects awk's sign-following $\%$. The program is a compact number-theory library — gcd, Bézout, modular inverse — and it makes concrete that the multiple-return idiom is not a workaround but awk's idiomatic way to return a tuple: the primary result through *return*, the rest through agreed globals.

n_queens.awk: backtracking and the undo discipline

The N-queens solver places N non-attacking queens on an $N \times N$ board, and it is the tour's cleanest demonstration of *backtracking* — try a choice, recurse, and on the way back undo it — together with the constant-time pruning that makes the search feasible.

```

function solve(row, c) {
    if (row > N) {
        sols++
        for (r = 1; r <= N; r++) {
            line = ""
            for (c = 1; c <= N; c++) line = line ((board[r] == c) ? "Q" : ".")
            print line
        }
        print ""
        return
    }
    for (c = 1; c <= N; c++) {
        if ((c in col) || ((row + c) in diag1) || ((row - c) in diag2)) continue
        board[row] = c
        col[c] = 1; diag1[row + c] = 1; diag2[row - c] = 1
        solve(row + 1)
        delete col[c]; delete diag1[row + c]; delete diag2[row - c]
    }
}

```

The recursion places one queen per row. The genius is the three *witness sets* that make conflict-checking constant-time: `col[c]` marks an occupied column, `diag1[row + c]` an occupied \ diagonal (the sum `row + c` is invariant along it), and `diag2[row - c]` an occupied / diagonal (the difference is invariant). Before placing a queen at `(row, c)`, the `continue` guard checks all three with membership tests that do not create keys — the Chapter 8 subtlety used deliberately. The undo discipline is the heart of backtracking: after recursing, the function `delete`s exactly the three keys it added, restoring the witness sets to their pre-placement state so the next column choice sees a clean board. This add-recurse-delete symmetry is what lets one set of global arrays serve every branch of the search tree without copying. When `row` passes `N`, a full solution has been found and is printed. The associative array as a set, membership-without-creation as a constant-time conflict test, and `delete` as the undo on the way back up the recursion — N-queens is backtracking distilled, and `awk` has exactly the primitives it needs.

boyer_moore.awk: right-to-left search with a skip table

Boyer-Moore is the string-search algorithm that can run *sub-linearly* by skipping ahead, and this example implements its bad-character heuristic — the single-table classic — to show the counterintuitive right-to-left scan that drives the skips.

```

function bm_search(t, n, m, s, j, ch, shift, hits, sep) {
  m = patlen; n = length(t)
  if (m == 0 || m > n) return ""
  hits = ""; sep = ""
  s = 0 # 0-based shift of pattern over text
  while (s <= n - m) {
    j = m
    while (j >= 1 && substr(p, j, 1) == substr(t, s + j, 1)) j--
    if (j == 0) {
      hits = hits sep (s + 1); sep = " "
      s++
    } else {
      ch = substr(t, s + j, 1)
      shift = j - ((ch in last) ? last[ch] : 0)
      if (shift < 1) shift = 1
      s += shift
    }
  }
  return hits
}

```

The preprocessing (not shown) builds `last[c]`, the rightmost position of each character in the pattern. The search aligns the pattern at shift `s` and compares *right to left* — the inner `while` walks `j` down from the pattern's end while characters match. On a mismatch at position `j`, the bad-character rule computes how far to jump: `shift = j - last[ch]`, where `ch` is the text character that failed, so the pattern slides until that character lines up with its rightmost occurrence in the pattern (or past it entirely if absent, since `last[ch]` is then `0`). The `if (shift < 1) shift = 1` guard guarantees forward progress. The payoff is that a character in the text not in the pattern at all lets the search skip the entire pattern width in one step — which is why Boyer-Moore is fast in practice. It is a subtle algorithm, and the example renders it faithfully with nothing but `substr` for character access and an associative array for the skip table.

lzw.awk: dictionary compression and the tricky decode

LZW compression builds a dictionary of substrings as it goes, and its decoder has a famous edge case — a code that refers to a dictionary entry not yet built — that this example handles correctly. It is the tour's most interesting codec because the encode and decode are not quite mirror images.

```

function lzw_enc(s, n, dict, next_code, w, i, c, wc, codes, sep) {
  delete dict
  for (i = 0; i < 256; i++) dict[chr(i)] = i
  next_code = 256
  n = length(s)
  if (n == 0) return ""
  w = substr(s, 1, 1)
  codes = ""; sep = ""
  for (i = 2; i <= n; i++) {
    c = substr(s, i, 1)
    wc = w c
    if (wc in dict) { w = wc; continue }
    codes = codes sep dict[w]; sep = " "
    dict[wc] = next_code++
    w = c
  }
  codes = codes sep dict[w]
  return codes
}

```

The encoder seeds the dictionary with all 256 single bytes, then greedily extends the current string `w` by

one character at a time: while `w c` is in the dictionary it keeps growing, and when `w c` is new it emits the code for `w`, adds `w c` to the dictionary as the next code, and restarts `w` at `c`. The decoder (in the program's `lzw_dec`) rebuilds the same dictionary from the codes — but it faces the edge case where a code arrives that equals `next_code`, one the decoder has not yet added. This happens when the encoder used a string immediately after defining it, and the standard resolution, which the example codes exactly, is that the entry must be the previous string plus its own first character (`prev substr(prev, 1, 1)`). Handling this case is what separates a correct LZW decoder from a broken one, and seeing it in `awk` — `else if (k == next_code) entry = prev substr(prev, 1, 1)` — makes the subtlety concrete. The round trip (encode then decode recovering the original) is the program's own correctness check, and the whole compressor is built from associative-array dictionaries keyed by string.

manacher.awk: the longest palindrome in linear time

Manacher's algorithm finds the longest palindromic substring in linear time, and it is the most intricate string algorithm in the tour — its linear bound depends on reusing previously-computed palindrome radii across a moving center, the same mirror trick the Z-function used, applied to palindromes.

```
function manacher(s, T, n, i, j, center, right, mirror, best_len, best_center,
    < raw_start) {
    T = "|"
    for (i = 1; i <= length(s); i++) T = T substr(s, i, 1) "|"
    n = length(T)
    delete p
    center = 0; right = 0
    for (i = 1; i <= n; i++) {
        if (right > i) {
            mirror = 2 * center - i
            p[i] = (right - i < p[mirror]) ? right - i : p[mirror]
        } else {
            p[i] = 0
        }
        while (i - p[i] - 1 >= 1 && i + p[i] + 1 <= n \
            && substr(T, i - p[i] - 1, 1) == substr(T, i + p[i] + 1, 1)) {
            p[i]++
        }
        if (i + p[i] > right) { center = i; right = i + p[i] }
        ...
    }
}
```

The first move is the classic pre-processing: insert a separator `|` between every character and at both ends, transforming `s` into `T`, so that every palindrome — odd or even length in the original — becomes odd-length around a center in `T`, eliminating the even/odd case split. The array `p[i]` is the palindrome half-radius at center `i`, and `[center, right]` track the rightmost palindrome found. For each position inside the current rightmost palindrome, `p[i]` is seeded from its mirror `p[2*center - i]`, clamped to the remaining width — the reuse that avoids redundant comparisons — and then the inner `while` extends the palindrome character by character past what was known. The linear bound, as with the Z-function, comes from `right` only ever moving forward. Recovering the original substring from the transformed index requires the `int((best_center - best_len) / 2) + 1` arithmetic that maps a position in `T` back to `s`. It is competitive-programming-grade string processing, and it runs on `awk`'s `substr` and one array.

segment_tree.awk: an iterative range-query tree

The segment tree answers range-sum queries and point updates in logarithmic time, and this example builds the *iterative* (bottom-up) variant — the one without recursion — which makes the tree's array layout and its

climb-from-the-leaves structure especially clear.

```
function range_sum(l, r, sum, a, b) {
    sum = 0
    a = N + l - 1
    b = N + r - 1
    while (a <= b) {
        if (a % 2 == 1) { sum += t[a]; a++ }
        if (b % 2 == 0) { sum += t[b]; b-- }
        a = int(a / 2)
        b = int(b / 2)
    }
    return sum
}
```

The tree is one array `t[1 .. 2N-1]`, with the `N` leaves at indices `N` through `2N-1` and each internal node `i` holding the sum of its children `t[2i]` and `t[2i+1]` — the implicit-binary-tree layout where a node's children are at double its index, the same indexing `dijkstra.awk`'s heap used. The build sums children into parents from the leaves up. `range_sum` is the elegant iterative query: it starts at the two leaf endpoints and climbs toward the root, and at each level it *includes* a boundary node into the sum exactly when it is a right child (`a % 2 == 1`) on the left or a left child (`b % 2 == 0`) on the right — because those are the nodes whose whole subtree lies inside the query range — then moves both pointers up a level with `int(a / 2)`. Point updates (`point_set`, `point_add`) climb the same way, updating each ancestor's sum. The iterative form fits `awk` especially well because `awk` has no native recursion-free tree type anyway — the array *is* the tree — and the index arithmetic that a lower-level language would write with pointers is here just `2 * i` and `int(i / 2)`.

sql_like.awk: a query engine in awk

The most elaborate program in the tour is a small SQL-like query engine: it parses a `SELECT ... WHERE ... GROUP BY ... ORDER BY` query, then runs it over a CSV body, supporting projection, filtering, aggregation, and sorting. It is `awk` doing the job `awk` is least expected to do — interpreting a query language — and it ties together the tokenizer, the pattern-action dispatch, and the parallel-arrays aggregation from across the tour.

```
function tok_query(s, i, c, t, n) {
    qn = 0
    n = length(s)
    i = 1
    while (i <= n) {
        c = substr(s, i, 1)
        if (c == " " || c == "\t") { i++; continue }
        if (c == ",") { qt[++qn] = ","; i++; continue }
        t = ""
        while (i <= n && substr(s, i, 1) != " " && substr(s, i, 1) != "\t" && substr(s, i, 1) != ",") {
            t = t substr(s, i, 1); i++
        }
        qt[++qn] = t
    }
}
```

The query is tokenized into the global array `qt` by the same hand-written-lexer pattern as the calculators — skip whitespace, emit commas as their own tokens, accumulate everything else into word tokens. A set of `parse_*` functions then consume the token stream clause by clause, each returning the index where it stopped so the next clause picks up there — `parse_select` reads the column list up to a keyword, `parse_where` reads the comparison triples, `parse_group` and `parse_order` read their clauses. This is recursive descent's iterative cousin: a sequence of clause parsers threading a shared position, exactly the structure of a real query parser.

```

function row_passes( i, ci, v, op, rv, lhs) {
  for (i = 1; i <= nwh; i++) {
    ci = col_idx(wh_col[i]); if (ci == 0) err("unknown WHERE col " wh_col[i])
    lhs = row[ci]; op = wh_op[i]; rv = wh_val[i]
    if (op == "==") { if (lhs != rv && lhs + 0 != rv + 0) return 0 }
    else if (op == "<") { if (!(lhs + 0 < rv + 0)) return 0 }
    ...
    else if (op == "~") { if (!(lhs ~ rv)) return 0 }
  }
  return 1
}

```

`row_passes` evaluates the `WHERE` clause against the current row, dispatching on each operator — and notice the `==` case tries both a string compare *and* a numeric compare (`lhs != rv && lhs + 0 != rv + 0`), the engine's pragmatic way to match either a textual or a numeric equality, and the `~` operator threads straight through to awk's own regex match so `WHERE col ~ pattern` is a real regex filter for free. The main record rule loads each data row, skips it if it fails `row_passes`, and then either accumulates it into the grouped aggregates (the parallel-arrays idiom — `grp_count`, `grp_sum` keyed by group value, exactly as `csv_pivot.awk`) or stores it for projection. The `END` rule emits the result, sorting with `asort` or a hand-written insertion sort depending on the path, and computing `AVG` as `grp_sum / grp_count`. It is a teaching-grade engine — no quoted-comma CSV, comma `FS` — but it is a *complete* one: parse, filter, group, aggregate, order, project. That a query interpreter fits in one awk file, built from the same tokenizer, dispatch, and aggregation primitives the rest of the tour used, is the strongest possible statement of the book's thesis: awk's small, composable vocabulary, fully internalized, reaches astonishingly far.

base64.awk: encoding with pure byte arithmetic

Base64 encoding packs three input bytes into four output characters, and this example does it with nothing but integer arithmetic — no `system`, no external tool, no bit operators even — using division and modulo to slice the bits. It is the clearest demonstration that awk can do binary-level encoding with ordinary `/`, `%`, and `int`.

```

function b64_encode(s, n, i, b1, b2, b3, out, t) {
  n = length(s); i = 1
  out = ""
  while (i <= n) {
    b1 = ord(substr(s, i, 1)) + 0
    b2 = (i + 1 <= n) ? ord(substr(s, i + 1, 1)) + 0 : -1
    b3 = (i + 2 <= n) ? ord(substr(s, i + 2, 1)) + 0 : -1
    t = enc[int(b1 / 4)]
    if (b2 == -1) {
      t = t enc[(b1 % 4) * 16] "=="
    } else if (b3 == -1) {
      t = t enc[(b1 % 4) * 16 + int(b2 / 16)]
      t = t enc[(b2 % 16) * 4] "="
    } else {
      t = t enc[(b1 % 4) * 16 + int(b2 / 16)]
      t = t enc[(b2 % 16) * 4 + int(b3 / 64)]
      t = t enc[b3 % 64]
    }
    out = out t
    i += 3
  }
  return out
}

```

The `BEGIN` rule (not shown) builds four tables: `enc[0..63]` mapping a six-bit value to its Base64 character, `dec` the inverse, and `chr/ord` the byte-to-character pair. The encoder reads three bytes at a time and computes

the four six-bit groups by arithmetic rather than bit-shifting: `int(b1 / 4)` is the top six bits of the first byte (dividing by four drops two bits), `(b1 % 4) * 16 + int(b2 / 16)` splices the low two bits of `b1` with the high four of `b2`, and so on — every shift expressed as a multiply or divide by a power of two, every mask as a modulo. The padding logic handles the two tail cases (one or two leftover bytes) with the `=` and `==` padding the standard requires. The decoder reverses it, reading four characters and reconstructing three bytes, recognizing `=` padding by the sentinel `-1`. That Base64 — a binary-to-text codec — is implementable in awk with only `/`, `%`, and `int`, no bitwise builtins at all, is a quiet demonstration that powers-of-two arithmetic *is* bit manipulation, and awk has all of it.

shunting_yard.awk: infix to postfix and back to a value

The shunting-yard algorithm is Dijkstra's method for converting infix arithmetic to postfix, respecting operator precedence and associativity, and this example implements the full pipeline — tokenize, convert to RPN, then evaluate the RPN — making it a companion to `calc_rd.awk`'s recursive-descent approach to the same problem.

```
function to_rpn( i, t, top, out_n, sep) {
    out_n = 0; sp = 0
    for (i = 1; i <= tn; i++) {
        t = tk[i]
        if (t ~ /^[0-9.]+$/ ) { out[++out_n] = t; continue }
        if (t == "(") { stk[++sp] = t; continue }
        if (t == ")") {
            while (sp > 0 && stk[sp] != "(") { out[++out_n] = stk[sp--] }
            if (sp == 0) { print "mismatched "; return 0 }
            sp--
            continue
        }
        if (is_op(t)) {
            while (sp > 0 && is_op(stk[sp]) \
                && (prec(stk[sp]) > prec(t) \
                    || (prec(stk[sp]) == prec(t) && !rassoc(t)))) {
                out[++out_n] = stk[sp--]
            }
            stk[++sp] = t
            continue
        }
    }
    while (sp > 0) {
        if (stk[sp] == "(") { print "mismatched ("; return 0 }
        out[++out_n] = stk[sp--]
    }
    rpn_n = out_n
    return 1
}
```

The algorithm maintains an operator stack `stk` and an output queue `out`, both arrays with integer top-of-stack indices. Numbers go straight to the output; an operator pops higher-or-equal-precedence operators to the output before pushing itself — the condition `prec(stk[sp]) > prec(t) || (prec(stk[sp]) == prec(t) && !rassoc(t))` is the precise rule that makes `^` right-associative (it does *not* pop an equal-precedence `^`) while `+`, `-`, `*`, `/` are left-associative (they do). Parentheses push and, on a closing paren, pop until the matching open, with mismatch detection on both sides. The helper functions `is_op`, `prec`, and `rassoc` encode the operator table, and the tokenizer distinguishes unary minus from binary minus by tracking whether the previous token was an operator — the `u-` token. The `eval_rpn` function (not shown) then evaluates the postfix with a value stack, the same stack-machine evaluation as `rpn.awk`. Two different parsing philosophies — recursive descent in `calc_rd.awk`, shunting-yard here — solve the same operator-precedence problem, and seeing both in awk shows the language is comfortable with either, on nothing but arrays used as stacks and the

regex-classified tokenizer.

modexp.awk: modular exponentiation and Miller-Rabin

The final number-theory example computes modular exponentiation and uses it for a deterministic Miller-Rabin primality test — the algorithm real cryptographic libraries use — showing awk doing serious modular arithmetic within the precision limits its double-precision numbers allow.

```
function pow_mod(base, ex, m, r) {
    r = 1
    base = base % m
    while (ex > 0) {
        if (ex % 2 == 1) r = mul_mod(r, base, m)
        base = mul_mod(base, base, m)
        ex = int(ex / 2)
    }
    return r
}
```

`pow_mod` is exponentiation by squaring done modularly: it walks the exponent's binary digits (via `ex % 2` and `int(ex / 2)`), multiplying the running result by the current squared base whenever the bit is set, keeping every intermediate reduced mod `m` so the numbers never grow. This is the standard fast-exponentiation algorithm, and it is the core of both modular power queries and the primality test. `is_prime` runs deterministic Miller-Rabin: it factors $n - 1$ as $d * 2^s$, then tests a fixed set of witnesses (2, 3, 5, ... 37) that the program's comment notes is deterministic for all n below $3.3e24$, far beyond the f64-safe integer range. Each witness either confirms possible primality or proves compositeness via `miller_witness`, which checks whether the witness raised to `d` reaches 1 or -1 through the squaring sequence. The program's header is careful about the precision boundary — inputs must keep $(a * a) \% m$ within f64's exact integer range — which is exactly the honesty the book has praised throughout: the algorithm is correct *within stated limits*, and for larger numbers you would reach for -M bignum mode (Chapter 19) and the `bigint_mul.awk` and `modexp` examples that exercise it. A correct primality test, the kind cryptography depends on, written in awk with ordinary modular arithmetic — bounded honestly by the precision of a double.

brainfuck.awk: a complete language interpreter

The Brainfuck interpreter is the tour's proof that awk can be a *host* language — it implements another language's full execution model: an instruction pointer, a data tape, eight operations, and balanced loops. It is a compact but complete interpreter, and it shows the bracket-matching pre-pass that makes loops constant-time.

```

function bf_run(src, prog_in, ip, dp, n, c, j, sp, ii, out) {
  n = length(src)
  sp = 0
  # Pre-compute matching brackets so loops are O(1).
  for (ip = 1; ip <= n; ip++) {
    c = substr(src, ip, 1)
    if (c == "[") { stack[++sp] = ip; continue }
    if (c == "]") {
      if (sp == 0) return "ERR: unbalanced ]"
      j = stack[sp--]
      jmp_fwd[j] = ip
      jmp_back[ip] = j
    }
  }
  if (sp != 0) return "ERR: unbalanced ["
  ...
}

```

The interpreter has two phases. First, a pre-pass scans the source once with a stack to match every `[` with its `]`, recording `jmp_fwd` and `jmp_back` so that a loop jump is an O(1) array lookup rather than a scan — the standard bytecode-VM trick of pre-resolving jump targets, here applied to Brainfuck inside awk. Imbalanced brackets are caught here. Second, the execution loop walks the instruction pointer `ip`, dispatching on each of the eight commands: `>` and `<` move the data pointer, `+` and `-` adjust the current cell modulo 256 (the `+ 256` before `% 256` keeping it positive), `.` outputs a character via `sprintf("%c", ...)`, `,` reads from the buffered input, and `[/]` jump using the precomputed tables when the current cell is zero or nonzero. The tape is a sparse associative array `tape[dp]`, so it is effectively unbounded — cells spring into existence as the pointer reaches them, the natural awk representation of an infinite tape. That a Turing-complete language's interpreter fits in fifty lines of awk, with correct loop semantics and an unbounded tape, is the strongest form of the claim that awk's model hosts arbitrary computation: it can run a programming language.

diff_lcs.awk: longest common subsequence and a recursive traceback

The diff example computes a unified-style diff between two blocks of lines using the longest-common-subsequence DP, and its standout technique is a *recursive* traceback that emits the diff in order without an explicit stack — recursion used as the output-ordering mechanism, not just the computation.

```

function emit(i, j) {
  if (i > 0 && j > 0 && a[i] == b[j]) {
    emit(i - 1, j - 1)
    printf "  %s\n", a[i]
    return
  }
  if (j > 0 && (i == 0 || c[i, j - 1] >= c[i - 1, j])) {
    emit(i, j - 1)
    printf "+ %s\n", b[j]
    added++
    return
  }
  if (i > 0 && (j == 0 || c[i, j - 1] < c[i - 1, j])) {
    emit(i - 1, j)
    printf "- %s\n", a[i]
    removed++
    return
  }
}

```

The `lcs` function (not shown) fills the standard DP table `c[i, j]` — the length of the longest common subsequence of the first `i` lines of A and the first `j` of B — in a `SUBSEP` two-dimensional array, exactly like `levenshtein.awk` and `knapsack.awk`. The elegant part is `emit`, the traceback: it recurses *first* (toward the top-left of the table) and *then* prints, so the print statements unwind in forward order even though the traceback walks backward — the recursion stack reverses the order for free. At each cell it decides, by comparing the table values, whether the two lines matched (context, `,`), whether B's line was an addition (`+`), or whether A's line was a removal (`-`), the same three-way DP-traceback decision `knapsack.awk` made for items. The input is split into the A and B blocks by two range-like marker rules. Using recursion to emit a backward traceback in forward order is a technique worth keeping — it turns the awkward “build a list then reverse it” into a single recursive walk — and it is exactly the kind of move that the call stack makes natural and an explicit loop makes fiddly.

suffix_array.awk: build, sort, and the LCP array

The suffix array lists every suffix of a string in sorted order, the foundation of many string-indexing algorithms, and this example builds it the simple way — generate all suffixes, sort them — while also computing the longest-common-prefix array between consecutive suffixes. It is the example where `asort` does the heavy lifting and the composite-key-in-a-string trick reappears.

```
{
  s = $0
  n = length(s)
  delete suf
  for (i = 1; i <= n; i++) suf[i] = sprintf("%s\t%d", suffix_at(s, i), i)
  asort(suf)
  sa = ""; lp = ""
  for (i = 1; i <= n; i++) {
    split(suf[i], parts, "\t")
    sa = sa (i == 1 ? "" : " ") parts[2]
    ...
  }
}
```

The build is three steps. First, every suffix is generated with `substr(s, i)` and stored in `suf[i]` — but not as the bare suffix: as `sprintf("%s\t%d", suffix, i)`, the suffix *tab-joined with its starting position*. This is the key idea: when `asort` sorts the array, it sorts by the whole string, which begins with the suffix, so the suffixes order lexicographically — and because each carries its original position after a tab, splitting the sorted entry back on the tab recovers where each suffix started. It is the pack-the-payload-into-the-sort-key technique, the same instinct as `huffman.awk`'s zero-padded composite key, here pairing a suffix with its index so a value-sort doubles as an index-sort. The LCP between consecutive sorted suffixes is computed by the `lcp` helper, a simple character-by-character common-prefix count. The program is honest in its header that this is the trivial $O(n^2 \log n)$ construction rather than a linear-time one — “plenty fast for the demo inputs and easy to read” — the same honest-about-complexity posture the book has praised. A sorted suffix array and its LCP array, built from `substr`, `asort`, and the pack-into-the-key trick: the data structure behind full-text search, in awk.

rabin_karp.awk: the rolling hash

Rabin-Karp is the string search that hashes a window of text and slides it, updating the hash in constant time rather than recomputing it — the *rolling hash*, a technique distinct from the comparison-based KMP and Boyer-Moore. This example implements it with a polynomial hash and the modular arithmetic to roll the window.

```

function rk_search(pat, txt, m, n, i, ph, th, top, hits, sep, ok) {
    m = length(pat); n = length(txt)
    if (m == 0 || m > n) return ""
    ph = 0; th = 0
    for (i = 1; i <= m; i++) {
        ph = (ph * BASE + ord[substr(pat, i, 1)]) % MOD
        th = (th * BASE + ord[substr(txt, i, 1)]) % MOD
    }
    top = pow_mod(BASE, m - 1, MOD)
    hits = ""; sep = ""
    for (i = 1; i + m - 1 <= n; i++) {
        if (ph == th) {
            if (substr(txt, i, m) == pat) { hits = hits sep i; sep = " " }
        }
        if (i + m <= n) {
            th = (th - ord[substr(txt, i, 1)] * top) % MOD
            if (th < 0) th += MOD
            th = (th * BASE + ord[substr(txt, i + m, 1)]) % MOD
        }
    }
    return hits
}

```

The hash is a polynomial: each character contributes its ordinal times a power of the base `BASE`, all modulo a prime `MOD`. The pattern's hash and the first window's hash are computed once; then the window *rolls* — to advance by one character, subtract the leftmost character's contribution (`ord[...] * top`, where `top` is `BASE^(m-1)`), shift up by multiplying by `BASE`, and add the new rightmost character. That update is $O(1)$ regardless of the pattern length, which is Rabin-Karp's whole point. The crucial correctness detail, which the program's header calls out, is that a hash match is only a *candidate*: hashes can collide, so every `ph == th` is verified with an actual `substr(txt, i, m) == pat` comparison before being reported — so the output is byte-exact even though the search is hash-driven. The constants are chosen, the comment notes, so the intermediate products stay within `f64`'s exact-integer range. The rolling hash is a genuinely different idea from the other string searches in the tour — it trades exactness for speed and then buys the exactness back with a verification — and `awk`'s modular arithmetic expresses it directly.

mandelbrot.awk: floating-point iteration and ASCII art

The Mandelbrot renderer is the tour's floating-point showcase: it iterates the complex-plane escape-time formula over a grid of pixels and shades each by how fast it escapes, producing ASCII art. It is also the example most relevant to Chapter 17's JIT, because its inner loop is exactly the tight numeric kernel the JIT compiles.

```

for (px = 0; px < W; px++) {
    cx = xmin + (xmax - xmin) * px / (W - 1)
    x = 0; y = 0; n = 0
    while (n < ITER && x * x + y * y <= 4) {
        x_new = x * x - y * y + cx
        y      = 2 * x * y + cy
        x      = x_new
        n++
    }
    if (n == ITER) idx = 1
    else {
        idx = int(n * (ramp_n - 1) / ITER) + 1
        if (idx > ramp_n) idx = ramp_n
    }
    line = line substr(ramp, idx, 1)
}

```

The math is the Mandelbrot iteration $z \rightarrow z^2 + c$ over complex $z = x + yi$, written out in real and imaginary parts: $x_{\text{new}} = x^2 - y^2 + cx$ and $y = 2xy + cy$ are the real and imaginary components of squaring and adding. The escape test $x*x + y*y \leq 4$ checks whether the point has left the radius-two disc; the iteration count n at escape (or the cap `ITER`) maps through the shading ramp " .:-+*#%@" to a character, lighter for fast escape and denser for slow. This is pure double-precision floating-point arithmetic in a tight loop — $x * x - y * y + cx$ run up to `ITER` times per pixel, for $W \times H$ pixels — which is precisely the chunk shape the fusevm JIT of Chapter 17 is eligible to compile to native code, since it touches only numeric scalars, arithmetic, and a comparison. A program like this, run at a large resolution and iteration count, is where the JIT's fourteen-to-a-hundred-fold speedup on numeric loops would actually appear in an awk program — and it is bit-for-bit identical whether the JIT engages or not. The Mandelbrot set rendered as text, from awk's ordinary floating-point arithmetic and `substr` for the shading lookup.

gauss_elim.awk: linear algebra with partial pivoting

Gaussian elimination solves a system of linear equations $Ax = b$, and this example implements the numerically-careful version with *partial pivoting* — choosing the largest available pivot to limit error — over a matrix stored, naturally, in a `SUBSEP` two-dimensional array. It is the tour's linear-algebra entry, and it shows awk handling matrices and floating-point row operations.

```

for (k = 1; k <= N; k++) {
    best = k
    for (i = k + 1; i <= N; i++) {
        if (((a[i, k] < 0) ? -a[i, k] : a[i, k]) > ((a[best, k] < 0) ? -a[best, k] : a[best, k])) best = i
    }
    if (a[best, k] == 0) { print "SINGULAR"; exit 1 }
    if (best != k) {
        for (c = 1; c <= N + 1; c++) { t = a[k, c]; a[k, c] = a[best, c]; a[best, c] = t }
    }
    for (i = k + 1; i <= N; i++) {
        f = a[i, k] / a[k, k]
        for (c = k; c <= N + 1; c++) a[i, c] -= f * a[k, c]
    }
}

```

The augmented matrix `a[row, col]` holds A in its first N columns and b in column $N+1$, the standard augmented form, in one `SUBSEP` array. Forward elimination walks the columns, and for each it first finds the *pivot* — the row with the largest-magnitude entry in that column (the inline $(x < 0) ? -x : x$ computing absolute value, since awk has no `abs` builtin) — and swaps it into place. Partial pivoting is what keeps the elimination numerically stable: dividing by the largest available pivot minimizes the amplification of rounding error, the

same f64 rounding the book has discussed. A zero pivot means the matrix is singular and the system has no unique solution, reported and exited. The elimination then zeroes the column below the pivot by subtracting scaled multiples of the pivot row, and back-substitution recovers the solution into `x`. This is numerical linear algebra — matrices, pivoting, floating-point row reduction — done in `awk` with a two-dimensional array and ordinary arithmetic, and the `%.6f` output formatting shows the six-decimal precision the doubles carry.

date_calc.awk: calendar arithmetic and Zeller's congruence

The date calculator does day-of-week computation, calendar rendering, and date differences, and it is the tour's calendar-arithmetic example — the domain where the rules are irregular (leap years, varying month lengths) and the algorithms are classic. It shows `awk` handling the messy arithmetic of the Gregorian calendar.

```
function day_of_week(y, m, d, K, J, h) {
    if (m < 3) { m += 12; y-- }
    K = y % 100
    J = int(y / 100)
    h = (d + int(13 * (m + 1) / 5) + K + int(K / 4) + int(J / 4) + 5 * J) % 7
    return (h + 6) % 7 # rotate: 0=Sun, 6=Sat
}
```

`day_of_week` is Zeller's congruence, the closed-form formula for the weekday of any Gregorian date — note the January-and-February-are-months-13-and-14 adjustment (`if (m < 3) { m += 12; y-- }`) that the formula requires, and the integer-division terms `int(13 * (m + 1) / 5)`, `int(K / 4)`, `int(J / 4)` that encode the month and century corrections. The result is rotated so Sunday is zero. The companion `fixed_from_gregorian` converts a date to a day count from a fixed epoch using the leap-year-aware formula, so date *differences* are plain subtraction — the standard technique of mapping dates to a serial day number. `is_leap` encodes the full Gregorian leap rule (divisible by four, except centuries, except every four hundred years), and `days_in_month` and the calendar renderer `cal` build a formatted month grid with `printf` column alignment and the week-wrapping logic. Calendar arithmetic is notoriously fiddly — every off-by-one in the month adjustment or leap rule produces a wrong weekday — and seeing it correct in `awk`, built from `int` division and modulo and a few lookup tables split from strings, shows the language handles irregular real-world arithmetic as readily as the clean kind.

heap_sort.awk: the heap as a sorting algorithm

The heapsort example builds a min-heap and then drains it, which sorts the input — a clean demonstration that a priority queue *is* a sorting algorithm, and a more focused look at the binary-heap operations that `dijkstra.awk` embedded in a larger algorithm.

```

function hpop( r, i, l, rt, sm, t) {
    r = hp[1]
    hp[1] = hp[hn]
    delete hp[hn]
    hn--
    i = 1
    while (1) {
        l = 2 * i; rt = l + 1; sm = i
        if (l <= hn && hp[l] < hp[sm]) sm = l
        if (rt <= hn && hp[rt] < hp[sm]) sm = rt
        if (sm == i) break
        t = hp[i]; hp[i] = hp[sm]; hp[sm] = t
        i = sm
    }
    return r
}

```

The heap is one array `hp[1..hn]` with the implicit-tree layout — children of `i` at `2i` and `2i+1` — and two operations: `hpush` adds at the end and sifts up while smaller than its parent, and `hpop` removes the root, moves the last element to the top, and sifts it down toward its smaller child. The sift-down here is the canonical version: pick the smallest of the node and its two children, and if it is not already the node, swap and continue. The sort is then trivial — push every input number, then repeatedly `hpop` the minimum, which emits them in ascending order. This is heapsort: a heap is exactly a structure that yields its elements in sorted order one at a time, so draining it sorts. Seeing the heap operations isolated here, without the distance-tracking and lazy-deletion machinery `dijkstra.awk` wrapped around them, makes the data structure itself clear — and it is the same $2 * i / \text{int}(i / 2)$ index arithmetic that the segment tree used, because both are implicit binary trees in an array.

coin_change.awk: unbounded DP and predecessor traceback

The coin-change solver finds the fewest coins that sum to a target, with each denomination usable any number of times — the *unbounded* knapsack variant — and it reconstructs which coins were chosen via a predecessor table. It is a compact companion to `knapsack.awk`, showing the unbounded DP recurrence and a different traceback style.

```

for (s = 1; s <= A; s++) {
    for (i = 1; i <= nc; i++) {
        c = cn[i]
        if (c <= s && dp[s - c] + 1 < dp[s]) {
            dp[s] = dp[s - c] + 1
            prev[s] = c
        }
    }
}

```

`dp[s]` is the minimum number of coins to make the sum `s`, initialized to a large sentinel for “unreachable” and filled bottom-up: for each sum `s` and each coin `c`, if using that coin (`dp[s - c] + 1`) beats the current best, take it and record `c` as the predecessor in `prev[s]`. The unbounded part is subtle and lies in the loop order — because `dp[s - c]` for a smaller `s` may *already* include coin `c`, a denomination can be reused freely, which is exactly what distinguishes unbounded from 0/1 knapsack (where the item loop is outermost precisely to prevent reuse). The traceback walks the predecessor chain from the target: `chosen[++ncoins] = prev[s]; s -= prev[s]` repeatedly, collecting the coins, then an insertion sort orders them for deterministic display. The `dp[A] >= INF` test detects the impossible case. It is the same DP-with-rollback shape as `knapsack.awk`, with the recurrence tweaked for unbounded reuse and the reconstruction done through a predecessor array

rather than by comparing table cells — two idioms for the same job, both built on [SUBSEP](#)-free one-dimensional arrays here since the state is a single sum.

convex_hull.awk: computational geometry

The convex-hull example computes the smallest convex polygon containing a set of points, using Andrew's monotone chain algorithm, and it brings computational geometry into the tour — cross products to test turn direction, and the shoelace formula for area. It is the example where awk does coordinate geometry.

```
function cross(ox, oy, ax, ay, bx, by) {
    return (ax - ox) * (by - oy) - (ay - oy) * (bx - ox)
}

# Lower hull.
k = 0
for (i = 1; i <= n; i++) {
    p = idx[i]
    while (k >= 2 \
        && cross(px[hull[k-1]], py[hull[k-1]], px[hull[k]], py[hull[k]], px[p], py[p]) <= 0) {
        delete hull[k]; k--
    }
    k++; hull[k] = p
}
```

The geometric primitive is `cross`, the two-dimensional cross product of the vectors from `O` to `A` and `O` to `B` — positive when `A` is to the left of the directed line `OB`, the sign that tells you whether three points turn left, right, or are collinear. Andrew's algorithm first sorts the points by `(x, y)` — with a hand-written insertion sort on a parallel index array, because, as the comment notes, `asort` would sort by value and lose the `x-y` pairing, the same parallel-arrays-need-an-index-sort situation `convex_hull` shares with `dijkstra` and `convex_hull`'s own `idx`. Then it builds the lower hull and upper hull each as a stack: walk the sorted points, and while the last two hull points plus the new point make a non-left turn (`cross(...) <= 0`), pop the middle one — the `delete hull[k]; k--` that discards a point the hull bends the wrong way around. The two chains together form the full hull. Finally the shoelace formula sums the cross terms around the polygon to get twice the signed area, halved and absolute-valued for the output. Cross products, a convex-hull scan, and the shoelace area — the staples of a computational-geometry course — implemented in awk with parallel coordinate arrays and arithmetic, no geometry library in sight.

permutations.awk: Heap's algorithm

The permutations generator emits all `n!` orderings of its input using Heap's algorithm, the classic method that produces each permutation from the previous with a single swap. It is the tour's combinatorial-generation example, and it shows recursion driving an exhaustive enumeration.

```

function permute(k, i, line) {
    if (k == 1) {
        line = arr[1]
        for (i = 2; i <= N; i++) line = line " " arr[i]
        print line
        count++
        return
    }
    for (i = 1; i <= k; i++) {
        permute(k - 1)
        if (k % 2 == 0) swap(i, k)
        else swap(1, k)
    }
}

```

Heap's algorithm is subtle and compact: to generate all permutations of the first k elements, it recursively generates all permutations of the first $k-1$, and between recursive calls performs one swap — `swap(i, k)` when k is even, `swap(1, k)` when odd — chosen so that every one of the $k!$ arrangements is produced with exactly one swap between consecutive outputs. The base case $k == 1$ emits the current arrangement. The elegance is that there is no copying of the array and no explicit “used” tracking — the array `arr` is permuted in place, and the precise swap pattern guarantees completeness. Each leaf of the recursion prints one permutation; the recursion depth is N and the total work is $N!$ swaps. It is the kind of algorithm that is hard to derive but short to write once known, and seeing it in `awk` — recursion plus an in-place swap on a global array — shows the language handles backtracking-style enumeration as naturally as it handles the DP and graph examples.

markdown_basic.awk: the line-oriented transformer

The Markdown converter is, in a sense, the most *characteristically awk* program in the entire tour, because it is the job `awk` was born for: read text line by line, classify each line by a pattern, and emit a transformed version. It converts a subset of Markdown to HTML, and it is built almost entirely from pattern-action rules with a little state.

```

function inline(s) {
    s = gsub(/\*(^*)\*/, "<strong>\1</strong>", "g", s)
    s = gsub(/\*([^\s]*)\*/, "<em>\1</em>", "g", s)
    s = gsub(/\`([^\`]+)\`/, "<code>\1</code>", "g", s)
    s = gsub(/\[([^\[\]]+)\]\([^\(\)]+\)/, "<a href=\1>\2</a>", "g", s)
    return s
}

/^#{1,6} / {
    flush_ul()
    level = 0
    while (substr($0, level + 1, 1) == "#") level++
    rest = substr($0, level + 2)
    printf "<h%d>%s</h%d>\n", level, inline(rest), level
    next
}

/^- / {
    if (!in_ul) { print "<ul>"; in_ul = 1 }
    printf "<li>%s</li>\n", inline(substr($0, 3))
    next
}

```

Read the structure as the pattern-action grammar it is. Each *block* kind gets its own rule keyed by a regex: `/^#{1,6} /` matches a heading and counts the `#` characters to get the level, `/^- /` matches a list item,

`/^```/` toggles a fenced code block, a blank line breaks a paragraph, and the fall-through rule wraps anything else in `<p>`. The *inline* formatting — bold, italic, code spans, links — is the `inline` function, four `gensub` calls each rewriting a Markdown construct to its HTML with capture-group backreferences (`\\1`, `\\2`), exactly the `gensub` capability of Chapter 10 doing real work: `[text](url)` becomes `text` by capturing the two parts and reordering them. The one piece of state is the `in_ul` flag, which tracks whether the converter is inside a list so consecutive `-` items merge into one `` and the `flush_ul` helper closes it when a non-item line arrives — a small state machine layered over the line-by-line rules. This is awk's home ground: stateful, pattern-driven, line-oriented text transformation, where the patterns classify and the actions emit, and the whole converter reads top to bottom as a description of what each kind of line becomes. If the algorithmic examples proved awk reaches further than its reputation, this one is a reminder of what that reputation was *right* about — text transformation is what awk does, and it does it beautifully.

lru_cache.awk: a doubly linked list in arrays

The LRU cache is the tour's most pointer-heavy data structure: it implements constant-time get and put by combining a hash map with a *doubly linked list*, and it builds that linked list — next and previous pointers, sentinel nodes, splicing — out of nothing but associative arrays. It is the clearest proof that awk's arrays can express pointer structures.

```
function unlink(n) {
    nxt[prev[n]] = nxt[n]
    prev[nxt[n]] = prev[n]
}
function link_after(n, target) {
    nxt[n] = nxt[target]; prev[n] = target
    prev[nxt[target]] = n
    nxt[target] = n
}
function touch(n) { unlink(n); link_after(n, 0) }
```

The linked list is two arrays, `nxt[node]` and `prev[node]`, mapping a node id to its neighbors' ids — pointers as array indices, the awk representation of a linked structure. Two sentinel nodes, id 0 (head) and id 1 (tail), bracket the list so the splicing code never has to special-case the ends, the classic sentinel trick. `unlink` removes a node by pointing its neighbors at each other; `link_after` splices a node in after a target; and `touch` — the LRU operation — moves a node to the front (just after the head sentinel) by unlinking and relinking, which is how an access marks an entry most-recently-used. A second array, `id_of`, is the hash map from key to node id, giving $O(1)$ lookup. On a `put` that overflows the capacity, the least-recently-used node is the one just before the tail sentinel (`prev[1]`), which is unlinked and its key deleted from the map. Every operation — get, put, eviction — is constant time, and the whole thing is three arrays (`nxt`, `prev`, `val`) plus the key map. A doubly linked list with sentinels and $O(1)$ splicing, the structure a systems-programming course builds with pointers and `malloc`, here built with associative arrays and integer node ids — the final word on Chapter 8's claim that the associative array is every data structure.

regex_engine.awk: a regex matcher written in awk

In a book about an awk whose regex engine is the Rust `regex` crate, there is something fitting about an example that implements a *small regex engine in awk itself* — a recursive matcher in the style of Kernighan's "Software Tools," supporting anchors, the dot, character classes, and the `*`, `+`, `?` quantifiers. It is the tour's most self-referential program, and a clean demonstration of recursive matching.

```

function match_here(pat, text, t,    c1, al, atom, rest, q) {
    if (length(pat) == 0) return 1
    c1 = substr(pat, 1, 1)
    if (c1 == "$" && length(pat) == 1) return (t == length(text) + 1)
    al = atom_len(pat, 1)
    atom = atom_at(pat, 1)
    rest = substr(pat, al + 1)
    q = substr(rest, 1, 1)
    if (q == "*") return match_star(atom, substr(rest, 2), text, t)
    if (q == "+") return match_plus(atom, substr(rest, 2), text, t)
    if (q == "?") return match_qmark(atom, substr(rest, 2), text, t)
    if (t > length(text)) return 0
    if (!atom_match(atom, substr(text, t, 1))) return 0
    return match_here(rest, text, t + 1)
}

```

The matcher is mutually recursive. `match_here` tries to match the pattern against the text at position `t`: it peels off the next *atom* (a literal character, the dot, or a bracketed class, with `atom_len/atom_at` measuring and extracting it), checks whether a quantifier follows, and dispatches accordingly — `match_star` for `*`, `match_plus` for `+`, `match_qmark` for `?`, or a plain atom-then-recurse otherwise. `match_star` implements greedy zero-or-more by trying to match the rest after consuming as many atoms as possible and backing off, exactly the recursive structure the K&R original uses. `re_match` handles the `^` anchor by deciding whether to try matching only at the start or at every position. This is a genuine, if small, regex engine — pattern parsing, atom matching, quantifier semantics, anchoring — written entirely in awk with `substr` for character access and recursion for the backtracking. That awk can implement a regex engine, in a book whose whole point is that awk *has* a fast regex engine, is a pleasing demonstration that the language is expressive enough to build the very feature it is famous for using.

intervals.awk: merging ranges and the padded sort key

The interval-merging example collapses a set of overlapping ranges into the minimal set of disjoint ranges that cover the same points — a staple of scheduling and computational geometry — and it showcases, one final time, the padded-sort-key technique that several tour programs have leaned on for deterministic numeric ordering through string comparison.

```

END {
    for (i = 1; i <= ni; i++) keys[i] = sprintf("%012d %012d %d", s[i], e[i], i)
    asort(keys, sorted)
    cs = 0; ce = 0; have = 0; covered = 0
    for (i = 1; i <= ni; i++) {
        split(sorted[i], parts, " ")
        idx = parts[3] + 0
        ts = s[idx]; te = e[idx]
        if (!have) { cs = ts; ce = te; have = 1; continue }
        if (ts <= ce + 1) {      # overlap or adjacent
            if (te > ce) ce = te
        } else {
            printf "[%d, %d]\n", cs, ce
            covered += ce - cs + 1
            cs = ts; ce = te
        }
    }
    ...
}

```

The algorithm is the standard one: sort the intervals by start, then sweep, extending the current merged interval whenever the next one overlaps or abuts it (`ts <= ce + 1`) and emitting and restarting when it does not.

The interesting awk-specific move is the sort. Each interval is encoded as `sprintf("%012d %012d %d", start, end, index)` — start and end zero-padded to twelve digits, followed by the original index — so that `asort`'s string comparison orders the intervals numerically by start then end, regardless of magnitude, because zero-padded decimal strings compare in numeric order. This is the same pack-the-sort-key-into-a-padded-string technique `huffman.awk` and `suffix_array.awk` used, and it is awk's standard answer to "sort numerically when the only sort I have compares strings": pad to a fixed width so lexical order is numeric order, and tuck the payload (here the original index) after the key so a value sort doubles as an index sort. Interval merging with a clean sweep, deterministic through the padded key — and a fitting close to the tour's recurring lesson that a handful of awk idioms, the padded key among them, compose into solutions for a remarkable range of problems.

email_extract.awk: text mining with match in a loop

The email extractor pulls every email address out of free-form text, and it is the tour's text-mining example — the data-cleaning job awk is reached for constantly. Its technique is calling `match` repeatedly in a loop to find *all* occurrences on a line, advancing past each match, which is the standard awk idiom for "find every match" that the single-shot `match` does not provide directly.

```
function find_emails(line, start, m, addr, hits, sep) {
    hits = ""; sep = ""
    start = 1
    while (match(substr(line, start), /[A-Za-z0-9._+\-]+\@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}/)) {
        addr = substr(line, start + RSTART - 1, RLENGTH)
        hits = hits sep addr
        sep = " "
        tally[addr]++
        start += RSTART + RLENGTH - 1
    }
    return hits
}
```

The loop is the find-all-matches pattern. `match` finds the first email in the remaining tail of the line (`substr(line, start)`), setting `RSTART` and `RLENGTH`; the address is cut out with `substr(line, start + RSTART - 1, RLENGTH)`, converting the position-within-the-tail back to a position within the whole line; and `start` advances past the match so the next iteration searches what remains. When `match` finally fails, the line is exhausted. The regex itself is a realistic email pattern — a local part, an `@`, a domain, and a top-level domain of at least two letters, the `{2,}` interval quantifier that awkrs always has on. Each address is also tallied into a global counter, and the `END` rule reports the deduplicated counts in sorted order. This loop-over-`match` idiom is one every awk programmer needs, because awk's `match` finds only the first occurrence — to find them all you slide a window with `RSTART + RLENGTH`, exactly as `url_parser.awk` peeled components but here repeated until the line is consumed. Extracting structured data from unstructured text, one regex and a sliding match: this is awk doing the job it is most often actually hired for.

bst.awk: a binary search tree and three traversals

The binary search tree example inserts keys and prints the three classic traversals, and it is the tour's cleanest illustration of a *recursive* tree built in arrays — node records as parallel arrays, child links as integer ids, and the inorder/preorder/postorder walks as three nearly-identical recursive functions.

```

function inorder(p) { if (p == 0) return; inorder(lt[p]); printf " %d", rk[p];
    ~ inorder(rt[p]) }
function preorder(p) { if (p == 0) return; printf " %d", rk[p]; preorder(lt[p]);
    ~ preorder(rt[p]) }
function postorder(p) { if (p == 0) return; postorder(lt[p]); postorder(rt[p]); printf "
    ~ %d", rk[p] }

```

The tree is three arrays indexed by node id: `rk[i]` the key, `lt[i]` the left child id, and `rt[i]` the right child id, with `0` standing for the nil child — the same node-records-in-parallel-arrays representation as `trie.awk` and `lru_cache.awk`, here for a binary tree. `bst_insert` walks down from the root comparing keys, going left for smaller and right for larger, until it finds an empty child slot to attach the new node. The three traversals are the textbook trio, and the example lays them side by side to make their single difference visible: they differ *only* in where the `printf` of the current node sits relative to the two recursive calls — before both (preorder), between them (inorder, which for a BST yields the keys in sorted order), or after both (postorder). Seeing the three on three lines, identical but for the placement of one statement, is the clearest possible illustration of what traversal order *is*: the same recursive structure, visiting the node at three different moments. A binary search tree with all three traversals, in a dozen lines, on three arrays and recursion.

sudoku.awk: constraint backtracking with witness sets

The Sudoku solver is the tour's constraint-satisfaction capstone: it solves a 9×9 grid by recursive backtracking, using three witness sets — one per row, column, and box — to test a candidate placement in constant time, the same place-recurse-unplace discipline as `n_queens.awk` but with a richer constraint structure.

```

function solve( r, c, v ) {
    if (!find_blank()) return 1
    r = empty_r; c = empty_c
    for (v = 1; v <= 9; v++) {
        if ((r SUBSEP v) in rowused) continue
        if ((c SUBSEP v) in colused) continue
        if ((bx(r, c) SUBSEP v) in boxused) continue
        place(r, c, v)
        if (solve()) return 1
        unplace(r, c, v)
    }
    return 0
}

```

The grid is `g[r, c]`, a `SUBSEP` two-dimensional array, and the constraints are three more two-dimensional arrays: `rowused[r, v]` records that value `v` is already used in row `r`, and likewise `colused` and `boxused`, where `bx(r, c)` computes which of the nine 3×3 boxes a cell belongs to. `solve` finds the next blank cell and tries each value `1` through `9`, skipping any that the three witness sets say would conflict — three constant-time membership tests, `(r SUBSEP v) in rowused` and its siblings, using the explicit `SUBSEP` join to build the composite key and the membership test that does not create it. The place-recurse-unplace structure is backtracking's signature: `place` sets the cell and marks all three witness sets, `solve` recurses, and if the recursion fails `unplace` clears the cell and *deletes* the three witness entries, restoring the state for the next candidate. This is exactly `n_queens.awk`'s add-recurse-delete discipline, scaled up to three constraint families, and it shows that awk's arrays-as-sets plus *delete-as-undo* handle constraint satisfaction as readily as they handle the simpler search. A complete Sudoku solver — find a blank, try the legal values, recurse, undo on failure — in forty lines of awk, on `SUBSEP` arrays and the membership test.

floyd_warshall.awk: all-pairs shortest paths

Where `dijkstra.awk` found shortest paths from one source, Floyd-Warshall finds them between every pair of nodes, and it does so with a strikingly simple triple loop over a distance matrix — a different shape of shortest-path algorithm, dynamic programming over intermediate vertices rather than a frontier search.

```
for (k = 1; k <= n; k++) {
    K = order[k]
    for (i = 1; i <= n; i++) {
        I = order[i]
        if (dist(I, K) == INF) continue
        for (j = 1; j <= n; j++) {
            J = order[j]
            if (dist(K, J) == INF) continue
            cand = dist(I, K) + dist(K, J)
            if (cand < dist(I, J)) setd(I, J, cand)
        }
    }
}
```

The distances live in `d[u, v]`, a `SUBSEP` two-dimensional array, with a missing entry treated as infinity by the `dist` accessor. The algorithm's whole idea is in the loop order: the outermost loop over `k` considers each node as a possible *intermediate* point on a path, and asks, for every pair `(i, j)`, whether routing through `k` (`dist(I, K) + dist(K, J)`) beats the best path found so far. After all `n` intermediate nodes have been considered, every pair holds its true shortest distance. That `k` must be the outermost loop is the subtle correctness requirement — it is what lets each round build on the previous one's results. The example handles negative edge weights (which Dijkstra cannot) and detects a negative cycle by checking whether any node ends up with a negative distance to itself. It iterates nodes in a sorted order captured into `order[]` so the output matrix is deterministic. The all-pairs shortest-path matrix, with negative-weight support and cycle detection, in a triple loop over a `SUBSEP` matrix — Floyd-Warshall is three nested loops and an array, and `awk` expresses it almost verbatim from the textbook.

bellman_ford.awk: relaxation and negative cycles

Bellman-Ford is the other negative-weight shortest-path algorithm, single-source like Dijkstra but able to handle negative edges by *relaxing every edge repeatedly* rather than greedily expanding a frontier. It is the example that shows the relaxation-pass structure and the elegant negative-cycle test that falls out of it.

```
for (pass = 1; pass < nn; pass++) {
    relaxed = 0
    for (i = 1; i <= ne; i++) {
        u = eu[i]; v = ev[i]; w = ew[i]
        if (dist[u] == INF) continue
        cand = dist[u] + w
        if (cand < dist[v]) { dist[v] = cand; relaxed = 1 }
    }
    if (!relaxed) break
}
```

The graph is stored as an edge list — parallel arrays `eu`, `ev`, `ew` for the endpoints and weight of each edge — rather than an adjacency list, because Bellman-Ford iterates over *edges*, not neighbors. The core is the relaxation: for up to `n-1` passes, walk every edge and, if going through it gives a shorter distance to its destination, take it. The key fact is that after `n-1` passes every shortest path (which can have at most `n-1` edges) has been found, and the early-exit `if (!relaxed) break` stops as soon as a pass changes nothing. The negative-cycle test is the beautiful part: after the `n-1` passes, if *one more* relaxation would still improve some distance, then there must be a negative cycle, because a graph without one is fully relaxed after

`n-1` passes. That single extra check — does any edge still relax? — is a complete negative-cycle detector. Bellman-Ford trades Dijkstra's speed for the ability to handle negative weights and detect negative cycles, and the edge-list relaxation that buys that ability is, in `awk`, one doubly-nested loop over parallel edge arrays.

`scc_tarjan.awk`: strongly connected components

The most sophisticated graph algorithm in the tour is Tarjan's strongly-connected-components algorithm, which finds the maximal groups of mutually-reachable nodes in a single depth-first traversal using the famous low-link bookkeeping. It is the tour's hardest algorithm to derive and a showcase of deep recursion with an explicit auxiliary stack — all in `awk`.

```
function strongconnect(v, k, w, smallest, members, sorted, line, n, arr, i, m) {
    idx[v] = idx_counter
    low[v] = idx_counter
    idx_counter++
    stack[++sp] = v
    onstack[v] = 1
    for (k = 1; k <= (outdeg[v] + 0); k++) {
        w = adj[v, k]
        if (!(w in idx)) {
            strongconnect(w)
            if (low[w] < low[v]) low[v] = low[w]
        } else if (w in onstack) {
            if (idx[w] < low[v]) low[v] = idx[w]
        }
    }
    if (low[v] == idx[v]) {
        # Pop one SCC off the stack.
        ...
    }
}
```

Tarjan's algorithm assigns each node a discovery `idx` (the order it was first visited) and a `low` value (the smallest index reachable from its subtree), maintaining a stack of nodes in the current search path with `onstack` marking membership. When it recurses into an unvisited neighbor, it updates its own low-link from the child's; when it sees a neighbor already on the stack, it updates from that neighbor's index. The defining moment is `if (low[v] == idx[v])` — a node whose low-link equals its own index is the *root* of a strongly connected component, and everything above it on the stack down to itself forms that component, which is popped off. This is genuinely intricate — the low-link invariant is the kind of thing that takes a careful proof to believe — and seeing it transcribed into `awk`, with `idx`, `low`, `onstack`, and the explicit `stack` array all as associative arrays and the recursion carrying the DFS, is the strongest demonstration in the tour that `awk` is a real programming language. The recursion cap of Chapter 24 keeps a pathological input from overflowing the host stack; within that bound, Tarjan's algorithm runs in `awk` exactly as it would in any language, on nothing but arrays and recursion.

`tsp_dp.awk`: Held-Karp bitmask dynamic programming

The travelling-salesman solver uses the Held-Karp algorithm — dynamic programming over *subsets* of cities, with each subset encoded as a bitmask — and it is the tour's bitmask-DP example, the technique where the DP state is indexed by a set represented as the bits of an integer. It is also where the bitwise builtins and `SUBSEP` arrays meet.

```

function bit(i) { return lshift(1, i - 1) }
function has_bit(mask, i) { return and(mask, bit(i)) != 0 }
function set_bit(mask, i) { return or(mask, bit(i)) }

for (mask = 1; mask <= full; mask += 2) {
  for (last = 1; last <= N; last++) {
    if (!has_bit(mask, last)) continue
    if (!((mask SUBSEP last) in dp)) continue
    base = dp[mask, last]
    for (nxt = 2; nxt <= N; nxt++) {
      if (has_bit(mask, nxt)) continue
      new_mask = set_bit(mask, nxt)
      cand = base + d[last, nxt]
      if (!((new_mask SUBSEP nxt) in dp) || cand < dp[new_mask, nxt]) {
        dp[new_mask, nxt] = cand
        par[new_mask, nxt] = last
      }
    }
  }
}

```

The DP state is `dp[mask, last]`: the cheapest cost to start at city one, visit exactly the cities whose bits are set in `mask`, and end at city `last`. The three helper functions wrap the bitwise builtins of Chapter 11 — `lshift` to make a city's bit, `and` to test membership, `or` to add a city — turning an integer into a *set* of visited cities, which is the heart of bitmask DP. The main loop extends each partial tour by one unvisited city, the `mask += 2` stepping only over masks that include city one (bit zero), which is always the start. After all subsets are filled, the best tour closes the loop back to city one, and a traceback through the `par` predecessor array reconstructs the route, using `and` with a cleared bit to walk the masks backward. Held-Karp is exponential — the program caps `N` at fifteen because the state space is 2^N — but it is the textbook exact-TSP algorithm, and seeing it in `awk` shows the bitwise builtins doing exactly what they do in C: representing a set as the bits of an integer, indexed into a DP table. The `SUBSEP` array keyed by `(mask, last)` is a DP table over (set, endpoint) pairs, and `awk` handles it as naturally as any other two-dimensional state.

pollard_rho.awk: integer factorization

The final number-theory example factors integers using Pollard's rho algorithm with Brent's cycle-detection improvement, falling back on the Miller-Rabin primality test — the same machinery real factorization libraries use for moderate inputs. It is the tour's most algorithmically dense numeric program, combining several sub-algorithms with a work stack.

```

function factor(n) {
  ...
  delete stack
  sp = 0
  if (n > 1) stack[++sp] = n
  while (sp > 0) {
    v = stack[sp]; sp--
    if (v == 1) continue
    if (is_prime(v)) { primes_found[++pf_n] = v; continue }
    d = brent_split(v)
    stack[++sp] = d
    stack[++sp] = int(v / d)
  }
  ...
}

```

The structure is a recursive factorization made iterative with an explicit stack. After stripping small primes by

trial division, each remaining composite is pushed onto a work stack; the loop pops a value, and if Miller-Rabin proves it prime it is recorded, otherwise Pollard's rho (`brent_split`) finds a nontrivial divisor and pushes *both* the divisor and the quotient back to be factored further. This divide-and-conquer-via-stack is the same pattern as the SCC and DFS examples, here driving a factorization tree. `brent_split` implements Brent's variant of Pollard's rho — a cycle-detection method that finds a factor by following a pseudo-random sequence modulo `n` and computing a GCD, the `gcd` helper being the iterative Euclid from `gcd_extended.awk`. The program is honest in its header about the precision bound: the intermediate `(a * a) % m` must stay within `64`'s exact-integer range, so it factors small to moderate numbers, and beyond that you would reach for `-M` bignum mode. It is a remarkable thing to find in an `awk` example — a real factorization algorithm with primality testing, cycle detection, and a work-stack-driven divide-and-conquer — and it is the strongest numeric counterpart to the SCC algorithm's claim that `awk` hosts genuinely advanced computation, bounded only, and honestly, by the precision of its doubles.

kruskal.awk: minimum spanning tree and union-find

Kruskal's minimum-spanning-tree algorithm sorts the edges and adds each one that connects two previously-separate components, and its engine is the *union-find* (disjoint-set) data structure — the near-constant-time “are these two in the same group?” structure that no tour example has shown until now. It is the data-structure highlight of the graph examples.

```
function find(x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]]    # path compression (halving)
        x = parent[x]
    }
    return x
}

function unite(a, b, ra, rb) {
    ra = find(a); rb = find(b)
    if (ra == rb) return 0
    if (rank[ra] < rank[rb]) { parent[ra] = rb }
    else if (rank[ra] > rank[rb]) { parent[rb] = ra }
    else { parent[rb] = ra; rank[ra]++ }
    return 1
}
```

Union-find is two arrays: `parent[x]` points each element toward its set's representative, and `rank[x]` bounds the tree height. `find` walks the parent chain to the root, applying *path compression* on the way — `parent[x] = parent[parent[x]]`, the halving variant that flattens the tree as it climbs, so future finds are faster. `unite` joins two sets by attaching the lower-rank root under the higher-rank one — *union by rank* — keeping the trees shallow. Together these two optimizations make the disjoint-set operations effectively constant time, which is what makes Kruskal's algorithm efficient. The main routine sorts the edges by weight (the padded-key idiom again, `sprintf("%012d %s %s %d", ...)`), then walks them, calling `unite` on each — an edge is added to the spanning tree exactly when its endpoints were in different components, because then joining them does not create a cycle. The algorithm stops once `n-1` edges are chosen. Union-find is one of the most elegant data structures in computer science, and seeing it in `awk` — two arrays, path compression, union by rank — is the final proof that `awk`'s arrays express not just the obvious structures but the subtle ones, the near- $O(1)$ disjoint-set among them.

lis.awk: longest increasing subsequence by binary search

The longest-increasing-subsequence example finds the longest run of increasing values (not necessarily contiguous) in $O(n \log n)$ time using the patience-sorting method — a binary search over a “tails” array — which is a more sophisticated technique than the quadratic DP, and the tour’s showcase of binary search inside an algorithm.

```
for (i = 1; i <= n; i++) {
    lo = 1; hi = tail_count
    while (lo <= hi) {
        mid = int((lo + hi) / 2)
        if (a[tail[mid]] < a[i]) lo = mid + 1
        else hi = mid - 1
    }
    pos = lo
    tail[pos] = i
    prev_idx[i] = (pos == 1) ? 0 : tail[pos - 1]
    if (pos > tail_count) tail_count = pos
}
```

The key insight of the patience method is the `tail` array: `tail[k]` holds the index of the smallest possible tail value of any increasing subsequence of length k . For each new element, a *binary search* finds the position where it would extend or improve a subsequence — the `while (lo <= hi)` loop with `mid = int((lo + hi) / 2)` is a textbook binary search over the tails — and the element either lengthens the longest subsequence or replaces a tail with a smaller value, keeping future extensions possible. The `prev_idx` array records each element’s predecessor so the actual subsequence can be reconstructed by traceback, the same predecessor-chain reconstruction `coin_change.awk` used. The binary search is what turns the naive $O(n^2)$ LIS into $O(n \log n)$, and seeing it written out in awk — the halving search over `tail`, the predecessor tracking, the traceback — shows that awk handles the divide-by-two index arithmetic of binary search as readily as the linear scans of the simpler examples. Longest increasing subsequence in log-linear time, on three arrays and a binary search.

aho_corasick.awk: multi-pattern matching with failure links

Aho-Corasick searches a text for *many* patterns at once in a single linear scan, by building an automaton — a trie of the patterns augmented with failure links that let the scan recover from a mismatch without restarting. It is the multi-pattern generalization of KMP, and the most elaborate automaton in the tour, built entirely in associative arrays.

```
function ac_scan(s, i, n, c, cur, hits, name, sep, nx) {
    hits = ""; sep = ""
    cur = 0
    n = length(s)
    for (i = 1; i <= n; i++) {
        c = substr(s, i, 1)
        while (cur != 0 && !((cur SUBSEP c) in goto_)) cur = fail[cur]
        if ((cur SUBSEP c) in goto_) cur = goto_[cur, c]
        if (out[cur] != "") { ... }
    }
}
```

The automaton is three arrays: `goto_[node, char]` is the trie’s child links (a `SUBSEP` array, exactly `trie.awk`’s `edge`), `fail[node]` is the suffix-failure link, and `out[node]` lists the patterns ending at a node.

Building it has two phases. `tr_add` inserts each pattern into the trie, the same insertion as `trie.awk`. `tr_build` then computes the failure links by a breadth-first traversal — the `q` array used as a queue with head and tail indices — where each node's fail link points to the longest proper suffix that is also a trie prefix, and output links are merged so a node reports every pattern that ends there or at any node its fail chain reaches. The scan, shown above, is then linear: at each character it follows `goto_` if it can, and on a mismatch it follows `fail` links until it can proceed or reaches the root — the same fall-back-on-mismatch idea as KMP's failure function, generalized to a whole dictionary of patterns. Aho-Corasick is the algorithm behind tools like `grep -f` with many patterns and intrusion-detection signature matching, and seeing its goto/fail/output automaton built and run in `awk` — a BFS over a `SUBSEP` trie with a queue and failure links — is the high-water mark of the tour's string-algorithm examples.

tictactoe.awk: minimax game-tree search

The final example is a tic-tac-toe solver that plays perfectly by *minimax* — searching the full game tree to find the move that maximizes the current player's outcome assuming the opponent also plays optimally. It is the tour's game-AI example, and it shows the recursive try-move, recurse-for-opponent, undo-move structure of adversarial search.

```
function minimax(player, w, i, sc, best, best_idx, saved) {
    w = winner()
    if (w != ".") return (w == root_player) ? 1 : -1
    if (full()) return 0
    best = (player == root_player) ? -100 : 100
    best_idx = 0
    for (i = 1; i <= 9; i++) {
        if (board[i] != ".") continue
        saved = board[i]
        board[i] = player
        sc = minimax(opponent(player))
        board[i] = saved
        if (player == root_player) {
            if (sc > best) { best = sc; best_idx = i }
        } else {
            if (sc < best) { best = sc; best_idx = i }
        }
    }
    ...
    return best
}
```

`minimax` is adversarial recursion. At a terminal position — a win, loss, or full board — it returns a score from the root player's perspective (+1 win, 0 draw, -1 loss). Otherwise it tries every legal move, recursing into the opponent's turn, and *maximizes* the score when it is the root player's move but *minimizes* it when it is the opponent's — the defining alternation of minimax, the assumption that each side plays to its own advantage. The move-and-undo is the same place-recurse-restore as the backtracking examples: set `board[i]`, recurse, restore the saved value — here restoring rather than `delete`ing because every cell always holds a value. The `winner` function checks the eight winning lines (split from strings, the lookup-table idiom), and the driver reports the optimal move and its score. Because tic-tac-toe is a known draw under perfect play, the empty board returns score zero — a small built-in correctness check. A game-tree search with optimal play for both sides, in fifty lines of `awk`, on a board array and recursion — the same recursive-search machinery as the puzzle solvers, turned to a two-player game.

bigint_mul.awk: arbitrary-precision multiplication by hand

The big-integer multiplier multiplies integers of *any* length by storing them as digit arrays and doing schoolbook multiplication digit by digit — the algorithm you learned in grade school, implemented to sidestep the f64 precision limit entirely. It is the example that shows awk doing exact arithmetic on numbers far larger than a double can hold, without `-M`.

```
function mul_pos(a, b, la, lb, i, j, carry, k, prod, out) {
    la = length(a); lb = length(b)
    for (i = 1; i <= la; i++) ra[i] = substr(a, la - i + 1, 1) + 0
    for (i = 1; i <= lb; i++) rb[i] = substr(b, lb - i + 1, 1) + 0
    for (i = 1; i <= la + lb; i++) acc[i] = 0
    for (i = 1; i <= la; i++) {
        for (j = 1; j <= lb; j++) {
            acc[i + j - 1] += ra[i] * rb[j]
        }
    }
    for (k = 1; k <= la + lb; k++) {
        if (acc[k] >= 10) { carry = int(acc[k] / 10); acc[k] %= 10; acc[k + 1] += carry }
    }
    ...
}
```

The numbers are *strings of digits*, exploded into arrays `ra` and `rb` least-significant-digit first (handy for accumulation), so the value is never held as a single double and the length is unbounded. The multiplication is the textbook two-loop algorithm: every digit of `a` times every digit of `b` accumulates into position `i + j - 1` of `acc`, exactly as the columns line up when you multiply by hand. A separate carry-propagation pass then normalizes each column to a single digit. This is the right way to think about awk's relationship with large numbers: a double holds about fifteen exact digits, and beyond that you either switch to `-M` bignum mode (Chapter 19), which makes the *language's* arithmetic arbitrary-precision, or — as here — you represent the number as digits yourself and do the arithmetic explicitly. The example is the manual counterpart to `-M`: where `-M` makes `a * b` exact for any size transparently, `bigint_mul.awk` shows the machinery `-M` automates, schoolbook multiplication over digit arrays, exact at any length on the ordinary double-precision engine.

horner.awk: polynomial evaluation and synthetic division

The Horner example evaluates polynomials, differentiates them, and divides them by a linear factor, all through Horner's method — the nested-multiplication scheme that evaluates a degree-`n` polynomial in `n` multiplications instead of recomputing powers. It is the tour's numerical-methods example, and it shows the same compact recurrence serving three related tasks.

```

function horner(coef, n, x, i, r) {
    r = 0
    for (i = 1; i <= n; i++) r = r * x + coef[i]
    return r
}

function horner_div(coef, n, x0, i, r, out) {
    delete quot
    r = coef[1]
    quot[1] = r
    for (i = 2; i <= n; i++) { r = r * x0 + coef[i]; quot[i] = r }
    return r
}

```

`horner` is the whole idea in one loop: starting from zero, repeatedly multiply the running result by x and add the next coefficient, so $a_3x^3 + a_2x^2 + a_1x + a_0$ is evaluated as $((a_3 \cdot x + a_2) \cdot x + a_1) \cdot x + a_0$ — no exponentiation, just n multiply-adds, which is both faster and more numerically stable than computing powers. The elegant part is `horner_div`: the *same* recurrence, if you keep the intermediate results, simultaneously yields $p(x_0)$ (the final value, which is the remainder of dividing by $x - x_0$) and the coefficients of the quotient polynomial (the intermediate values) — this is synthetic division, and it falls straight out of Horner's scheme. The example also computes the derivative by building the derivative's coefficient array and running Horner on it. One four-line recurrence, evaluating, dividing, and (with a coefficient tweak) differentiating polynomials — Horner's method is a small gem of numerical computing, and awk expresses it as cleanly as the array-and-loop language it is.

rule30.awk: an elementary cellular automaton

The Rule 30 example runs Wolfram's elementary one-dimensional cellular automaton, the famous chaos-from-simplicity rule, on a wrap-around tape. It is the one-dimensional companion to `conway.awk`'s two-dimensional Game of Life, and it shows the lookup-table-driven update that defines an elementary CA.

```

function step(i, l, c, r, idx, n) {
    for (i = 1; i <= W; i++) {
        l = cell[(i == 1) ? W : i - 1]
        c = cell[i]
        r = cell[(i == W) ? 1 : i + 1]
        idx = l * 4 + c * 2 + r
        if (idx == 0) n = 0
        if (idx == 1) n = 1
        ...
        if (idx == 7) n = 0
        nxt[i] = n
    }
    for (i = 1; i <= W; i++) cell[i] = nxt[i]
}

```

Each cell's next state depends on itself and its two neighbors — three bits, eight possible patterns — and the “rule number” is the eight-bit lookup table mapping each neighborhood pattern to the next state. The code computes the pattern index $l * 4 + c * 2 + r$ (treating the three neighbor cells as a three-bit number) and reads the next state from the rule, which for Rule 30 is the bit pattern `00011110`. The wrap-around boundary $((i == 1) ? W : i - 1)$ makes the tape a ring. Like `conway.awk`, it uses the double-buffer discipline — compute the whole next generation into `nxt` before copying back — because every cell's update must see the *current* generation's neighbors. Rule 30 is famous for producing apparent randomness from this trivial deterministic rule, and watching it emerge from a few lines of awk — a neighborhood index, an eight-entry lookup, a double buffer on a ring — is a small demonstration that complexity is a property of iteration, not of the rule, which awk's simple arrays render as plainly as anything could.

graph_bfs.awk: breadth-first search and the array queue

Breadth-first search explores a graph level by level, finding the shortest unweighted path to every node, and it is the example that shows the *queue* — an array with a head and tail index — driving a traversal, the breadth-first counterpart to the depth-first recursion of the SCC and tree examples.

```
qh = 1; qe = 0
q[++qe] = src
while (qh <= qe) {
    u = q[qh++]
    for (k = 1; k <= outdeg[u]; k++) {
        v = adj[u, k]
        if (v in dist) continue
        dist[v] = dist[u] + 1
        parent[v] = u
        q[++qe] = v
    }
}
```

The queue is the array `q` with `qh` (head) and `qe` (tail) indices: enqueue by `q[++qe] = v`, dequeue by `q[qh++]`. This is the FIFO counterpart to the LIFO stack idiom — same array, but advancing the head as well as the tail — and it is what makes the traversal breadth-first rather than depth-first. The search dequeues a node, and for each unvisited neighbor (the `v in dist` membership test marking visited), records its distance as one more than the current node's, sets its parent for path reconstruction, and enqueues it. Because BFS visits nodes in order of distance, the first time a node is reached is via a shortest path — which is why BFS solves unweighted shortest paths, where Dijkstra's weighted machinery would be overkill. The `parent` chain reconstructs each path, the same predecessor-walk as Dijkstra and the DP examples. A breadth-first traversal with shortest-path distances and path reconstruction, on an adjacency array and a queue made of one array and two indices — the queue completing the trio of array-as-stack, array-as-queue, and array-as-heap that the tour's traversals are built from.

avl_tree.awk: a self-balancing tree with rotations

The AVL tree is the tour's most demanding data structure: a self-balancing binary search tree that keeps itself shallow through *rotations* triggered by balance factors, guaranteeing logarithmic operations where a plain BST can degenerate to a list. It is `bst.awk` with the balancing machinery added, and it shows pointer rotations expressed in arrays.

```

function rot_right(y, x) {
    x = left[y]
    left[y] = right[x]
    right[x] = y
    update_h(y); update_h(x)
    return x
}

function rebalance(n, b) {
    update_h(n)
    b = bf(n)
    if (b > 1 && bf(left[n]) >= 0) return rot_right(n)
    if (b > 1 && bf(left[n]) < 0) { left[n] = rot_left(left[n]); return rot_right(n) }
    if (b < -1 && bf(right[n]) <= 0) return rot_left(n)
    if (b < -1 && bf(right[n]) > 0) { right[n] = rot_right(right[n]); return rot_left(n) }
    return n
}

```

The tree is four parallel arrays — `key`, `left`, `right`, and `h` (the cached subtree height) — indexed by node id, extending `bst.awk`'s three-array representation with a height. The *balance factor* `bf(n)` is the height difference between a node's subtrees, and an AVL tree maintains the invariant that every node's balance factor is in `{-1, 0, +1}`. When an insertion pushes a node out of balance, `rebalance` performs one of four rotation cases — left, right, left-right, or right-left — restoring the invariant. A rotation is a local restructuring: `rot_right` makes the left child the new subtree root, moving subtrees and updating heights, all by reassigning the `left/right` array entries — pointer surgery done as array-index assignment. The `insert` recursion rebalances on the way back up, so a single insertion does at most a constant amount of rebalancing. AVL trees are the data structure where balancing is genuinely intricate — the four rotation cases and the height bookkeeping take care to get right — and seeing it in `awk`, with rotations as array reassignments and the recursion threading the rebalance back up the tree, is the final demonstration that `awk`'s arrays express not just data structures but *self-maintaining* ones, the balanced trees that database indexes and language runtimes are built from.

vigenere.awk: ciphers and the ordinal-table pattern

The Vigenère cipher closes the tour, and it is a fitting end because it combines techniques from across the chapter — the precomputed ordinal tables of `hexdump.awk`, the character-classification regex of `roman.awk`, and the positive-modulo arithmetic of `gcd_extended.awk` — into a clean, reversible cipher.

```

function shift_char(c, k, dir, o, base, n) {
    o = ord[c]
    if (o >= 65 && o <= 90) base = 65
    else if (o >= 97 && o <= 122) base = 97
    else return c
    n = ((o - base) + dir * k + 2600) % 26 # +2600 keeps modulus positive
    return chr[base + n]
}

```

The `BEGIN` rule builds the two tables every byte-level `awk` program wants — `ord` mapping each character to its code and `chr` the inverse — by walking 0 to 255, the same precompute the `hexdump` used. `shift_char` is the Caesar shift at the heart of Vigenère: it classifies the character by its code into the uppercase or lowercase range (returning non-letters unchanged), shifts within the twenty-six-letter alphabet, and the `dir` parameter — `+1` to encrypt, `-1` to decrypt — makes one function serve both directions. The `+ 2600` before the `% 26` is the same positive-modulo guard the `gcd` example needed, ensuring a backward shift during decryption does not produce a negative index, since `awk`'s `%` follows the sign of its left operand. The `vig` driver walks the text, advancing the key index only on letters (so spaces and punctuation do not consume key characters), folding the key to uppercase, and threading each letter through `shift_char`. Encryption and decryption differ only in

the `dir` argument, so the round trip is built in. It is a complete, correct, case-preserving Vigenère cipher in about forty lines, and every technique in it has appeared earlier in the tour — the ordinal tables, the regex classification, the modular arithmetic — which is the quietly fitting note to end on: the tour's sixty-six programs are not sixty-six unrelated tricks but a small, composable vocabulary of awk idioms, used and reused, on nothing but the associative array, the string builtins, the pattern-action loop, and the arithmetic that Chapter 6 laid out. That vocabulary, fully internalized, is what the rest of this book has been arguing awk always was: smaller than its reputation fears and larger than its reputation admits.

The tour as a whole

Step back from the sixty-six programs and a single shape is visible. Every one of them — the heaps and trees, the DP solvers, the graph traversals, the string matchers, the parsers, the ciphers, the games — is built from the same dozen parts: the associative array, used as a map, a set, a stack, a queue, a heap, a grid, and a graph; the `SUBSEP` join and split that make it multidimensional; the string builtins `substr`, `index`, `match`, `gsub`, `sprintf`; user functions with parameter-locals and the out-parameter for multiple returns; recursion, capped for safety; the pattern-action loop that classifies and dispatches; and the small kit of idioms — the `+ 0` coercion, the membership test, the parallel arrays, the padded sort key, the place-recurse-undo — that Chapter 40 catalogues. Not one of these programs reaches outside that kit. A Sudoku solver and a SQL engine and an arbitrary-precision multiplier look nothing alike as problems, but as awk programs they are arrangements of the same handful of moves. That is the deepest thing the tour has to teach, and it is why the book has returned to it so often: awk's surface is small enough to hold entirely in the head, and a small surface fully internalized is not a limitation but a multiplier, because every problem you meet decomposes into parts you already know. The programs are not sixty-six feats of cleverness; they are sixty-six proofs that a model from 1977, taken seriously, reaches as far as you are willing to carry it — and awkrs runs every one of them fast, debugs them with modern tooling, and checks them byte-for-byte against gawk, so the model's reach and the engine's speed are, finally, the same story told from two ends.

Chapter 27 — The String and Substitution Builtins, in Full

Chapters 11 through 13 toured the builtin library in guided form, grouping the functions by family and dwelling on the corner cases that distinguish a faithful `awk` from an approximate one. This chapter and the two that follow are the other thing: a *catalogue*, one function at a time, in the order a reader looking something up would want it — signature, what it does, the edge cases `awkrs`'s parity work pinned, and a small real example for each. The signatures here are the ones `awkrs`'s own Language Server reports on hover and its `gen-docs` binary renders into `reference.html`, drawn from the single doc corpus of Chapter 20, so the catalogue and the editor and the published reference all agree. This first reference chapter covers the string and substitution builtins — the heart of a text tool — eleven functions plus the substitution family.

`length(s)`

`length([s])` returns the length of the string `s`, or of `$0` when called with no argument, and — the `gawk` extension `awkrs` implements — the *element count* of an array when `s` is an array name. The bare `length` with no parentheses is the idiomatic “length of the current record,” parsed specially so that `length` alone means `length($0)`. By default the count is in characters; under the `-b/--characters-as-bytes` flag it is in bytes, which matters for multibyte UTF-8 text where a character may be several bytes.

```
{ if (length($0) > 80) print FNR, "long line" } # flag lines over 80 chars
END { print length(seen), "distinct keys" }    # array element count
```

The two uses in one program show the overload: `length($0)` measures the record, and `length(seen)` counts the keys of an associative array. The array form is what makes a distinct-value count a single call, and it is `gawk`'s, not POSIX's — a traditional `awk` would make you maintain a counter.

`substr(s, m [, n])`

`substr(s, m, n)` returns the `n`-character substring of `s` beginning at the one-based position `m`; with `n` omitted it returns everything from `m` to the end. The edge cases are where `awks` diverge, and `awkrs` follows `gawk` deliberately: when the start `m` is less than one, `awkrs` clamps it to one and leaves `n` unchanged, where strict POSIX would instead shorten the result by `1 - m`. A start past the end of the string yields the empty string, and a negative or zero length yields the empty string. Positions and lengths are rounded toward zero after truncation, as `awk`'s integer coercion requires.

```
{ print substr($0, 1, 3) } # first three characters
{ print substr($0, index($0, "=") + 1) } # everything after the first '='
```

The second line is the universal “value after the delimiter” idiom — find the delimiter with `index`, take the substring after it — and it is exactly how `ini_parser.awk` in Chapter 26 splits a key from a value.

`index(s, t)`

`index(s, t)` returns the one-based position of the first occurrence of the string `t` within `s`, or `0` if `t` does not occur. An empty needle returns `1`, matching gawk — the empty string is found at the front of every string. `index` is byte-or-character oriented per the `-b` flag like `length`, and it is the tool to reach for when you want a literal substring search without the overhead or the metacharacter hazards of a regex.

```
{ if (index($0, "ERROR")) errors++ } # count records containing the literal ERROR
```

Note this is a *literal* search — `index($0, "a.b")` looks for the three characters `a`, `.`, `b`, not a regex — which is precisely why you choose `index` over `~` when the needle is plain text.

split(s, arr [, fs [, seps]])

`split(s, arr, fs)` breaks the string `s` into the array `arr` — populating `arr[1]`, `arr[2]`, and so on — using the field separator `fs`, and returns the number of fields produced. When `fs` is omitted, the current `FS` is used, so `split` honors the same separator rules as field splitting: a single space means whitespace-with- trimming, a single other character is literal, and a multi-character `fs` is a regex. awkrs implements the four-argument gawk extension where `seps` is filled with the actual separator text that appeared between consecutive fields, so `seps[i]` is the separator between `arr[i]` and `arr[i+1]` — which lets you reconstruct the original string with its exact delimiters. A defined corner case awkrs honors: `split("", arr, fs)` returns `0` and produces no empty pseudo-field, and `split` clears `arr` of any prior contents before filling it.

```
{ n = split($0, parts, ","); for (i = 1; i <= n; i++) print i, parts[i] }
```

`split` is how you parse a delimited field into sub-fields without changing `FS` for the whole record — the natural tool when one column is itself a comma-separated list, as `ini_parser.awk` and `markov.awk` both use it.

patsplit(s, arr [, fpat [, seps]])

`patsplit(s, arr, fpat, seps)` is the `FPAT` analogue of `split`: instead of splitting *on* separators, it splits *by matching* the field pattern `fpat`, taking each match as a field, and returns the count. Its four-argument form fills `seps` with the gaps *between* the matches. This is the right tool for data where you can describe what a field looks like more easily than what separates them — the classic case being numbers embedded in noise. awkrs accepts two to four arguments for `patsplit` and rejects other arities at compile load time with a clear error, the arity discipline of Chapter 24.

```
BEGIN { n = patsplit("a12b345c6", nums, /[0-9]+/); print nums[1], nums[2], nums[3] }
```

Here `patsplit` extracts the three numeric runs `12`, `345`, `6` directly, ignoring the interspersed letters — a one-call extraction that `split` could not do, because there is no single separator to split on.

sub(re, repl [, target])

`sub(re, repl, target)` replaces the *first* match of the regex `re` in `target` — or in `$0` when `target` is omitted — with the replacement string `repl`, modifying the target in place and returning the number of substitutions made (`0` or `1`). In `repl`, an unescaped `&` stands for the whole matched text, and `\&` is a literal ampersand. Because `sub` writes back to its target, the target must be an lvalue — a variable, a field, or an array element — and when the target is `$0` or a field, the substitution triggers the record rebuild of Chapter 5.

```
{ sub(/^[\t]+/, ""); print } # strip leading whitespace from each line
```

This is half of the universal `trim` idiom; pairing it with a trailing-whitespace `sub` gives the full trim that `ini_parser.awk` defines as a helper.

gsub(re, repl [, target])

`gsub(re, repl, target)` is the *global sub*: it replaces every non-overlapping match of `re` with `repl`, returning the count of replacements. `awkrs` implements gawk's zero-width-match behavior, so `gsub(/,/, "-")` inserts a dash at every position including the ends, and `gsub(/x*/, ...)` matches the empty string between characters where `x` is absent. The same `&` and `\&` conventions as `sub` apply in `repl`. `awkrs` heavily optimizes the common `gsub` cases — when the target is `$0` the substitution and the record rebuild happen in one step, literal needles reuse a cached substring finder, and the memory-mapped fast path recognizes a literal-pattern `gsub(...); print` and copies a non-matching line straight through without running the VM.

```
{ gsub(/[0-9]/, "#"); print }      # mask every digit
{ n = gsub(/,/, ";"); print n }    # count and replace commas, print the count
```

The returned count makes `gsub` double as a counter — `gsub(/,/, ",")` replacing each comma with itself returns the number of commas without changing the string, a known `awk` idiom for “how many of these are there.”

gensub(re, repl, how [, target])

`gensub(re, repl, how, target)` is gawk's more powerful substitution, and it differs from `sub/gsub` in three ways: it does *not* modify the target in place but *returns* the new string; the `how` argument selects which match to replace — a number `N` for the Nth match, or `"g"/"G"` for all; and the replacement may contain `\1` through `\9` to refer to captured parenthesized groups, as well as `&` for the whole match. `awkrs` implements the `\N` backreference expansion in `repl`, a parity fix its report records. Because `gensub` returns rather than mutates, it composes into larger expressions where `sub/gsub` cannot.

```
{ print gensub(/(\w+)@(\w+)/, "\\2.\\1", "g", $0) } # swap user and host around @
```

The replacement `\\2.\\1` (doubled backslashes because the string literal eats one) rebuilds each match from its second then first captured group — the kind of field-reordering rewrite that the in-place `sub/gsub` cannot express because they have no capture references.

match(s, re [, arr])

`match(s, re)` finds the first match of the regex `re` in `s`, returns its one-based position (or `0` for no match), and as a side effect sets `RSTART` to that position and `RLENGTH` to the match length (or `RSTART` to `0` and `RLENGTH` to `-1` on no match). The three-argument gawk form `match(s, re, arr)` additionally captures the submatches into `arr`: for each successful capture group `i`, it writes `arr[i]` (the matched text), `arr[i, "start"]` (the one-based character index), and `arr[i, "length"]`, and — matching gawk — an *unmatched* optional group produces *no* entries rather than empty ones, so a program can test `(i, "start") in arr` to know whether group `i` participated.

```
{ if (match($0, /([0-9]+)\.([0-9]+)/, m)) print m[1], "point", m[2] }
```

This single call gives the position, the length through `RLENGTH`, and every group's text and offset — which is why `url_parser.awk` in Chapter 26 uses `match` with `RSTART/RLENGTH` to bite the scheme off the front of a URL.

sprintf(fmt, ...)

`sprintf(fmt, ...)` formats its arguments exactly as `printf` does — every conversion, flag, width, precision, and the dynamic and positional forms of Chapter 13 — but *returns* the formatted string instead of printing it. It

is the general string-construction tool: any text whose shape a format string captures (zero-padded, aligned, based, rounded) is built with `sprintf` and then stored, concatenated, or used as a key.

```
{ key = sprintf("%04d-%02d", $1, $2); count[key]++ } # build a sortable composite key
```

`huffman.awk` in Chapter 26 leans on exactly this — `sprintf("%012d %s %d", ...)` — to build a zero-padded key whose string comparison orders by several fields at once, the multi-key-sort idiom that string comparison alone makes possible.

tolower(s) and toupper(s)

`tolower(s)` returns a copy of `s` with every uppercase letter lowercased, and `toupper(s)` the reverse. They operate on the case-mappable letters and leave other characters unchanged. They are the tools for case-insensitive grouping when you want to normalize the data itself rather than turn on `IGNORECASE` for matching — folding a key to one case before counting, as `anagrams.awk` and `markov.awk` both do with `tolower` before building their tables.

```
{ count[tolower($1)]++ } # case-insensitive tally of the first field
```

Folding the key with `tolower` collapses `Error`, `ERROR`, and `error` into one bucket — the canonical-key technique applied to case, and the reason `markov.awk` lowercases every word before counting bigrams.

The -b flag: bytes versus characters

Three of the string builtins — `length`, `substr`, and `index` — count in *characters* by default but in *bytes* under the `-b/--characters-as-bytes` flag, and the distinction matters the moment the input is multibyte UTF-8, where a single character may occupy several bytes. By default, `length("café")` is 4 (four characters) even though the string is five bytes in UTF-8, and `substr("café", 4, 1)` is the `é`; under `-b`, `length` reports 5 and `substr` indexes by byte, so the same call would cut into the middle of the multibyte character. The default is the right one for text processing — you almost always mean characters when you say “the third character” — and `-b` exists for the cases where you are doing genuinely byte-level work and need byte offsets to line up with positions in a binary or fixed-width format. The choice is process-wide, set once by the flag, and it threads through all three builtins consistently so that `length`, `substr`, and `index` agree on what a “position” means. This is one more place `awk` follows `gawk`’s design: the same flag, the same scope, the same default, so a program that depends on character counting on one engine depends on it identically on the other.

Chapter 28 – The Numeric, Bitwise, and Type Builtins, in Full

The catalogue continues with the functions that operate on numbers rather than text: the POSIX math library, the random-number pair, the non-decimal and integer-division helpers, gawk's six bitwise operators, and the type-and-array introspection builtins. As in the previous chapter, each entry gives the signature awkrs's own Language Server reports, the precise semantics, the edge cases its parity work pinned, and a small example. Several of these functions are also the ones the JIT of Chapter 17 can lower to native code, and where that matters the entry says so.

`sin(x)`, `cos(x)`, `atan2(y, x)`

`sin(x)` and `cos(x)` return the sine and cosine of `x` interpreted in radians; `atan2(y, x)` returns the arctangent of `y/x` in radians, in the full range $-\pi$ to π , using the signs of both arguments to place the result in the correct quadrant. All three are eligible for the JIT — they lower to native fusevm ops (`AwkSin`, `AwkCos`, `AwkAtan2`) implemented as Cranelift libcalls that normalize a NaN result to `+nan`, the sign convention from Chapter 13 — so a tight loop calling them compiles to native code.

```
BEGIN { pi = atan2(0, -1); print sin(pi / 2) } # pi from atan2, then sin → 1
```

`atan2(0, -1)` is the standard awk idiom for π , since there is no π constant; from it every other trigonometric value follows.

`exp(x)`, `log(x)`, `sqrt(x)`

`exp(x)` returns `e` raised to the power `x`; `log(x)` the natural logarithm; `sqrt(x)` the square root. The edge cases are the domain errors, and awkrs follows gawk: `log` and `sqrt` of a *negative* argument emit a warning on stderr (awkrs: `warning:`) and return NaN rather than crashing or producing garbage, with the NaN sign normalized to `+nan`. The warning is gated so it fires when lint is active. This warning-on-negative behavior is exactly why `log` and `sqrt` stay on the interpreter and are *not* JIT-eligible — a pure native op cannot emit a host stderr warning — while the trap-free `exp` is.

```
BEGIN { print exp(1), log(exp(1)), sqrt(2) } # e, 1, 1.41421
```

`log(exp(1))` round-trips to `1`, and `sqrt(2)` shows the default `%.6g` output formatting of an irrational result.

`int(x)`

`int(x)` truncates `x` toward zero, discarding any fractional part — so `int(3.9)` is `3` and `int(-3.9)` is `-3`, *not* `-4` (it truncates, it does not floor). It is the tool for integer arithmetic in a language whose numbers are doubles, used constantly: `int(x / base)` in `base_conv.awk`, `int(i / 2)` for heap-parent indexing in `dijkstra.awk`, `int(old_r / r)` for the quotient in `gcd_extended.awk`. `int` is JIT-eligible — it lowers to the native `fusevm::Op::AwkInt`, a Cranelift truncation, and the README notes it as the first AWK builtin reached end-to-end into fusevm's native op set.

```
{ print int($1), int(-$1) } # truncation toward zero, both signs
```

rand() and srand(x)

`rand()` returns a pseudo-random number in the half-open interval `[0, 1)`. `srand(x)` seeds the generator with `x` and returns the *previous* seed; `srand()` with no argument seeds from the current time and returns the previous seed. The exact *sequence* a seed produces is, like every `awk`'s, not guaranteed to match any other engine — random sequences are explicitly unspecified by POSIX — but a given seed produces a *repeatable* sequence within `awk`s, which is what matters for reproducible runs. To get an integer in a range, scale and truncate: `int(rand() * n)` gives 0 to `n-1`.

```
BEGIN { srand(42); for (i = 0; i < 3; i++) print int(rand() * 100) }
```

Seeding with a fixed 42 makes this print the same three numbers every run — the pattern for a test that needs randomness but must be deterministic. Note that the parallel path of Chapter 15 derives a per-record seed deterministically from a base seed and the record index, so `rand()` is reproducible even under `-j`.

strtonum(s)

`strtonum(s)` converts the string `s` to a number, honoring a `0x/0X` hexadecimal prefix and a leading `0` octal prefix — so `strtonum("0x1F")` is 31 and `strtonum("010")` is 8. This is `gawk`'s function for reading non-decimal numeric text *explicitly*, the per-call equivalent of what the `-n/--non-decimal-data` flag turns on for input generally. Plain decimal text converts as ordinary numeric coercion would.

```
{ print strtonum($1) } # read $1 honoring 0x hex and 0 octal
```

Where `$1 + 0` would read `0x1F` as just `0` (stopping at the `x`), `strtonum($1)` reads it as 31 — the difference between `awk`'s default decimal coercion and explicit non-decimal parsing.

intdiv(a, b, arr) and intdiv0(a, b)

`intdiv(a, b, arr)` is `gawk`'s integer division: it computes the integer quotient and remainder of `a` divided by `b`, storing `arr["quotient"]` and `arr["remainder"]`, and returns the quotient. `awk`s additionally provides `intdiv0(a, b)`, a variant that returns 0 rather than raising the fatal division-by-zero error when `b` is zero — the “safe” integer division for code that would rather get a zero than abort on a zero divisor.

```
BEGIN { q = intdiv(17, 5, r); print q, r["remainder"] } # 3 2
```

`intdiv` is the clean way to get both quotient and remainder in one call, with the remainder delivered through the array out-parameter — the same out-parameter idiom the string builtins use for their result arrays.

and, or, xor, compl, lshift, rshift

`gawk`'s six bitwise operators treat their arguments as unsigned 64-bit integers. `and`, `or`, and `xor` are variadic — each takes two or more arguments and folds the operation across them — while `compl(v)` is the bitwise complement, `lshift(v, n)` shifts `v` left by `n` bits, and `rshift(v, n)` shifts right. The edge case `awk`s matches `gawk` on: `compl`, `lshift`, and `rshift` of a *negative* argument are fatal with the message *negative values are not allowed*, because a negative complement or shift count is a value-dependent trap. This distinction governs the JIT — the trap-free `and`, `or`, and `xor` lower to native `band/bor/bxor` over a saturating float-to-int conversion and are JIT-eligible, while the trapping `compl/lshift/rshift` stay on the interpreter because only it can faithfully raise the fatal.

```
BEGIN { print and(12, 10), or(12, 10), xor(12, 10), lshift(1, 4) } # 8 14 6 16
```

`fenwick.awk` in Chapter 26 builds its whole binary-indexed-tree index arithmetic on `and(x, x - 1)`, the low-bit trick — proof that the bitwise builtins do real algorithmic work, not just flag-masking.

typeof(x)

`typeof(x)` reports the runtime type of `x` as a string, using gawk 5.x vocabulary: `"number"`, `"string"`, `"strnum"` (a numeric string from input), `"array"`, `"regexp"` (a `@/.../` value), `"untyped"` for an unset variable, and `"unassigned"`. The subtle case awkrs pins: `typeof` of noisy numeric text like `"42abc"` reports `"string"` — a numeric *prefix* is not enough to make a value a `strnum` — while pure-numeric input text like `"42"` reports `"strnum"`. awkrs fixed `typeof` to return `"untyped"` for unset variables to match gawk.

```
{ print $1, typeof($1) } # "42" → strnum, "42abc" → string, "" → untyped(ish)
```

`typeof` is the tool for the rare program that needs to branch on whether a value came from input looking like a number, the distinction Chapter 6 built the comparison rules on.

isarray(x) and mkbool(expr)

`isarray(x)` returns `1` if `x` is an array and `0` otherwise — useful in a function that accepts either a scalar or an array parameter and must decide which it got. `mkbool(expr)` produces a *typed boolean* value (gawk’s boolean type) from the truth of `expr`, returning a value that is `1` or `0` but carries the boolean type that `typeof` can later report. Both are gawk extensions awkrs implements.

```
function dump(x) { if (isarray(x)) for (k in x) print k, x[k]; else print x }
```

`isarray` lets one function handle both kinds of argument — the array-or-scalar dispatch that Chapter 9’s strict array/scalar boundary otherwise makes fatal to get wrong.

asort(src [, dst [, how]]) and asorti(src [, dst [, how]])

`asort(src, dst, how)` sorts the *values* of the array `src` and stores them under integer keys `1, 2, 3, ...` in `dst` — or back into `src` if `dst` is omitted — returning the element count. `asorti` does the same for the *indices* of `src`. Both accept a `how` string selecting the comparison mode, the same `@ind_str_asc/ @val_num_desc` vocabulary as `PROCINFO["sorted_in"]`. The corner case awkrs pins: `asort/asorti` on an *unassigned* name treats it as an empty array and returns `0`, while a *scalar* argument raises the “first argument is not an array” fatal, and the compiler tracks these argument positions so the name is promoted to an array slot correctly. One parallelism note: `asort/asorti` in a record rule make the program not parallel-safe, because they impose a global ordering.

```
END { n = asort(vals, sorted); for (i = 1; i <= n; i++) print sorted[i] }
```

`asort` is how you get a *sorted copy* when you need the values in order but want to keep the original array intact — distinct from `PROCINFO["sorted_in"]`, which orders an *iteration* without producing a new array. `ini_parser.awk` uses `asort` to order a section’s keys before printing them.

The missing math, supplied by idiom

awk’s math library is the POSIX set plus gawk’s additions, and a few functions a programmer expects are conspicuously *absent* — there is no `abs`, no `floor`, no `ceil`, no `min/max`, no `pi` constant. They are absent because each is a one-expression idiom, and the tour supplies every one of them inline. Absolute value is the

ternary `(x < 0) ? -x : x`, written out in `gauss_elim.awk`'s pivot search, `pollard_rho.awk`'s `abs1`, and `convex_hull.awk`. Floor and ceiling come from `int`, which truncates toward zero: `int(x)` is floor for non-negative `x`, and `floor` for negatives is `(x == int(x)) ? x : int(x) - 1`, with ceiling the mirror — though most programs that want floor have non-negative values and use `int` directly, as `base_conv.awk`'s `int(x / base)` and every heap's `int(i / 2)` do. The minimum of two values is `(a < b) ? a : b`, spelled out in `levenshtein`, `manacher`, and the heap sift-downs. And π , when needed, is `atan2(0, -1)`, the standard trick, since there is no constant. The absence is deliberate minimalism: each of these is shorter to write inline than to look up, and adding them would grow the library Chapter 11 argued should stay small. The lesson for a programmer coming from a language with a fat math module is that `awk`'s math is the core plus a handful of one-liners you write yourself — and once `(x < 0) ? -x : x` and `int` and the two-argument ternary min are in your fingers, nothing is actually missing.

Chapter 29 – The I/O, Time, and gawk-Extension Builtins, in Full

The last catalogue chapter covers the functions that reach outside the computation — the output and input statements, the handle-management calls, the time builtins, and the substantial set of gawk extensions awkrs implements as native Rust builtins rather than as loadable `.so` modules. As Chapter 12 stressed, this is awkrs's central trade: gawk's *capabilities*, implemented natively inside the one binary, without gawk's dynamic-extension *mechanism*. Every I/O-touching builtin here routes through the sandbox check first, so `-S/--sandbox` (Chapter 24) can block them, and they set `ERRNO` on failure in the gawk manner.

print and printf

`print arg1, arg2, ...` writes its arguments to output separated by `OFS` and terminated by `ORS` — which is why you never write a trailing newline in a `print`, the separator being part of the statement. With no arguments, `print` writes `$0`. `printf fmt, ...` writes formatted output using the C-style format string of Chapter 13, with no implicit separator or terminator — it emits exactly the bytes the format produces. Both accept redirection: `> file` (overwrite), `>> file` (append), `| cmd` (pipe to a command), and `|& cmd` (coprocess). awkrs accepts the function-call form `printf("fmt", a, b)` as equivalent to the statement form, a parity fix, though a mixed `printf(a, b), c` is still rejected.

```
{ print $1, $3 > "out.txt" }      # redirect to a file (OFS between fields)
{ printf "%-10s %5d\n", $1, $2 } # formatted columns, explicit newline
```

The comma between `print` arguments inserts `OFS`; juxtaposition without a comma concatenates with no separator — `print $1 $2` glues them, `print $1, $2` spaces them, a distinction every awk program depends on.

getline and its forms

`getline` is the explicit record read, and it comes in the forms Chapter 7 detailed, each returning the protocol value `1` (read), `0` (end of input), `-1` (error), or awkrs's added `-2` (retryable, when a `PROCINFO` timeout is configured). Bare `getline` reads the next main-input record into `$0`, updating `NF`, `NR`, and `FNR`; `getline var` reads into a variable, updating `NR` and `FNR` but not the fields; `getline < "file"` reads from a named file into `$0`; `getline var < "file"` into a variable; `cmd | getline [var]` reads a command's output; and `cmd |& getline var` reads from a coprocess.

```
BEGIN { while ((getline line < "config") > 0) n++; close("config"); print n }
```

The parentheses around `getline line < "config"` are mandatory — without them the `<` would bind as a comparison rather than a redirection — the precedence trap Chapter 7 and Chapter 21 both flag.

close(file [, how]) and fflush([file])

`close(name)` closes the file or command previously opened under the exact string `name`, returning `-1` for a name that was never opened, and for a clean close the command's exit code or `0`. The two-argument

`close(cmd, "to")` / `close(cmd, "from")` closes one direction of a coprocess. `fflush()` flushes all buffered output; `fflush(name)` flushes one named handle. `awkrs` implements both with `gawk` parity, including the detail that `system()` flushes buffered stdout, pipes, and files before running its subprocess so output ordering across the boundary is correct.

```
END { for (h in seen) print h | "sort"; close("sort") } # flush the pipe to sort
```

Closing the pipe to `sort` is what forces `sort` to run and finish before the program exits — without the `close`, the piped data might still be buffered.

system(cmd)

`system(cmd)` runs `cmd` through the shell, waits for it to finish, and returns its exit status. Before launching the subprocess it flushes `awkrs`'s buffered output, so that anything `printed` before the `system` call appears before the subprocess's own output. Under `-S/--sandbox`, `system` is blocked.

```
{ if (system("test -f " $1) == 0) print $1, "exists" }
```

The return value is the subprocess's exit status, so testing it against `0` is how an `awk` program branches on whether an external command succeeded.

systeme(), strftime(fmt [, ts [, utc]]), mktime(spec [, utc])

`systeme()` returns the current time as seconds since the Unix epoch. `strftime(fmt, ts, utc)` formats the timestamp `ts` (default: now) using a `strftime`-style format string (default: `awkrs`'s `gawk`-equivalent `"%a %b %e %H:%M:%S %Z %Y"`, exposed as `PROCINFO["strftime"]`), interpreting the time as UTC when the third argument is truthy. `mktime(spec, utc)` converts a `"YYYY MM DD HH MM SS [DST]"` specification string to a timestamp, with the optional second argument forcing UTC interpretation when truthy and the one-argument form using local time.

```
BEGIN { print strftime("%Y-%m-%d", systeme()) } # today's date, ISO style
```

Together these three are `awk`'s date arithmetic: `mktime` turns a broken-down time into a number, `systeme` gives the current number, and `strftime` turns a number back into formatted text — so date differences become plain subtraction.

gettimeofday(arr) and getlocaltime(arr [, ts])

`gettimeofday(arr)` fills `arr` with high-resolution current-time fields, and `getlocaltime(arr, ts)` fills `arr` with the broken-down local-time components (year, month, day, hour, minute, second, and the rest) of the timestamp `ts` or the current time. These are `awkrs`'s native equivalents of `gawk`'s bundled time extension, delivering their results through the array out-parameter.

```
BEGIN { getlocaltime(t); print t["year"], t["hour"] }
```

The broken-down form is what you reach for when you need a specific component of the current time — the hour, the day of week — without formatting and re-parsing a string.

The filesystem builtins: stat, statvfs, readdir, fts, chdir, rename, readfile

These are awkrs's native implementations of gawk's `filefuncs` and `readdir` extensions, which is why `@load "filefuncs"` is a harmless no-op — the functions are already present. `stat(path, arr)` fills `arr` with a file's metadata (type, size, and on Unix the full `stat` fields — device, inode, mode, link count, uid, gid, rdev, block size, blocks, and the access/modify/change times), returning `0` or `-1`. `statvfs(path, arr)` fills `arr` with filesystem statistics. `readdir(path, arr)` reads a directory into `arr`, each entry as "filename/filetype" where the type is `d`, `l`, `f`, or `u`. `fts(root, arr)` walks a directory tree recursively into a sorted array. `chdir(path)` and `rename(old, new)` perform the directory and file operations, returning `0/-1` and setting `ERRNO`. `readfile(path)` returns a whole file's contents as a single string.

```
BEGIN { n = readdir(".", entries); for (k in entries) print entries[k] }
```

`readdir` turns awk into a directory lister; combined with `stat` it becomes a tiny `find`, all within the one binary and all gated by the sandbox check.

Array persistence: writea(path, arr) and reada(path, arr)

`writea(path, arr)` writes the array `arr` to disk in awkrs's own tab-separated "rarray" format with a magic header (`awkrs-rarray-v1`), and `reada(path, arr)` reads it back. Together they let an awk program *checkpoint* a table between runs — build up a counter array over one input, persist it, and reload it on the next run to continue.

```
END { writea("counts.rarray", count) } # persist the tally for next run
```

This is the gawk `rarray` extension, native in awkrs, and it is the closest awk comes to a persistent data store without leaving the language.

ord(s), chr(n), and sleep(seconds)

`ord(s)` returns the code point of the first character of `s`, and `chr(n)` returns the single-character string for code point `n` — the inverse pair that `vigenere.awk` builds by hand in `BEGIN` but that awkrs also provides as builtins. `sleep(seconds)` suspends the program for a fractional number of seconds.

```
BEGIN { print ord("A"), chr(97) } # 65 a
```

`ord` and `chr` are the bridge between characters and their numeric codes, the operation at the heart of every cipher, hex dump, and byte-level transform.

In-place editing: inplace_tmpfile and inplace_commit

`inplace_tmpfile(path)` and `inplace_commit(tmp, dest)` are the primitives for safe in-place file editing — the first creates a uniquely-named temporary in the same directory as `path`, and the second atomically renames the temporary over the original. The atomic-rename discipline means a reader either sees the whole old file or the whole new one, never a half-written state, the same durability trick the bytecode cache uses.

```
BEGIN { tmp = inplace_tmpfile(FILENAME) }  
{ print transform($0) > tmp }  
END { inplace_commit(tmp, FILENAME) }
```

These give awk gawk's `inplace` capability — editing a file in place without a separate `>` redirect and `mv` — with the safety of an atomic swap.

The gettext builtins and the demo extensions

`bindtextdomain`, `dcgettext`, and `dcngettext` are the gettext internationalization builtins, backed by `.mo` catalogs through the Rust `gettext` crate, with `TEXTDOMAIN` selecting the active domain — so an awk program's user-facing strings can be localized. Finally, `revoutput` and `revtwoway` are gawk's demo extensions that reverse a string, kept for completeness. The compatibility matrix marks the gettext builtins `Part`, an honest signal that the localization path is present but not exhaustively verified.

```
BEGIN { TEXTDOMAIN = "myapp"; print dcgettext("Hello") }
```

The gettext family is niche but real, and its presence means a localized awk program — one that prints translated messages — runs under awkrs without modification.

The sandbox boundary, applied to the catalogue

A unifying property of this chapter's builtins is that every one that reaches outside the process routes through awkrs's sandbox check first, so the `-S/--sandbox` flag of Chapter 24 can close them as a group. Knowing exactly which builtins are gated is useful when running an untrusted program. Blocked under `-S`: `system` (no subprocess), redirected `print/printf` and `getline` (no file writes or reads beyond the given input), the pipe and coprocess forms (no `| cmd` or `|&`), the `/inet` socket connections (no network), and the filesystem builtins `stat`, `statvfs`, `readdir`, `fts`, `chdir`, `rename`, `readfile`, `writeta`, `reada`, and the in-place editing primitives (no filesystem inspection or mutation). *Not* blocked: the pure-computation builtins — the string, math, bitwise, time, type, and array functions of Chapters 27 and 28 — because they touch nothing outside the process; `ord`, `chr`, and `sleep`; and reading the program's given input, since that is the data you handed it. The boundary is exactly “does this builtin reach outside the process,” and because awkrs routes every such builtin through one check, the flag is a single, reliable firewall rather than a per-feature patchwork. This is what makes `-S` a real capability for running a program you did not write over data you care about: you get its computation — all of the arithmetic, the text processing, the algorithms of the tour — while denying it the ability to run commands, write files, or open sockets, with the gating implemented once at the point every outside-touching builtin already passes through.

Chapter 30 – The Special Variables, in Full

awk's special variables are the dials of the machine — the built-in globals that control how input is split, how output is joined, and what the runtime reports about itself. Appendix A is the compact list; this chapter is the narrative version, each variable walked with its default, its precise behavior, and the edge cases awk's parity work pinned against gawk. The descriptions are the ones awk's Language Server shows on hover, drawn from the same doc corpus as the rest of the catalogue. They fall into six groups, and reading them in order is the fastest way to hold the whole control surface in mind.

Record and field state: \$0, 1...n, NF, NR, FNR, FILENAME

`$0` is the whole current record. Assigning to it re-splits the fields and recomputes `NF`. `$1`, `$2`, ... `$n` are the fields, split from `$0` by `FS`; `$NF` is therefore always the last field. Assigning to a field rebuilds `$0` by joining the fields with `OF5`, and assigning to a field past `NF` extends the record, creating intervening empty fields. Referencing a negative field number is a fatal error, matching gawk. `NF` is the number of fields in the current record; assigning to it truncates the record (dropping trailing fields and rebuilding `$0`) or extends it with empty fields, and an invalid `NF` assignment is fatal like gawk. `NR` is the total number of records read so far across all input files; `FNR` is the record number within the current file, reset to zero at each new file. `FILENAME` is the name of the current input file — empty during `BEGIN`, and set per file thereafter.

The bidirectional consistency among `$0`, the fields, and `NF` is the live record model of Chapter 5: these are not independent variables but mutually-maintained views of one record, and awk implements the full POSIX rebuild-and-re-split semantics in every direction.

Input control: FS, RS, RT, FIELDWIDTHS, FPAT, SUBSEP, IGNORECASE

`FS` is the input field separator, default a single space — which is *special*, meaning split on runs of whitespace with leading and trailing whitespace trimmed. A single other character is a literal separator; a multi-character `FS` is a regex. `RS` is the input record separator, default a newline; a single character makes that the terminator, `RS = ""` is paragraph mode (records separated by blank lines, with newline an additional field separator), and a multi-character `RS` is a regex (a gawk extension), all streamed correctly over stdin and files with trailing-newline trimming that matches gawk. `RT` is the text that actually matched `RS` for the current record — gawk's window into what terminated each record, set precisely even in paragraph mode, where it captures the full run of trailing newlines plus the separating blank lines. `FIELDWIDTHS`, when non-empty, switches to fixed-width field splitting, accepting gawk's `width`, `skip:width`, and `*` tokens, with the last field clamped to its declared width. `FPAT`, when non-empty, defines fields by a regex they *match* rather than by separators, with leftmost-longest semantics. `SUBSEP` is the subscript separator that joins multidimensional array keys into one string, defaulting to the non-printing byte `0x1c` precisely so it is unlikely to collide with real key text. `IGNORECASE`, when true, makes regex matching and comparison case-insensitive, with gawk's exact scope — it applies to multi-character regex `FS`, to `match`, `sub`, `gsub`, `split`, and `gensub`, and to `~/!~`, but *not* to a single-character string `FS` or `split` separator, which stay literal.

These seven variables are the entire input-shaping surface: between them you control what a record is (*RS/RT*), what a field is (*FS/FIELDWIDTHS/FPAT*), how multidimensional keys compose (*SUBSEP*), and whether matching folds case (*IGNORECASE*).

Output control: OFS, ORS, OFMT, CONVFMT

OFS is the output field separator, inserted between the arguments of a *print* and used to join fields when *\$0* is rebuilt, default a single space. *ORS* is the output record separator, appended by *print*, default a newline. *OFMT* is the format used to convert a number to a string specifically in *print*, default *"%.6g"*. *CONVFMT* is the format for *general* number-to-string conversion — concatenation, array subscripts, the string-compare fallback — also default *"%.6g"*. The important exception *awkrs* implements for both: an *integer-valued* number bypasses *OFMT/CONVFMT* and prints exactly, so a large integer is never mangled by six-significant-digit rounding, and *print 100000000* shows *100000000*, not *1e+08*.

The symmetry is worth noting: *FS/RS* shape input, *OFS/ORS* shape output, and *CONVFMT/OFMT* govern the number-to-text boundary — input separators in, output separators out, formatting in the middle.

Match results: RSTART and RLENGTH

RSTART is the one-based start position of the most recent *match()* call, or *0* if that match failed. *RLENGTH* is the length of that match, or *-1* on failure. The two are set as a side effect of every *match()*, and they are how the two-argument *match(s, re)* reports *where* and *how long* the match was — the three-argument form adds the capture array, but *RSTART/RLENGTH* describe the whole-match position regardless.

```
{ if (match($0, /[0-9]+/)) print substr($0, RSTART, RLENGTH) }
```

This is the canonical “extract the first number” idiom: *match* finds it, and *substr(\$0, RSTART, RLENGTH)* cuts exactly the matched span out — which is how *url_parser.awk* peels the scheme off the front of a URL.

Arguments and environment: ARGV, ARGV, ARGIND, ENVIRON

ARGC is the count of command-line arguments and *ARGV* the array holding them, with *ARGV[0]* the engine name and *ARGV[1]* through *ARGV[ARGC-1]* the operands — the file paths and *var=value* assignments the implicit loop walks, as Chapter 3 detailed. A program can edit *ARGV* or lower *ARGC* to change what it reads. *ARGIND* is the index in *ARGV* of the *current* input file, a gawk extension that lets a program know which operand it is processing. *ENVIRON* is an array of the process environment variables, so *ENVIRON["HOME"]* reads the home directory — values that arrive as numeric strings when they look like numbers, and so compare numerically.

```
BEGIN { print ENVIRON["USER"], "running", ARGV[0] }
```

ENVIRON and *ARGV* are *awk*’s connection to the world it was invoked from — the environment it inherited and the arguments it was handed.

Introspection and diagnostics: PROCINFO, SYMTAB, FUNCTAB, ERRNO, LINT, TEXTDOMAIN, BINMODE

PROCINFO is the richest introspection array — process and runtime facts keyed by name: *platform* (mapped to gawk’s *posix/mingw* vocabulary), *version*, process and group ids, *errno*, the API numbers, *identifiers*

(the program's variable names), `FS` (the active split mode), `strftime` (the default time format), the supplementary group ids, and `mb_cur_max`. It also carries the `sorted_in` control that steers `for-in` order, the per-input read-timeout and retry keys, and — in `-M` mode — the MPFR configuration (`prec`, `roundmode`, `prec_min`, `prec_max`, `gmp_version`, `mpfr_version`). `SYMTAB` is the symbol table as an array: reading or assigning `SYMTAB["x"]` aliases the global `x`, and it can be iterated and measured with `length`. `FUNCTAB` is the analogous array of the program's function names, supporting membership and iteration, which pairs with the indirect-call mechanism for dispatch over the program's own functions. `ERRNO` carries a description of the most recent I/O error from a `getline`, `close`, or `system`. `LINT`, when truthy, enables runtime lint warnings. `TEXTDOMAIN` selects the active gettext domain. `BINMODE` controls binary-mode I/O. The compatibility matrix marks `PROCINFO`, `SYMTAB/FUNCTAB`, `LINT`, `BINMODE`, and `TEXTDOMAIN` as **Part** — present with a documented subset of gawk's full behavior rather than complete parity.

```
BEGIN { PROCINFO["sorted_in"] = "@ind_num_asc" } # iterate arrays in numeric order
END   { for (f in FUNCTAB) print "defined:", f } # list the program's functions
```

These seven turn awk from a language you only write *in* into one a program can, to a degree, inspect *about itself* — the modest but real reflection that Chapter 12's introspection section put to work in dispatch tables and self-configuring scripts.

A worked reconfiguration: reshaping input and output

The special variables are most instructive seen working together to reshape a stream. Suppose the input is colon-separated and you want tab-separated output with the fields reversed and the records numbered. Every part of that is a special-variable setting:

```
BEGIN { FS = ":"; OFS = "\t"; ORS = "\n" }
{
    line = NR
    for (i = NF; i >= 1; i--) line = line OFS $i
    print line
}
```

`FS = ":"` in `BEGIN` makes the input split on colons from the first record; `OFS = "\t"` sets the output field separator to a tab; `ORS` stays the default newline. The body reads `NF` to know how many fields there are, walks them in reverse, and joins with `OFS`. A simpler version of the same idea — reverse nothing, just convert the separator — is the famous `$1 = $1` idiom: assigning any field forces the `$0` rebuild, which re-joins with `OFS`, so `BEGIN { FS=":"; OFS="\t" } { $1=$1; print }` retabulates colon data as tab data in one line. The point is that the dials are *compositional*: `FS` and `FIELDWIDTHS/FPAT` decide how input becomes fields, `OFS` and `ORS` decide how fields and records become output, `CONVFMT/OFMT` decide how numbers become text, and `SUBSEP` decides how multidimensional keys compose — and a program reshapes a stream by setting the relevant dials in `BEGIN` and letting the implicit machinery do the rest. `awkrs` caches the separators as byte slices and refreshes them only on assignment (Chapter 34), so setting them costs nothing per record; the reshaping is expressed in `BEGIN` and executed for free in the loop.

Chapter 31 – The Keyword and Statement Reference

The builtins are functions; the keywords are the grammar. This chapter catalogues awk's reserved words — the special patterns, the function declaration, the control-flow statements, and the I/O verbs — each with the one-line definition awkrs's Language Server shows on hover and the behavior the earlier chapters established. Where a keyword is gawk-only rather than POSIX, the entry says so. Together these are the whole statement vocabulary; there is nothing else to learn beyond the expression operators of Chapter 6 and the builtins of the last three chapters.

The special patterns: BEGIN, END, BEGINFILE, ENDFILE

BEGIN is the special pattern whose action runs once, before any input is read — the place for setup, for setting **FS**, for printing a header. **END** is its mirror, running once after all input is consumed — the place for the summary. **BEGINFILE** and **ENDFILE** are gawk's per-file analogues: **BEGINFILE** runs before each input file with **FILENAME** set (and can inspect **ERRNO** and call **nextfile** to skip an unreadable file), and **ENDFILE** runs after each file is exhausted. All four require an explicit action block — an action-less **BEGIN** is meaningless — and **next/nextfile** are invalid inside **BEGINFILE**, a constraint awkrs enforces in the VM.

```
BEGIN { FS = "," } END { print NR, "records" }
```

The declaration: function

function name(params) { body } defines a user function. Its parameters are its locals — awk has no separate local-variable syntax, so extra locals are declared as extra parameters the caller never supplies, conventionally set off by a run of spaces. Functions may be mutually recursive and defined in any order, since awk reads the whole program before running it. Scalars pass by value, arrays by reference. A bare **return** or falling off the end yields the empty string / numeric zero.

```
function max(a, b) { return (a > b) ? a : b }
```

Conditionals and loops: if, else, while, for, do

if (cond) stmt runs **stmt** when **cond** is true, with an optional **else stmt** for the false branch. **while (cond) stmt** repeats while **cond** is true. **for (init; cond; incr) stmt** is the C three-clause loop, and **for (key in array) stmt** iterates an array's keys (in unspecified order unless **PROCINFO["sorted_in"]** is set). **do stmt while (cond)** runs the body at least once, then repeats while **cond** holds. awkrs compiles each to jump-patched bytecode, fusing the counted-loop test into a single comparison-and-branch opcode where it can.

```
for (i = 1; i <= NF; i++) print i, $i
for (k in count) print k, count[k]
```

Loop control: break, continue

break exits the innermost **for**, **while**, or **do** loop immediately. **continue** skips to the next iteration of the innermost loop. Both are exactly C's, and both interact correctly with the **for-in** iteration protocol — **continue** returns to the key fetch, **break** jumps past the loop's teardown.

```
for (i = 1; i <= n; i++) { if (a[i] == "") continue; if (a[i] == "STOP") break; use(a[i])  
    }
```

Record control: next, nextfile, exit

next stops processing the current record and reads the next one, restarting the rule tests from the top — “I’m done with this line.” **nextfile** abandons the rest of the current input file and advances to the next, as if the file had ended. **exit [expr]** stops reading input and proceeds to the **END** rules before terminating, setting the exit status to **expr** — and crucially, **exit** from anywhere but **END** still runs the **END** rules first, so “bail out early but still print the summary” works. Only **exit within END** terminates immediately.

```
/^#/ { next }           # skip comment lines  
$1 == "END_OF_DATA" { exit } # stop here, but END still runs
```

The function return: return

return [expr] returns from a user function, optionally with a value. A function returns exactly one scalar through **return**; to yield several results it fills an array passed by reference or writes to agreed globals, the out-parameter idiom of Chapter 9.

```
function abs(x) { return x < 0 ? -x : x }
```

Array and I/O statements: delete, getline, print, printf, in

delete arr[k] removes one key; **delete arr** empties the whole array while leaving it an array; **delete** on a scalar is fatal, and on an unassigned name a silent no-op. **getline** reads a record explicitly in the several forms of Chapter 7, returning the 1/0/-1/-2 protocol. **print** and **printf** write to output as the previous chapter catalogued. **in** is both the membership test — **key in arr** is true when the key exists, without creating it — and the iteration construct **for (key in arr)**. The multidimensional membership form **(i, j) in arr** tests a **SUBSEP**-joined key.

```
{ count[$1]++ }  
END { if ("root" in count) print count["root"]; delete count }
```

switch, case, default

switch (expr) { case ...: ... } is gawk’s multi-way branch, and awkrs implements it in gawk’s dialect with two distinguishing features: a **case** can be a *regular expression* (**case /^POST/**) as well as a constant value, and — contrary to C — there is *no* implicit fall-through, so each **case** body runs only its own statements. **default** is the fallback taken when no **case** matches. The **break** statements remain the idiomatic way to close a case even though fall-through does not occur.

```
switch ($1) {  
  case "GET": case "HEAD": reads++; break  
  case /^POST|PUT$/:      writes++; break  
  default:                other++  
}
```

`switch`, `case`, and `default` are gawk extensions; a program targeting strict POSIX awk would write the same dispatch as a chain of `if/else if`, which is exactly how the tour's command-dispatch examples — `rpn.awk`, `trie.awk` — are written for portability.

Chapter 32 – The Compatibility Matrix, Walked

Chapter 14 introduced awkrs’s compatibility matrix and its honest framing — a feature matrix, not a proof of correctness. This chapter reads the matrix row by row, because for a tool whose entire promise is to run whatever awk invocation you throw at it, the exact shape of “how faithful, where” is reference material a reader genuinely looks up. The matrix compares awkrs against BSD awk, mawk, and gawk across the command line, source directives, language constructs, special variables, builtins, formatting, and regular expressions, and it grades each cell with five values: **Match** (intended to follow the reference, covered by tests or design), **Part** (a subset or different edge cases), **Ext** (an extension POSIX awk may lack), **No** (unsupported), and **—** (not applicable). The matrix is explicit that awkrs does not claim bit-identical behavior, zero defects, or complete coverage, and that where POSIX leaves behavior unspecified — random sequences, hash iteration order, subtle rounding — differences are expected and acceptable.

The executive stance

The matrix opens with a one-paragraph stance per engine. Against the *POSIX core*, awkrs implements a large subset, with `-P/--posix` toggling some ordering rules such as `for (i in a)` iteration without gawk-style sorting. Against *BSD awk* (nawk), many of awkrs’s features are gawk-only and so are marked **Ext** — they are extensions BSD awk lacks, not incompatibilities. Against *mawk*, the fast traditional awk, the extension sets differ, and awkrs accepts some of mawk’s `-W` tokens for CLI compatibility only. Against *gawk*, awkrs has the highest overlap, implementing many gawk builtins and globals directly or as native Rust builtins. Two awkrs-specific rows sit outside the comparison entirely: `@load` inlines `.awk` sources or maps known gawk module names but does *not* load arbitrary `.so` extensions, and the parallel-records path under `-j` is an awkrs-only execution mode that can diverge from any sequential reference for unsafe programs.

The command-line surface

On the command line, the POSIX core — `-f`, `-F`, `-v`, and the program-plus-operands form — is **Match** across the board. The gawk source flags `-e/-i` are **Match**, including the repeatable multiple-fragment form. A long run of gawk flags is graded **Part**, each for a documented reason: `-b` (characters-as-bytes) is wired into the runtime but flagged for verification against the release I/O paths; `-c` (traditional) is reserved with incremental strictness; `-d` (dump-variables) dumps after the run in an awkrs-specific format; `-D` (debug) produces a listing rather than gawk’s interactive debugger; `-l` (load) does library search but no dynamic `.so`; `-L` (lint) has warn and fatal modes; `-M` (bignum) implements the MPFR path with `PROCINFO["prec"]/roundmode`; `-o` (pretty-print) emits an AST listing rather than gawk’s reformatter text; `-p` (profile) gives an awkrs wall-clock summary rather than gawk’s profiler format; `-P` (posix) is a runtime flag with incremental strictness; `-S` (sandbox) blocks `system()` and the other outside-touching operations; and `-t` (lint-old) is the older lint. Several gawk flags are full **Match**: `-g` (the POT generator), `-k/--csv` (CSV mode), `-N` (use-lc-numeric, with the documented format-not-parse split), `-n` (non-decimal-data), `-O` (optimize, accepted with the JIT on unless `-s`), `-r` (re-interval, a no-op since intervals are always on), and `-s` (no-optimize, which disables the JIT). The mawk `-W` flag is **Part** — `help/version/exec=` are honored and other tokens accepted. And the two awkrs-original flags, `-j/--threads` and `--read-ahead`, are **Ext**, the parallel pool and its stdin chunking that no other awk has.

Source directives, namespaces, and language constructs

The source directives are mostly **Match**: `@include "file"` expands before parsing, `@namespace "ns"` and `ns::name` qualified identifiers work through the namespace pass, and only `@load` is **Part** — `.awk` sources inline but `.so` extensions do not load, the deliberate single-binary boundary. Among language constructs, `BEGIN/END`, range patterns, regex record patterns and compound `/re/ && expr` forms, `next/nextfile/exit`, user functions and `return`, `delete a[k]/delete a`, and `switch` are all **Match**. `BEGINFILE/ENDFILE` are **Match** as gawk-style extensions (with the `next/nextfile`-invalid-in-`BEGINFILE` rule enforced in the VM). Three constructs are **Part** for honest reasons: `for (i in a)` order (hash order versus `PROCINFO["sorted_in"]`, with `-P` skipping gawk ordering), indirect function calls (implemented, with some edge cases differing from gawk), the coprocess `|&` (the runtime has the types but full parity is not guaranteed), and the `getline` variants including the `PROCINFO` timeout/retry path.

Special variables and builtins

The core special variables — `NR`, `FNR`, `NF`, `$0`, `$n`, `FS`, `RS`, `OF5`, `ORS`, `OFMT`, `CONVFMT`, `FILENAME`, `ARGC`, `ARGV`, `ENVIRON`, `SUBSEP`, `RSTART`, `RLENGTH` — are **Match**, with the notable refinements that invalid `NF` and negative fields are fatal like gawk, that `RS` handling covers streaming multi-character and regex separators and paragraph mode over both stdin and files, that `FIELDWIDTHS` accepts the full `width/skip:width/*` token vocabulary with last-field clamping, and that `IGNORECASE` has gawk's exact scope. `RT`, `ARGIND`, and `ERRNO` are **Match**. `PROCINFO`, `SYMTAB/FUNCTAB`, `BINMODE`, `LINT`, and `TEXTDOMAIN` are **Part** — a documented subset rather than full coverage. On the builtin side, the math core (`atan2`, `cos`, `sin`, `exp`, `log`, `sqrt`, `int`) is **Match** with the warn-and-NaN behavior on negative `log/sqrt`; `length`, `index`, `substr`, `sprintf`, `tolower/toupper`, `system/close` are **Match**; `match/sub/gsub/split` are **Match/Part** because the regex engine is the Rust `regex` crate (so subtle differences are possible, while `gsub(//,...)` zero-width matches and the `split` fourth `seps` argument are implemented); `rand/srand` are **Part** (sequence not guaranteed to match any one engine); and the gawk-only families — `asort/asorti`, the time builtins, the bitwise operators, `isarray/typeof/mkbool`, `intdiv/intdiv0`, the gettext builtins, and the filesystem/OS extensions — are graded **Match**, **Part**, or **Ext** as their coverage warrants.

The formatting details

The matrix's longest section is `printf/print`/numeric formatting, and it is worth reading in full because byte-exact output is what its parity suite checks hardest. Every row here is **Match**: `%g/%G` use total significant digits with the fixed-versus-`e` decision made from the *rounded* exponent (so `%.1g` of `9.5` is `1e+01`); `%u` of a negative value wraps via two's complement rather than clamping; the `0` flag pads `%s/%c` with spaces, since POSIX restricts zero-padding to numeric conversions; unknown conversion letters emit the literal `%X` without consuming an argument; `%a/%A` hex floats are gawk-parity confirmed; non-finite floats spell as `+inf/-inf/+nan/-nan` (with `INF/NAN` for uppercase variants) consistently across `printf` and `print`; the `%'` thousands-grouping flag consults `localeconv` regardless of `-N`; a number compared against a string literal stringifies via `CONVFMT`; `a % 0` is fatal; `"inf"/"nan"` coercion accepts only the signed three-letter forms; the negative-argument bitwise fatal holds; `typeof` of noisy numeric text reports `"string"`; `match`'s capture subscripts write `start/length` with no entries for unmatched groups; `mktime`'s optional UTC argument works; assignment in a ternary else-branch parses; `asort` on an unassigned name returns zero; numeric `==` is bit-exact; paragraph-mode `RT` captures the full newline run; the `PROCINFO["strftime"]` default is gawk's date-equivalent; the `printf("fmt", ...)` function-call form is accepted; wrong builtin arity gives a uniform non-panicking error; `delete`/scalar-as-array confusion is fatal; and `%u` past `2^64` falls back to `%g`-style. Only the MPFR `-M` row is **Part**. This density of pinned corner cases is the matrix's strongest evidence that awkrs's formatting was built test-then-fix against gawk rather than approximated.

Regular expressions and the intentional divergences

On regular expressions, the engine is the Rust `regex` crate, not a GNU-regex copy. Interval quantifiers `{m,n}` are always enabled (so `-r` is a no-op); `IGNORECASE` is supported for the split/match contexts that consult the runtime; `.` matches newline via `dot_matches_new_line(true)`, the gawk ERE convention; POSIX character classes are **Match**. The one **No** in the whole matrix is pattern-side backreferences like `(.)\1` — the linear-time Rust engine does not support them — though replacement-text backreferences in `gensub` (`\1..\9, 6`) are fully supported. NUL bytes and binary input are **Part**, gated behind `-b/BINMODE`. Finally, the matrix closes with the known intentional or unavoidable divergences, which the book has met one at a time: the JIT must match the interpreter (a mismatch is a JIT bug, not “gawk is wrong”), and its eligibility is an allowlist of numeric ops with regex match lowering to a fusevm extended op that runs on the interpreter; parallel mode can diverge for side-effecting or order-dependent programs; dynamic `.so` extensions have no equivalent; `PROCINFO["platform"]` uses a `posix/mingw` style rather than every host’s gawk string; and `for-in` order differs across engines when sorting is off. Reading these divergences is the fastest way to calibrate expectations: everywhere else the matrix intends fidelity, and these are the handful of places where fidelity is either impossible (a different host string), unsafe to promise (parallel reordering), or deliberately declined (loading foreign native code) — each named openly rather than papered over, which is the matrix’s, and the book’s, governing posture.

Chapter 33 — Formatting Internals

Chapter 13 covered `printf` from the language's side — the conversions, the flags, the corner cases a program can observe. This chapter goes underneath, into `format.rs`, the twenty-three-hundred-line module that implements the formatting, because byte-exact output is the single most heavily parity-tested part of `awk` and the implementation is correspondingly careful. The module's own one-line description is modest — “`sprintf/printf` formatting (POSIX-ish; common `awk` conversions)” — but the care inside it is the reason the compatibility matrix's longest section is almost entirely **Match**.

One spec at a time

Formatting is a walk over the format string, copying literal bytes through and, at each `%`, parsing one conversion specification and consuming the matching argument. The parser for a single spec recognizes, in order: an optional positional selector `m$`; the flags `-`, `+`, space, `#`, `'`, and `0`; a field width that may be a literal, a `*` taking the width from an argument, or a `*n$` positional width; a precision introduced by `.` that may likewise be literal or argument-supplied; the length modifiers `h`, `l`, and `L`, which `awk` parses and ignores because `awk` has one numeric type at the language level; and finally the conversion letter. Each piece is parsed by a small focused routine, and the whole spec is then formatted by `format_one_spec`, which dispatches on the conversion letter to the right rendering. The two entry points are `awk_sprintf`, which uses the C-locale radix `.`, and `awk_sprintf_with_decimal`, which takes an explicit decimal point and thousands separator for the `-N` locale path of Chapter 19 — so the locale-aware and locale-free paths share one engine, differing only in the radix character threaded through.

The CONVFMT preprocessing pass for %s

A subtle `gawk`-parity requirement is that `printf "%s", 3.14159` under `CONVFMT="%.3f"` must show the `CONVFMT`-formatted string `3.142`, not the raw float's display form. The module handles this with a preprocessing pass, `convfmt_preprocess_for_percent_s`, that runs before formatting: it scans the format string to find which argument indices are consumed by a `%s` (carefully accounting for positional `%m$s` and the `*` width and precision specifiers that also consume arguments, mirroring the main parser just enough to count correctly), and for each such argument that is a number, it pre-converts that number to a string via `CONVFMT`. Other conversions keep the original numeric value, so `printf "%d %s", 3.7, 3.7` formats the first `3.7` as the integer `3` and the second through `CONVFMT`. The pass has a quick reject — if the format contains no `%s` at all it returns the arguments untouched, borrowed rather than cloned — so the common case pays nothing. This is the kind of precise, narrowly-scoped fix that turns a **Part** into a **Match**: the number-to-string boundary is honored exactly where POSIX requires it and nowhere it would be wrong.

%g and the rounded exponent

The trickiest conversion is `%g/%G`, and Chapter 13 named the subtlety: the choice between fixed and scientific form is made from the *rounded* exponent, not the raw one. The implementation computes the value's exponent, applies the requested precision (the number of significant digits, with a precision of zero treated as one per C99), and — critically — re-derives the exponent *after* rounding, because rounding can push a value across the threshold. The textbook case is `%1g` of `9.5`: rounded to one significant digit it becomes `10`, whose exponent is `1`, which crosses into scientific form, so the output is `1e+01` and not `10`. Exponents are always signed and

always at least two magnitude digits (`e+03`, never `e+3`), matching C. Getting the rounded-exponent decision right is what separates a `%g` that agrees with gawk on the boundary cases from one that does not, and it is one of the parity cases the suite pins explicitly.

`%u`, the two's-complement wrap, and the 2^{64} boundary

`%u` formats a value as an unsigned integer, and the boundaries are where awks diverge. The implementation follows gawk: a negative value wraps via an `i64`-to-`u64` reinterpretation (two's complement), so `%u` of `-5` is `18446744073709551611`, not a clamp to zero. Values in the representable `u64` range cast directly. The exact boundary 2^{64} renders as `u64::MAX`'s digit string, a saturating-cast parity point, and a value *strictly greater* than 2^{64} falls back to `%g`-style formatting, so `2^65` prints as `3.68935e+19`. An earlier awkr saturated everything over `u64` at `u64::MAX`; the current behavior matches gawk's transitions exactly, which the compatibility matrix documents.

`%a`, the hex float

`%a/%A` render a float in C99 hexadecimal form `[-]0xh.hhhhp±d`, and `format_hex_float` implements the full specification: it normalizes a subnormal by its leading one, truncates the hex mantissa digits to the requested precision with round-half-to-even, shows all significant hex digits trimming trailing zeros in the default (no-precision) form, uses `0X` and `P` for the uppercase variant, and honors the locale radix character. It is a rarely-used conversion, but it is the exact, round-trippable representation of a float's bits, and the matrix confirms gawk parity on it — a striking amount of care for a conversion most programs never touch, included because “drop-in replacement” means *every* conversion, not the common ones.

Non-finite values, spelled once

Infinities and NaNs must be spelled somehow, and the spelling has to be identical between `printf` and `print` or a program that mixes them gets inconsistent output. The module's `format_non_finite` produces gawk's spelling — `+inf`, `-inf`, `+nan`, `-nan` in lowercase contexts, `INF` and `NAN` for the uppercase `%E/%G` variants — and the same spelling is used by `runtime.rs`'s `format_number` for `print` and plain numeric-to-string conversion. So `print x` and `printf "%s", x` agree on how an infinity looks, a consistency that only happens because both paths were deliberately routed through the same convention. The NaN *sign* is normalized to `+nan` at the math-function source, so it is predictable too.

The traditional-mode flag and fuzz-hardening

Two more details round out the module. A process-wide atomic flag, `AWK_TRADITIONAL_MODE`, is set once from `--traditional` at startup and read with relaxed ordering inside `format_one_spec`; it opts into the older Bell-Labs/BSD behavior of zero-padding `%0Ns` and `%0Nc` string conversions, which POSIX forbids (the `0` flag is for numeric conversions only). Defaulting it off keeps gawk/POSIX parity for every caller that does not flip it, and implementing *both* behaviors rather than picking one is the drop-in-replacement discipline again. And the whole digit-parsing path uses saturating arithmetic with the field width capped at one hundred thousand, the fuzz-hardening of Chapter 24, so a malformed or hostile format string — a gigantic width built from untrusted input — produces bounded, defined output instead of an overflow or a runaway allocation. Formatting is exactly the place attacker-influenced data meets integer arithmetic, and `format.rs` is hardened for it.

Chapter 34 – The Runtime and the Field Engine

If `format.rs` is where output is shaped, `runtime.rs` is where everything else lives: at roughly five thousand lines it is the largest module after the parser, and it holds the central mutable state the VM operates on — the fields and `$0`, the variables and slots, the separators, the open I/O handles, the regex cache, and the MPFR integration. This chapter walks the runtime's structure and the field engine at its core, because the field engine is both the correctness story of Chapter 5 and a large part of the speed story of Chapter 16, and seeing the data structures makes both concrete.

The state and its hash map

The runtime's variables and arrays live in an `AwkMap`, which the module defines as a `rustc_hash::FxHashMap` — `FxHash` rather than the default `SipHash`, chosen because it is roughly twice as fast for the short string keys typical of awk programs and the DoS-resistance `SipHash` buys is irrelevant for a local text tool. This is a small, characteristic decision: the hash function is a hot path in every array-heavy program, and `awkrs` picks the fast one knowingly. The user *scalar* variables do not live in this map at all — they live in the flat slot vector of Chapter 16, indexed by `u16` — and the map holds the special variables, the arrays, and anything the slot allocator could not prove was a plain scalar. So a tight loop over a counter and an accumulator touches only the flat slot array, while the occasional `NR` or array access takes the `FxHashMap` path.

The print buffer

Output does not go to `stdout` per record. The runtime accumulates `print` output in a `Vec<u8>` that starts at half a megabyte (`DEFAULT_PRINT_BUF_CAPACITY = 512 KiB`) and flushes at file boundaries, so a million `print` statements become a handful of large `write` calls rather than a million small syscalls. The large initial capacity is deliberate — a big `END` block doing `for (k in a) print ...` grows the buffer heavily, and starting large avoids repeated reallocation — with no hard upper bound, so output size is never artificially capped. The `OF5` and `OR5` separators are cached as byte slices and refreshed only when the program assigns to them, so the common case of unchanged separators pays nothing to look them up.

The field engine

The field engine is the runtime's heart, and its central design choice is that a field is a *slice*, not a copy. A field is represented as an `(offset, length)` byte range into the record string, so reading `$3` does not allocate — it returns a view into `$0`. A field only becomes an owned `String` when you *assign* to it, at which point the record-rebuild machinery of Chapter 5 engages. Field splitting is *lazy*: the fields are not parsed until the program first references a field or `NF`, so a rule that never touches a field never pays to split the record. And number parsing has a fast path — `parse_number` tries a plain-decimal-integer parse before falling back to the general float parse, and `longest_f64_prefix` extracts the numeric prefix for string-to-number coercion. A thread-local flag controls whether parsing honors `0x/0` non-decimal prefixes, set by the `-n/--non-decimal-data` flag through `set_numeric_parse_mode`, so the same parse routine serves both the default decimal mode and `gawk`'s non-decimal mode without branching on a global.

Locale-aware comparison

String comparison is not always a plain byte compare. On Unix, `awk_locale_str_cmp` routes ordering comparisons through the C library's `strcoll`, so `for-in` value sorts and string comparisons honor the active locale's collation — and when `IGNORECASE` is set, the comparison folds case in a locale-aware way too. This is the gawk behavior, and implementing it means a program sorting strings under a non-C locale orders them the way the locale says, not the way raw bytes would. It is a detail invisible until you process non-ASCII text, and then it is exactly what you want.

I/O handles, sockets, and MPFR

The runtime owns the open I/O handles — the files and pipes that redirected `print` and `getline` open, keyed by the exact string used to open them, which is how `close(name)` finds the right one. It carries the coprocess child-process handles for `|&`, and — the extension that surprises people — it can open network sockets through the `socket2` crate, so a `/inet/tcp/0/host/port` special file becomes a real TCP connection (Chapter 12). And it integrates MPFR: the `Value` type includes an `Mpfr` variant holding a `rug::Float`, and under `-M` the arithmetic routes through it at the working precision `MPFR_PREC` (256 bits by default), with `value_to_mpfr` converting as needed. All of this — the buffered output, the slice-based fields, the `FxHashMap` variables, the locale comparison, the handle table, the socket support, and the MPFR values — is the single mutable state the VM reads and writes as it runs, which is why `runtime.rs` is large: it is the entire world the bytecode acts upon.

Field mutation: the materialize-rebuild-republish cycle

The slice representation above is a read optimization, and it has a direct consequence: the moment a program *writes* a field, the slices stop being a valid representation, because there is no longer a single backing string that contains every field's bytes. The runtime handles this with an explicit two-state model and a `fields_dirty` flag, and tracing an assignment through it explains several of awk's documented behaviors at once.

The state before any write is: `record` holds `$0`, `field_ranges` holds `(start, end)` byte offsets into it, and `fields` — the vector of owned strings — is unused. The state after a write is: `fields` holds owned strings, `fields_dirty` is set, and `record` has been regenerated from them.

`set_field` performs the transition. It first checks the index: a write to `$0` delegates to `set_record_str`, and an index below one is a runtime error with the message `attempt to access field number -1`. Then, if `fields_dirty` is not already set, it *materializes* — clears `fields` and pushes an owned `String` for each `(start, end)` range, copying out of the record — and sets the flag. This is the one allocation-heavy step, and it happens once per record no matter how many fields are then written, because subsequent writes see the flag already set.

With owned fields in hand, the write is straightforward: if the target index is past the current length, `fields` is resized with empty strings, which is how `$10 = "x"` on a three-field record creates fields four through nine as empty. Then the element is assigned.

Then two things must happen, and their order is fixed. `rebuild_record` joins the fields with the *current* `OF5` and stores the result as `record`, so `$0` reflects the write. And `NF` is republished into the variable map from `fields.len()`.

Three well-known awk behaviors fall directly out of this sequence.

Assigning to a field rebuilds `$0` with `OF5`, not with the original separators. A tab-separated record read with `FS = "\t"`, given `$1 = $1`, comes back joined with the default single space, because the join is unconditional and uses `OF5`. This surprises people, it is standard awk, and here it is visibly a consequence of there being no representation of the original inter-field bytes to restore — the ranges are gone the moment the fields are materialized.

Assigning past `NF` extends the record. The `resize-with-empties` plus the `rebuild` produce a `$0` with the intervening `OF5` separators present and the intervening fields empty, and an `NF` reflecting the new length.

And the `OF5` used is the one live at `rebuild` time. Changing `OF5` and *then* touching a field applies the new separator; changing it after the last field write does not retroactively rejoin.

Assigning to `NF`, and to `$0`

`set_nf` is the same machinery driven from the other end. A negative value is a runtime error. Otherwise the fields are split if not already, materialized if not already dirty, and then either truncated to the new count or resized upward with empty strings — the same `Vec` operation in both directions. Then `rebuild_record` and the `NF` republish. So `NF = 2` genuinely discards fields three onward and shortens `$0`, and `NF = 10` genuinely appends eight empty fields with separators, in both cases through the ordinary `rebuild` rather than a special case.

`set_record_str` runs the cycle in reverse. Assigning to `$0` reads the current `FS`, re-splits the new string under it, forces the split to complete, and republishes `NF` from the resulting count. So `$0 = "a:b:c"` under `FS = ":"` yields `NF` of three, and the fields are once again slices into a fresh record — the dirty state is left behind and the cheap representation is restored.

The numeric field write

`set_field_num` exists as a separate entry point from `set_field`, and the reason is a small allocation-shaping decision worth noting because it is representative of how the runtime is tuned.

The naive path for `$3 = x + 1` is: compute the number, wrap it in a `Value::Num`, call `as_str()` on it to get a `String`, and pass that to `set_field`, which stores it. That is one allocation for the intermediate string plus whatever `set_field` does with it.

`set_field_num` takes the `f64` directly and formats it *into the existing String* at that index — `clear()` on the current element, then `write!` into it — reusing the allocation the field already owns. In a program assigning a computed value to a field on every record, that is one saved allocation per assignment per record, on a path that runs as often as records exist.

The formatting rule is the same in both entry points and is worth stating because it is observable: a finite value with no fractional part formats through `{:.0}`, producing integer text with no decimal point, and everything else formats through the default float rendering. So `$3 = 4.0` puts `4` in the field, not `4.0` — the integer-valued float convention `awk` uses throughout (Chapter 6).

Why this design rather than always-owned fields

The alternative is to always hold owned field strings, which would delete the `fields_dirty` flag, the materialize step, and the dual representation. It would also allocate one `String` per field on every record of every program, including the enormous majority that only ever *read* fields.

The measured shape of real `awk` programs is that field reads outnumber field writes by a wide margin — `{ print $1, $3 }` and `$3 > 100 { print }` never write a field at all — and that record counts are large. Under those conditions, paying a copy only on the records that are actually mutated, at the cost of a branch on a flag in the write path, is the right side of the trade. The lazy-split decision compounds it: a rule that references no field never even builds the ranges. The dual representation is therefore not premature optimization but a direct encoding of the read/write asymmetry of the language's actual usage, and the `fields_dirty` flag is the one bit of bookkeeping that makes the two states safe to move between.

Chapter 35 – The Namespace and Source-Expansion System

Two small modules handle the pre-parse transformations that let an awk program span multiple files and avoid name collisions: `source_expand.rs` expands the `@include` and `@load` directives, and `namespace.rs` applies gawk's `@namespace` prefixing. Both run before the parser sees the final program text or the final AST, and both are governed by the same single-source-of-truth name tables the rest of the engine reads. This chapter covers them together, because they are the machinery behind Chapter 12's source directives and Chapter 9's namespaced functions.

Source expansion: `@include` and `@load`

`source_expand.rs` runs a pre-parse pass over the program source, processing the `@`-prefixed directives gawk added. `@include "file"` textually includes another awk source file, expanded in place before parsing — the awk equivalent of a C `#include` — with a set of already-included paths tracked so a diamond of includes does not expand a file twice or loop forever. The directive parser, `take_double_quoted`, reads the quoted filename, handling the escape sequences `\n`, `\t`, `\r`, `\\`, and `\"` within it. The firm boundary is `@load`: a `@load` of a `.awk` path is inlined exactly like an include, and a `@load` of one of gawk's *bundled* extension names is accepted as a no-op because the functionality is already a native builtin — but a `@load` of an arbitrary `.so` shared object errors rather than pretending to load foreign native code. This is the single-binary discipline drawn at the source level: awkrs will assemble your program from multiple awk files, but it will not load compiled extensions, by design.

Namespace prefixing: the two name tables

`namespace.rs` implements `@namespace "ns"`, which prefixes unqualified identifiers inside a namespace with `ns::` so a library can use short internal names without clashing with the calling program. The prefixing pass walks the whole AST, and the rule is: an unqualified name becomes `ns::name` except for names that are already qualified (containing `::`), function parameters and locals (which stay local), the special global variables, and the builtin function names. The two exception tables are the project's single source of truth for “what names are special,” and they are exactly the tables the rest of the engine consults. `BUILTIN_NAMES` lists the builtin function names — the catalogue of Chapters 27 through 29, in one alphabetized array — and the namespace pass must leave every one of them unprefixed so that `length` inside a namespace is still the builtin `length`, not `ns::length`. `SPECIAL_GLOBAL_NAMES` lists the special variables of Chapter 30, which must likewise stay unprefixed so `NR` inside a namespace is still the record count. The same `BUILTIN_NAMES` table feeds the LSP's completion list and the arity checker, and the same special-name table feeds the compiler's slot-versus-map decision — so a builtin or special added in one place is known everywhere, and the namespace pass can never disagree with the runtime about which names are reserved.

Why prefixing at the AST level

The choice to prefix at the AST level rather than the source text is what makes the mechanism robust. A textual prefix would have to re-implement awk's lexer to know what is an identifier and what is a string or a

comment; walking the parsed AST means the pass sees genuine identifier nodes and can prefix exactly those, leaving string literals and the insides of regex literals untouched. The pass renames a namespaced function's definition and its qualified call sites while leaving the function's own parameters unprefixed, because they are locals. A qualified name like `awk::x` — the default namespace gawk uses — stays as written. The result is gawk's solution to the library-collision problem, implemented as a clean AST transformation: a function library written under `@namespace "graph"` exposes its `bfs` as `graph::bfs` to the outside while using bare `bfs` internally, and the prefixing pass makes the two views consistent.

Cycle detection, and why it uses a scoped set

The include expander tracks visited files in a `HashSet<PathBuf>` of canonicalized paths, and the way it manages that set is the detail that makes diamond includes work while still catching real cycles.

A path is inserted before its contents are expanded and **removed after**. So the set is not a record of “every file included anywhere in this program” — it is the set of files currently *on the expansion stack*. A cycle is detected when a file appears while it is already being expanded, which is the definition of a cycle, and the error names the offending path: `@include cycle: <path>`.

The scoping matters because the alternative — a permanent visited set, which is what C's `#pragma once` implements — would silently *drop* the second inclusion of a shared library file. In awk, where an included file is a set of function definitions, dropping the second inclusion happens to be harmless; but including it twice would be a duplicate function definition and a parse error. awkrs takes neither behavior implicitly: a diamond where two included files each include a common third expands that third file twice, and the resulting duplicate definitions are a parse error the user sees, at parse time, with a message about the duplicate rather than a mysterious inclusion that vanished. Whether that is the friendlier choice is arguable; that it is the *legible* choice is not, because a silently-skipped include is invisible and a duplicate-definition error is not.

Path canonicalization is best-effort: `fs::canonicalize` is attempted and the unresolved path used on failure. That matters because canonicalization requires the file to exist, and using the resolved path when it does exist means two spellings of one file — a relative path and an absolute one, or a path through a symlink — are recognized as the same file rather than expanding twice.

Relative paths resolve against the including file

`resolve_include_path` implements the rule that makes multi-file programs relocatable. An absolute path is used as written. A relative path resolves against the *directory of the file doing the including* — not the process's working directory — and only the top-level program, which has no including file, resolves relative paths against the current directory.

The effect is that a library directory can be moved wholesale: a file at `lib/graph.awk` that includes `"util.awk"` gets `lib/util.awk` regardless of where the program is run from, which is the behavior every include mechanism converged on for the same reason. The recursion carries the base directory forward as the expander descends, so the rule holds at every depth.

The @load boundary, enumerated

Chapter 12 described `@load`'s policy; the implementation makes the three branches explicit, and the list in the middle branch is worth having in full.

A `@load` whose path ends in `.awk` — case-insensitively — is treated exactly like an `@include`: resolved, cycle-checked, read, and expanded in place.

A `@load` naming one of gawk's *bundled* extension modules is accepted and ignored. The recognized names are `filefuncs`, `readdir`, `time`, `inplace`, `ordchr`, `readfile`, `revoutput`, `revtwoway`, `rvarray`, and `intdiv`.

The match is on the path's file stem, lowercased, so `@load "filefuncs"`, `@load "filefuncs.so"`, and a path with a directory prefix all recognize the same module. Ignoring rather than erroring is correct here because `awkrs` implements these modules' functionality as native builtins already present for the whole run — the directive's *effect* is satisfied before the directive is read, so the right response is to consume the line and continue.

Anything else — an arbitrary third-party `.so` — is a parse error, and the error text states the whole policy in one message: `awkrs` inlines `.awk` source or recognizes `gawk`'s bundled extension names implemented natively, and does not load arbitrary `gawkapi` modules. This is the single-binary discipline (Chapter 39) enforced at the point of violation, with an error that explains the rule rather than merely refusing.

The directive line parser

Both `@include` and `@load` take a double-quoted path, parsed by `take_double_quoted`, which is small enough to describe completely and has two behaviors that matter.

It handles the escapes `\n`, `\t`, `\r`, `\\`, and `\"` inside the quoted string, and any other backslash-escaped byte passes through as itself. And it *rejects* a raw newline inside the quotes, returning nothing — so an unterminated quote does not consume the rest of the program looking for a closing delimiter. The failure surfaces as a “malformed `@include`” parse error on the directive's own line number, which is where the mistake is, rather than as a confusing error hundreds of lines later.

The `@namespace` trailing-source case

`@namespace` accepts either a quoted string or a bare identifier — `gawk` permits both — and the expander records the name and removes the directive from the emitted text. The last `@namespace` in the source wins, which the module's tests pin directly.

The non-obvious part is what happens to text *after* the directive on the same line. `gawk` permits `@namespace "name"; BEGIN { ... }`, and a naive implementation that dropped the whole directive line would silently discard the `BEGIN` rule — a program that compiles, runs, and does nothing, with no diagnostic. The expander instead captures whatever follows the namespace name, strips one leading `;` if present, and if the remainder is non-empty emits it as its own line. The parser then sees the rest of the program as if the directive line had ended where the directive ended.

This is a two-line detail guarding against a silent-wrong-answer failure, which is the category of bug that costs the most to find. It is also a good illustration of what the source-expansion pass is for: it operates on text, before the lexer, so every one of its edge cases is about not corrupting the text it hands onward.

Chapter 36 — The Bignum Engine in Depth

Chapter 19 covered the `-M` bignum mode from the language's side — what it changes, what it costs, where its scope ends. This chapter goes into `bignum.rs`, the module that implements the arbitrary-precision arithmetic on top of the Rust `rug` crate (which wraps GNU MPFR for floats and GMP for integers), because the bignum path has its own parsing and formatting rules that bypass `f64` entirely, and seeing them explains how a two-hundred-digit integer survives from input text to output text without ever touching a double.

Parsing without `f64`

The key function is `numeric_string_to_mpfr`, which converts a numeric string straight to an MPFR `Float` at a given precision and rounding mode, *never* going through `f64` — and that “never” is the whole point, because routing through a double would clamp the value to fifty-three bits of mantissa, destroying exactly the precision `-M` exists to provide. The parser handles the three numeric syntaxes directly: a `0x/0X` prefix parses through a GMP `Integer` from base-16 digits; a gawk-style octal (a leading `0` followed only by digits `0` through `7`, with no `8`, `9`, `.`, `e`, or `E` — so `01238` is decimal because the `8` disqualifies it) parses through a GMP `Integer` from base-8; and otherwise the longest numeric prefix is extracted by `longest_f64_prefix` and parsed at full MPFR precision via `Float::parse`. Each path falls back to a clean MPFR zero on a parse error rather than panicking. This is the same number-literal grammar as the `f64` path — the same hex, octal, and decimal rules of Chapter 6 — re-implemented to land in arbitrary precision instead of a double, so a program reads its numeric literals identically whether or not `-M` is active.

Integer truncation and integer division

Two operations need bignum-specific care. `float_trunc_integer` truncates an MPFR float toward zero, the `int()` builtin under `-M`, producing an exact arbitrary-precision integer result rather than a double's truncation — so `int()` of a huge value is exact. `intdiv/intdiv0` likewise compute their quotient and remainder in arbitrary precision. And `srand` under `-M` seeds from the low thirty-two bits of the previous seed, keeping the random path consistent. Each of these is the bignum counterpart of an `f64` operation, routed so that the exactness `-M` promises holds all the way through.

Formatting without loss

The output side matters as much as the input side. `mpfr_string_for_percent_s` formats an MPFR value for `%s` and plain string conversion: an exact integer prints as a plain decimal digit string with no `i64` or `f64` clamp, so a two-hundred-digit number prints all two hundred digits, and a non-integer trims its trailing zeros. The `printf/sprintf` integer conversions under `-M` print the exact arbitrary-precision integer the same way. A consequence the README highlights: the default number-to-string conversion uses each value's *own* precision for the MPFR formatting path, so raising `PROCINFO["prec"]` actually widens the displayed precision rather than being undercut by a hardcoded display width. And `compl(0)` under `-M` yields the full unsigned sixty-six-bit `18446744073709551615`, the bignum-correct complement. The throughline of the module is that every place a value crosses the boundary between text and number — parsing in, formatting out — has a

bignum-aware path that avoids `f64`, which is what makes `-M` exact end to end rather than exact only in the middle.

The octal rule, spelled out

The octal detection inside `numeric_string_to_mpfr` deserves its own reading, because it is a five-condition test and every condition is there to exclude a specific string that looks octal and is not.

A string is parsed base-eight only when: its length exceeds one (so bare `0` is decimal zero, not an empty octal digit run); it starts with `0`; it does not start with `0x` or `0X` (the hex branch has already run, but the guard keeps the condition independently readable); it contains no `.`, no `e`, and no `E` (so `0.5` and `0e3` are decimal); and every byte is in the range `0` through `7`.

That last condition is the gawk-compatibility one and the reason the whole test is a conjunction rather than a prefix check. `01238` contains an `8`, which is not an octal digit, so the string is not an octal literal at all — it is decimal one thousand two hundred thirty-eight. An implementation that parsed the longest valid octal prefix would yield eighty-three and then be confused about the remaining `8`. Requiring the *entire* digit run to be octal makes the classification total: a string is either fully octal or not octal.

Each branch falls back to a clean MPFR zero on a parse failure rather than panicking or propagating an error, which matches awk's general posture toward numeric conversion — a non-numeric string is zero, not an error.

The bitwise family, and the u64 window

The bitwise builtins under `-M` have a structure that looks contradictory at first and is in fact exactly right, and it is worth explaining because it is the one place where arbitrary precision deliberately stops.

Every one of `and`, `or`, `xor`, `lshift`, `rshift`, and `compl` follows the same shape. Check `rt.bignum`; if off, delegate to the ordinary `f64` builtin. If on, convert each operand to MPFR at the runtime's working precision and rounding mode, then truncate it through `float_trunc_u64` — which truncates toward zero to an arbitrary-precision `Integer` and then takes `to_u64_wrapping` — perform the operation on `u64` values, and wrap the result back into an MPFR `Float`.

So the operands go *into* arbitrary precision and immediately come back *out* into sixty-four bits, the operation happens in sixty-four bits, and the result is promoted back. Why route through MPFR at all if the operation is going to be a `u64` operation?

Because the conversion path matters even when the final width does not. A string operand reaching a bitwise builtin under `-M` is converted by `value_to_mpfr`, which uses `numeric_string_to_mpfr` — the no-`f64` parser above. A two-hundred-digit numeric string therefore reaches the truncation with its value intact and wraps deterministically, rather than having been clamped to a double's fifty-three-bit mantissa first and wrapping from a *different*, already-lossy value. The MPFR leg is not there to widen the operation; it is there to keep the *parse* exact so the truncation is the only lossy step and it is a defined one.

And sixty-four bits is the right width because that is the width gawk's bitwise operations have. The module's documentation says so directly: bitwise operands use gawk's unsigned-64 reinterpretation of the signed truncated value. Widening awk's bitwise operations to arbitrary precision would produce different answers from gawk for negative operands and for values above the sixty-four-bit range — a divergence in the opposite direction from the one `-M` exists to fix. `-M` is for making *arithmetic* exact, not for redefining the bitwise builtins' domain.

The shift operations mask their shift count with `0x3f`, giving the sixty-four-bit modular shift semantics, so a shift by sixty-four is a shift by zero rather than undefined behavior.

`compl` is the clearest demonstration of the boundary: complementing zero yields `18446744073709551615`, the full unsigned sixty-four-bit value, because the complement is taken on the `u64` and *then* promoted to MPFR. In non-bignum mode the same expression goes through the `f64` builtin. The value is *bignum-correct* in the

sense that it prints exactly, with every digit, rather than being clamped by a double's mantissa on the way out — which is the actual `-M` improvement here, and it is a formatting improvement rather than a width improvement.

intdiv, and doing the division in the right domain

`awk_intdiv_values` shows the opposite decision — a place where the arbitrary precision runs all the way through — and the contrast is instructive.

In non-bignum mode it converts both operands to `f64`, checks the divisor against zero, casts both to `i64`, and divides. In bignum mode it converts both to MPFR, checks the divisor with `is_zero`, truncates *both* to arbitrary-precision `Integer` values, checks the truncated divisor against zero a second time, divides in `Integer` arithmetic, and promotes the quotient back to MPFR.

The double zero check is not redundant. A divisor of `0.5` is not zero as a float, but truncates to zero as an integer, and dividing by it would be an integer division by zero. The first check catches an explicit zero; the second catches a nonzero value that becomes zero under the truncation the operation performs. Both produce the same `intdiv: division by zero` runtime error.

The reason the division itself stays in arbitrary precision, unlike the bitwise family, is that `intdiv` is arithmetic and its answer is meaningful at any magnitude — dividing a two-hundred-digit number by a three-digit one has an exact two-hundred-digit-ish answer, and truncating the operands to sixty-four bits would destroy it for no compatibility reason. gawk's `intdiv` under `-M` is likewise arbitrary-precision. The rule the two cases jointly express: `-M` widens what gawk widens and does not widen what gawk does not, because the point of the mode is exactness *within* the reference semantics, not a different set of semantics.

The delegation pattern

Every function in the module opens with a `rt.bignum` check and a delegation to the `f64` implementation when the flag is off. That is not incidental structure; it is what makes `-M` a *mode* rather than a fork.

The alternative designs are a separate bignum interpreter, which doubles the surface area that can diverge, or making every value MPFR unconditionally, which imposes MPFR's cost on every program that did not ask for it. The delegation pattern gives one code path per operation with two arms, so the bignum and non-bignum behaviors of a builtin are written next to each other, in the same function, and the question “what does this do under `-M`” is answered by reading fifteen lines rather than by locating a parallel implementation. It also means adding a new numeric builtin requires deciding its `-M` behavior at the moment of writing it, because the function will not compile without both arms.

Chapter 37 — Tests as Specification

A claim of fidelity is only as good as the tests behind it, and `awkrs` is tested heavily — the source tree carries a large suite of Rust integration tests alongside the cross-engine parity harness of Chapter 25. This chapter surveys how the behavior the book has described is *pinned*, because the test suite is, in a real sense, the executable version of this book: every “matching `gawk`” sentence corresponds to a test that would fail if it stopped being true.

The two kinds of test

`awkrs`'s tests come in two flavors with different purposes. The *integration tests* — Rust files like `integration.rs`, `massive_integration.rs`, `more_integration.rs`, `extra_integration.rs`, `cross_feature_integration.rs`, and `new_features_integration.rs` — run `awk` programs through `awkrs`'s own engine and assert on the output, exercising the language features, the builtins, the edge cases, and the interactions between features. The *parity tests* — the `parity/` harness with its general, `gawk`-specific, and portable case suites — run the same programs through `awkrs` and through `gawk`, `mawk`, and `BSD awk` and diff the output byte-for-byte, so they catch any divergence from the reference engines rather than merely confirming `awkrs` is self-consistent. The integration tests answer “does `awkrs` do what we intend?”; the parity tests answer “does what we intend match what `gawk` does?” Both run in CI on every push, so a regression in either fails the build.

Contract tests

A distinctive category is the *contract tests* — files like `contract_printf_getline_lint.rs` and `contract_round_split_flow.rs` — which pin the precise behavioral contracts the book has leaned on: how `printf` rounds, how `getline` reports its return protocol, how `split` and the flow-control verbs interact, how lint warnings fire. These are the tests behind the corner cases that fill the compatibility matrix's formatting section: the bit-exact numeric equality, the `%.1g` rounded-exponent rule, the `%u` two's-complement wrap, the fatal division by zero, the arity-error wording. Each is a contract `awkrs` promises to honor, and each has a test that breaks if the promise breaks.

The example programs as tests

The seventy-odd example programs of the tour are themselves part of the test surface: each is run with its paired `.in` input and its output is verified, in CI, against `gawk`. This is why so many of the tour examples go out of their way to be deterministic — `csv_pivot.awk` sorting with `PROCINFO["sorted_in"]`, `markov.awk` using a greedy walk instead of `rand()`, `huffman.awk` building a zero-padded composite sort key — because byte-stable output is what lets them serve as parity cases. An example that depended on hash iteration order could not be a parity test, so the examples are written to the same determinism discipline the parity suite demands, which is why they double as both documentation and regression coverage.

Why this is the real specification

POSIX and the gawk manual are the *nominal* specification of awk, but they leave much unspecified and they cannot, by themselves, say what *this* implementation does on the boundary cases. The test suite can, and does. When the book says awkrs clamps `substr`'s start index the gawk way, or that paragraph-mode `RT` captures the trailing newlines, or that an unmatched capture group produces no array entries, those are not aspirations — they are assertions in a test file that ran green on the last push. The compatibility matrix is careful to call itself a feature matrix and not a proof of correctness precisely because it is backed by tests rather than by a formal proof: the cases that are checked are checked hard, and the matrix's honesty about where it claims only `Part` is the honesty of a project that knows exactly which behaviors it has pinned and which it has not. That is the right relationship between a book like this and the code it describes — the book explains the intent, and the test suite holds the code to it, push after push.

The shape of the suite

It is worth putting numbers on “tested heavily,” because the distribution across files says something about how the suite was built.

The in-crate unit tests in `vm_tests.rs` carry 509 `#[test]` functions — the single largest concentration, and the one closest to the machine, since it can construct bytecode and runtime state directly rather than going through the CLI. The out-of-crate integration files add roughly a thousand more across nine files, and their sizes are uneven in an informative way: `extra_integration.rs` holds 289 tests, `massive_integration.rs` 200, `more_integration.rs` 172, `new_features_integration.rs` 73, `integration.rs` 70, `cross_feature_integration.rs` 54, and `batch_integration.rs` 53. The four small files are the interesting ones — `contract_printf_getline_lint.rs` with 8, `contract_round_split_flow.rs` with 8, `format_g_rounding_carry.rs` with 4, and `begin_only_no_input.rs` with 2.

That distribution is the signature of a suite that grew from two directions. The large files are *breadth*: sweeps across the language surface, added in batches as features landed. The small files are *depth*: each one exists because a specific behavior was gotten wrong, or was subtle enough that someone wanted it pinned in isolation where a failure names the problem. `format_g_rounding_carry.rs` is four tests about one thing — the carry case in `%g` rounding, where rounding up changes the exponent and therefore the chosen representation (Chapter 13). `begin_only_no_input.rs` is two tests about a program with only a `BEGIN` rule and no input, which is the case where an awk must decide whether to block reading stdin. Neither belongs in a thousand-test sweep, because the point of having them separate is that the file name states the invariant.

The parity corpus is larger still and is organized by which reference engines a case can run against. `parity/cases/` holds 1,054 `.awk` programs, `parity/cases_portable/` another 1,000 written to run identically on gawk, mawk, and BSD awk, and `parity/cases_gawk/` 62 that exercise gawk-only extensions and therefore only have gawk as a reference. A small number of cases carry a paired `.in` input file; the rest are self-contained, generating their own data in `BEGIN` — a deliberate choice, since a case with no external input has no way to be flaky about file state.

How a parity run works

`parity/run_parity.sh` is the harness, and its contract is stricter than “the output looks right.” It runs each case through the reference awk and through awkrs and compares **exact stdout and stderr bytes**, under `LC_ALL=C` and `LANG=C`.

Both halves of that matter. Comparing stderr as well as stdout means the *error text* is part of the contract, which is why the compatibility matrix can make claims about matching gawk's arity-error wording (Chapter 32) — those claims are byte comparisons that would fail on a reworded message. And forcing the C locale removes the largest source of environment-dependent divergence: with locale-aware collation active (Chapter

34), a sort comparison's result depends on the machine's locale, and a parity harness that let that vary would produce failures that reproduce on one developer's machine and not another's.

The harness takes the reference engine as an argument — `gawk`, `mawk`, `bsd`, or `all` — and runs the same case set against each. The BSD reference is resolved by search: an explicit `BSD_AWK` if set, then `nawk`, then `original-awk` for the Linux packaging of the Bell Labs awk, then `/usr/bin/awk` on Darwin. That fallback chain exists because “BSD awk” is not one binary with one name across systems, and hardcoding any single name would make the suite silently unrunnable on the others. Under `all`, the first failing engine exits nonzero rather than continuing, so a CI failure names the engine that diverged.

Determinism as a design constraint on the examples

The 72 example programs in `examples/` double as parity cases, and that dual role constrains how they are written in a way worth making explicit, because it explains choices in the tour (Chapter 26) that would otherwise look like stylistic preferences.

A parity case must produce byte-identical output on awkr and on the reference, which means it must produce byte-identical output on *any* correct awk. Three sources of nondeterminism are therefore excluded by construction. Hash iteration order is one: a bare `for (k in a)` is unordered and every implementation is free to differ, so an example that iterates an array either sorts explicitly or sets `PROCINFO["sorted_in"]`. Randomness is another: `rand()` is seeded and sequenced differently across implementations, so an example that would naturally use it — a Markov chain text generator, say — uses a deterministic greedy walk instead. Tie-breaking is the third and subtlest: a sort whose comparison function admits ties can order the tied elements differently on different implementations, so an example that sorts by frequency builds a zero-padded composite key that makes the ordering total.

None of those three is a compromise for the sake of testing. They are the same discipline a program written to be *reproducible* would follow anyway, and the examples are better programs for it. But the reason they are written that way is that a nondeterministic example cannot be a parity case, and an example that is not a parity case is documentation nobody is checking.

What a test suite can and cannot establish

The chapter's claim is that the tests are the real specification, and the claim needs its boundary stated as carefully as the compatibility matrix states its own.

What the suite establishes is that a specific, enumerable set of behaviors matched a specific set of reference implementations at a specific commit, byte for byte, in the C locale, on the platforms CI runs. That is a strong statement, and it is stronger than what most implementations of anything can say about themselves, because it is mechanical and rerun on every push rather than asserted once in a document.

What it does not establish is that untested behaviors match. There is no proof, no model checking, no exhaustive input generation — the suite is a large finite set of cases, and awk's input space is not finite. A behavior with no case covering it is unverified, regardless of how confident the implementation is about it.

That gap is precisely why the compatibility matrix's `Part` entries exist and why the book keeps saying “matching gawk” rather than “identical to gawk.” The honest formulation is: the behaviors that are pinned are pinned hard, the boundary between pinned and unpinned is knowable by reading the suite, and the project's claims are scoped to the first side of that boundary. A book that described awkr as simply gawk-compatible would be making a claim the tests do not support; describing it as compatible in the ways the matrix enumerates and the suite checks is a claim that can be audited by running `run_parity.sh`.

Chapter 38 – The Benchmark Tables, Read in Full

Chapter 14 sampled awkrs’s benchmark numbers and Chapter 1 set the honest framing; this chapter reads the full benchmark suite — all ten workloads — because the numbers are the concrete form of the performance thesis and they reward careful reading. The tables are generated by `scripts/benchmark-vs-awk.sh`, which runs `hyperfine` across five engines — BSD awk, gawk 5.4.0, mawk 1.3.4, and awkrs in two configurations (JIT-default and `AWKRS_JIT=0` bytecode-only) — on the same one-million-line input on an Apple M-series machine, and copies the results verbatim into the repository. Every number below is from that generated file; none is rounded for effect.

The methodology, stated plainly

Three things about the methodology matter for reading the numbers honestly. First, each benchmark section is a *single* hyperfine invocation across all five engines on the same input, so the relative column is apples-to-apples within a table. Second, the input is one million lines — the generated file’s own note warns that inputs below about five hundred thousand lines put a `{ print $1 }` workload under hyperfine’s shell-startup noise floor, so the measurements need a large input to be meaningful, and the timings *include* process startup, which is the right measure for a tool invoked per-pipeline. Third, the two awkrs rows — JIT-default and `AWKRS_JIT=0` — are the same binary with the fusevm offload toggled by an environment variable, so comparing them isolates exactly what the JIT contributes on each workload. The headline finding, visible in every table, is that those two awkrs rows are nearly identical: these are string-and-field workloads, the JIT does not engage for them, and turning it off changes nothing — which is precisely the point Chapter 1 insisted on, that awkrs’s everyday speed comes from the bytecode VM and its fast paths, not the JIT.

Throughput and field extraction (§1–§4)

The first four workloads are the bread and butter of awk — pull a field, sum a column, print several fields. On `{ print $1 }` over a million single-field lines (§1), BSD awk takes 195.0 ms, gawk 100.8 ms, mawk 66.2 ms, and awkrs about 15.7 ms — roughly four times faster than mawk, the fastest traditional awk, and twelve times faster than BSD awk. This is the inline-fast-path-plus-raw-byte-field workload of Chapter 16, where for a single-rule `{ print $1 }` program there is essentially no interpreter in the loop. The CPU-bound `BEGIN` loop (§2), a four-hundred-thousand-iteration loop with no input, is the one workload where the traditional awks are competitive — mawk at 9.7 ms against awkrs’s 9.2 ms — because it is a tight interpreter loop with no field work to exploit, and notably this is exactly the kind of numeric loop where the JIT *could* help but the benchmark’s bytecode and JIT rows are close because the loop body and iteration count here are modest. Summing the first column (§3), `{ s += $1 } END { print s }`, runs in 12.9 ms for awkrs against gawk’s 62.9 and mawk’s 37.5 — the raw-byte field path parsing the first field straight from the mapped buffer and the `AddFieldToSlot` fusion of Chapter 22. Multi-field print (§4), `{ print $1, $3, $5 }` over five-field lines, is 55.8 ms for awkrs against gawk’s 266.1 and mawk’s 156.6 — the multi-field-print fusions and direct-to-buffer output at work.

Regex, arrays, and conditionals (§5–§7)

The regex filter (§5), `/alpha/ { c += 1 }` with no matches, is where the linear-time Rust engine shines and gawk's backtracking engine is the slowest of the whole suite: awkrs at 11.1 ms, mawk at 19.3, BSD awk at 191.8, and gawk at a striking 351.4 ms — gawk is more than thirty times slower than awkrs here, the largest gap in the table, because its regex engine handles even a non-matching literal scan far less efficiently than a finite automaton. The associative-array workload (§6), `{ a[$5] += 1 } END { for (k in a) print k, a[k] }`, is the honest counterpoint: awkrs at 340.0 ms is at *parity* with gawk's 342.4, ahead of mawk's 610.0 and BSD awk's 826.2, but not ahead of gawk — because the work here is irreducible hash-table churn with no fast path to exploit, and the README includes this row rather than hiding it. It is the one workload where awkrs does not lead, and saying so plainly is the same honesty the compatibility matrix shows. The conditional-field workload (§7), `NR % 2 == 0 { print $2 }`, is 30.7 ms for awkrs against gawk's 116.1 and mawk's 71.1.

Field computation, concatenation, and gsub (§8–§10)

The last three round out the picture. Field computation (§8), `{ sum += $1 * $2 }`, runs in 18.8 ms for awkrs against gawk's 100.5 and mawk's 57.7 — the `AddMulFieldsToSlot` fusion of Chapter 22, the `sum += $1 * $2` idiom collapsed to one instruction. String-concatenation print (§9), `{ print $3 "-" $5 }`, is 50.5 ms against gawk's 182.2 and mawk's 121.0 — the `ConcatPoolStr` fusion and the zero-copy field slices. And the no-match `gsub` (§10), `{ gsub("alpha", "ALPHA"); print }`, is 13.9 ms for awkrs against mawk's 74.3, BSD awk's 291.5, and gawk's 436.3 — the literal-needle slurp-inline path of Chapter 16 that copies a non-matching line straight from the mapped buffer without running the VM, which is why awkrs is more than thirty times faster than gawk on a workload that is, for the other engines, a full regex substitution pass over every line.

What the suite says

Read together, the ten workloads tell a consistent story. On the field-and-text workloads that are awk's daily work — extraction, summation, projection, filtering, substitution — awkrs is several times to thirty-odd times faster than the traditional awks, and the advantage comes entirely from the bytecode VM, the peephole fusions, the raw-byte field path, and the direct-to-buffer output, because the JIT rows are indistinguishable from the bytecode rows on every one of them. On the one workload that is irreducible hash-table work, awkrs is at parity with gawk, which the project reports rather than hides. And the JIT's large wins — the fourteen-to-a-hundred-fold speedups of Chapter 17 — appear on *none* of these tables, because none of these is a tight numeric loop; they would show up on a program like the Mandelbrot renderer of Chapter 26, not on field extraction. The benchmark suite is the empirical backing for the book's central performance claim: awkrs is fast on the common case because of its engine, honest where the work is irreducible, and its JIT is a separate, narrower story measured elsewhere. The README leads with "world's fastest awk bytecode engine," and the tables are generated, real, and reproducible — and the project is scrupulous to note that the historical "JIT" benchmark rows were generated against a former in-tree Cranelift module since removed, that "JIT" now means the fusevm offload, and that the offload does not engage for these string-and-field programs anyway. The slogan is backed; the caveats are kept; the numbers are the ones above.

Chapter 39 – Deployment and the Single Binary

A language’s deployment story is part of its design, and `awk`’s has always been its quiet advantage: one small program, everywhere, with nothing to install alongside it. `awkrs` keeps that discipline exactly, and this short chapter gathers the deployment picture — the single binary, the two names, the install paths, the container story, and the AOT endpoint — because for a tool meant to live in shell pipelines and build scripts, how it ships is as important as how it runs.

One binary, two names

The reference implementation is one Rust binary with no separate runtime, no shared library to keep in sync, and no package tree to resolve before a program runs. `cargo install awkrs` or `brew install awkrs` puts a single executable on your path. That executable answers to two names — `awkrs`, the full spelling, and `aw`, the short one — because, as Chapter 2 noted, every keystroke in a one-liner is one you type a thousand times, and an `awk` that makes you type four characters before any program carries a small daily tax that `aw` removes. Both names are the identical engine; the second is pure ergonomics, the same instinct that has `awk` users alias things. The single-artifact discipline is the same one that makes `awk` itself so deployable, and `awkrs` holds to it: there is exactly one thing to copy to a machine, and a program that leans only on the language and its builtins has no dependencies to resolve because there are none — the whole library is statically linked into the binary.

The cold start as a deployment property

The cold-start time of single-digit milliseconds, measured in Chapter 38’s §1 at about sixteen milliseconds for a full million-line run including startup, is not merely a performance number — it is a deployment property. A language you reach for in a pipeline pays its startup on every invocation, and a build that runs an `awk` script once per file across thousands of files pays it thousands of times. `awkrs`’s fast start is what lets it occupy `awk`’s niche — cheap enough to drop into a shell loop a hundred times an hour — rather than merely reimplement `awk`’s language. The binary memory-maps at startup, the parallel pool is not spun up unless `-j` asks for it, and the `-f script.awk` form skips compilation entirely on the second run via the bytecode cache of Chapter 18. None of this requires any action from the user: the ordinary, idiomatic invocation is also the fast one.

Containers and reproducibility

The single-binary design makes containerization trivial: an image needs nothing in it but the binary and your script. There is no base runtime to layer in, no language-version pin to manage, no dependency lockfile, because the dependency graph for a program that uses only the language and its builtins is empty. This is the “works on my machine” gap closed by construction — the binary built on one machine behaves identically on another because there is no externally-pinned transitive dependency to drift. The bytecode cache is per-machine and self-invalidating, so it never travels in an image to cause a stale result; the JIT’s machine-code cache is likewise local and regenerated as needed. What you ship is the binary and the script, and what runs is exactly what you shipped.

The AOT endpoint

For the narrow case of a self-contained numeric computation — a **BEGIN**-only program with no input — the `--aot` flag of Chapter 17 is the logical endpoint of the deployment story: it ahead-of-time-compile the program to a standalone native executable, linking fusevm’s AOT-compiled objects against the awk runtime (which is why the crate builds as a `staticlib` producing `libawkrs.a`). The result is a binary that contains the compiled computation and needs no awkrs at all to run. It is an advanced, optional path — most awk programs are not input-free **BEGIN** computations — but it completes the spectrum: from a script interpreted fresh, to a script whose bytecode is cached, to a numeric loop JIT-compiled at run time, to a self-contained program compiled ahead of time to its own executable. Across that whole spectrum the deployment promise is the same one awk has always made and awkrs keeps: the thing you move to the machine is small, self-contained, and starts instantly, because being cheap enough to reach for is the property that made awk last, and the property awkrs refuses to give up.

Chapter 40 – The Idiom Catalogue

The tour of Chapter 26 walked sixty-six programs, and the same handful of idioms appeared again and again across them — the small, composable moves that, combined, reach astonishingly far. This chapter gathers those idioms into one place, each named, explained, and cross-referenced to the programs that use it, because once you can see the vocabulary as a vocabulary the programs stop looking like sixty-six separate tricks and start looking like sixty-six arrangements of a dozen reusable parts. Every idiom here is drawn directly from a real example in the repository.

Coercion: the `+ 0` and the `""` tricks

The most ubiquitous idiom is forcing a value's type. `$1 + 0` coerces a numeric-string field to a number — the `N = $1 + 0` that opens `prime_sieve.awk`, the `$3 + 0` of `csv_pivot.awk`, the `$2 + 0` of nearly every command-dispatch example — because a field arrives as a numeric string and adding zero makes it unambiguously numeric for arithmetic and numeric comparison. Its mirror is `x ""`, concatenating the empty string to force a value to its string form, used when you need string comparison or a string key. These two — `+ 0` and `""` — are how an awk programmer controls the type duality of Chapter 6 by hand, and they appear in essentially every program in the tour.

Membership without creation

`if (k in arr)` tests for a key without creating it, and choosing it over `if (arr[k])` is the idiom that keeps arrays from silently growing. `prime_sieve.awk`'s `if (!(i in is)) continue`, `csv_pivot.awk`'s `!(h in min_cpu)` first-value test, and the witness-set checks in `n_queens.awk` and `sudoku.awk` (`(r SUBSEP v) in rowused`) all rely on it: membership is a pure query, and constant-time. The companion is the “is this key new?” pattern — `!(h in seen)` to detect the first occurrence of a group key and initialize a running min or max, which `csv_pivot.awk` and `markov.awk` both use.

Parallel arrays as a struct

awk has no record type, so a structure with several fields becomes several arrays sharing an index — `dijkstra.awk`'s `hd/hk` for a heap entry's distance and node, `dijkstra` and `topo_sort`'s `adj/outdeg` adjacency, `huffman.awk`'s five `ch/fr/lo/lc/rc` arrays for a tree node, `bst.awk`'s `rk/lt/rt`, `lru_cache.awk`'s `nxt/prev/val`. The index is the “pointer,” and the parallel arrays are the fields. This is the awk representation of every pointer-based data structure, and it is why a heap, a tree, a linked list, and a graph all look alike underneath: an integer id indexing a family of arrays.

The SUBSEP grid and the round trip

`a[i, j]` joins the subscripts with `SUBSEP` into one key, giving a two-dimensional array — the DP tables of `levenshtein.awk`, `knapsack.awk`, `diff_lcs.awk`, the grids of `conway.awk` and `sudoku.awk`, the matrices of `floyd_warshall.awk` and `gauss_elim.awk`, the bitmask-DP state of `tsp_dp.awk`. The inverse, `split(key, parts, SUBSEP)`, cracks a composite key back into its subscripts when iterating, as

`markov.awk` does to recover the two words of a bigram. The join on the way in and the split on the way out are the complete multidimensional round trip.

The padded sort key

When you must sort numerically but only have string comparison, pack the sort key into a fixed-width zero-padded string so lexical order *is* numeric order — `huffman.awk`'s `sprintf("%012d %s %d", ...)`, `intervals.awk`'s padded start/end, `suffix_array.awk`'s suffix-plus-index. Tuck the payload (an original index, a node id) after the key, and a value sort doubles as an index sort. This is `awk`'s standard answer to multi-key and numeric sorting through `asort` or string comparison.

The out-parameter and the global-pair return

A function returns one scalar, so to return more it fills an array passed by reference or writes to agreed globals. `split`, three-argument `match`, and `asort` fill arrays; `dijkstra.awk`'s heap pop writes `popd/popk`, and `gcd_extended.awk` writes `gx/gy` for the Bézout pair. The out-parameter is `awk`'s tuple return, and the by-reference array passing of Chapter 9 is what makes it work.

The find-all-match loop

`match` finds only the first occurrence, so to find them all you loop, cutting out each match with `RSTART/RELENGTH` and advancing past it — `email_extract.awk`'s `while (match(substr(line, start), ...)) { ...; start += RSTART + RELENGTH - 1 }`. The same slide-the-window idea drives `url_parser.awk`'s component-peeling. It is the idiom for repeated extraction from a single string.

Place-recurse-unplace

Backtracking in `awk` is add-the-choice, recurse, and on the way back delete-the-choice, restoring state for the next branch — `n_queens.awk`'s witness-set add and `delete`, `sudoku.awk`'s `place/unplace`, `pollard_rho.awk`'s stack-driven divide. The undo on return is what lets one set of global arrays serve every node of the search tree without copying, and `delete` is the undo.

The precomputed ordinal table

Byte-level work wants a character-to-code map and its inverse, built once in `BEGIN` — `hexdump`, `vigener`, `base64`, `lzw`, `rabin_karp`, and `brainfuck` all build `ord/chr` tables by walking 0 to 255 with `sprintf("%c", i)`. `awkrs` also provides the `ord/chr` builtins, but the hand-built table is portable and makes the byte-to-value lookup a simple array index, the operation at the heart of every cipher and dump.

The hand-written tokenizer

Several parsers — `calc_id`, `shunting_yard`, `json_pretty`, `sql_like` — share one tokenizer shape: walk the string with an index, skip whitespace, accumulate runs of a character class into tokens, and emit punctuation as single tokens, storing the result in a global array. It is the micro-lexer of Chapter 21 written in `awk`, and once you have read one you have read them all.

The arrays-as-stack and the implicit field-stack

An array plus an integer top-of-stack index is a stack — `rpn.awk`'s `st/sp`, `shunting_yard.awk`'s operator and output stacks, `brainfuck.awk`'s bracket stack. And for a space-separated line, the *fields themselves* are an already-split token stack, which is why `rpn.awk` walks `for (i = 1; i <= NF; i++)` over `$i` with no tokenizer at all. Recognizing when the fields are already your tokens is the idiom that makes the simplest programs the shortest.

The two-pass join with `FNR==NR`

The single most useful idiom that never appears in the tour's algorithm programs — because it is a property of `awk`'s *input* model rather than of any data structure — is the two-file join keyed on `FNR == NR`. `NR` counts records across all files; `FNR` resets to 1 at the start of each file; so `FNR == NR` is true exactly while the first file is being read and false forever after. A program that opens with `FNR == NR { table[$1] = $2; next }` builds a lookup table from the first file and, because of the `next`, does nothing else while reading it; every rule after that point is reached only when the second file is being read, and can consult the table the first pass built — `{ print $0, table[$1] }`. This is the shell-native cousin of the file-redirected `getline` of Chapter 7, and it is how `awk` does a relational join, a translation-table substitution, or a filter-against-a-set without any tool but itself. The one caveat is the degenerate case: if the first file happens to be empty, `NR` and `FNR` advance together into the second file and the test misreads, which is why the disciplined form checks `FILENAME == ARGV[1]` or the number of arguments when an empty side file is possible.

The guarded divide

Chapter 6 makes division by zero fatal — a deliberate loud failure — which means any division whose denominator is computed from data must be guarded, and the idiom is a conditional that supplies a defined answer for the zero case rather than letting the program die. The mean one-liner `{ s += $1 } END { print s / NR }` is safe only because `END` runs after at least the header of the loop; a per-group average is not, and the disciplined form is `END { for (k in n) print (c[k] ? t[k] / c[k] : 0), k }`, which tests the count before dividing by it. The same guard protects rate and ratio computations, normalizations, and any `a / b` where `b` is a field, a count, or a difference that can vanish. The idiom is not a workaround for a limitation; it is the correct expression of intent — you are stating what the answer *is* when the denominator is zero, which a language that returned a silent infinity would have let you leave unsaid and wrong.

The accumulate-and-dump report

The most common shape of a real `awk` program that is more than a one-liner is: accumulate per-key statistics across the record loop into one or more associative arrays, then walk the keys in `END` and emit a report. `csv_pivot.awk` is the tour's archetype — `count[$1]++, sum[$1] += $3`, a first-seen guard for the running min and max — but the shape is universal: the record loop is pure accumulation, touching only arrays keyed by the grouping field, and the `END` block is pure output, iterating the keys (often through `PROCINFO["sorted_in"]` for ordered output) and printing one line per group with the derived figures. Recognizing this shape is what lets you write a log summarizer, a per-user tally, a histogram, or a pivot table as a handful of lines: decide the key, decide the statistics to accumulate, choose the guarded derivations for the `END` walk, and the boilerplate — the loop, the splitting, the end-of-input handling — is the language's. It is also, not coincidentally, the shape that is *not* parallel-safe (it assigns to globals), which is why Chapter 15's refactor splits it into a parallel map and a sequential reduce; the accumulate-and-dump report is the sequential reduce in its natural habitat.

Why a vocabulary, not a list

The point of cataloguing these is the same point the tour's closing made: a dozen idioms, freely combined, are the whole of what the sixty-six programs do. A heap is parallel arrays plus stack indexing plus the padded comparison; a DP solver is the SUBSEP grid plus the out-parameter plus the traceback; a parser is the tokenizer plus the arrays-as-stack plus recursion. Learn the idioms once and the programs read themselves, because they are not invented anew each time — they are assembled from these parts, which is exactly why `awk`'s small surface, fully internalized, reaches as far as it does.

Chapter 41 – Field Recipes: One-Liners by Task

awk's reason to exist is the one-liner, and a book about awk owes a chapter of them — the small, complete programs you type at a shell prompt to answer a question about a text file. This chapter is a recipe collection organized by task, every one a real, valid awk program that runs unchanged under awkrs. They are not drawn from the example directory — they are the everyday one-liners that are awk's daily work — and each is the shortest path from a question to an answer that the language affords.

Selecting and projecting

The foundational moves are pulling fields and filtering rows.

```
awkrs '{ print $1 }' file           # first whitespace field of every line
awkrs '{ print $NF }' file         # last field
awkrs '{ print $(NF-1) }' file     # second-to-last field
awkrs -F, '{ print $3 }' file.csv  # third comma-separated field
awkrs '$3 > 100' file              # rows whose third field exceeds 100
awkrs '/error/' file              # rows containing "error" (default action prints)
awkrs '!/^#/' file                # rows that are not comments
awkrs 'NR % 2 == 0' file           # even-numbered rows
awkrs 'NR >= 10 && NR <= 20' file  # rows 10 through 20
awkrs 'NF > 5' file                # rows with more than five fields
```

Each is the pattern-action model at its most compact: a pattern selects, and either an explicit `{ print ... }` projects or the default action prints the whole row. The `$NF` and `$(NF-1)` idioms — computed field indices — are how you address columns from the right.

Counting and summing

Accumulation across rows, reported in `END`, is awk's second core job.

```
awkrs 'END { print NR }' file      # count lines
awkrs '{ s += $1 } END { print s }' file  # sum the first column
awkrs '{ s += $1 } END { print s / NR }' file  # mean of the first column
awkrs '$1 > max { max = $1 } END { print max }' file  # maximum first field
awkrs '{ c[$1]++ } END { for (k in c) print c[k], k }' file  # frequency by first field
awkrs '!seen[$0]++' file           # remove duplicate lines, keep first
awkrs 'seen[$0]++' file            # print only the duplicated lines
awkrs '{ n[$1] } END { print length(n) }' file  # count distinct first-field values
```

The dedupe idiom `!seen[$0]++` is the most famous awk one-liner of all: `seen[$0]++` returns the old count (zero, falsy, on first sight) and increments, so `!` makes the rule fire exactly once per distinct line. `length(n)` counting distinct keys uses the array-length extension of Chapter 27.

Transforming text

Rewriting fields and lines is the third job, the one `markdown_basic.awk` scaled up.

```
awkrs '{ $2 = toupper($2); print }' file           # uppercase the second field, rebuild
↳ $0
awkrs '{ gsub(/\t/, ","); print }' file           # turn tabs into commas
awkrs '{ sub(/ +$/, ""); print }' file            # strip trailing whitespace
awkrs '{ $1 = $1; print }' file                   # normalize whitespace to single OFS
awkrs 'BEGIN { OFS = "\t" } { $1 = $1; print }' file # re-emit as tab-separated
awkrs '{ print NR ": " $0 }' file                 # number every line
awkrs '{ for (i = NF; i >= 1; i--) printf "%s%s", $i, (i > 1 ? OFS : ORS) }' file #
↳ reverse fields
```

The `$1 = $1` idiom is a classic: assigning a field to itself triggers the `$0` rebuild of Chapter 5, which re-joins the fields with `OFS` and so normalizes the separators — change `OFS` first and it becomes a separator converter.

Ranges, sections, and multi-file work

The range pattern and the per-file variables handle structured input.

```
awkrs '/BEGIN/,/END/' file                        # everything between the two markers
awkrs 'FNR == 1 { print "==" FILENAME "==" } { print }' f1 f2 # header per file
awkrs 'FNR == NR { a[$1] = $2; next } { print $0, a[$1] }' lookup data # join two files
awkrs 'END { print FILENAME }' f1 f2              # name of the last file
```

The two-file join — `FNR == NR` is true only while reading the first file, so the first rule builds a lookup table and the second rule (reached only for the second file) uses it — is the shell-native version of `getline`'s side-file idiom from Chapter 7, and it is one of `awk`'s most quietly powerful one-liners.

Field surgery: inserting, deleting, and reordering columns

Rewriting the column layout of a record is a job all its own, and it turns on the `$0`-rebuild rule of Chapter 5: change any field, or assign `NF`, and `awkrs` re-joins the record with `OFS`.

```
awkrs '{ $2 = ""; sub(/ +/, " "); print }' file   # blank the second column
awkrs -F, -v OFS=, '{ print $3, $1, $2 }' file.csv # reorder to columns 3,1,2
awkrs '{ NF = 3; print }' file                   # keep only the first three
↳ fields
awkrs '{ $(NF+1) = NR; print }' file              # append the line number as a new
↳ column
awkrs -F, -v OFS=, '{ $1 = $1; print }' file.csv  # rewrite CSV through OFS
↳ (normalizes commas)
awkrs '{ tmp = $1; $1 = $NF; $NF = tmp; print }' file # swap the first and last fields
awkrs '{ for (i = 2; i <= NF; i++) $(i-1) = $i; NF--; print }' file # drop the first
↳ column
```

Assigning `NF = 3` truncates the record to three fields, and assigning to `$(NF+1)` extends it — both trigger the rebuild, so the printed line reflects the new shape joined by `OFS`. Setting `OFS` before the rebuild, as the CSV lines do with `-v OFS=,`, is what turns a field edit into a format-preserving rewrite rather than a whitespace one.

Whitespace, encoding, and cleanup

The unglamorous work of normalizing messy text — the job that precedes most real analysis — is a family of substitutions and the `$1 = $1` normalizer.

```
awkrs '{ gsub(/^[\t]+|[\t]+$/, ""); print }' file      # trim leading and trailing
↳ blanks
awkrs 'NF' file                                     # drop blank lines (NF is 0 on an
↳ empty line)
awkrs '{ gsub(/ +/, " "); print }' file             # collapse runs of spaces to one
awkrs '{ gsub(/\r$/, ""); print }' file             # strip a trailing CR (DOS to
↳ Unix)
awkrs 'BEGIN { RS = ""; FS = "\n" } { print NF " lines in paragraph" }' file #
↳ per-paragraph work
awkrs '{ print length($0), $0 }' file                # prefix each line with its
↳ length
awkrs '!NF && p { next } { p = NF } 1' file          # squeeze consecutive blank lines
↳ to one
```

The bare `NF` pattern drops empty lines because an empty line splits into zero fields and `0` is falsy, so the default print action never fires — a one-character filter that reads as an assertion about the record. The paragraph example switches `RS` to the empty string to read blank-line-separated blocks and `FS` to newline so each line becomes a field, the reconfiguration Chapter 5 covers; it is how `awk` handles input that is structured in stanzas rather than rows.

Why these stay short

What every recipe here shares is that the boilerplate — opening the file, looping lines, splitting fields, handling end of input — is the language's job, not the program's, so what remains on the command line is only the part specific to the question. That compression is `awk`'s whole proposition, and it is why, fifty years on, the shortest path from a text file to an answer is still so often an `awk` one-liner — now running on `awkrs`, which keeps the one-liner exactly as terse and makes the machine underneath it as fast as the chapters on the engine described.

Chapter 42 – The Interactive REPL

awk has no interactive mode. That is not an oversight in the original design; it follows from what awk *is*. A program is a set of pattern/action rules compiled once and then driven by an input stream, and the compilation unit is the whole program — there is no notion of a partial program, no notion of a session, no notion of state that survives from one invocation to the next. Every other language that grew a REPL had somewhere to put the session state: Lisp has the image, Python has the module namespace, Ruby has the top-level binding. awk has a compiled rule table and a record loop, and when the loop ends the process ends. So the traditional way to explore awk interactively is to type `awk 'BEGIN { print 2 ^ 10 }'` at the shell, read the answer, and type it again with a change — a REPL implemented by the shell's history, one process per expression, no continuity at all.

awkrs ships `--repl`, an actual interactive session, and this chapter is how it works and what it can honestly promise given that the language underneath it still has no session concept. The short version is that the REPL *fakes* continuity in exactly one dimension — user-defined functions — and is transparently stateless in every other, and the reason it draws the line there is worth understanding, because a REPL that pretended to have more continuity than the engine can give would be lying about results.

Faking a session with an accumulator

The REPL keeps one piece of state across turns: a `String` holding every `function` definition typed this session. That is the entire session model. Each entered line is classified into one of three shapes and handled accordingly.

If the trimmed line begins with `function` followed by whitespace, it is a definition. The REPL parses that line *in isolation* with the ordinary `parse_program`, and on success appends it to the accumulator and prints nothing; on failure it prints the parse error and the accumulator is untouched. Nothing runs — a definition is not an evaluation — so the visible effect of defining a function is silence, which is the correct feedback for “I recorded that.”

If the line is a complete rule — one containing a top-level `{` (an action block), or starting with `BEGIN` or `END` — it runs as typed, with the accumulated functions prepended, against empty input. So typing `BEGIN { print length("hello") }` behaves exactly like putting those characters in a file and running the file over `/dev/null`, and typing `/error/ { print }` compiles, runs over an empty input, and prints nothing — which is the honest answer, because there were no records.

Anything else is a bare statement or expression, and this is the case that needed a design decision. In awk, a bare expression at top level is a *pattern*, and a pattern over empty input produces no output; typing `1 + 2` into a naive awk-backed REPL would print nothing, which is useless. So the REPL tries the line first as a printed expression, wrapping it as `BEGIN { print (<line>) }`, and if that parses, runs it — so `1 + 2` prints `3` and `substr("hello", 2, 3)` prints `ell`. If the expression form does not parse, the line is retried as a statement body, `BEGIN { <line> }`, so `print "hi"` and `for (i = 0; i < 3; i++) print i` work. If *both* forms fail to parse, the REPL surfaces the error from the *statement* form, not the expression form, because the statement form is the more general grammar and its error message is therefore the more informative one — an expression-form error on `for (i=0;i<3;i++) print i` would complain about something irrelevant.

The parenthesization in the expression wrapper is load-bearing. `print (<line>)` rather than `print <line>` means that a line containing a `>` — say `x > y` — is parsed as a comparison inside the parentheses rather than as `print x` redirected to file `y`, which is exactly the ambiguity Chapter 7 describes in the `print` grammar. Wrapping in parentheses resolves it toward “expression,” which is what the user typing at a prompt meant.

What the accumulator does and does not carry

Functions accumulate; nothing else does. Assign `x = 5` at the prompt and it runs inside its own `BEGIN` block in its own process-equivalent evaluation, and the next line does not see `x`. This is a real limitation and the REPL does not disguise it. The reason it is drawn here rather than further out is that a function definition is *source text that is safe to re-parse*: prepending it to the next entry costs a re-lex and a re-parse of a few lines and produces identical semantics every time. A variable binding is not source text — it is a runtime value in a `Runtime` that ends when the evaluation ends — and carrying it forward would require either keeping a `Runtime` alive across turns (and then reconciling it with each turn's freshly compiled program, whose global slot layout may differ) or serializing values back into synthesized assignment source, which would round-trip a numeric value through its string form and silently change its strnum-ness (Chapter 6). Either path introduces a way for the REPL to disagree with a script, and a REPL whose answers differ from the language's answers is worse than no REPL. Accumulating only definitions keeps the invariant that anything the REPL prints is what a script containing the same text would print.

The accumulator is also what feeds completion. After every turn the REPL re-scans the accumulated function text, extracts each definition's name with a small parser that strips the `function` keyword and takes the following run of identifier characters (and `:`, so a namespaced `ns::helper` is captured whole), and installs those names into the completer's dynamic word list. So a function you defined three lines ago is Tab-completable on the next line.

The line editor

The REPL is built on reedline, and the integration is where most of the interactive behavior lives.

Completion draws from two sources: the static vocabulary, which is `lsp::completion_words()` — the *same* keyword, builtin, and special-variable list the Language Server serves to editors (Chapter 20) — and the dynamic list of session-defined function names. The completer finds the word under the cursor by scanning backward from the cursor for the first character that is not `[A-Za-z0-9_]`, which means awk's punctuation — `$`, `(`, `{`, operators — all act as word boundaries. The practical consequence is that `$N` completes on the bare tail after the `$`, so typing `$NF` and hitting Tab completes `NF` as an identifier rather than trying to complete a sigil-prefixed token. This differs deliberately from a sigil-aware language like stryke, where the sigil is part of the name; in awk `$` is an operator applied to an expression, and the completion boundary reflects that.

Matches are prefix-filtered, deduplicated against a seen-set so a name that appears in both the static and dynamic lists offers one suggestion, and sorted alphabetically, and they are presented in a four-column menu. Tab opens the menu or advances within it — the binding is `UntilFound([Menu("completion_menu"), MenuNext])`, so the first Tab opens and subsequent Tabs cycle — and Shift+Tab (and bare BackTab, since terminals disagree about which they send) steps backward.

Multi-line continuation is a brace-balance validator. The REPL keeps reading while there are more open `{` than close `}`, so a `function` definition or an action block spread over several lines is submitted as one unit. The balance scan is not a full lex but it is not naive either: it tracks double-quoted strings with backslash escapes and stops at a `#` comment, so a `{` inside `"a { b"` or after a `#` does not open a level. It returns "complete" on an excess of closers as well as on balance — only a positive open-brace surplus means "keep typing" — so a stray `}` produces a parse error you can see rather than a prompt that will not return.

Edit mode is emacs by default with vi available, and the resolution order is `AWKRS_REPL_MODE` (accepting `emacs`, `vi`, or `vim`), then `~/.awkrs/config.toml`'s `[repl] mode`, then emacs. The completion-menu bindings are installed on the emacs keymap and on the vi `insert` keymap — deliberately both, so Tab-completion works in vi mode without having to leave insert. Vi normal mode keeps reedline's default bindings untouched.

History is a file-backed history at `~/.awkrs/history` with a five-thousand-entry cap. If the history file cannot be opened — a read-only home directory, a sandbox — the REPL reports the reason on stderr and falls back to an in-memory history rather than refusing to start, and only a failure to create even the in-memory history is fatal.

The config file that documents itself

On first REPL launch, `awkrs` writes `~/.awkrs/config.toml` if it does not already exist. Every setting in the seeded file is *commented out*, with the available values and the default described in the comments. The file therefore changes no behavior — it is purely a discovered-by-existing schema document — and uncommenting a line is how you override. Deleting the file regenerates it. `AWKRS_NO_CONFIG=1` suppresses the seeding entirely, which is what CI and sandboxed environments set so that running `awkrs` never writes to a home directory it does not own.

This is a small thing with a deliberate shape. The alternative designs are a config file with live defaults written in (which makes the file a second source of truth that drifts from the code's defaults on every upgrade) or no seeded file at all (which makes the settings undiscoverable without reading the manual). A fully-commented seed has neither problem: the in-code default is the only default, and the schema is visible in the place you would look for it.

The prompt furniture

Each turn draws a status bar sized to the terminal — the current local time on the left, a per-session command counter in the middle, and the binary name and version on the right — followed by the prompt line. The time is computed through `libc`'s `localtime_r` rather than pulling in a date-formatting crate for the interactive path, with a fallback to UTC modulo arithmetic if the call fails, so the bar always renders something. The terminal width comes from a `TIOCGWINSZ` ioctl on stdout, falling back to `$COLUMNS` and then to eighty, floored at forty so the bar never collapses into nonsense on a very narrow terminal.

Startup prints the shared ASCII banner (Chapter 46) and a single dim hint line naming `exit`, Ctrl-D, and Tab. `exit` and `quit` are matched case-insensitively against the trimmed line and leave the loop; Ctrl-D leaves; Ctrl-C abandons the current line and returns to the prompt rather than exiting, which is the convention every interactive tool follows and the one users' fingers already know.

Why this is a genuine addition rather than a convenience

The REPL does not make `awk` a different language, and it is not a debugger — that is DAP's job (Chapters 20 and 45). What it changes is the cost of the innermost loop of actually writing `awk`, which is not “run the program,” it is “find out what this expression does.” The traditional answer to “what does `substr` do with a negative start” is to construct a whole `awk 'BEGIN { ... }'` invocation at the shell for each guess. In the REPL it is three keystrokes and Enter, with completion for the builtin name, history for the previous guess, and the accumulated helper functions still in scope. That is not a new capability in the sense of a language feature; it is the removal of a per-question fixed cost, and given that `awk`'s whole proposition is compressing the distance between a question and an answer, removing that cost is squarely on-mission.

The honest limitation restated: this is a REPL over a language with no session state, so variables do not persist and every entry is a complete program. Knowing that, the tool is predictable — everything it prints is what the corresponding script prints — and predictability is what a REPL for a language with `awk`'s coercion rules most needs to be.

Chapter 43 – The Linter, Diagnostic by Diagnostic

`-L/--lint` was described in Chapter 20 as one of the static-analysis flags. This chapter opens it, because the linter is a real static-analysis pass over the AST — several passes, in fact — and knowing precisely which classes of mistake it can catch and which it structurally cannot is the difference between a flag you trust and a flag you run once and ignore.

The first thing the linter emits is a disclaimer: the pass announces itself as best-effort static checks, explicitly not full gawk `--lint` parity. That line is not padding. gawk's linter has decades of accreted checks tied to gawk's own internals, and claiming parity would set an expectation awkr's would then quietly violate. Stating the scope up front is the same discipline the compatibility matrix applies (Chapter 32): say what is checked, do not imply what is not.

Three activation paths, one implementation

Lint diagnostics fire from three different triggers and all three run the same pass. `-L/--lint` with an optional sub-mode is the static flag; `-t/--lint-old` turns on the additional old-awk checks; and a truthy `LINT` variable turns on the runtime lint warnings, which also enables the static pass. Internally, `emit_lint_warnings` computes two booleans — whether to warn at all, and whether lint is `fatal` — and then hands a closure to `emit_lint_diagnostics`. The closure writes `<binary>: lint: <message>` to stderr and, when the mode is `fatal`, calls `std::process::exit(2)` immediately.

That structure has a consequence worth naming: under `-L fatal` the linter stops at the *first* diagnostic, because the emit closure never returns. You get one message and exit status 2, not a full report with a failing status. If you want the complete list, run plain `-L` first and escalate to `fatal` in CI once the list is empty. The design is deliberate — `fatal` exists to gate a build, and a gate does not need to enumerate.

The passes are decoupled from output by that same closure: `emit_lint_diagnostics` takes any `impl Fn(&str)`, which is why the module's own tests can collect diagnostics into a `RefCell<Vec<String>>` and assert on them without capturing stderr or spawning a process. The lint pass is unit-testable because its output is a parameter.

Pass one: gawk-extension and structure warnings

The first sweep walks the rules looking for constructs that are gawk extensions or that have order-sensitive semantics.

A `BEGINFILE` or `ENDFILE` pattern warns that it is a gawk extension. More than one `BEGIN` rule, or more than one `END` rule, warns that gawk merges them and that the merge order is significant — this is not an error, since merging is well-defined, but it is a genuine source of confusion in a program assembled from `@included` fragments where two files each contribute a `BEGIN`. A `switch` statement in a rule body warns that `switch` is a gawk extension, emitted once per rule via a `break` after the first hit rather than once per statement, so a rule with three switches produces one message. A `nextfile` inside a *user function* warns that it may be invalid depending on context — because whether `nextfile` is meaningful there depends on whether the function was called from a record rule, which is a dynamic property the static pass cannot resolve.

Pass two: possibly-uninitialized variables

The uninitialized-variable check is the linter's most useful pass and also the one whose limits most need stating. It works in two phases. First, `collect_program_defines` walks the entire program — every rule's pattern and body, every function body — and collects the name of everything that is *ever assigned*: plain assignments, compound assignments, index assignments, increments and decrements, `getline` targets, and the array names that `split`, `patsplit`, `asort`, and the other array-writing builtins populate. That set, unioned with `SPECIAL_GLOBAL_NAMES`, is the “defined” universe. Second, a read-walk visits every variable *read* in the program and warns for any name not in the defined set, not in the enclosing function's parameter list, and not a builtin name.

Three suppressions shape the false-positive rate. A name starting with `_` is skipped entirely — the conventional spelling for awk's local-parameter idiom, where extra parameters past the caller's arity serve as locals (Chapter 9), and where a read of an intentionally-empty local is the whole point. Builtin names are skipped, because a bare `length` is a call and not a variable read. And a `warned` set means each offending name produces exactly one message no matter how many times it is read, so a typo used in a loop does not print a thousand lines.

The check is *flow-insensitive and program-global*, and that is the important qualification. An assignment anywhere in the program marks the name defined everywhere, so this will not catch a variable read in `BEGIN` that is only assigned in a record rule — a real ordering bug the pass is blind to. What it *does* catch, reliably, is the class of bug it was built for: a misspelled variable name. `total` assigned and `totl` read is a silent empty string in awk, one of the language's sharpest edges, because undefined variables are not an error — they are the empty string, which is `0` numerically and `" "` in a comparison, so the program runs and produces a wrong number with no diagnostic at all. That failure mode is why the check exists and why the best-effort caveat is acceptable: catching every misspelling with occasional false positives is a far better trade than catching none.

Pass three: regex literals

Two literal-level checks run over every regex in the program — pattern position, range endpoints, and the regex arguments to the matching and substitution builtins.

An *empty* regex warns that it matches every position. This is well-defined behavior, not a bug, but `//` as a pattern matching every record, or `gsub(/ /, "x")` inserting at every position, is almost never intended and is usually the residue of an edit that deleted the pattern's contents.

A regex ending in a *trailing backslash* warns of a possible mistake. In a regex literal this is nearly always a quoting accident — a `\` that was meant to escape something in the shell, or a path separator that lost its partner — and the resulting pattern is at best surprising.

Deliberately absent is any attempt to validate the regex against awk's engine at lint time. The runtime already fails on a genuinely malformed pattern with a real error (Chapter 24); duplicating that check in the linter would produce two different error texts for one condition and give the pass a way to disagree with the engine. The linter restricts itself to patterns that *compile* but probably do not mean what was written.

Pass four: printf arity, checked by execution

The `printf/sprintf` argument check is the cleverest pass and the one that best illustrates the book's recurring single-source-of-truth theme.

When the format argument is a *string literal* — the common case, and the only case a static pass can reason about — the linter must determine how many value arguments the format requires. The obvious implementation is a second parser for the format-string grammar: walk it, count `%` conversions, handle `%%`, handle `*` width and precision which each consume an argument, handle the flags and the length modifiers. That parser would then be a duplicate of the one in `format.rs` (Chapter 33) and would drift from it the first time a conversion's argument consumption changed.

Instead, `printf_min_args_for_format` calls the *real formatter*. It builds a vector of dummy numeric values and calls `awk_sprintf(fmt, &dummies[..n])` for *n* from zero upward, returning the first *n* that succeeds. The answer is by construction exactly what the runtime requires, because it was obtained by asking the runtime. If a future conversion consumes two arguments, or `*` handling changes, the linter's answer changes with it and no second implementation exists to fall out of sync. The loop is bounded at 256, and a format that never succeeds within that bound yields no diagnostic rather than a wrong one — the fail-safe direction.

When the count is known and the call supplies fewer, the diagnostic states both numbers: the minimum the format needs and the count present. The check runs on `printf` as a statement and on `printf/sprintf` in expression position, and the expression walk is exhaustive — it descends into binary operands, unary operands, assignment right-hand sides, field and index subscripts, ternary branches, tuple parts, `getline` command and redirection expressions, and the arguments of other calls — so a `sprintf` buried inside a nested expression inside a `print` redirection target is still checked. It also descends into indirect calls' callee expressions and arguments, even though it cannot know what an indirect call resolves to, because the arguments themselves may contain checkable `sprintf` calls.

The corresponding over-supply is not warned about, because supplying extra arguments to `printf` is legal and ignored, and warning about legal behavior trains people to ignore the linter.

Pass five: the old-awk checks

`-t/--lint-old` adds a pass aimed at a specific question: will this program run on a traditional awk? It announces itself, then walks every statement and expression looking for gawk-only constructs.

At statement level, `switch/case` warns as a gawk extension not present in traditional awk. At expression level, a call to any of `gensub`, `patsplit`, `mkbool`, `mktime`, `strftime`, `system`, `isarray`, `typeof`, `strtonum`, `and`, `or`, `xor`, `compl`, `lshift`, or `rshift` warns that the named function is a gawk extension. The walk descends through conditionals, loop bodies and `for` init/update clauses, `print` and `printf` argument lists, blocks, binary operands, and ternary branches.

The value of this pass is entirely situational and it is worth being clear about when it is worth running. If `awk` is your `awk`, the gawk extensions are not a problem — `awk` implements them, that is a stated positioning goal (Chapter 32). `-t` matters when the program has to run somewhere else too: a script destined for a BusyBox environment, a BSD `nawk`, or a minimal container where the `awk` on `PATH` is not gawk. Under those constraints `-t` gives you the portability inventory statically, which beats discovering it from a syntax error on a machine you are not sitting at.

What the linter structurally cannot do

Being explicit about the boundary is what makes the tool trustworthy. The linter is a pure AST walk with no type inference, no dataflow, and no execution, so:

It cannot detect a type confusion, because `awk` has no static types — a variable holding a string in one branch and a number in another is ordinary `awk`, not an error.

It cannot detect a use-before-assignment in execution order, only a use-with-no- assignment-anywhere, as the flow-insensitivity above explains.

It cannot check `printf` arity when the format is computed rather than literal, because there is no literal to hand to the formatter.

It cannot know what an indirect call `@f()` invokes, so it cannot check its arity or its effects — the same limitation that drops indirect calls out of the parallel-safe class (Chapter 15), for the same reason.

And it does not attempt semantic regex analysis beyond the two literal checks.

Each of those is a consequence of being a static pass over a dynamically-typed language with no declarations, and the right response is the one the linter takes: state the scope, catch the classes it can catch reliably, and do not manufacture confidence with checks that would be wrong often enough to be ignored.

Chapter 44 – The Reformatter, the POT Generator, and the Profiler

Three of awkrs's diagnostic flags produce *artifacts* rather than warnings: `-o/--pretty-print` writes a reformatted version of the program, `-g/--gen-pot` writes a gettext translation template, and `-p/--profile` writes a run summary. They share an implementation neighborhood — all three live in `cli_effects.rs`, all three consume the AST — and they share a design stance that this chapter is mostly about: each one announces, in its own output, exactly how it differs from the gawk flag whose letter it borrows.

The reformatter: printing the AST back as awk

`-o` runs `format_program` over the parsed AST and emits awk-like source. This is a *re-render from the tree*, not a token-stream reformat, which sets both its strengths and its limits.

The renderer is a recursive descent over the AST mirroring the parser. Rules print their pattern — `BEGIN`, `END`, `BEGINFILE`, `ENDFILE`, an expression, a `/regex/`, or a `start, stop` range — followed by a brace-delimited action block, with statements indented by depth. Functions print after rules, and they print in *sorted* order, because the AST stores them in a map keyed by name; the original definition order is not in the tree to recover.

Several details in the renderer exist to make the output re-parse to the same tree, which is the property that makes a reformatter worth anything.

Field expressions are special-cased. `$1` in the tree is a `Field` node wrapping a literal, and the renderer emits `$1` when the inner expression is a non-negative integer literal — but for any other inner expression it emits `$(...)` with explicit parentheses. This matters because `$` binds tighter than arithmetic: `$NF-1` is `($NF)-1`, and a renderer that emitted `$NF-1` for a tree meaning `$(NF-1)` would produce source that parses to a different program. Parenthesizing the general case makes the round-trip safe.

Regex literals are re-escaped on the way out: an unescaped `/` inside the pattern is escaped so the emitted `/.../` delimiters close where they should. String literals are re-escaped for their own delimiters and control characters.

Increment and decrement render through target-specific helpers — one for variables, one for fields, one for subscripted array elements — because each target's textual form differs and because the field form needs the same parenthesization discipline as field reads.

What the reformatter explicitly does not promise is byte-identical or structurally identical output to gawk's `--pretty-print`, and it says so in the file it produces: the output opens with two `#` comment lines naming it an AST-derived listing and warning against diffing it against GNU awk. That warning is doing real work. gawk's `--pretty-print` is a documented canonical form some toolchains diff against; if awkrs emitted something similar-but-different with no marker, someone would eventually wire it into a formatting check and get confusing failures. Marking the output in the output itself is the version of that warning that survives being piped into a file and read six months later.

The other structural consequence of rendering from the tree is that **comments are gone**. The lexer discards them; they are not AST nodes; the renderer cannot invent them. So `-o` is a tool for *reading* an unfamiliar or badly-formatted program — see its real structure, its true nesting, its actual rule boundaries — not a `gofmt`-style in-place formatter you run on your own source. Pointing it at a commented program and overwriting the original

destroys the comments. That is not a bug to fix in the renderer; it is a property of AST-based rendering, and the correct response is to know which of the two jobs the tool does.

The POT generator: string extraction

`-g/--gen-pot` emits a GNU gettext PO template. It walks every rule body and every function body collecting string literals into a `BTreeMap<String, usize>` — the ordered map giving deterministic, sorted output so regenerating the template produces a stable diff — and writes them as `msgid/empty-msgstr` pairs under a standard PO header.

The collection walk is the same exhaustive expression descent the `printf` linter uses: into call arguments, binary and unary operands, assignment right-hand sides, index subscripts, ternary branches, tuple parts, `getline` command and redirection expressions, print and `printf` arguments and redirection targets. Every string literal anywhere in the program is a candidate.

Extraction escapes each string for the PO format on the way out — the quoting rules PO files require, not `awk`'s.

The header is the conventional PO skeleton with `YEAR`, `PACKAGE`, and translator placeholders left as literal placeholder text, marked `#`, `fuzzy` as newly-generated templates are. The `Project-Id-Version` is filled in as `awkrs`; everything identifying a person or a project remains a placeholder for the user to fill.

The gettext integration on the *runtime* side — `dcgettext`, `bindtextdomain`, `TEXTDOMAIN` — is Chapter 12's material. The relationship between them is the ordinary gettext workflow: `-g` produces the template, a translator produces the catalogues, and the runtime functions look up the translations at execution. `awkrs`'s participation is the two ends; the middle is standard gettext tooling.

The variable dump

`-d/--dump-variables` writes the global variables after the program finishes. Each entry prints its name, a tag distinguishing scalars from arrays, and a rendering of the value; the output is ordered so successive runs are diffable.

Its use is narrow and specific: it answers “what did the accumulation actually end up as” for a program whose `END` block does not print everything, and it does so without editing the program to add prints. For a script that builds several tables and reports on one, `-d` shows the others.

The debug listing

`-D/--debug` writes a static listing of the program's structure — its rules with their pattern kinds, its functions with their parameters. It is not an interactive debugger, and unlike `gawk`'s `-D` it does not attempt to be one; the interactive debugging role belongs to the DAP server (Chapters 20 and 45). The listing's purpose is orientation: for an unfamiliar script, what rules fire on what, and what functions exist.

The profiler and its honest header

`-p/--profile` writes a run summary, and its output header is a four-line statement of what it is not. It says it is wall time plus per-record-rule invocation counts for sequential runs only; that it is not `gawk`'s full profiler and has no per-function or per-line counts; that the layout is `awkrs`-specific key/value text that will not diff against `gawk`'s; and that it is not `gawk`'s annotated profile format with per-statement execution counts.

Four disclaimers on a five-line report is a ratio that deserves comment. `gawk`'s `--profile` produces a well-known artifact: the program re-printed with execution counts annotated per statement. Anyone reaching for `-p`

on an awk knows that artifact and will expect it. awkrs's `-p` gives something genuinely different — a wall-clock total and a hit count per record rule — and the header exists so nobody spends time trying to feed it to a tool expecting gawk's format.

The body is: `wall_seconds` at microsecond resolution; `records_processed` when the record count is known; and `record_rule_hits`, a per-rule invocation count indexed by the rule's position in the program.

Under parallel execution (`-j` above 1) the per-rule counts are *skipped*, and the report says so in place of the numbers rather than printing counts that might be wrong. The reason is direct: the parallel path (Chapter 15) distributes records across workers, and maintaining a shared exact hit counter would mean either a contended atomic on the hottest path in the engine — a real cost imposed on every parallel run to serve a diagnostic flag — or per-worker counters summed at the end, which is defensible but was not built. Rather than pay the cost or report an approximation, the profiler tells you to re-run with `-j1` for rule counts. The wall time and record count are still reported, because those are meaningful in parallel.

The destination is a path argument, with an empty string or `-` meaning stderr, so the report can go to a file for archiving or to the terminal for a quick look without disturbing the program's own stdout.

What `-p` is good for, given its scope, is the rule-level question: which of my twelve rules is actually firing, and on how many records? For a script whose rules have overlapping patterns, that is often the thing you actually wanted to know, and it is cheap to compute — a counter increment per rule match, on the sequential path only. What it is not good for is finding the hot statement inside a rule; that is a job for a system profiler on the awkrs process, which sees the VM's dispatch loop and the builtin implementations directly.

The common thread

All five artifacts share the same stance toward gawk: implement the useful function, give it the same flag letter so muscle memory works, and state in the artifact itself where the output differs from gawk's. The alternative stances are both worse. Silently producing a different format under the same flag sets a trap for anyone who diffs. Refusing to implement the flag because full parity is out of reach removes a useful tool to protect a claim nobody made. Announcing the difference in the output costs four lines in a report and eliminates the entire class of confusion — which is the same trade the compatibility matrix makes at the scale of the whole language.

Chapter 45 – The Debug Adapter, Internals

Chapter 20 described what the debugger does from the outside: breakpoints, stepping, scopes, variable expansion, two transports. This chapter is the inside — the threading architecture, the handshake between the protocol reader and the VM, the ordering hazard in the step commands, and the variable-reference allocation scheme. The reason it deserves its own chapter is that a debug adapter is a *concurrent* program with a specific and easy-to-get-wrong synchronization structure, and *awkr's* is small enough to describe completely.

Why there must be threads

The requirement that forces the design: while the VM is stopped at a breakpoint, the adapter must still answer requests. The editor asks for the stack trace, then the scopes, then the variables in each scope, then perhaps evaluates a watch expression — all while the program is paused. If the adapter processed protocol messages on the same thread that runs the VM, then a stopped VM would mean a stopped message loop and the editor would hang the moment it hit a breakpoint. So message reading and program execution must be on different threads, and the two must synchronize.

awkr's structure is three roles across two threads. The **main thread** spawns the reader, waits for the **launch** request, and then becomes the VM thread, running the program in place. The **reader thread** parses incoming DAP messages for the process lifetime, mutating shared state and signaling the VM through a condition variable. The **VM thread** — the main thread after launch — executes, and at each line stop captures a state snapshot, emits a **stopped** event, and *condvar-waits* for a resume decision.

Running the VM on the main thread rather than a spawned one is a small decision with real consequences: the program's own signal disposition, its process-level state, and its stack are the ones a non-debugged run would have. A debugged run differs from a normal run as little as possible.

The framing and the wire

DAP uses the same framing as LSP: **Content-Length: N**, CRLF, blank line, then N bytes of JSON. The adapter reads headers, parses the length, reads exactly that many bytes, and deserializes. Sharing framing with the LSP means the header parsing and the write path are the same shape as the language server's, which is a genuine simplification — the two servers in the binary have one wire discipline between them.

The write side is behind a mutex because both the reader thread (responses) and the VM thread (events) emit messages. Every write is a complete framed message under the lock, so a **stopped** event emitted by the VM cannot interleave into the middle of a response being written by the reader.

The condvar handshake

The synchronization core is a mutex-and-condvar pair in the shared state. When the VM reaches a stop, it captures a snapshot, publishes it into the shared state, emits the **stopped** event, and then waits on the condvar. When the reader thread receives a resume command — **continue**, **next**, **stepIn**, **stepOut**, **terminate**,

`disconnect` — it stores the resulting `DebugAction` and notifies the condvar, and the VM wakes and acts on it.

`terminate` and `disconnect` are handled together: both set a disconnected flag, resume the VM with `DebugAction::Quit`, respond, and emit `terminated`. Resuming with a quit action rather than simply killing the process matters — a VM blocked on a condvar with nobody left to notify it would hang the process on editor disconnect, so the teardown path must wake it with an action that means “stop running.”

A `pause` request sets a pause-request flag rather than interrupting anything; the VM notices it at the next line marker and stops there. Interrupting mid-instruction would mean stopping at a point with no source line, which is not a location the editor can render.

The ordering hazard in the step commands

The subtlest thing in the module is the ordering inside `next`, `stepIn`, and `stepOut`. Each of the three does exactly two things in a fixed order: first record the step kind into the breakpoint state, then resume the VM.

The order is load-bearing and the source comments say why: the VM reads the pending step kind immediately after the condvar wakes it. Resume first and set the step kind second, and there is a window in which the VM has woken, read no pending step, and continued as if the command had been `continue` — a step command that silently runs to the next breakpoint. It is a classic wake-before-publish race, it would be intermittent, and it would look like “sometimes stepping doesn’t work,” which is among the worst bug signatures to chase. Writing the step kind before the notify closes it.

Capability negotiation as scope declaration

The `initialize` response advertises what the adapter supports, and awkrs’s list is mostly `false`. It supports configuration-done, function breakpoints, evaluate-for-hovers, and terminate. It declares *not* supported: conditional breakpoints, hit-conditional breakpoints, restart, step-in targets, set-variable, completions, loaded-sources, exception info, log points, modules, restart-frame, goto-targets, and step-back.

Being explicit rather than omitting the fields is the right move, because the DAP client adapts its UI to the declaration. An adapter that does not declare `supportsSetVariable` gets an editor that does not offer an editable value field, rather than one that offers it and silently discards the edit. The capability list is the adapter’s honest scope statement, and the same instinct that produces awkrs’s compatibility matrix produces this list: enumerate what is not there so nothing is discovered by failure.

Two of the false entries are worth explaining. **Conditional breakpoints** would require evaluating an arbitrary awk expression in the paused frame’s context, which — see below — the evaluator does not do. **Set-variable** would require writing a value back into a live `Runtime` from the protocol thread while the VM thread owns it, which is a substantially larger synchronization commitment than the read-only snapshot model the adapter is built on.

The snapshot model

The adapter never reads the live `Runtime` from the protocol thread. At each stop the VM thread — which owns the runtime — builds a `PauseSnapshot`: the file, the line, the stop reason, the call frames, and captured locals and globals. Every subsequent `stackTrace`, `scopes`, `variables`, and `evaluate` request is served from that snapshot, cloned out of the shared state under the mutex.

This is the decision that keeps the concurrency tractable. The alternative — protocol requests reaching into the runtime — would require the runtime to be `Sync`, would require every read to be locked against a VM that is stopped but not conceptually frozen, and would open questions about what a read observes mid-instruction. The snapshot makes the paused state immutable, which makes every read trivially safe, at the cost of a

capture on each stop. Given that a stop is a human-scale event and a capture is a walk of the current scopes, the trade is not close.

Variable references and array expansion

DAP addresses expandable values with an integer `variablesReference`: zero means a leaf, non-zero means the client can request that reference's children. `awkr5` partitions the number space by convention. The `scopes` response hands out 1000 for Locals and 2000 for Globals — fixed, so those two never collide with anything. Expandable containers are allocated from a base of 10,000 upward by a small per-pause allocator, and the map from reference to children is rebuilt on every snapshot. References are therefore stable within one pause and meaningless across pauses, which is exactly what DAP's model expects.

Each captured value becomes one row. A scalar is a leaf: its value is rendered through the shared `debugger::format_value` — the same renderer the rest of the debug path uses, so strings quote and regexp values render as `@/re/` consistently — and truncated to two hundred characters with an ellipsis, so a record holding a megabyte of text does not push a megabyte through the protocol.

An array gets a reference and a summary value of the form `array (N elements)`, with the singular spelled correctly for one element. Its children are its elements, each with the same truncation, sorted by key so the editor's tree view is stable across expansions, and capped at five thousand elements so a large table does not flood the client.

The key rendering is a small but real usability detail. `awk` joins multidimensional subscripts with `SUBSEP`, which defaults to the control character `0x1c`. Passing that byte through to the editor produces a key that renders as a control glyph or nothing at all. The adapter substitutes a comma, so `a[1,2]` displays with the key `1,2` — reading the way the program wrote it rather than the way the runtime stores it.

Scope rows are sorted by name, for the same stability reason as the element sort.

Evaluate, and its deliberate limit

The `evaluate` request — driving watch expressions and hover-evaluate — resolves a bare variable name against the snapshot, checking Locals first and then Globals, and returns its rendered value. Anything that is not a bare name returns a bracketed cannot-evaluate message.

The limit is deliberate and the alternative is worth stating. Full expression evaluation would mean compiling the typed expression and running it in the paused frame's context — that is, re-entering the VM from the protocol path while the VM thread sits in a condvar wait holding the runtime. That is a substantially harder concurrency problem than anything else in the module, and it introduces the possibility that inspecting a value *changes* program state, since `awk` expressions can assign, call functions, run `getline`, and invoke `system`. A watch expression with side effects is a debugger that alters the program it is observing. Name lookup against an immutable snapshot cannot do that, and for the overwhelmingly common use — watch a variable, hover a name — it is what the user wanted anyway.

Transports, and why the distinction is not cosmetic

Bare `--dap` speaks the protocol on stdin and stdout. That is convenient for testing an adapter by hand, and it is wrong for real use, because the debugged program's own `print` output goes to the same stdout. A single `print` from the program injects unframed bytes into the protocol stream and the client's parser desynchronizes.

`--dap host:port` connects a TCP socket, carries the protocol there, and leaves stdout to the program. This is the mode IDE plugins use, and it is the only mode where a program that prints anything can be debugged.

Protocol tracing is behind the `AWKR5_DAP_LOG` environment variable and writes to stderr, off by default. The default-off is not tidiness: in stdio mode, stdout is the protocol, and a debug print that reached it would corrupt

the very stream being debugged. Gating on an explicit environment variable and routing to stderr means the tracing cannot break the thing it exists to help fix.

Chapter 46 — Reflection: the Banner, the Help HUD, and the Generated Reference

This chapter is about three surfaces that display facts about `awkrs` — the REPL startup banner, the `--help` output, and the published HTML reference — and about the property they share: none of them contains a number or a description that is written down twice. Every count is computed from a live table at render time, and every documentation string comes from one corpus. The chapter is short on new machinery and long on one idea, because that idea is the one thing that keeps a project's self-description from going stale, and `awkrs` applies it with unusual consistency.

The banner, and where its numbers come from

`banner.rs` renders the ASCII logo plus a boxed statistics panel, and it is a single source of truth in two directions at once. The logo glyphs are a `const` in that module and are used by *both* the REPL's startup and the `--help` HUD, so the art exists in one place. And the statistics inside the box are read from the language's own reflection tables at render time.

The panel shows a system row (status, OS, architecture, process ID), a resources row (core count from `available_parallelism`, available and total memory from `sysinfo`), and a language row — the keyword count, the builtin count, and the special-variable count. Those three come from `AWK_KEYWORDS.len()`, `BUILTIN_NAMES.len()`, and `SPECIAL_GLOBAL_NAMES.len()`: the same tables the namespace prefixing pass consults, the LSP serves for completion, and the arity checker reads. The module documentation states the reason directly — the banner never goes stale after a build adds a keyword, a builtin, or a special variable.

This is enforced, not merely intended. A test renders the plain banner and asserts that it contains the current `.len()` of each of the three tables. A hardcoded count would survive exactly until someone added a builtin, and then that test would fail — which is the point. The book's own numbers are subject to the same discipline: it is why this chapter names the tables rather than quoting the counts.

Getting the box right

Rendering a box with ANSI color inside it is a well-known way to produce misaligned output, because the escape sequences occupy no columns but do occupy string length. `banner.rs` handles it with `visible_width`, which walks the bytes and skips CSI sequences — an ESC followed by `[`, then parameter bytes, terminated by a byte in the final-byte range `0x40..=0x7E` — counting one column per remaining character. Padding is computed from that width, not from the string's length.

The function's documentation is explicit about the limit it accepts: it counts one column per `char`, which is right for the box-drawing glyphs and Latin labels the banner uses and wrong for East-Asian wide characters, and the `wcwidth`-style lookup that would fix that is deliberately skipped. Stating the boundary is better than a comment claiming correctness the code does not have.

Three tests hold the rendering. One checks that `visible_width` ignores CSI sequences and handles a lone ESC and multi-byte characters. One anchors the expected width to the box's *top border* and then asserts that

every interior row matches it — so retuning the box size does not require editing the test, and a stats row that overflows still fails. One asserts the logo's six rows all share one visible width. Together they mean the box cannot silently drift, which is the failure mode of every hand-padded ASCII layout that was correct when written.

The colored and plain paths are both tested: the plain render must contain no ESC byte at all, and the colored render must contain both an escape introducer and a reset.

The help HUD

`cyber_help.rs` renders `--help`. It sources the logo from `banner.rs` rather than duplicating the art, draws a fixed-width status box and a tagline, prints two dim lines naming the CLI's positioning and its three modes (`--repl`, `--lsp`, `--dap`), and then renders clap's own usage and argument blocks through a custom template and style set — colored headers, usage, literals, placeholders, and errors — with section rules framing the options and positionals. A footer carries the version, the MenkeTechnologies attribution, and the license tag.

The color handling is the part that matters operationally. `color_on()` returns true only when `NO_COLOR` is unset *and* either stdout is a terminal or `CLICOLOR_FORCE` is set. That is the full conventional contract: respect the user's opt-out, colorize interactively by default, allow forcing for a pipe that will render the escapes.

The template contains literal ANSI sequences for its section rules, so honoring `NO_COLOR` needs one more step: when color is off, the rendered body is passed through `strip_ansi`, which drops each ESC and everything up to the terminating `m`. The clap color choice is set to `Never` in the same branch. Both are necessary, because the template's escapes are not clap's and clap's setting does not remove them.

Every count in the help output that could go stale — the version — comes from `CARGO_PKG_VERSION` at compile time, so the help cannot report a version the binary is not.

The generated reference

`gen_docs.rs` is a separate binary that writes `docs/reference.html` offline, run before publishing. Its input is the LSP documentation corpus in `lsp.rs` — `builtin_signature`, `special_doc`, `keyword_doc`, and `AWK_KEYWORDS` — the same functions the language server calls to build a hover tooltip.

That sharing is the whole architectural point, and it is stated in the module's own documentation: the generator and the LSP hover path render from the exact same data, so the static page never drifts from what editors show on hover. A builtin whose documentation is corrected in the corpus is corrected in both places by construction, because there is only one place.

The page's structure is a list of chapters, each a title and an explicit ordered list of corpus names — string functions, numeric functions, and so on. The explicit list exists for a specific reason the source names: the corpus is stored as `match` arms, which are not enumerable at runtime. There is no way to ask “what names does this corpus document”; you can only ask “what does it say about this name.” So the generator supplies the ordering and the grouping, and every name in its lists is resolved through the shared lookups so a builtin, special variable, or keyword renders exactly as its hover markdown does.

The page header line is itself computed: it reports the version from `CARGO_PKG_VERSION`, the total topic count, and the chapter count, each derived from the data being rendered rather than typed in. A chapter added to the generator updates the counts on the page automatically.

The markdown-to-HTML converter is written in the file and is deliberately minimal. Its scope is what the corpus actually uses — fenced `awk` code blocks, inline backticks, **`**bold**`**, paragraph breaks, `###` headings, and bullet lists — and everything else is HTML-escaped and passes through as text. The rationale for not pulling in a markdown crate is proportionality: a full CommonMark implementation is a dependency with its own version churn and its own security surface, bought to render a corpus whose syntax is enumerable in one sentence. Under the long-horizon dependency posture the project takes, a converter you can read in one screen beats a crate you must track.

Anchors are slugified per topic and per chapter, so every builtin has a stable URL fragment, and the chapter index links to each section — the properties that make a reference page linkable from an issue or a chat message.

The pattern, and why it is worth naming

Three surfaces, one rule: nothing describes awkrs except by reading awkrs.

The banner's language counts come from the tables the engine dispatches on. The help's version comes from the build's version. The reference's every entry comes from the corpus the editor reads. None of the three can report something the binary does not have, because none of them holds an independent copy of the fact.

The failure mode this avoids is unglamorous and universal: a README claiming a builtin count that was true two releases ago, a help text listing a flag that was renamed, documentation describing a behavior that changed. These are not caused by carelessness; they are caused by a fact being written in two places and only one of them being updated, which is what happens by default over a long enough time. Deriving instead of duplicating removes the possibility rather than relying on discipline to catch it.

The same rule runs through the parts of the system the earlier chapters covered: one parser serving the runtime, the LSP diagnostics, and the include expander; one `BUILTIN_NAMES` table serving the namespace pass, completion, and the arity checker; one `awk_sprintf` serving the runtime and the linter's arity check (Chapter 43); one JIT, fusevm's, rather than a second one written for awk. Reflection in the banner is the most visible instance because you see it every time the REPL starts, but it is the same principle applied at every layer — and it is the reason a project of awkrs's size can describe itself accurately without a documentation process to keep it honest.

Chapter 47 — Lineage: `awk` and Its Implementations

`awkrs` is one entry in a long line, and it is worth closing the reference movement by placing it in that line — the history of the language it implements, the family of other implementations it is measured against, and the family of *its own* sibling projects that share its engine. None of this changes how `awkrs` behaves, but all of it explains why it is shaped the way it is, which is the book’s recurring concern.

The language’s origin

AWK was written at Bell Labs in 1977 by Alfred Aho, Peter Weinberger, and Brian Kernighan — the three initials are the name. It was designed for the single most common task in computing that is not arithmetic: take text laid out in lines and columns, look at each line, and do something with the columns. The design was radical in its compression — the loop over records, the splitting into fields, the output separators, all implicit, so the program on the page is only the part specific to the problem — and that compression is exactly why the language outlived nearly every contemporary it shared a decade with. The “one true `awk`,” the Bell Labs implementation maintained by Kernighan to this day (often called BWK `awk` or `nawk`), is a tree-walking interpreter, and it remains the reference for what POSIX `awk` *is*. The language was standardized by POSIX, and that standard is the common denominator every implementation targets.

The family of implementations

`awk` is unusual in having many independent implementations, and `awkrs`’s compatibility matrix measures against three of them. *BSD `awk`* (the `nawk` shipped on the BSDs and macOS) is close to the Bell Labs original and the most conservative — it lacks most of the `gawk` extensions, which is why `awkrs` marks those features `Ext` against it. *`gawk`*, the GNU implementation, is the richest: thirty years of accreted extensions — `gensub`, the bitwise and time builtins, `PROCINFO`, `@namespace`, the `/inet` sockets, coprocesses, the typed regexp value — and it is the `awk` most people actually write in 2026, which is why `awkrs`’s stated positioning is to implement “the `gawk` extensions that show up in real scripts” and why its highest compatibility overlap is with `gawk`. *`mawk`*, Mike Brennan’s implementation, is the speed-focused traditional `awk`, compiling to bytecode and running fast on the classic workloads — it is the engine `awkrs` most often measures itself against on throughput, and the one it most consistently beats on the benchmark suite of Chapter 38.

There is also a newer generation of `awks` built for performance, which `awkrs`’s own cache survey (Chapter 18) names: *`goawk`*, an `awk` in Go with a bytecode VM; *`frawk`*, an `awk` with a Cranelift/LLVM JIT, the closest prior art to `awkrs` on JIT; and *`zawk`*. `awkrs`’s specific competitive claim — to be the first `awk` pairing a bytecode VM with a *persistent on-disk bytecode cache* — is made carefully against exactly this set: `frawk` recompiles on every invocation with no disk-persisted artifacts, and `gawk`’s `pm-gawk` persists a heap of variables and functions rather than compiled bytecode, so the bytecode-cache-across-invocations capability appears, on `awkrs`’s survey, to be `awkrs`’s alone. The point is not to disparage the others — each made sensible choices for its goals — but to locate precisely what `awkrs` adds to a crowded field.

The fusevm family

`awkrs` also belongs to a second family, one defined not by the language it implements but by the engine it shares. The JIT of Chapter 17 is not `awkrs`'s own — it is a bridge to *fusevm*, a separate bytecode VM published as its own crate, with Cranelift-backed JIT tiers and a persistent machine-code cache. *fusevm* is shared machinery: it also powers the Rust-rewritten shell *zshrs*, the language *strykelang*, and the editor ports *vimlr*s and *elisp*rs. This is the same single-source-of-truth instinct that runs through `awkrs`'s own internals (the builtin tables, the doc corpus, the parser), applied one level up — rather than build a fifth JIT, `awkrs` reuses the one *fusevm* provides, so an improvement to *fusevm*'s code generation benefits all five projects and they cannot drift in their numeric semantics. The `rkyv`-outer/bincode-inner cache architecture of Chapter 18 is likewise shared with *zshrs* and *strykelang*. `awkrs` is, in this light, one application of a common runtime substrate — the `awk`-shaped front end (lexer, parser, the bytecode VM, the field engine) sitting atop a shared JIT-and-cache foundation that several sibling tools also sit atop.

Where `awkrs` sits

The placement, then, is precise. `awkrs` implements the POSIX `awk` language plus the `gawk` extensions that appear in real scripts, faithful enough to be checked byte-for-byte against `gawk`, `mawk`, and BSD `awk` by a two-thousand-case parity suite. It rebuilds the engine underneath that language as a fused-superinstruction bytecode VM with a persistent bytecode cache — the claimed first among `awks` — and bridges to the shared *fusevm* JIT for numeric loops and to *fusevm*'s AOT path for self-contained computations. It adds the one capability no traditional `awk` has, parallel record processing, behind a static safety check. And it does all of this as a single fast-starting binary, because the thing `awk` was always good at — being cheap enough to drop into a pipeline a hundred times an hour — is the thing `awkrs` refuses to give up. It is, in one sentence, the forty-year-old AWK programming model, kept bit-for-bit faithful at the surface, on a new and unusually fast engine — which is where the book began, and where, having walked the language, the machine, the catalogue, and the lineage, it ends.

Chapter 48 – The Introspection Arrays, Built and Refreshed

Chapter 30 catalogued `PROCINFO`, `FUNCTAB`, and `SYMTAB` as names a program can read. This chapter is about who writes them, when, and what the write strategy implies for a script that tries to use them as live views of the running process. The short version — and the part that surprises people porting gawk scripts — is that these arrays are snapshots taken at specific moments, not windows onto the runtime, and knowing which moments is the difference between an introspection script that works and one that silently reports stale values.

One entry point, three builders

Everything funnels through `Runtime::refresh_special_arrays`, which takes the compiled program and the binary name and calls three private builders in sequence: `procinfo_refresh`, `functab_refresh`, and `symtab_mirror_refresh`. The compiled program is a parameter rather than runtime state because two of the three need compile-time facts — the slot table, the array-name list, the function map — that the `Runtime` does not carry.

The interesting question is where that entry point is called from. In the CLI path it fires twice: once immediately before `BEGIN` runs, with a comment noting that the arrays must exist during `BEGIN` because gawk scripts branch on the pid and the platform, and once immediately after `BEGIN` returns. The debug adapter does the same pair around its own `BEGIN` invocation. The library entry point `run_program` calls it once, before `BEGIN`, and never again.

No call site is inside a record loop. That is the load-bearing fact. Every value in `PROCINFO` that could change while records flow — the `errno` mirror, the field-splitting mode, the process's supplementary groups — is frozen at the moment `BEGIN` ended. A program that sets `FPAT` in a record rule and then reads `PROCINFO["FS"]` expecting to see `FPAT` will see whatever the mode was when `BEGIN` finished. The refresh after `BEGIN` exists precisely so that a `BEGIN` block which sets `FIELDWIDTHS` or flips on `bignum` has its effect reflected; there is no equivalent hook for the record loop.

Overwrite, guard, or defer: three write disciplines in one function

`procinfo_refresh` starts by copying every key from the previous `PROCINFO` into a fresh map, so user-set keys survive a refresh by default. It then writes the engine's own keys back over that base — and it does so with three different disciplines, which is the part worth reading closely, because the discipline determines whether a user assignment sticks.

Unconditional insertion wins over anything the user wrote. The keys `version`, `api`, `api_major`, `api_minor`, `program`, `platform`, `pid`, `errno`, the Unix identity block of `ppid`, `uid`, `euid`, `gid`, `egid` and `pgrp`, the supplementary `group1`, `group2`, ... entries, `FS`, `strftime`, `argv`, `mb_cur_max`, and `awkrs_binmode` are all plain inserts. Assign to `PROCINFO["pid"]` in `BEGIN` and the post-`BEGIN` refresh throws your value away. Under `bignum`, the four MPFR identity keys — `gmp_version`, `mpfr_version`, `prec_min`, and `prec_max` — are also unconditional.

A `contains_key` guard is used for `prec` under `bignum`, for `roundmode`, and for `READ_TIMEOUT`. These are the keys where the user's value is a control input rather than a report: setting `PROCINFO["prec"]` before the refresh is how you ask for a specific MPFR precision, so the refresh must not stomp it. The `READ_TIMEOUT` guard has a second layer — the engine only supplies a default from the environment when the environment actually specifies a positive value, so an absent key stays absent rather than becoming a spurious zero.

The `entry(...).or_insert` idiom covers `nproc`, `sorted_in`, the non-`bignum` `prec`, and the per-input composite keys built from a path, `SUBSEP`, and either `READ_TIMEOUT` or `RETRY`. Functionally this is the same “user wins” behavior as the `contains_key` guard; the difference is stylistic, and the presence of both forms in one function is a small archaeology marker showing the function grew in layers.

The `nproc` key is worth calling out because it has no gawk counterpart. It is seeded from `std::thread::available_parallelism`, falling back to one when the platform will not answer, and it exists so an awk program can size its own work against the machine it landed on. Because it uses `or_insert`, a user who sets `PROCINFO["nproc"]` in `BEGIN` keeps the override.

platform is a compatibility decision, not a lookup

`gawk_platform_string` in `procinfo.rs` does not return `std::env::consts::OS`. Its doc comment says so explicitly, naming gawk's vocabulary — `posix`, `mingw`, `vms` — against Rust's `macos` and `linux`. The body checks `cfg!(unix)` first and returns `posix`, then `cfg!(windows)` for `mingw`, then `unknown`. There is a `target_os = "vms"` early return above both, documented as reserved for a future OpenVMS Rust target and unused on every tier-one platform today.

This is the right call, and it is worth understanding why. A gawk script that branches on `PROCINFO["platform"] == "posix"` is asking whether it is on a Unix; it is not asking which Unix. Returning `macos` would break that script for no gain, because the script has no `macos` arm. The module backs the decision with a test that asserts the Unix answer is `posix`, and the assertion message states the rule directly: this key must follow gawk, not Rust's OS constant.

PROCINFO["FS"] and the precedence of splitting modes

`field_split_mode` reports which field-splitting discipline is live, and its precedence order is a compact statement of the record model. CSV mode wins outright and reports `API` — gawk's name for splitting supplied by an extension rather than by the language. Otherwise `FIELDWIDTHS` is checked, then `FPAT`, then the fallback `FS`.

The check on each is not “is the variable set” but “is the variable non-blank after trimming”. A `FIELDWIDTHS` consisting of a space and a tab does not activate fixed-width splitting, and the module has a test named for exactly that case, asserting the mode falls through to `FPAT`. This matters because clearing a splitting mode in awk is conventionally done by assigning the empty string, and whitespace is the natural sloppy version of empty.

The precedence is exercised in both directions. One test asserts `FIELDWIDTHS` beats a simultaneously-set `FPAT`; another walks a runtime through all four states in sequence — default, `FPAT` set, `FIELDWIDTHS` set, CSV on — asserting the reported mode after each transition.

FUNCTAB is asymmetric on purpose

`functab_refresh` builds its map in two passes. User functions go in first, each mapped to a nested map carrying `type` set to `user` and `arity` set to the parameter-list length. Builtins go in second, guarded by `contains_key` so a user function that shadows a builtin name keeps its own entry, and each builtin's nested map carries `type` set to `builtin` and nothing else.

Builtin entries have no `arity`. That is not an oversight to route around; it is the honest answer. `length` takes zero or one argument, `substr` takes two or three, `split` takes two or three, `sub` and `gsub` take two or three — a single integer cannot describe them, and writing one would be worse than writing none. A script iterating `FUNCTAB` must therefore test for the `arity` subscript rather than assume it, and the shape of the data tells it to.

The builtin list comes from `namespace::BUILTIN_NAMES`, whose own doc describes it as the built-in function names that must not be prefixed with a namespace qualifier. As of this tree it holds 63 names. That one table is read by the banner (which prints its length), by the linter, by the language server's completion and hover paths, by `PROCINFO["identifiers"]`, and by `FUNCTAB` — so a builtin added to the language appears in each of those surfaces without anyone remembering to update any of them.

`PROCINFO["identifiers"]` and the shadowing order

`merge_procinfo_identifiers` builds gawk's nested identifier map — every known name tagged `builtin`, `scalar`, `array`, or `user`. It inserts in exactly that order: first every entry of `BUILTIN_NAMES` as `builtin`, then every non-empty name in the compiled program's slot-name table as `scalar`, then every name in the array-name list as `array`, then every user function name as `user`.

Because these are plain inserts into one map, later passes overwrite earlier ones. A program that uses `index` as a scalar variable name produces an identifier entry tagged `scalar`, not `builtin` — the second pass wins. A name used both as a scalar and as an array ends up tagged `array`, because the array pass runs after the scalar pass. The ordering is the specification here; there is no conflict resolution beyond it.

The empty-name skip in the scalar pass is not cosmetic. Slot tables are indexed by position and can carry gaps, so an unnamed slot would otherwise insert an entry keyed by the empty string.

`SYMTAB` is a placeholder, and that is the design

`symtab_mirror_refresh` does two things: it clones the compiled program's slot map into the runtime's own slot map, and it inserts an empty array under the name `SYMTAB`. The comment states the reason — subscripts resolve live through the VM context and `Runtime::symtab_elem_get`, so the stored map is a placeholder.

This is the opposite strategy from `PROCINFO`, and it is the better one for the job. `SYMTAB["x"]` must see the current value of `x`, not the value `x` had when `BEGIN` ended, so materializing a snapshot would be actively wrong. The cost is that `SYMTAB` cannot be a normal array: iteration needs a separate path, which is `symtab_keys_reflect`.

That function unions four sources — the runtime's own variable map, the read-only shared globals a parallel worker sees, the slot map's keys, and every entry of `SPECIAL_GLOBAL_NAMES` — deduplicates through a hash set, and sorts. The first two loops skip the names `SYMTAB`, `FUNCTAB`, and `PROCINFO`, which is what stops `for (k in SYMTAB)` from finding `SYMTAB` inside itself. The companion predicate `symtab_has_key` applies the same three-name exclusion to its variable-map arm, so membership tests agree with iteration.

Including `SPECIAL_GLOBAL_NAMES` unconditionally means `SYMTAB` enumerates every special variable whether or not the program has touched it. That matches the mental model where specials always exist, and it means a script can discover the special-variable surface by iterating `SYMTAB` rather than carrying a hardcoded list of its own.

MPFR and GMP identity, and where the strings come from

Under `bignum` the refresh reports the exact library versions it linked against, and it gets them by calling into the C libraries rather than reading a Cargo version string. One helper reads GMP's version symbol; another

calls MPFR's version function. Both go through a single small routine that null-checks the pointer, returning an empty string rather than dereferencing, and otherwise builds a `String` through `CStr::to_string_lossy`.

The null check is not defensive noise. These are C globals from a dynamically linked library, and the failure mode of skipping the check is a segfault inside a `PROCINFO` refresh — a crash in introspection code, which is the worst place for one, because introspection is what you reach for to diagnose a crash. An empty version string is a bad answer a script can see; a segfault is not an answer at all.

The PMA key that is deliberately absent

`AWKRS_PMA_VERSION` is a public constant of type `Option<&'static str>` set to `None`, and the refresh only inserts `PROCINFO["pma"]` when it is `Some`. gawk's persistent memory allocator is a build-time feature, and a gawk built without it omits the key entirely rather than setting it empty. awkrs reproduces the omission.

A test asserts the constant is `None`, with a message explaining that awkrs is built without gawk's PMA, so the key should stay unset unless explicitly wired. That is a pin on a negative — the test exists so that someone who later sets the constant has to consciously delete an assertion saying “we do not claim this feature”. Pinning absence is underused, and it is exactly right for a compatibility surface where the difference between a missing key and a present empty key is a script's `if` branch.

Chapter 49 – The Parallel-Safety Analyzer, Read Precisely

Chapter 15 gave the list of constructs that disqualify a program from the parallel record path. That list is accurate as a summary and incomplete as a specification, because the analyzer in `src/ast/parallel.rs` is more conservative than the summary suggests in ways that determine whether real programs get the speedup. This chapter reads the analyzer itself.

The shape of the pass

`record_rules_parallel_safe` takes a whole `Program` and returns a `bool`. It has no configuration, no diagnostics channel, and no partial results — a program is either eligible or it is not, and the caller learns nothing about why. The warning the CLI prints when threads are requested for an ineligible program therefore has to enumerate the possibilities rather than name the actual cause; it lists range patterns, `exit`, `getline` without a file, `getline` from a coprocess, and cross-record assignments, then says it is running sequentially. Everything downstream is built on that single bit.

The traversal is two loops. The first walks the program's rules. A range pattern in any rule returns false immediately, before its statements are even examined — range state is inherently ordered, so nothing in the body can rescue it. A `BEGIN`, `END`, `BEGINFILE`, or `ENDFILE` pattern hits a `continue` and its statements are skipped entirely. Everything else — expression patterns, regexp patterns, the empty pattern — has its statement list scanned.

The second loop walks the function table and scans every function body, unconditionally and regardless of whether any record rule can reach it. A function defined and never called still disqualifies the program if its body contains a blocking construct. The module has a test named for that shape, `begin_only_still_checked_for_functions`, which parses a program whose only rule is an empty `BEGIN` and whose function body contains `exit 1`, and asserts the program is not parallel-safe.

That is the correct trade. Call-graph reachability from a record rule would be more precise, but awk has indirect calls, so reachability is not decidable in general — and a partial reachability analysis that is wrong once is worse than a whole-program scan that is conservative always. The whole-program scan also has the property that adding a hostile helper function to an included library changes the program's eligibility in a way a user can find by reading, rather than by reasoning about who calls what.

What blocks, from the match arms

The statement predicate matches on the `Stmt` enum, which has 19 variants in this tree, and the expression predicate matches on `Expr`, which also has 19. Both are written as exhaustive matches with no wildcard arm, so a new AST node forces a compile error here rather than defaulting into either answer. For an analysis whose false-negative mode is silent data corruption across threads, that is the only acceptable structure.

The statements that return true outright are `Exit`, `NextFile`, and `Delete`. `Delete` is unconditional — deleting a single subscript is treated exactly like deleting a whole array, with no attempt to prove the array is record-local.

`GetLine` is decomposed rather than blanket-rejected. A form carrying a pipe command blocks. A form with no pipe reading from the primary input blocks. A form reading from a coprocess blocks. Everything else —

which is the file-redirectioned `getline x < "f.txt"` — falls through to a catch-all arm and returns false. The module tests both sides of that split explicitly, with one test asserting the file form is safe and a cluster of others asserting the bare, variable, and pipe forms are not.

`Print` and `Printf` block when a redirection is present, and otherwise recurse into their argument expressions. Unredirected output is safe because the parallel path reassembles per-record output buffers in input order; a redirect escapes that machinery and would interleave.

`Break`, `Continue`, `Next`, `Return`, and the source-line marker are all false. `Next` being safe is the non-obvious one: it terminates processing of this record only, which is per-record control flow, not cross-record. A test pins it.

The compound statements — `If`, `While`, `DoWhile`, `ForC`, `ForIn`, `Block`, and `Switch` — recurse structurally into every child list. The `Switch` arm is the most involved: it checks the scrutinee expression, then each arm, and for a case arm it checks the label too, analyzing an expression label and treating a regexp label as trivially safe.

Every assignment blocks, including to locals

The expression predicate opens with a four-pattern group — `Assign`, `AssignField`, `AssignIndex`, and `IncDec` — all returning true. There is no inspection of the assignment target.

This is stricter than “any assignment to a global”, which is how Chapter 15 puts it. The analyzer does not know what a global is. Assigning to a function’s own parameter — awk’s idiom for local variables, since extra parameters are the only locals the language has — blocks parallelism exactly as assigning to a program-wide global does. A helper as ordinary as this one is disqualifying:

```
function trim(s, t) {
    t = s
    sub(/^ +/, "", t)
    return t
}
```

The body’s `t = s` is an assignment expression, so the whole program drops to the sequential path even though `t` cannot be observed by any other record.

The module’s own test for this shape parses `function f(x) { g = 1 } { f($1) }` and carries the comment `// g is global`. The assertion holds, but not for the reason the comment gives — the analyzer would return the same answer if `g` were declared as a fourth parameter of `f`. The comment describes a distinction the code does not draw.

Field assignment blocking is the same story from the other side. Assigning to `$1` mutates only the current record, which is per-record state in the parallel model, but the field-assignment arm returns true unconditionally. Likewise the increment and decrement forms on fields, pinned unsafe by a test whose comment says field modifications make the rule unsafe for parallel execution.

What passes that you might not expect

The call arm blacklists exactly two names, `asort` and `asorti`, and otherwise recurses into the arguments. Everything else in the builtin library is treated as ordering-neutral, and the module pins several of these as safe: `rand`, `srand`, `systemtime`, `strftime`, `mktime`, the full bitwise set, `match`, and both `sub` and `gsub`.

`srand` passing is a real consequence of the blacklist’s narrowness. Seeding the generator is process-global state, and a test asserts it does not block. The parallel path gives each worker runtime its own seed, which is what makes this defensible rather than a bug — but the analyzer is not what makes it safe, and reading the analyzer alone would not tell you that.

`sub` and `gsub` pass for a structural reason. Their target argument, when present, is a field expression, and the field arm recurses into the index expression rather than treating the field as a write. A `gsub` whose target is `$1` is therefore analyzed as a read of a constant index and returns safe. That is defensible on the same per-record grounds as `$1 = "x"` would be — and inconsistent with field assignment blocking, which is the sharper observation: two syntaxes for mutating a field get opposite answers.

The indirect-call arm returns true with a one-line comment stating the reason plainly: the callee is dynamic, so parallel-safety cannot be proven. That is the whole justification and it is sufficient.

Two tests whose names disagree with their assertions

The test named `parallel_safe_delete_array_element_v3` asserts the *negation* of `record_rules_parallel_safe`. Its body carries the comment noting that the current implementation treats `delete` as unsafe, so the assertion is deliberate and the name is stale — presumably written when the author expected element-level `delete` to be permitted, then inverted when the code said otherwise without renaming.

The test named `parallel_safe_match_v2` asserts safety and carries a comment saying the current implementation seems to consider `match` safe, question mark included. That is a test written by observation rather than by specification. It still has value — it pins the behavior, so a change to the call arm that accidentally starts blocking `match` fails a test — but the comment is an honest record that nobody decided `match` should be safe; the blacklist simply does not name it.

Both are worth knowing before you use test names as documentation for this module. The assertions are the specification; the names are approximately right.

What the analyzer's shape says about the feature

The set of programs that survive is: pattern-action rules with no range pattern, no `exit`, no `nextfile`, no `delete`, no redirected output, no primary or piped `getline`, no `asort` or `asorti`, no indirect call, and — the binding constraint — no assignment or increment anywhere, in any rule or any function, to any target.

That is the pure filter-and-project class. Printing selected fields, filtering on a regexp, filtering on a numeric comparison, case-folding a field, computing a distance from two fields and printing it — the last pinned safe by a test — all qualify. Anything that computes into a temporary does not, which excludes most programs longer than one line.

The honest framing is that the parallel path aims at a narrow, high-volume class rather than at awk programs generally, and the analyzer's structure is what makes that concrete. Widening it would mean distinguishing function-local targets from globals, which needs the compiler's slot resolution rather than the raw AST — a real project, and a different one from what this module does.

Chapter 50 – The fusevm-Native Migration: Host, Backend, and the Plan to Delete `vm.rs`

Chapter 17 described the fusevm bridge as an accelerator: eligible numeric chunks are translated from awkrs bytecode into fusevm ops and handed to fusevm's interpreter and Cranelift JIT, with the hand-written interpreter in `vm.rs` remaining the engine. That is one of two relationships awkrs has with fusevm, and it is the older one. The other is a staged replacement of `vm.rs` entirely, and it lives in two modules the earlier chapter did not open: `fusevm_host.rs` and `fusevm_compile.rs`.

The stages, as the source names them

The module headers state the plan and number it. `fusevm_host.rs` is labelled stage 1 and describes the goal directly: awkrs is migrating off its hand-written interpreter onto fusevm's VM, the way zshrs runs on fusevm. fusevm carries first-class awk operations — field get and set, `NF`, `print`, `getline`, `split`, `sub` and `gsub` — that dispatch to an `AwkHost` trait. Stage 1 is implementing that trait against awkrs's `Runtime`.

`fusevm_compile.rs` is labelled stage 3: compile the awk AST directly to fusevm bytecode, replacing `compiler.rs` and `vm.rs` together. Its header names stage 4 as the point where execution flips over to it, and stage 5 as the removal of `vm.rs`. Stage 2 is not labelled in either header; from the ordering it is the period during which host methods migrate out of the interpreter one category at a time, which is what the stage-1 header describes in its closing sentence — ops move into the host implementation one category at a time, each gated by the parity suite.

Naming the end state in the source is unusual and useful. A reader who opens `fusevm_compile.rs` and finds it does not support `getline` now knows that is a gap in an in-progress backend, not a design boundary.

The thread-local runtime pointer

fusevm's host methods take a mutable reference to the host. awkrs's state lives in `Runtime`, which the surrounding executor owns as a mutable borrow. There is no way to hand the same mutable borrow to both the executor and the host without aliasing, so the host does not hold it: the host type is zero-sized, and the `Runtime` is reached through a thread-local raw pointer.

The header names the precedent — this mirrors zshrs's current-executor thread-local — which is worth noting, because it means the pattern was already load-bearing in a sibling project before it appeared here.

Two thread-locals carry the state. One is a `Cell` holding a raw pointer to the `Runtime`, initialized null. The other is a `RefCell` holding an optional `Error`, initialized to `None`. A guard type's constructor sets the pointer and clears the error slot; the guard's `Drop` sets the pointer back to null. The guard's documentation states the safety contract in one sentence: while the guard is alive, the `Runtime` must not be touched through its original mutable borrow, only through the accessor from inside host methods during the VM run — and the guard brackets the run exactly, so no aliasing occurs.

The accessor asserts the pointer is non-null before dereferencing. That assertion is a real check, not a debug aid, and the module tests it: a test named for calling outside a guard expects the panic. Pinning the panic

matters because the alternative failure mode — a null dereference inside a host callback during a VM run — produces a signal, not a diagnosable error.

The error channel, and why it exists

The host methods return values, not results. A method that hits a fatal awk condition — a negative field index, a failed `gensub`, a sandbox violation — therefore cannot propagate. The error-recording helper stashes the error in the thread-local slot, keeping only the first, with the doc stating the reason: to match the interpreter's fail-fast propagation. The method then returns a benign placeholder, typically an empty string or `-1`, and execution continues until the VM finishes the chunk.

After the run, a drain function empties the slot and the wrapper turns a present error into the failure the interpreter would have returned. The observable result is the same error at the same exit status; the difference is that operations after the fatal one execute against a runtime in an unexpected state before the error surfaces. That is a real semantic gap during migration, and it is the price of a trait whose methods return plain values.

Not every failure is recorded. The host's `getline` shows the discrimination: an error from the underlying read is inspected, and only a message beginning with a sandbox prefix is promoted to a host error, with a comment stating that sandbox redirection violations stay fatal for gawk parity. Every other read error yields `-1`, which is what awk's `getline` returns for a failed read — an ordinary value, not a fatal.

The four registered builtins, and why each is necessary

The host installer runs just before every VM run. It sets the awk host, sets a minimal shell host, and registers four builtins by numeric id. Each of the four exists because a fusevm primitive does not carry an awk semantic.

The comparison builtin, id 2000, backs the relational operators. Its doc gives the reason: fusevm's value model of integer, float, and string cannot carry awk's `strnum` tag, so comparison cannot be a native op. Every awk relational lowers to a call to this builtin, which returns `-1`, `0`, or `1` per awk's rule that two numeric operands compare numerically and anything else compares as strings, followed by a numeric compare against zero. The comparison itself delegates to the same sorting comparator `asort` uses, passed the runtime's `IGNORECASE` flag, so relational operators and array sorting cannot drift apart.

The array-keys builtin, id 2001, pops an array name and returns its keys as a fusevm array for the compiler to iterate by index. Its doc notes awk leaves iteration order unspecified, matching the runtime's hash order. The array-length builtin, id 2002, pops that array and returns its element count, returning zero for any non-array value rather than failing. Together they are how `for (k in a)` compiles without fusevm needing an awk-array iteration op.

The concatenation builtin, id 2003, exists because fusevm's native concat op does not apply awk's number-to-string rule. The doc gives the worked example: with `CONVFMT` set to two decimal places and a variable holding `pi`, concatenating the variable with an empty string must produce the two-decimal rendering. A generic concat produces the full precision. So concatenation is a host call.

The shell host implements exactly one method, backing the match operators through the runtime's compiled-regex cache. Its doc explains why the rest of the trait keeps its defaults — awk never emits glob, tilde-expansion, or parameter-expansion ops. A regex that fails to compile returns false rather than raising, which is a deliberate softening relative to the interpreter path.

What the host actually implements

The host implementation carries 48 methods. They cover fields, the special variables, the array surface, the numeric library, the string library, the bitwise set, the conversion and gawk-extension builtins, substitution and matching, `split`, the time trio, `getline`, the sort comparator, and the three output entry points.

Almost every one is a thin wrapper: enter the runtime accessor, convert fusevm values to awk values, call the existing function in `builtins.rs` or `runtime.rs`, convert back. The doc comments say so — several read as ports of a named interpreter arm, citing the `vm.rs` opcode they replace. That is the right shape for a migration. The host is a translation layer over the same implementations the interpreter uses, so parity is structural rather than maintained by hand.

The value conversions are where the layer leaks. The awk-to-fusevm direction maps the uninitialized value to fusevm's undefined, the three string-shaped variants to a fusevm string, a number to a float, and an array to undefined. The MPFR arm maps to a float built by narrowing — an arbitrary-precision value truncated to 64 bits on the way into fusevm. Bignum programs therefore lose precision at any boundary crossing this function, which is a concrete reason the native path is not yet the default.

`getline` is the other visible gap, and the host names it. When the source is one of the into-a-variable forms, the method records a host error whose text states the cause: `getline` into a variable is not yet supported on fusevm, because the dispatch does not thread the target variable. The bare forms work — main input, file, and command — and the method updates `NF`, re-splits fields, and bumps `NR` and `FNR` for the main-input case only. A test named for surfacing the fusevm gap pins the error rather than the eventual behavior, which is the correct way to test a known gap: when the gap closes, the test fails and forces a rewrite.

The stage-3 backend, and the shape of its bytecode

`fusevm_compile.rs` compiles the AST straight to a fusevm chunk. Its header enumerates coverage honestly and closes with the rule that matters: unsupported constructs return an error, and there is no silent miscompile. Every unsupported path in the module produces an error whose message is prefixed with the module name, so a fallback is always traceable to a named construct.

Control flow is carried by an awk-signal op with a one-byte code. The `next` signal is 0, `exit` is 1, `nextfile` is 2, and a halt sentinel is 255. The first three are the awk statements. The sentinel's doc explains a non-obvious fusevm behavior: a top-level return makes fusevm re-execute the whole chunk, so its host side effects fire twice. A chunk's main body therefore terminates with the halt signal — a clean stop — rather than a return, and the main-loop driver ignores that code.

That is a bug-shaped detail preserved as a comment, and it is the kind of thing that gets rediscovered expensively if it is not written down. Doubled `print` output from a chunk ending in a return is a symptom nobody would trace to “top-level return means restart”.

Builtin calls compile to first-class ops through a match on the name, and the match has 14 arms: `length`, `substr`, `sprintf`, `index`, `match`, `tolower`, `toupper`, `int`, `sqrt`, `sin`, `cos`, `exp`, `log`, and `atan2`. `sub` and `gsub` are handled before the match by a dedicated routine, because their target is an lvalue whose name rather than value is passed, with the host writing the result back and `$0` as the default target. Every other builtin name falls to the error arm.

User calls are recognized before builtins, by looking the name up in the compiler's parameter-list map. Arguments are pushed, then missing parameters are padded with empty-string constants up to the declared arity, so the callee prologue always pops a fixed count. awk's rule that omitted arguments become locals is implemented as call-site padding rather than as callee defaulting.

Range patterns compiled without a runtime mechanism

The range-rule compiler is the most interesting piece of codegen in the module, because a range pattern needs state that survives between per-record chunk runs and the chunks are separate.

The solution is a host-backed variable whose name is generated from a control-character prefix followed by the literal text `range` and the rule index. The comment states the reason: the flag name uses a control-character prefix no user variable can have. Since every awk scalar in this backend is host-backed through the runtime's

symbol table, the flag persists across chunk boundaries for free — no new mechanism, just a name the language cannot produce.

The emitted code has two halves. The in-range half runs the action, evaluates the end pattern, and clears the flag if it matches. The not-in-range half evaluates the start pattern, and if it matches, sets the flag, runs the action, evaluates the end pattern, and clears the flag if that matches too — which is how a range whose start and end match the same record terminates immediately. Both halves call the same action compiler, so the action's code is emitted twice and a range rule's bytecode is roughly double the size of the same action in an ordinary rule. For a backend not yet on the hot path, that is the right trade against a shared block with a jump into it.

The driver, and why it is not yet the default

The program compiler partitions rules into three chunk lists — `BEGIN`, `main`, and `END` — and compiles each. The partition uses a predicate that returns true for anything that is not a `BEGIN` or an `END` pattern. `BEGINFILE` and `ENDFILE` therefore land in the `main` list, where the main-rule compiler's pattern match has arms only for the empty, expression, regexp, and range patterns — so they reach the catch-all and produce an unsupported-pattern error. A program using `BEGINFILE` cannot compile on this backend, which is correct behavior (an error, not a miscompile) reached by an indirect route.

Compilation happens before any input is read. The doc on the compiled-program type says why: so the caller can detect unsupported constructs and fall back to the interpreter without having consumed stdin. That is a real constraint on the design — a backend that discovered its gaps lazily could not be a fallback at all, because the bytes it had already read would be gone.

The chunk runner is where the current cost lives. For each chunk it constructs a fresh fusevm VM from a clone of the chunk, installs the hosts, enters the guard, runs, reads the signal, and drains the host error. Because the driver calls it once per main chunk per record, a program with two rules over a million records builds two million VMs and clones two million chunks. Nothing about the architecture requires this — the VM and the installed hosts are reusable — but it is the reason the backend is a correctness harness rather than a performance path today.

The record loop itself is a plain nested iteration over the source list and the content's lines. Splitting on lines means `RS` is not honored: the backend processes newline-delimited records regardless of what a `BEGIN` block set. `FS`, by contrast, is re-read from the runtime on every record, with a comment noting that a rule changing it affects the next record — matching `awk`'s actual semantics. Signal handling is a labelled-loop dispatch: `next` continues the record loop, `nextfile` continues the file loop, `exit` breaks out of both. `END` chunks then run unconditionally, with a comment stating that `END` always runs even after `exit`, and that an `exit` inside `END` just stops it.

The gate

The whole path is behind one environment check. Setting the native-backend variable to `1` makes the CLI try the native run before the bytecode cache is consulted, and the comment at the call site states the role: this is the validation harness for the migration, and it becomes the default once parity is proven, after which `vm.rs` is removed.

The native-run entry point falls back — returning a “did not run it” result — in four situations. An ARGV entry containing an equals sign, because inline assignments between files are not handled. A program that fails to parse. A program that fails to compile. And a file that cannot be read as UTF-8, checked by reading each input fully into a `String` before the run begins.

That last one is a consequence of the driver taking filename-and-content pairs rather than readers. The whole input is resident before the first record is processed, which is fine for a harness and unacceptable for a shell pipeline — another item on the list between stage 3 and stage 4.

The AOT module uses the same backend from a different angle. The `--aot` flag parses, checks that every rule is a `BEGIN` — rejecting anything else with a message stating that only BEGIN-only programs are supported because per-record rules and `END` need the record-loop driver — calls the BEGIN-only compiler, and rejects an empty chunk. The AOT path predates the full driver and is restricted to exactly the subset the driver was not needed for.

Chapter 51 – The Environment Surface

awkrs reads a small, closed set of environment variables, and each one is a deliberate escape hatch rather than a general configuration mechanism. There is no options variable, no dotfile of default flags, no config flag. This chapter walks the set, because the variables are scattered across the modules that own the behavior they control and there is no single place in the source that lists them.

The engine switches

`AWKRS_JIT` is read in `lib.rs` alongside the `--no-optimize` flag, and the two combine: the JIT is enabled when `--no-optimize` was not passed and `AWKRS_JIT` is either unset or set to something other than `0`. Any other value — `1`, `yes`, an empty string — leaves it on. This is an opt-out, not a tri-state.

`AWKRS_FUSEVM` is read in `vm.rs` by the fusevm dispatch predicate, with the same not-zero polarity, and it controls whether eligible chunks are offloaded to fusevm's shared VM at all. The two switches are independent, and both are required: the dispatch predicate is conjoined with the JIT flag at the call site, so setting either to `0` forces the awkrs interpreter regardless of the other.

The dispatch predicate caches its answer in a `OnceLock`. The doc gives the reason: the execute function is the per-record hot path, and reading the answer once keeps it allocation- and syscall-free. Reading an environment variable allocates a `String` and, on some platforms, takes a lock; doing that per record would be a measurable tax to support a variable nobody changes mid-run. The consequence is that the variable is sampled once per process and cannot be changed by a program that alters its own environment.

`AWKRS_FUSEVM_NATIVE` is the stage-4 gate from Chapter 50, and it is the only one of the three tested for exact equality with `1`. Anything else, including `true`, leaves the native backend off. The asymmetry is defensible — the other two are opt-outs from a default-on feature, this is an opt-in to an in-progress one — but it is an asymmetry a user will hit.

`AWKRS_CACHE` disables the bytecode cache, and it is the most permissive of the four: the predicate returns false for `0`, `false`, or `no`, and true for everything else including unset. The value flows into a lazily-initialized static holding the process-wide cache handle, which is absent when the cache is disabled or when the cache file could not be opened — the two failures are indistinguishable downstream by design, since a program that cannot open its cache should behave exactly like one that was told not to.

`AWKRS_AOT_RUNTIME_LIB` overrides the search for the awkrs runtime static library. Without it, the resolver looks for the archive beside the running executable and returns an error naming the variable if it is not there. Pointing an error message at the variable that fixes it is a small thing that saves a support round trip.

The tooling switches

`AWKRS_NO_CONFIG` suppresses the REPL's first-run config seeding. The doc explains the audience directly: CI and sandbox environments that should not touch the user's home directory. The check tests for presence only, so the value is ignored — setting it to `0` still disables seeding. That is the conventional Unix reading for a variable meaning “do not”, and it differs from the not-zero convention the engine switches use. Both conventions are defensible; having both in one binary is a genuine inconsistency.

`AWKRS_REPL_MODE` selects the REPL's editing keymap and sits at the top of a three-level precedence chain the resolver documents: the variable, then a mode key under a `[repl]` table in the REPL config file, then the

emacs default. It accepts `vi` or `vim` for the vi keymap and `emacs` for emacs, lowercased before comparison, and falls through to the config file for any other value rather than erroring — so a typo silently yields the configured or default mode.

`AWKRS_DAP_LOG` gates the debug adapter's tracing. The logging macro checks for the variable's presence on every invocation and writes to stderr when set. The doc gives the reason it must be off by default and must go to stderr: in stdio mode, stdout carries the DAP protocol stream, so a stray log line corrupts the wire format and the editor disconnects. Presence-based again, matching the config-suppression variable.

The compatibility variables

`GAWK_READ_TIMEOUT` is gawk's, and awkrs honors it with gawk's semantics: milliseconds, consulted when `PROCINFO["READ_TIMEOUT"]` is absent. The reader parses it as a 64-bit integer, clamps to the range zero through the 32-bit signed maximum, and returns zero for anything unparseable. Every degenerate input is tested — missing, empty, non-numeric, negative, and larger than the 32-bit maximum — and the negative and huge cases assert the clamp endpoints rather than the raw value.

`AWKPATH` is narrower here than in gawk. It is consulted by exactly one resolver, the one backing `-l / --load`, and it defaults to the current directory when unset. The resolver tries each directory in the path with the library name suffixed by `.awk` and then bare, and its not-found message says so. Notably `@include` does *not* consult it: the include resolver takes an absolute path as given, joins a relative path onto the including file's directory when there is one, and otherwise joins it onto the current working directory. Directory-relative resolution for includes, path-search resolution for loads.

The locale variables — `LANGUAGE`, `LC_ALL`, `LC_MESSAGES`, and `LANG` — are consulted by the gettext catalog loader in the priority order Chapter 53 covers.

The terminal variables

`NO_COLOR` and `CLICOLOR_FORCE` govern whether the help output is colorized, and the predicate composes them precisely: color is on when `NO_COLOR` is absent *and* either stdout is a terminal or `CLICOLOR_FORCE` is present. `NO_COLOR` therefore wins over `CLICOLOR_FORCE`, which is what the `NO_COLOR` convention specifies — an unconditional disable, not a vote. Both are presence-based, and the module also carries a helper that strips the color escape sequences, so a colorized help body can be emitted plain rather than built twice.

`COLUMNS` is read by the REPL when sizing its banner. Everything else about terminal geometry is either queried from the tty or fixed.

What is not here

There is no environment variable that sets `F5`, `OF5`, `RS`, or any other awk special. There is no variable that adds default command-line flags. There is no variable that changes parsing or compatibility mode — `--posix` and `--traditional` are flags only, and `POSIXLY_CORRECT` is not read anywhere in this tree.

That last omission is a decision worth stating. gawk honors `POSIXLY_CORRECT`, and a user who sets it globally would reasonably expect awkrs to follow. awkrs does not, which means a script's behavior does not change based on a variable set somewhere in a login profile. For a tool whose value proposition is byte-for-byte reproducibility against a parity suite, an ambient variable that silently changes language semantics is a liability, and the parity suite itself would be unable to test both modes without process isolation.

Chapter 52 — Errors, Flow, and Limits: Three Small Modules That Set the Rules

Three of the smallest files in the crate define constraints every larger module operates inside. None of them is complicated. All three are worth reading because their choices are visible everywhere else.

The error taxonomy

`error.rs` defines one enum with 6 variants, derived through `thiserror`, plus a result alias. The variants are `Io`, `Parse`, `Runtime`, `ProgramFile`, `InputFile`, and `Exit`.

The first thing to notice is what is not there. There is no separate lex error — lexical faults arrive as parse errors. There is no type error, no arity error, no regex-compilation error, no division-by-zero variant. Everything the engine detects while running is a runtime error carrying a formatted string and nothing structured.

That is a real design position, and it costs something. A caller cannot match on “division by zero” without string matching, and one place in the tree does exactly that: the fusevm host’s `getline` checks whether a runtime error’s message starts with a sandbox prefix to decide whether the failure is fatal. String-prefix dispatch on an error message is fragile, and it exists because the taxonomy has no variant to carry the distinction.

The benefit is that adding a new runtime diagnostic requires no change to the error type, no new match arm anywhere, and no decision about whether the new condition deserves its own variant. For an engine whose error surface is essentially “whatever POSIX awk says is fatal”, a flat string variant is proportionate.

`Parse` is the one structured variant, carrying a 1-based line number and a message, each with its own field doc. It is structured because the line number is consumed rather than merely printed: the language server destructures the variant to place a diagnostic at a position, and a test pins that a parse error is reported with its line.

The two file variants exist to distinguish the program from its data. One renders as “cannot read program file”, the other as “cannot open file ... for reading”, and the latter’s doc explains the wording is deliberately phrased like gawk’s to keep error messages consistent across implementations. This is a place where matching another tool’s prose is the compatibility requirement, because scripts and CI pipelines grep error output.

`Exit` is the odd one — not a failure but awk’s `exit` statement, threaded through the same result channel so it can unwind out of a function call or a nested expression without a separate mechanism. Its rendering is just the word and the code, and the module tests it at zero, at a negative code, and at the 32-bit maximum.

The tests read like a display-format specification. One asserts the exact string for a runtime error with a message. Another asserts the rendering of a runtime error with an *empty* message, trailing space included, pinning that an empty message does not collapse the separator. A third checks that the standard error source chain is non-empty for a wrapped I/O error, which pins the conversion behavior that makes the chain traversable. A fourth asks what a line number of zero renders as and answers “line 0” — a slightly awkward output, pinned rather than special-cased, which is the right call for a case that should not occur.

The flow enum

`flow.rs` defines a control-flow enum with 4 variants: `Normal`, `Next`, `NextFile`, and `ExitPending`. It is what the VM returns to the record loop after running a rule body, and it is deliberately not part of the error type — these are ordinary control-flow outcomes, not failures, and routing them through the result channel would mean every rule invocation returns an error on a `next`.

The variant docs are uneven in a way that tells you which ones needed explaining. `Normal` and `Next` have placeholder docs restating their names. `NextFile` carries a real one: skip to the next input file, invalid in `BEGIN`, `END`, `BEGINFILE`, and `ENDFILE`. `ExitPending` carries the POSIX rule — run `END`, then exit with the runtime's exit code — which is the semantic most often gotten wrong by reimplementations, because `exit` in `awk` does not exit.

`ExitPending` being a pending state rather than an exit is the whole design in one name. The record loop sees it, stops reading records, and proceeds to `END`; only after `END` completes does the process terminate. A second `exit` inside `END` produces the same variant, and the loop running `END` chunks breaks rather than recursing.

The module's tests are structural: they assert the debug rendering of each variant contains its own name, and that `Next` and `NextFile` do not match each other. Testing debug strings is usually a smell, but here it pins that the variant names are part of the interface — they appear in the debugger's state display, so renaming one is an observable change.

The recursion cap, and its build-dependent value

`limits.rs` holds one constant, the maximum nesting depth for user-defined function calls in the VM. It is declared twice: under a test configuration it is 32, and under a non-test configuration it is 150.

The substantive doc is attached to the test arm, and it explains the mechanism. Each depth level consumes native stack in the VM, so a very large limit can overflow the host stack before the check ever trips; the comment reports that empirically, on the default eight-megabyte Rust thread stack, overflow begins around two hundred frames in unoptimized builds, and states that the production cap stays comfortably below that. The non-test declaration — the one that ships — carries only a placeholder doc restating the constant's name, so a reader of generated documentation for a release build sees the number without the reasoning.

The split value is itself an interesting choice. Tests run at 32 so a recursion-limit test can actually reach the limit quickly, without a test function recursing 150 deep and taking real time in a debug build. The production value is 150. The gap means the recursion-limit path is exercised against a different constant than the one users hit, which is fine for the control flow — hitting the cap does the same thing at either value — and is not fine for anything depending on the number itself.

Three tests guard the constant. One asserts the test build sees 32, which is a pin on the configuration split working: if someone deleted the test-only declaration, the test build would see 150 and this test would fail rather than silently changing what every other recursion test exercises. The other two assert the value is positive and under a thousand, both routed through `std::hint::black_box` to stop the optimizer from constant-folding the comparison away and making the assertion vacuous.

That last detail generalizes. A test asserting a property of a compile-time constant is at risk of being optimized into nothing; the assertion still passes, but it stops testing anything. Wrapping the constant is the cheap fix, and its presence here is a sign someone thought about whether the test was real.

Chapter 53 — Locale, Message Catalogs, and the Limits of Both

Chapter 19 paired `bignum` with `locale`. This chapter takes `locale` on its own, because `awkrs`'s `locale` support has a sharp and deliberately documented boundary — output is `locale`-aware, input is not — and because the message-catalog path is a separate mechanism with its own search order.

`setlocale` is called exactly once, and the comment says why

`locale_numeric.rs` handles `-N / --use-lc-numeric`. Its entry point wraps a single `setlocale` call for the numeric category in a `std::sync::Once`, and the comment above it is one of the most useful in the tree: `setlocale` is process-global mutable state, calling it concurrently from multiple threads is undefined behavior and produces `SIGSEGV` or `SIGBUS` inside `libc`, and `Runtime::new` calls this on every construction — so under parallel tests the race is hit.

That is a real bug report preserved as a rationale. `Runtime::new` running on every worker in the parallel record path, and on every test in a multi-threaded test binary, turns a once-per-process C call into a contended one. `Once` gives at-most-once semantics with internal synchronization: the first caller activates the numeric locale and every subsequent caller is a no-op.

The consequence is that the locale is fixed at first `Runtime` construction for the life of the process. Nothing can change it afterward, which is exactly right given the constraint, and is worth knowing if you expect a second run inside an embedding to pick up a changed environment.

The two `localeconv` readers, and their different fallbacks

The decimal-point reader and the thousands-separator reader both call `localeconv` and both null-check twice — once on the struct pointer and once on the field pointer — before building a Rust string. Their return types differ, and the difference is the whole design.

The decimal point returns a plain `char` and falls back to `.` at every failure point: null struct, null field, empty string, invalid UTF-8, or empty character sequence. There is always a decimal point, so there is always an answer.

The thousands separator returns an `Option<char>` and returns `None` specifically when the locale supplies an empty separator. The doc explains what `None` means and why it is not an error: empty means “no separator” in the C and POSIX locales, and `gawk`'s grouping flag is then skipped entirely rather than defaulting to a comma.

The doc then records a platform divergence found by testing rather than by reading a standard: Apple's `libc` reports a comma even in the C locale, while `glibc` reports empty. Using the locale's actual value rather than a hardcoded comma is what matches `gawk`'s documented behavior — and it means the grouping conversion in the C locale groups on macOS and does not group on Linux, because the underlying `libcs` disagree and `awkrs`

is faithful to each. The doc names the workaround for callers wanting unconditional grouping: set the numeric locale to one that provides a separator.

The non-Unix implementations are unconditional: a period for the decimal point, a comma for the separator, and an empty locale initializer. The Unix test for the separator asserts the result is one of a small set — comma, period, space, non-breaking space, or narrow non-breaking space — which is a tolerance-band assertion rather than an equality, because the value depends on whichever locale the test machine happens to be in. That is the correct shape for a test over environment-dependent data: assert the contract, not the value.

Output is locale-aware, input is not

The module header states the boundary in bold, and then states the exception in bold as well. The numeric locale affects `sprintf`, `printf`, `print`, `CONVFMT`, `OFMT`, and gawk's grouping flag. It does not affect coercing field strings and other input text to numbers — a field compared as a number, or `strtonum`, still treats `.` as the decimal radix. The header notes this matches the README's own description of the flag.

This asymmetry is the pragmatic choice and should be understood rather than worked around. Locale-aware numeric input would mean that in a German locale the field `3.14` parses as three hundred fourteen — correct for a German data file and catastrophic for the far more common case of an ASCII data file processed on a machine that happens to have a European locale set. Every awk in wide use has landed in roughly the same place for the same reason. Stating it in the module header, rather than leaving it to be discovered, is what makes it a documented boundary rather than a bug.

The message-catalog search order

`gettext_util.rs` is a separate mechanism, reached through `bindtextdomain` and consumed by `dcgettext` and `dcngettext`. It loads GNU message catalogs in pure Rust — no `libintl` linkage — through the `gettext` crate's catalog parser.

Language resolution walks four environment variables in a fixed priority: `LANGUAGE` first, and if set, only its first colon-separated component is used; then `LC_ALL`; then `LC_MESSAGES`; then `LANG`; and the C locale if none is set. From that string it derives two shortened forms — everything before the first period, stripping a codeset suffix, and then everything before the first underscore, stripping the territory.

The candidate paths are then tried in order: the full language string directly under the given directory with a messages component; the same under a `locale` subdirectory; the codeset-stripped form under `locale`; the territory-stripped form under `locale`; and finally the C locale under `locale`. The first path that both opens and parses wins; a file that opens but fails to parse does not abort the search, it falls through to the next candidate.

The two layouts — with and without a `locale` subdirectory — exist because both appear in the wild. The progressive shortening means a program binding a fully-qualified Canadian French locale finds a catalog installed under the territory-qualified name and, failing that, one installed under the bare language code. That degradation is what makes catalog installation forgiving enough to be usable, and it is why the candidate list has five entries rather than one.

The function returns an optional reference-counted catalog. The reference count is because catalogs are shared with parallel workers — the worker-runtime constructor takes a map from domain name to catalog by value, and cloning a handle rather than a parsed catalog is what makes that cheap.

Every failure returns nothing. There is no error channel and no diagnostic, which means a `bindtextdomain` pointing at a mistyped directory is indistinguishable from one pointing at a directory with no matching catalog: both leave the program emitting untranslated strings, which is exactly what gettext does and exactly what a translation layer should do when it cannot find its data.

The tests are all negative — every one asserts the absent case — because the module cannot construct a real catalog file without fixture data. One of them says so in a comment: actual loading cannot easily be

tested without a real catalog, so the test covers the environment-variable fallback logic instead. That is an honest statement of coverage limits, and the tests that remain do have value: they pin that the function never panics on a missing directory, an empty domain, an invalid path, or any of the four language variables set to a plausible value.

The `LANGUAGE`-first ordering is the one to remember. It is GNU gettext's rule, not POSIX's, and it is why a user with `LANGUAGE` naming French ahead of German and `LANG` naming American English gets French: `LANGUAGE` is checked first and its first component wins. A test sets exactly that pair and exercises the path.

Chapter 54 – Reading the Tree Critically

Every claim in this book is meant to be checkable against the source. Working that way over a tree this size surfaces something worth a chapter of its own: the places where the source's own comments disagree with the source's code.

None of what follows is a defect in `awkrs`'s behavior. Every one is a documentation artifact — a comment that was true once, or that describes an intent the code did not end up implementing. They are collected here because the lesson generalizes past this codebase: a comment is a claim by a past author, and a claim is not evidence.

A path that moved and left a comment behind

`lib.rs` describes the bytecode-cache fast path in a block comment that names the cache file with a `.bin` extension. `script_cache.rs` — the module that owns the file — names it with a `.rkyv` extension in its own header, in the doc on the default-path function, in the path that function actually joins, in the lock-file and temporary-file names, and in every test that constructs a cache directory.

The code wins: the file is `~/.awkrs/scripts.rkyv`, and the extension is meaningful, naming the `rkyv` zero-copy serialization format the shard uses. The stale name looks like a rename that updated the owning module and missed a comment in a caller. It is harmless in that nothing dispatches on it, and harmful in that a reader debugging a cache problem may go looking for a file that does not exist.

The general form of this one: when two files name the same artifact differently, believe the one that constructs it.

A doc comment attached to the wrong item

`procinfo.rs` contains a doc line describing the locale-dependent maximum number of bytes per multibyte character, marked best-effort. The item immediately below it is not the multibyte function. It is a test, guarded for test-and-Unix builds, asserting the platform string is `posix`. The function the comment describes appears further down, after the test, carrying no doc of its own.

Rust attaches a doc comment to whatever item follows it, so the documentation for that test — if tests were documented — would carry a description of a different function entirely. The most likely history is a function moved without its comment, or a test inserted between a comment and its function.

The reason this one is worth naming is the failure mode it creates for a reader skimming rather than compiling: the comment reads as a description of the item below it, and here that reading is simply wrong. Verify by checking what the item does, not by trusting the adjacency.

A test name that contradicts its own assertion

`src/ast/parallel.rs` has a test whose name begins with the word describing a parallel-safe deletion of an array element. It asserts the negation — that the program is not safe — and carries an inline comment stating that the current implementation treats `delete` as unsafe.

The assertion and the comment agree with each other and with the analyzer's code, where the delete statement returns true unconditionally. Only the name disagrees. It looks like a test written under the expectation that element-level deletion would be permitted, then corrected in the body when the code said otherwise.

The practical consequence is for anyone building a mental model from test names, which is a normal and usually productive way to read an unfamiliar module. Here it produces the wrong answer. The same file also contains a test whose comment ends in a question mark, observing that the current implementation seems to consider `match` safe — a test written by watching behavior rather than by deciding it.

A comment that describes a distinction the code does not make

The same module's test for global mutation inside a function parses a program whose function body assigns to an undeclared name and comments that the name is global. The assertion passes. The reasoning does not transfer, because the analyzer's assignment arm returns true without examining the target at all. Change the name to an extra parameter of the function — a local by awk's rules — and the answer is unchanged.

This is the subtlest case here, because the comment is not false about the program it annotates. The name is global. The comment is false about *why the test passes*, and a reader taking it as a statement about the analyzer will conclude the analyzer distinguishes globals from locals, which it does not, and will then be surprised when a helper using a scratch parameter kills parallel eligibility.

A doc that lives on the wrong build configuration

`limits.rs` declares its call-depth constant twice under opposing configuration attributes. The test-guarded declaration carries the full rationale — native stack consumption per frame, the empirical overflow threshold, why the production cap sits below it. The declaration guarded for non-test builds, the one that ships in every release, carries a placeholder doc restating the constant's name.

Generated documentation for a normal build therefore renders the constant with no explanation. The reasoning exists, is good, and is invisible in exactly the configuration where a reader most needs it. Nothing is wrong; the documentation is filed under the arm nobody generates docs for.

Placeholder docs as a category

`error.rs` and `flow.rs` both mix real documentation with mechanical placeholders. The I/O variant is documented as “Io variant.” while the parse variant gets a real sentence and per-field docs, and the input-file variant gets a paragraph explaining that its wording deliberately mirrors gawk's. In the flow enum, `Normal` and `Next` get placeholders while `NextFile` and `ExitPending` get real explanations. In the pattern enum, `Begin`, `End`, `Expr`, `Regexp`, and `Empty` get placeholders while `BeginFile`, `EndFile`, and `Range` get real ones.

The pattern across all three is consistent and rational: a lint requiring every public item to be documented was satisfied mechanically, and then real prose was written for the items where a reader would actually have a question. `Flow::Next` needs no explanation to anyone who knows awk. `Flow::ExitPending` does, because `exit` not exiting is genuinely surprising.

Reading placeholder docs as an index of authorial attention is useful. Where the placeholder survives, nobody thought the item needed explaining — which is usually correct, and is occasionally a gap.

The method that generalizes

Four rules cover every case above, and they cost very little to apply.

When a comment states something checkable — a filename, a constant, a count, a test name, a behavior — check it. The comment is a hypothesis with good priors, not a fact. Where comment and code disagree, the code is what runs.

When a comment gives a reason, verify the reason applies. A comment can be attached to a passing test and still describe a mechanism the code does not use, which is the hardest version to catch, because nothing is broken and nothing fails.

When a test name asserts a property, read the assertion. Names drift; assertions are compiled and executed.

When two files describe the same artifact, prefer the one that owns it. The module that constructs the path, defines the constant, or implements the trait is the authority; every other mention is a copy that may not have been updated.

None of this is unique to awkrs, and none of it is criticism of a tree unusually well commented by the standards of systems software — the volume of real, load-bearing rationale in these modules is why the discrepancies are findable at all. A codebase with no comments has no stale comments. The trade is obviously worth it, and the cost is that a careful reader has to verify rather than trust.

Chapter 55 – The Compiled Program: What Survives Compilation

The bytecode cache of Chapter 18 stores one thing: a `CompiledProgram`. Chapter 22 walked the opcode catalogue that fills its chunks. Neither opened the container itself, and the container is where several decisions become permanent — what the VM can see at run time, what the cache has to serialize, and what a newer binary must still be able to read from an older shard.

Thirteen fields, three purposes

`CompiledProgram` carries 13 fields, and they sort into three groups.

Four are chunk lists, one each for `BEGIN`, `END`, `BEGINFILE`, and `ENDFILE`. They are `Vecs` rather than single chunks because `awk` permits multiple blocks of each kind and defines them to run in source order — a rule the linter warns about precisely because the order matters and is easy to lose track of across included files. Storing them as an ordered list rather than merging at compile time keeps the source correspondence intact for the debugger.

Two are the executable program proper: a vector of `CompiledRule` for the record rules and a map from name to `CompiledFunc` for user functions. Functions are a map because calls resolve by name at run time; rules are a vector because they run in order.

The remaining seven are metadata the VM or the driver needs and would otherwise have to recompute: the string pool, the slot count, the slot-name vector, the reverse name-to-slot map, the array-name list, and two cached analysis results.

The string pool, and the cost of interning

`StringPool` holds a `Vec<String>` and a `HashMap<String, u32>`. `intern` returns an existing index when the string is already present and otherwise pushes onto the vector and inserts into the map; `get` indexes the vector.

Every string constant and variable name in the program lives here once, and every op that needs one carries a `u32` index instead. The module header states the purpose: the VM refers to strings by cheap index rather than by pointer or by value.

The implementation stores each string twice — once as a vector element and once as a map key. That is the ordinary cost of a bidirectional intern table without unsafe self-referencing, and it is fully paid at compile time, which for a cached program means paid once ever. For a program with a hundred distinct strings the duplication is invisible; the alternative, a map keyed by index into the vector, would need a borrow the map cannot express safely.

Note the type: indices are `u32`, so a program with more than four billion distinct interned strings would overflow. That is not a practical limit, and choosing `u32` over `usize` halves the size of every op that carries one, which is the point.

Patterns compiled into five shapes

`CompiledPattern` has 5 variants, and the set is a compact record of which pattern forms the VM special-cases.

`Always` matches every record. `Expr` carries a chunk whose truthiness decides the match. `Range` carries two endpoints, with state tracked outside the pattern via the rule's original index. Those three are unsurprising.

The interesting pair is `Regex` and `LiteralRegex`, both holding a string-pool index. The split is decided at compile time by `is_literal_regex`, which returns true when the pattern contains none of 14 metacharacter bytes: period, asterisk, plus, question mark, both brackets, both parentheses, both braces, pipe, caret, dollar, and backslash. A pattern passing that test compiles to `LiteralRegex`, whose doc says what the VM does with it — use `str::contains` instead of the regex engine.

This is worth appreciating for what it avoids. A pattern like `/error/` is a regex by syntax and a substring search by meaning, and running it through a compiled regex automaton per record is real work for no benefit. The check is a single pass over the pattern bytes with no allocation, done once at compile time, and it converts an entire class of the most common awk one-liners onto a substring path. The same split is repeated for range endpoints, which have their own `LiteralRegex` variant for the same reason.

`CompiledRangeEndpoint` has more variants than `CompiledPattern` because a range endpoint can be something a top-level pattern cannot. Beyond the shared `Always`, `Regex`, `LiteralRegex`, and `Expr` forms, it adds `Never` for a `BEGIN`, `END`, `BEGINFILE`, or `ENDFILE` used as an endpoint, and `NestedRangeError` for a range nested inside a range, whose doc names the VM function that turns it into an error.

`NestedRangeError` is the notable one: an invalid construct encoded as a *value* in the compiled program rather than rejected at compile time. The compiler records that the endpoint is malformed and lets the VM raise when the endpoint is actually evaluated. That defers the diagnostic to run time, which is a real cost, and it buys a compiler that never fails on a program the parser accepted — every parse-clean program compiles, and errors surface uniformly through the runtime channel.

The special patterns that compile to `Always`

`compile_pattern` maps `BEGIN`, `END`, `BEGINFILE`, and `ENDFILE` to `CompiledPattern::Always`. That looks wrong on first read — a `BEGIN` pattern does not match every record.

It is correct because those rules never reach record matching. The compiler routes them into the four dedicated chunk lists, and the `record_rules` vector holds only rules the record loop evaluates. The `Always` mapping is what the match arm has to return to be exhaustive, and the value is never consulted. Returning `Always` rather than adding an unreachable variant keeps the enum honest about what the record loop can actually see.

Two cached analyses, and the reason they are fields

`parallel_safe` holds the result of the analyzer from Chapter 49, and `prog_rules_len` holds the original rule count. Both are computed in `compile_program` and both carry docs stating why they are stored rather than recomputed: a cache hit must not have to re-walk the AST, and the range-state vector must be sizable from a cached program without re-parsing.

This is the cache's shadow falling across the data model. On a cache hit there is no AST — the parser never ran — so anything derived from the AST either lives in the `CompiledProgram` or is unavailable. `prog_rules_len` exists solely because range state is indexed by original rule position, and the original positions include the `BEGIN` and `END` rules that were routed elsewhere. Without the cached length, a cache hit could not allocate a correctly-sized range-state vector.

Both fields carry `#[serde(default)]`. That attribute is the whole forward-compatibility story for this struct in one line: a shard written by a binary that predates either field deserializes with the field defaulted rather

than failing. A defaulted `parallel_safe` is false, which costs a program the parallel path until the shard is rewritten, and a defaulted `prog_rules_len` is zero, which sizes the range-state vector at zero.

That second default is the one to understand, and it is worth reading against the cache's actual validity rules. The shard header carries a magic value, a format version, an awkrs version string, and a pointer width, and a shard is only accepted when the stored awkrs version equals the running binary's package version. A shard written by any other release is therefore rejected wholesale before deserialization gets far enough for a default to matter.

So `serde(default)` here is not a cross-release compatibility mechanism — the header check makes cross-release reads impossible by construction. It is a development convenience, covering the window where a field is added without a version bump, plus a general safety net against a struct evolution nobody remembered to pair with a version change. That is a legitimate use, and it is a different use than the attribute usually signals. Marking a field defaultable is cheap; reasoning about which validity layer actually protects the read is the work, and it has to be redone every time a field joins a serialized structure.

CompiledRule keeps its original index

`CompiledRule` carries the compiled pattern, the body chunk, and an `original_index` documented as the index into the original rules vector, used for range-state tracking.

The index is necessary because compilation reorders. Rules are partitioned by pattern kind into the four special lists and the record-rule vector, so a record rule's position in `record_rules` is not its position in the source. Range state has to be keyed by something stable across that partition, and the source position is the only identifier that survives. It is also what lets the debugger map a running rule back to a line, and what makes the cached `prog_rules_len` meaningful as a bound.

`CompiledFunc` by contrast holds only a parameter-name vector and a body chunk. Functions need no index — they are keyed by name in a map — and no arity field, because the parameter vector's length is the arity. That is why `FUNCTAB` can report an arity for user functions and not for builtins: the data is there for one and structurally absent for the other.

Ninety-six opcodes, one flat vector

`Chunk` is a newtype over `Vec<Op>` with a hand-written `Debug` implementation and a single constructor. There is no jump table, no basic-block structure, no metadata — a chunk is a flat instruction stream and jumps are absolute indices into it.

The `Op` enum has 96 variants in this tree. That number is large for a bytecode instruction set and small for an awk implementation, and both halves matter. It is large because the set includes fused superinstructions — the compiler's peephole pass, which runs over every chunk and every pattern expression before the chunk is built, rewrites common op sequences into single ops. It is small relative to awk's surface because the builtin library is *not* in the opcode set; builtins dispatch by name through a single call op, which is why adding a builtin does not change the bytecode format and therefore does not invalidate every cached shard.

That last property is the one worth stating plainly. The opcode set is a serialization format with a persistent on-disk representation, so every variant added to it is a compatibility event. Keeping the builtin library outside the opcode set means the library can grow freely while the format stays still — which is exactly the boundary a cached bytecode design has to get right, and getting it wrong shows up as a cache that invalidates on every release.

Appendix A – Special Variables

This appendix lists awkrs's special variables — the built-in globals that control input, output, and the runtime, and that the engine maintains as records flow past. These are the twenty-eight names awkrs tracks as special globals (mirrored in the compiler's `SPECIAL_VARS` and the namespace system's `SPECIAL_GLOBAL_NAMES`), plus the field variables. For the full prose treatment, follow the chapter reference in each entry.

Record and field state:

- `$0` — the whole current record. Assigning to it re-splits the fields and updates `NF` (Chapter 5).
- `$1, $2, ... $n` — the fields. `$NF` is the last field. Assigning to a field rebuilds `$0` with `OFS` (Chapter 5).
- `NF` — number of fields in the current record. Assigning to it truncates or extends the record (Chapter 5).
- `NR` — number of records read so far, across all files (Chapter 3).
- `FNR` — number of records read in the current file, reset per file (Chapter 3).
- `FILENAME` — the name of the current input file.

Input control:

- `FS` — field separator. Single space (default) means whitespace-split with trimming; a single other character is literal; multi-character is a regex (Chapter 5).
- `RS` — record separator. Newline (default), single char, `" "` (paragraph mode), or multi-char regex (Chapter 5).
- `RT` — the actual text that matched `RS` for the current record (Chapter 5).
- `FIELDWIDTHS` — when non-empty, fixed-width field splitting; accepts gawk's `width`, `skip:width`, and `*` tokens (Chapter 5).
- `FPAT` — when non-empty, defines fields by a regex they match, leftmost- longest (Chapter 5).
- `IGNORECASE` — when true, case-insensitive regex matching and comparison (Chapter 10).
- `SUBSEP` — the subscript separator joining multidimensional array keys; defaults to byte `0x1c` (Chapter 8).

Output control:

- `OFS` — output field separator, used to join fields in `print` and on `$0` rebuild. Default a space.
- `ORS` — output record separator, appended by `print`. Default a newline.
- `OFMT` — format for converting a number to string in `print`. Default `"%.6g"` (Chapter 13).
- `CONVFMT` — format for general number-to-string conversion. Default `"%.6g"` (Chapter 13).

Matching results:

- `RSTART` — 1-based start position of the last `match`, or 0 (Chapter 10).
- `RLENGTH` — length of the last `match`, or -1 (Chapter 10).

Arguments and environment:

- `ARGC / ARGV` — the argument count and array; `ARGV[0]` is the executable, `ARGV[1..]` the file paths, set before `BEGIN` (Chapter 3).
- `ARGIND` — index into `ARGV` of the current file (gawk).
- `ENVIRON` — array of the environment variables.

Introspection and diagnostics (gawk):

- `PROCINFO` — array of process and runtime facts, including `sorted_in`, `platform`, ids, `errno`, read-timeout keys, and (under `-M`) MPFR config (Chapters 8, 12, 19).
- `SYMTAB` — the global symbol table as an array (Chapter 12).

- **FUNCTAB** — the table of function names as an array (Chapter 12).
- **ERRNO** — a description of the most recent I/O error (gawk).
- **LINT** — when truthy, enables runtime lint warnings (Chapter 11, 14).
- **TEXTDOMAIN** — the active gettext domain (Chapter 12).
- **BINMODE** — binary-mode I/O control (gawk; partial).

Appendix B – Builtin Functions Summary

awkrs implements sixty-three builtin function names (the `BUILTIN_NAMES` table). This appendix summarizes them by category; Chapters 10, 11, 12, and 13 give the full behavior, and the rendered [reference.html](#) gives exact signatures.

String functions:

- `length([s])` — length of `s` or `$0`; bytes under `-b`, else characters.
- `substr(s, start [, len])` — substring; gawk clamp rule for `start < 1` (Chapter 11).
- `index(s, t)` — 1-based position of `t` in `s`, or 0; empty needle → 1.
- `split(s, arr [, fs [, seps]])` — split into `arr`, return count; 4th arg captures separators (Chapter 5).
- `patsplit(s, arr [, fpat [, seps]])` — split by field pattern (Chapter 5).
- `sub(re, repl [, target])` — replace first match, return count (Chapter 10).
- `gsub(re, repl [, target])` — replace all matches, return count (Chapter 10).
- `gensub(re, repl, how [, target])` — return substituted copy; `\1..\9` and `&` backrefs (Chapter 10).
- `match(s, re [, arr])` — find match, set `RSTART/RLENGTH`; 3-arg captures groups (Chapter 10).
- `sprintf(fmt, ...)` — formatted string (Chapter 13).
- `tolower(s) / toupper(s)` — case conversion.

Arithmetic functions:

- `sin, cos, atan2, exp, log, sqrt, int` — math; `log/sqrt` warn on negative and return NaN (Chapter 11).
- `rand() / srand([seed])` — pseudo-random number and seeding.
- `intdiv(a, b, arr) / intdiv0(a, b)` — integer division (Chapter 12).
- `strtonum(s)` — string to number honoring `0x/0` prefixes (Chapter 11).

I/O and general:

- `print, printf` — output statements with redirection (Chapters 2, 13).
- `getline` — explicit read in its several forms (Chapter 7).
- `close(name [, how])` — close a handle (Chapter 7).
- `fflush([name])` — flush output (Chapter 7).
- `system(cmd)` — run a command, return its status (Chapter 11).

Time functions (gawk):

- `systime()` — current Unix timestamp.
- `strftime([fmt [, ts [, utc]])` — format a timestamp (Chapter 11).
- `mktime(spec [, utc])` — build a timestamp; optional UTC flag (Chapter 11).
- `gettimeofday(arr), getlocaltime(arr [, ts])` — fill time arrays (Chapter 12).

Bitwise functions (gawk):

- `and, or, xor` — variadic bit operations (Chapter 11).
- `compl(x), lshift(x, n), rshift(x, n)` — complement and shifts; fatal on negative (Chapter 11).

Type and array functions (gawk):

- `typeof(x)` — type as a string (Chapter 11).
- `isarray(x)` — 1 if `x` is an array.
- `mkbool(expr)` — typed boolean.
- `asort(src [, dst [, how]])` / `asorti(...)` — sort values / indices (Chapter 8).

Filesystem and OS extensions (native; no `@load` needed):

- `stat`, `statvfs`, `readdir`, `fts`, `chdir`, `rename`, `readfile` — filesystem access (Chapter 12).
- `writea(path, arr)` / `reada(path, arr)` — persist and restore arrays (Chapter 12).
- `ord(s)` / `chr(n)` — character and codepoint conversion (Chapter 12).
- `sleep(seconds)` — fractional-second sleep.
- `inplace_tmpfile`, `inplace_commit` — safe in-place edit primitives.
- `revoutput`, `revtwoway` — gawk demo string reversers.

Internationalization (gawk):

- `bindtextdomain`, `dcgettext`, `dcngettext` — gettext catalogs (Chapter 12).

Appendix C — Command-Line Flags

A compact reference to awkrs's flags; Chapter 14 gives the full treatment and the compatibility notes. Where awkrs's behavior differs from gawk's, the difference is noted there.

POSIX core:

- `-f progfile` — read program from file (repeatable; the cached form).
- `-F fs` — set `FS`.
- `-v var=value` — set a variable before `BEGIN` (repeatable).

GNU/gawk sources and modes:

- `-e/--source code` — inline program fragment (repeatable).
- `-i/--include file` — include a library file (repeatable).
- `-E/--exec file` — run a program file, remaining args as data.
- `-b/--characters-as-bytes` — byte-length `length/substr/index`.
- `-c/--traditional` — traditional-compatibility behaviors.
- `-C/--copyright` — print license and exit.
- `-d/--dump-variables [file]` — dump globals after the run.
- `-D/--debug [file]` — static rule/function listing (not gawk's debugger).
- `-g/--gen-pot` — generate a gettext template and exit.
- `-k/--csv` — CSV input mode (Chapter 5).
- `-l/--load name` — load a library from `AWKPATH`.
- `-L/--lint <fatal|invalid|no-ext>` — static lint.
- `-M/--bignum` — MPFR arbitrary precision; disables the JIT (Chapter 19).
- `-N/--use-lc-numeric` — locale-aware numeric formatting (Chapter 19).
- `-n/--non-decimal-data` — recognize `0x/0` in input.
- `-o/--pretty-print [file]` — AST pretty-print.
- `-O/--optimize` — accept optimize request (JIT on unless `-s`).
- `-p/--profile [file]` — wall-clock/hit-count summary (single-thread).
- `-P/--posix` — stricter POSIX behavior in places.
- `-r/--re-interval` — no-op (intervals always on).
- `-s/--no-optimize` — disable peephole and JIT (same as `AWKRS_JIT=0`).
- `-S/--sandbox` — block `system`, redirects, pipes, coprocesses, sockets.
- `-t/--lint-old` — older-style lint.

mawk compatibility:

- `-W opt` — `help`, `version`, `dump`, `exec=path` honored; other tokens accepted (Chapter 14).

awkrs-specific:

- `-j/--threads N` — parallel worker count (default 1; Chapter 15).
- `--read-ahead N` — stdin parallel chunk size (default 1024).
- `--lsp` — start the Language Server over stdio (Chapter 20).
- `--dap [host:port]` — start the Debug Adapter, stdio or TCP (Chapter 20).
- `--aot out` — AOT-compile a `BEGIN`-only program to a native binary (Chapter 17).

Environment variables:

- `AWKRS_CACHE=0` — disable the bytecode cache (Chapter 18).
- `AWKRS_JIT=0` — disable peephole/JIT optimization (Chapters 16, 17).

- `AWKR5_FUSEVM=0` — force the bytecode interpreter for every chunk (Chapter 17).
- `FUSEVM_JIT_CACHE_DIR`, `FUSEVM_JIT_CACHE_MAX_BYTES` — control the machine-code cache (Chapter 17).
- `AWKR5_DAP_LOG` — trace the debug adapter to stderr (Chapter 20).
- `NO_COLOR` — force plain text in the help HUD.

Appendix D — Glossary

Action — the `{ ... }` body of a rule, the statements run when the pattern matches (Chapter 3).

AOT (ahead-of-time compilation) — compiling a `BEGIN`-only program to a standalone native executable via `--aot` and fusevm's AOT path (Chapter 17).

Associative array — awk's only data structure: a hash map from string keys to scalar values, the basis of every collection (Chapter 8).

Bytecode cache — awkrs's persistent, rkyv-backed cache of compiled programs at `~/.awkrs/scripts.rkyv`, which lets a second `-f` run skip compilation (Chapter 18).

Chunk — a flat vector of bytecode opcodes; the compiled form of a rule body, function, or special block (Chapter 16).

CONVFMT / OFMT — the formats for converting a number to a string in general contexts and in `print` respectively, both `"%.6g"` by default (Chapter 13).

Coprocess — a two-way pipe to a subprocess, written `|&`, allowing both writing to and reading from a filter (Chapters 7, 12).

DAP (Debug Adapter Protocol) — the editor-debugger protocol awkrs speaks with `--dap`, giving breakpoints, stepping, and variable inspection (Chapter 20).

Field — a column of a record, addressed `$1`, `$2`, ..., split by `FS` (Chapter 5).

FS / OFS / RS / ORS — the input field separator, output field separator, input record separator, and output record separator (Chapters 3, 5).

Fused superinstruction — a single bytecode opcode that replaces a common short sequence, the peephole-fusion optimization at the core of awkrs's interpreter speed (Chapter 16).

fusevm — the shared bytecode VM, with Cranelift JIT tiers, that awkrs offloads eligible numeric chunks to; also used by zshrs and strykelang (Chapter 17).

getline — the explicit record-read statement, with a defined return protocol (1 read, 0 EOF, -1 error, -2 retry) and several source forms (Chapter 7).

JIT (just-in-time compilation) — fusevm's Cranelift compilation of eligible numeric chunks to native code, on by default, marked experimental, a large win on numeric loops and inert elsewhere (Chapter 17).

LSP (Language Server Protocol) — the editor-tooling protocol awkrs speaks with `--lsp`, providing completion, hover, diagnostics, and navigation (Chapter 20).

Numeric string (strnum) — a value from input that looks like a number and therefore compares numerically; distinct from a program string constant, which does not (Chapter 6).

Parallel-safe — the static property, checked before a `-j` run, that a program's record rules have no cross-record dependence and so may run concurrently (Chapter 15).

Pattern — the condition of a rule: empty (always), a regex, an expression, a range, or a special pattern (Chapter 4).

Peephole fusion — see fused superinstruction (Chapter 16).

PROCINFO / SYMTAB / FUNCTAB — gawk's introspection arrays exposing process facts, the symbol table, and the function table (Chapter 12).

Record — the unit awk processes one at a time, a line by default, configurable through `RS` (Chapters 3, 5).

Rule — a `pattern { action }` pair; a program is a list of rules (Chapter 3).

Slot — a `u16`-indexed flat-array location for a user scalar variable, the fast access path distinct from the HashMap path for specials and arrays (Chapter 16).

SUBSEP — the separator joining multidimensional array subscripts into one key, the mechanism behind `a[i, j]` (Chapter 8).

Special pattern — `BEGIN`, `END`, `BEGINFILE`, or `ENDFILE`: patterns that fire at fixed points rather than per record (Chapters 3, 4).

typeof / strnum / untyped — gawk's runtime type vocabulary, reported by the `typeof` builtin (Chapter 11).

AOT — see the entry above; ahead-of-time compilation of a `BEGIN`-only program to a standalone native executable via `--aot` (Chapters 17, 39).

BEGINFILE / ENDFILE — gawk's per-file special patterns, running before and after each input file with `FILENAME` set; `next/nextfile` are invalid in `BEGINFILE` (Chapters 3, 31).

CSV mode — the `-k/--csv` field-splitting mode, an RFC-style parser handling quoting and the doubled-quote escape, aligned with `gawk --csv` (Chapters 5, 27).

FPAT / FIELDWIDTHS — alternative field-definition variables: `FPAT` describes a field by a regex it matches, `FIELDWIDTHS` gives fixed column byte widths (Chapter 5).

fusevm machine-code cache — fusevm's persistent on-disk cache of Cranelift-emitted native code at `~/.cache/fusevm-jit`, keyed by chunk op-hash, distinct from awkrs's bytecode cache (Chapters 17, 18).

IGNORECASE — gawk variable that, when true, makes regex matching and comparison case-insensitive, with a precise scope (on for regex contexts and multi-char separators, off for single-char literal separators) (Chapter 10).

Inline fast path — a hand-written loop that bypasses the VM's call machinery entirely for a single-rule program whose body is one fused opcode, such as `{ print $1 }` (Chapters 16, 22).

Numeric string parse mode — the `-n/--non-decimal-data` toggle (and the `strtonum` builtin) that makes string-to-number conversion honor `0x` hex and `0` octal prefixes (Chapters 14, 28).

Out-parameter — awk's idiom for returning more than one value: a function fills an array passed by reference or writes to agreed globals, since `return` yields one scalar (Chapters 9, 40).

Padded sort key — the idiom of packing a sort key into a fixed-width zero-padded string so lexical comparison orders it numerically, used with `asort` (Chapter 40).

Parity suite — the cross-engine test harness in `parity/` that diffs awkrs's output against gawk, mawk, and BSD awk byte-for-byte across more than two thousand cases (Chapters 25, 37).

Sandbox — the `-S/--sandbox` flag that blocks every outside-touching builtin — `system`, redirects, pipes, coprocesses, sockets, and the filesystem builtins — via a single routed check (Chapters 24, 29).

Single source of truth — awkrs's discipline of writing each fact (builtin names, special-variable names, the doc corpus, the grammar) in exactly one place consumed everywhere (Chapters 20, 25).

SYMTAB / FUNCTAB — gawk introspection arrays aliasing the program's globals and listing its function names, supporting reflection and dispatch (Chapters 12, 30).

Appendix E — The Opcode Families

This appendix is a compact map of the bytecode instruction set — the `Op` enum's ninety-six variants — grouped by family, the same organization Chapter 22 gave in prose. It is the engine's instruction-set architecture at a glance; the full behavior of each is in Chapter 22, and the fused superinstructions in Chapter 16.

Constants and literals:

- `PushNum(f64)` — push a float literal.
- `PushNumDecimalStr(idx)` — push a decimal-integer literal by digit-string index; parsed exactly under `-M`.
- `PushStr(idx)` — push an interned string constant.
- `PushRegexp(idx)` — push a gawk `@/.../` typed-regexp value.

Variable and field access:

- `GetSlot(u16) / SetSlot(u16)` — the fast flat-array path for user scalars.
- `GetVar(u32) / SetVar(u32)` — the `HashMap` path for special variables.
- `GetField / SetField` — read/write `$n` with the index on the stack.
- `GetArrayElem(arr) / SetArrayElem(arr)` — associative-array element access.
- `GetNR / GetFNR / GetNF` — read these hot specials without a map lookup.
- `SymtabKeyCount` — `length(SYMTAB)`.

Compound assignment and increment:

- `CompoundAssignSlot/Var/Field/Index(op)` — `lvalue op= rhs` for each lvalue kind.
- `IncDecSlot/Var/Field/Index, IncrVar / DecrVar` — `++/--` on each lvalue kind, with result-discarding statement variants.

Arithmetic, comparison, unary:

- `Add Sub Mul Div Mod Pow` — arithmetic; `Pow` is right-associative `^`.
- `CmpEq CmpNe CmpLt CmpLe CmpGt CmpGe` — POSIX number-or-string comparisons.
- `Neg Pos Not, ToBool` — unary operators and truth normalization.

String, regex, control flow:

- `Concat, RegexMatch / RegexNotMatch, MatchRegexp(idx)` — string join and the `~/!~/` bare-regex tests.
- `Jump(i), JumpIfFalsePop(i) / JumpIfTruePop(i)` — absolute-index branches.

Output, flow signals, calls:

- `Print { argc, redir } / Printf { argc, redir }` — output with a `RedirKind` of `Stdout/Overwrite/Append/Pipe/Coproc`.
- `Next NextFile ExitWithCode ExitDefault ReturnVal ReturnEmpty` — the flow-signal op-codes the record loop reads as `Flow`.
- `CallBuiltin / CallUser / CallIndirect / CallUserBindArrays` — builtin, user, indirect (`@name`), and array-by-reference calls.

Types, arrays, complex builtins:

- `TypeofVar/Slot/ArrayElem/Field/Value` — `typeof` for each operand kind.
- `InArray / DeleteArray / DeleteElem, JoinArrayKey(n)` — membership, deletion, and `SUBSEP` key joining.

- `GetLine { var, source, push_result }` — `getline` in its `Primary/File/ Coproc/Pipe` forms.
- `SubFn(target) / GsubFn(target)` — substitution by `SubTarget (Record/Var/SlotVar/Field/Index)`.
- `Split / Patsplit / MatchBuiltin, Asort / Asorti` — array-filling builtins and the sorters.
- `ForInStart/ForInNext/ForInEnd, Pop / Dup` — iteration and stack primitives.

Fused superinstructions (peephole, Chapter 16):

- `AddFieldToSlot, AddSlotToSlot, AddMulFieldsToSlot, ArrayFieldAddConst` — accumulator fusions.
- `IncrSlot / DecrSlot, PushFieldNum, JumpIfSlotGeNum` — counter and loop-test fusions.
- `PrintFieldStdout, PrintFieldSepField, PrintThreeFieldsStdout, ConcatPoolStr` — output and concatenation fusions.

A test asserts every variant fits in thirty-two bytes, keeping a `Chunk` — the flat `Vec<Op>` a rule body compiles to — compact and cache-friendly.

Appendix F — A Source Map

A module-by-module guide to awkrs's roughly fifty thousand lines of Rust, for the reader who wants to follow the book's citations into the code. Chapter 25 gives the narrative; this is the index.

The front end (Chapter 21):

- `lexer.rs` (~3,600 lines) — tokenizer; string escapes, number tokens, the division-versus-regex slash decision.
- `parser.rs` (~5,100 lines) — recursive-descent parser; the full grammar, the precedence ladder, the gawk extensions.
- `ast.rs` / `ast_fmt.rs` — the syntax-tree node types and the AST-to-awk reformatter behind `-o` (Chapter 44).
- `source_expand.rs` — `@include/@load` pre-parse source expansion.

The compiler and the machine (Chapters 16, 22):

- `compiler.rs` (~3,900 lines) — AST-to-bytecode lowering, slot allocation, peephole fusion, static checks, the parallel-safety flag.
- `bytecode.rs` — the `Op` enum, `Chunk`, `StringPool`, and `CompiledProgram`.
- `vm.rs` (~3,000 lines) — the stack-machine dispatch loop, fast paths, array call-by-reference.
- `runtime.rs` (~5,000 lines) — the central mutable state: fields, `$0`, variables, slots, separators, MPFR integration.
- `vm_builtins.rs` / `builtins.rs` — the builtin-function implementations.
- `flow.rs` / `error.rs` / `limits.rs` — the `Flow` enum, the error taxonomy, the recursion cap (Chapter 24).

I/O, formatting, and features:

- `record_io.rs` — the `RS`-aware record splitter (Chapter 23).
- `format.rs` (~2,400 lines) — `printf/sprintf` and number formatting (Chapter 13).
- `gawk_extensions.rs` — the filesystem/OS/time native builtins (Chapter 12).
- `namespace.rs` — `@namespace` prefixing (Chapter 12).
- `bignum.rs` — the `-M` MPFR integration (Chapter 19).
- `locale_numeric.rs` — the `-N` locale formatting (Chapter 19).
- `procinfo.rs` — the `PROCINFO` array population (Chapter 12).

The JIT, the cache, and the tooling (Chapters 17, 18, 20):

- `fusevm_bridge.rs` / `fusevm_compile.rs` / `fusevm_host.rs` (~4,100 lines together) — the fusevm offload: translation, lowering, host helpers.
- `aot.rs` — the `--aot` ahead-of-time path.
- `script_cache.rs` — the persistent rkyv bytecode cache.
- `lsp.rs` — the Language Server.
- `dap.rs` / `debugger.rs` — the Debug Adapter and its front-end-agnostic state machine (Chapters 20, 45).
- `cli.rs` / `cli_effects.rs` — flag parsing and mode dispatch, plus the linter, the reformatter driver, the POT generator, the variable dump, and the profile summary (Chapters 43, 44).
- `repl.rs` — the interactive REPL over reedline (Chapter 42).
- `banner.rs` / `cyber_help.rs` — the shared ASCII logo and live-stats banner, and the `--help` HUD that sources it (Chapter 46).

The driver and the binaries (Chapter 25):

- `lib.rs` — the crate root tying the pipeline together.
- `main.rs` — the `awks` binary; `bin/ars.rs` — the `aw` binary; `bin/gen_docs.rs` — the `gen-docs` reference-page generator.

Appendix G — Migrating from gawk, mawk, and BSD awk

awkrs is meant to be a drop-in: a program written for another awk should run unchanged, because the surface is the POSIX language plus the gawk extensions that show up in real scripts, verified byte-for-byte by the parity suite. Migration is therefore rarely about rewriting and almost always about *verifying* — checking the handful of places where two conforming awks are allowed to differ, which POSIX leaves unspecified and which awkrs's compatibility matrix is explicit about calling a feature matrix rather than a proof. This appendix is a porting checklist organized by the awk you are coming from; Chapters 14 and 32 give the full matrix.

From gawk. This is the smoothest path, because gawk is the awk awkrs tracks most closely. Almost everything carries: `gensub`, the bitwise and time builtins, `PROCINFO`, `SYMTAB`, `FUNCTAB`, `@namespace`, `@include`, the typed regexp value, `FPAT`, `FIELDWIDTHS`, CSV mode, the `/inet` sockets, and coprocesses. The things to verify are the ones POSIX leaves undefined and no awk promises across implementations: the iteration order of `for (k in a)` unless you pin it with `PROCINFO["sorted_in"]`; the exact pseudo-random sequence from `rand()` for a given `srand(seed)`, which is implementation-specific; and subtle last-digit rounding in `%g` formatting. If a gawk script depends on a *dynamically loaded* extension via `@load` of a compiled `.so`, that will not port — awkrs draws its single-binary line at native code by design and does not load arbitrary shared extensions, offering instead a fixed set of built-in filesystem and OS functions (Chapter 12). Run the script, diff the output against gawk on a representative input, and the differences that remain will be in exactly these unspecified corners.

From mawk. mawk is a fast, traditional awk without most of the gawk extensions, so migrating to awkrs is a strict superset: everything mawk runs, awkrs runs, and the gawk extensions mawk lacks become available if you want them. There is nothing to remove. The reason to migrate is usually performance or tooling — the Language Server and Debugger of Chapter 20, the parallel path of Chapter 15, the bytecode cache of Chapter 18 — none of which mawk offers. Verify the same unspecified corners (hash order, RNG sequence, rounding), and note that any mawk-specific behavior you were relying on in an unspecified area was never portable in the first place.

From BSD awk / the one-true-awk. This is the most conservative source: BSD `nawk` and Kernighan's one-true-awk lack most gawk extensions, so a script written for them uses a small, universal subset that awkrs runs without question. The migration is entirely additive — you gain the extensions, the speed, and the tooling, and lose nothing — and the only scripts that need a second look are those that leaned on a BSD-awk quirk in unspecified territory. If you want to keep a script maximally portable *back* to these awks, stay inside the POSIX subset and awkrs's traditional mode (`--traditional/--posix`) will hold you to it, disabling the gawk extensions so a program that runs clean under it is portable to the conservative awks too.

The checklist, gathered. Regardless of source: (1) run the program under awkrs on a real input and diff against the old awk; (2) if `for (k in a)` output order matters, set `PROCINFO["sorted_in"]` rather than relying on hash order; (3) if a fixed `rand()` sequence matters, do not assume it survives the port — re-derive any golden values; (4) expect last-digit `%g/OFMT` differences and format explicitly where exactness matters; (5) replace any dependence on a dynamically loaded gawk extension with awkrs's built-in equivalents or a different design; (6) if portability back to conservative awks is a goal, develop under `--posix`. Everything outside this list is designed to be identical, which is the whole point of a parity suite that runs on every push.

Appendix H — The Performance Field Guide

A compressed, operational version of Chapters 16, 17, 18, and 38 — what to do when a program is slower than you expect, and how to know which of `awkrs`'s fast paths it is or is not hitting. The rule that frames all of it: `awkrs`'s everyday speed comes from the bytecode VM and its fast paths, not the JIT, and those fast paths engage *for* string-and-field work, so most tuning is about keeping a program in the shape the fast paths recognize.

Measure first, and measure honestly. Use `hyperfine` (the tool the benchmark tables use) rather than a single `time`, because a single run is dominated by noise and cold-cache effects. Distinguish cold from warm: the *first* run of a script file pays the lexer, parser, and compiler; every run after pays none of it, because the bytecode cache (Chapter 18) mmap's the compiled program back zero-copy. If you are benchmarking steady-state throughput, warm the cache with a discarded run first; if you are benchmarking cold start — the thing that matters for a one-liner invoked a hundred times an hour — measure the first run deliberately and do not warm it. Isolate startup from throughput by comparing a tiny input against a large one: the difference is per-record cost, the floor is fixed overhead.

Diagnose the fast paths from the program's shape. The four levers of Chapter 16's synthesis: keep the program to one simple rule to hit the inline bypass; keep `FS` at its default and read from a real file (not a pipe) to hit the raw-byte field path; write the tight loops in the recurring shapes so the superinstruction fusions fire; and understand that the JIT is a bonus on numeric loops, not a lever you tune toward. If a filter-and-project program is slow, the first suspects are a non-default `FS` that forces the general splitter, an unnecessary `$0` rebuild from a stray field assignment, or input arriving on a pipe that cannot be memory-mapped.

Anti-patterns that defeat the fast paths. Assigning to a field you did not need to change forces a `$0` rebuild every record — the `$1 = $1` normalizer is deliberate, but an accidental field write is pure cost. A multi-character regex `FS` in a hot loop is slower than a single-character or default `FS`; use the simplest separator the data allows. Recompiling a dynamic regex from a string that changes every record defeats the compiled-regex cache — hoist a constant pattern into a `@/.../` typed value or a variable set once. And piping a file into `awkrs` (`cat f | awkrs ...`) when you could pass it as an argument (`awkrs ... f`) forfeits the memory-map and the parallel file path for no reason.

When the special modes help, and when they cost. Reach for `-j N` only on a large input with a *parallel-safe* program (Chapter 15) — a stateless filter or transform — and measure the thread count rather than assuming the core count; on small inputs or accumulating programs it is slower or inert, which is why it is off by default. Reach for `-M` (arbitrary precision, Chapter 19) only when you need exactness beyond a 64-bit float, and expect it to be *slower*, because MPFR arithmetic is not free — it buys correctness, not speed. The bytecode cache needs no action; it engages automatically for script files and is the reason a repeatedly-invoked script has no compile cost after the first run. The distilled guidance: simple rule, default `FS`, file argument, constant regexes, and let the cache and fusions do their work — and reserve `-j` and `-M` for the specific situations that call for them.

Appendix I — A Short Cookbook

Three complete programs, longer than a one-liner and shorter than the tour's algorithms, written for this book to show the accumulate-and-dump shape and the reconfiguration idioms doing everyday work. Each uses only features documented in the preceding chapters, and each runs unchanged under `awkrs`.

A log-level summary. Given a log whose second field is a level (`INFO`, `WARN`, `ERROR`), count the levels, track the first and last line each was seen on, and print a sorted report.

```
BEGIN { PROCINFO["sorted_in"] = "@ind_str_asc" }
{
    lvl = $2
    count[lvl]++
    if (!(lvl in first)) first[lvl] = NR
    last[lvl] = NR
}
END {
    for (lvl in count)
        printf "%-6s %6d (lines %d-%d)\n", lvl, count[lvl], first[lvl], last[lvl]
}
```

The record loop is pure accumulation into three arrays keyed by the level; the `!(lvl in first)` guard records the first sighting exactly once (the membership idiom of Chapter 40); the `END` walk sets a sorted iteration order and formats one padded line per level. It is the accumulate-and-dump report in miniature.

A fixed-width to CSV converter. Given input whose columns are fixed-width — say a 6-character id, a 20-character name, and the rest a value — use `FIELDWIDTHS` to split by position and re-emit as CSV.

```
BEGIN { FIELDWIDTHS = "6 20 *"; OFS = "," }
{
    id = $1; name = $2; val = $3
    gsub(/ +$/, "", id); gsub(/ +$/, "", name); gsub(/ +$/, "", val)
    print id, name, val
}
```

`FIELDWIDTHS` (Chapter 5) makes the fields positional rather than separator-delimited, and the `*` token takes the remainder of the line as the last field; the trailing-space `gsub`s trim the padding that fixed-width columns carry; `OFS = ","` joins the cleaned fields with commas. Switching an input format is a `BEGIN`-block reconfiguration, not a rewrite of the loop.

A tiny template filler. Given a data file of `key=value` lines and a template on standard input containing `{{key}}` placeholders, substitute the values into the template — the two-pass `FNR == NR` join applied to text rather than columns.

```

BEGIN { FS = "=" }
FNR == NR { val[$1] = $2; next }
{
    line = $0
    while (match(line, /\{\{[A-Za-z_]+\}\}/)) {
        key = substr(line, RSTART + 2, RLENGTH - 4)
        line = substr(line, 1, RSTART - 1) val[key] substr(line, RSTART + RLENGTH)
    }
    print line
}

```

The first pass — true while `FNR == NR` — loads the `key=value` file into `val` and stops there via `next`; the second pass runs the find-all-match loop of Chapter 40 over each template line, cutting each `{{key}}` out with `RSTART/RLENGTH`, looking the key up, and splicing the value in with `substr` surgery. It combines three catalogued idioms — the two-pass join, the match-and-splice, and membership-free lookup — into a working templater in a dozen lines, which is the point the whole book has been making: the idioms are few, they compose, and once you have them the programs read themselves.

Chapter 56 – Where the Eight Record Cutters Disagree

Chapter 23 introduced `record_io.rs` as two families of four: four streaming readers for the sequential path, four slice splitters for the memory-mapped and slurp paths, one `RS` interpretation each, and it closed on the claim that the record boundaries the two families produce are identical. That claim is the precondition for the parallel path being transparent, so it is worth doing what the summary did not: reading all eight cutters against each other, byte by byte, and finding out exactly where the correspondence holds and where it does not.

The short version is that it holds for three of the four `RS` modes and breaks in a specific, reproducible way in the fourth. There is also a second, quieter asymmetry inside the streaming family — the four readers do not agree on whether the record terminator stays in the output buffer — which is invisible today only because the call sites happen to compensate for it identically, one of them by open-coding the shared helper rather than calling it.

The dispatch, and why there are really five readers

`read_next_record` is the single streaming entry point. It clears the output and terminator buffers, locks the shared reader, and branches:

```
if rs == "\n" {
    return read_until_lf(r, out, rt_sep, leftover);
}
if rs.is_empty() {
    let ok = read_paragraph_record(r, out, rt_sep, leftover)?;
    return Ok(ok);
}
if let Some(re) = regex_rs {
    return read_until_regex_bytes(r, re, out, rt_sep, leftover);
}
read_until_bytes(r, rs.as_bytes(), out, rt_sep, leftover)
```

Four arms, four functions. But `read_until_bytes` is not one strategy; it is two, selected by a length test at its top. A single-byte delimiter goes down a `memchr` plus `BufRead::read_until` path. A multi-byte delimiter goes down a chunked scan-and-refill loop. The two share a function and share nothing else — different search primitives, different buffer handling, and, as the next section shows, different conventions about the delimiter itself. So the streaming family is four dispatch arms over five distinct algorithms.

That fifth algorithm is much harder to reach than the length test suggests, and the reason sits upstream in `runtime.rs`. `ensure_rs_regex_bytes` decides whether an `RS` becomes a compiled regex, and its bail-out is a *character* count:

```
if rs.chars().count() <= 1 {
    self.rs_regex_bytes = None;
    return Ok(());
}
```

Any `RS` of two or more characters is compiled to a regex, so by the time `read_next_record` runs its dispatch the `if let Some(re) = regex_rs` arm has already claimed it. `read_until_bytes` is reached only when `RS` is a single character — and its multi-byte branch, therefore, only when `RS` is a single character whose

UTF-8 encoding runs to more than one byte, such as `"."`. A two-character separator like `"XX"` never crosses that boundary; it crosses into the regex arm. So when you are chasing an `RS` behaviour difference between a one-character and a two-character separator, the boundary being crossed is literal-versus-regex, not single-byte-versus-multi-byte.

The four splitters are a cleaner set, because a complete byte slice admits no buffering questions: `split_lines_unix`, `split_paragraph_mmap`, `split_by_regex_mmap`, and `split_by_delimiter_mmap`, dispatched by `split_input_into_records` on the same three tests in the same order. The literal splitter does not fork on delimiter length — it builds one `memchr::memmem::Finder` and uses it for both cases, which is why the splitter family has four members where the reader family has five.

`split_by_delimiter_mmap` inherits the same reachability problem as its streaming counterpart, and for the same reason: `split_input_into_records` only routes to it when no regex was compiled, which after `ensure_rs_regex_bytes` means a single-character `RS`. Two of this module's nine code paths — the multi-byte streaming branch and the literal splitter's multi-byte case — are therefore reachable in the shipped pipeline only through multi-byte single-character separators. They are tested, and they are correct, but they are close enough to dead code that a change to either would not show up in ordinary use.

The terminator convention is not uniform

Every cutter has to answer the same question: does the separator it just found belong in the record it is returning? All eight say no, semantically — `awk` records never contain their own `RS`. But the streaming readers implement that answer in three different places.

The newline reader leaves the terminator in the buffer. Both the leftover branch and the `read_until` branch include it:

```
if let Some(pos) = memchr(b'\n', leftover) {
    out.extend_from_slice(&leftover[..=pos]);
    leftover.drain(..=pos);
    rt_sep.extend_from_slice(b"\n");
    return Ok(true);
}
```

The `..=pos` is inclusive; the newline goes into `out`. `BufRead::read_until` does the same by contract. So after `read_until_lf` returns, `out` ends with `\n`, and somebody else has to remove it.

The single-byte literal reader removes it in place:

```
if out.last() == Some(&delim[0]) {
    out.pop();
    rt_sep.push(delim[0]);
}
```

The multi-byte literal reader never puts it in — `out.extend_from_slice(&leftover[..start])` stops before the match. The regex reader is the same shape, stopping at `m.start()`. Paragraph mode does its own trimming through `trim_end_record_bytes` at each of its exit points.

So of five streaming strategies, exactly one hands back a record with its terminator attached, and the compensation for that lives in the callers. They do it by testing the same string:

```
let end = if rs == "\n" {
    crate::record_io::trim_end_record_bytes(&self.line_buf)
} else {
    self.line_buf.len()
};
```

That is `runtime.rs`'s primary-input reader; the main record loop in `lib.rs` threads the buffer through `set_record_from_line_buf` after the same `read_next_record` call. The convention is shared by nothing

stronger than repetition: there is no type-level or API-level enforcement that a new caller would have to follow it. The condition is also on the `RS` string, not on which reader ran — which is fine today because the mapping is one-to-one, but it means the invariant is stated in the callers and nowhere in `record_io`'s own interface.

The sharper version of that point is visible in how the two call sites differ. `trim_end_record_bytes` has six call sites — five inside `record_io.rs` itself (the newline reader's two branches, the multi-byte reader, and both exit points of `split_paragraph_mmap`) and one in `runtime.rs`. `set_record_from_line_buf` is not among them. It re-implements the strip loop inline instead:

```
if rs == "\n" {
    while end > 0 && self.line_buf[end - 1] == b'\n' {
        end -= 1;
    }
}
```

That is a byte-for-byte restatement of the helper's body, guarded by the same `rs == "\n"` test, in a different module. So the convention is not “one helper, many callers” — it is a helper with six callers plus one open-coded copy, and the copy is in the path that the main record loop actually uses. A change to the helper's definition of a terminator run would reach the six and miss the seventh.

`trim_end_record_bytes` is worth its own note here, because it strips a *run*:

```
while end > 0 && buf[end - 1] == b'\n' {
    end -= 1;
}
```

Its own test file pins that behaviour explicitly — `trim_end_record_bytes_strips_only_trailing_lf_not_cr` asserts `trim_end_record_bytes(b"abc\n\n") == 3`. For the newline reader that never matters, because `read_until` stops at the first `\n` and so `out` can only ever have one. The multi-newline case exists for paragraph mode, which calls the same helper on a buffer that has accumulated several content lines and needs the whole trailing run gone. Only the paragraph call sites exercise the loop past its first iteration.

The trailing-empty-record question

Here is the divergence. Take an input that *ends* with the separator, which is the normal case for a well-formed text file, and ask each cutter how many records it produces.

For the newline splitter, `split_lines_unix` advances `pos = eol + 1` after each line and its loop condition is `while pos < len`. On `"a\n"` the first iteration pushes `"a"` and sets `pos` to 2, which equals the length, so the loop ends. One record. The streaming newline reader agrees: the next call finds an empty buffer and `read_until` returns zero.

For the literal-delimiter splitter, the same structure holds — `start = abs + delim.len()` then `while start < data.len()` — so `"aXX"` split on `"XX"` yields one record and stops. The streaming literal readers agree, both of them: after consuming the delimiter the leftover is empty, the next `read` returns zero, and both the single-byte and multi-byte arms return `Ok(false)`.

For paragraph mode, `split_paragraph_mmap` only emits when it has a `start`, and `start` is only set on a non-blank line, so trailing blank lines cannot manufacture a record. The streaming paragraph reader tracks `saw_content` for exactly the same reason and returns `Ok(saw_content)` at end of input.

Now the regex splitter:

```
fn split_by_regex_mmap<'a>(data: &'a [u8], re: &BytesRegex) -> Vec<'a [u8]> {
    let mut out = Vec::new();
    let mut last = 0usize;
    for m in re.find_iter(data) {
        out.push(&data[last..m.start()]);
        last = m.end();
    }
    out.push(&data[last..]);
    out
}
```

The final `out.push(&data[last..])` is unconditional. When the last match ends at the end of the data, `last` equals the data length and that push adds an empty slice. So `"a;;"` split on a regex matching `";;"` yields two records — `"a"` and the empty string — where the guarded loops elsewhere in the module yield one. The module's own test suite pins this rather than treating it as a defect: `split_regex_rs_mmap` feeds `"axxbxx"` through the regex `x+` and asserts three records, the third empty.

It is tempting to reach for the literal splitter as the contrast case, but as the first section established, that comparison cannot be made from an `awk` program. `RS="XX"` is two characters, so `ensure_rs_regex_bytes` compiles it to a regex and `split_by_delimiter_mmap` never sees it. The literal splitter's guarded loop is correct, and it is unreachable for exactly the separators that would expose the difference. The contrast that a user can actually observe is the streaming reader, below.

The streaming regex reader does not do this. Its end-of-input branch is guarded:

```
let n = reader.read(&mut chunk).map_err(Error::Io)?;
if n == 0 {
    if leftover.is_empty() {
        return Ok(false);
    }
    out.extend_from_slice(leftover);
    leftover.clear();
    return Ok(true);
}
```

After the final match, `leftover` was drained to `m.end()`, which is the end of input, so it is empty, so the reader reports end-of-input rather than an empty record.

That is the break. For `RS` set to a multi-character value — which, per the first section, means any `RS` of two or more characters — an input terminated by a separator produces one more record on the memory-mapped and slurp paths than on the streaming path, and that extra record is empty.

The axis of the divergence is worth stating carefully, because the obvious guess is wrong. It is not parallel versus sequential. It is *file* versus *stdin*:

```
$ printf 'a;;b;;' > t.txt
$ awkrs 'BEGIN{RS=";;"}{printf "[%s]", $0}' t.txt
[a][b][]
$ cat t.txt | awkrs 'BEGIN{RS=";;"}{printf "[%s]", $0}'
[a][b]
$ gawk 'BEGIN{RS=";;"}{printf "[%s]", $0}' t.txt
[a][b]
```

Three records from a named file, two from the identical bytes on standard input, and `gawk` agrees with the streaming answer. Adding `-j1` or `-j4` to the file invocation does not change the record count: both still emit three. A named file is read by slurping or mapping it, and that is what selects the unconditional push, whether or not any worker threads are involved.

Sequential file input does not even reach `split_by_regex_mmap`. `process_file_slurp` in `lib.rs` carries a *third* implementation of regex record splitting, written inline:

```

if let Some(regex) = re_owned.as_ref() {
    let mut last = 0usize;
    for m in regex.find_iter(data) {
        let chunk = &data[last..m.start()];
        last = m.end();
        count += 1;
        if dispatch_slurp_record(cp, range_state, rt, &fs, chunk, m.as_bytes())? {
            return Ok(count);
        }
    }
    let chunk = &data[last..];
    count += 1;
    ...
}

```

Same structure, same unconditional trailing push, dispatching each record as it goes rather than collecting a vector. So the “one empty record too many” behaviour is implemented twice — once in `record_io.rs` for the splitter families and once in `lib.rs` for the sequential slurp loop — and a fix applied only to `split_by_regex_mmap` would leave the more commonly taken path unchanged.

The consequences are concrete: `NR` at the end of a run differs by one between file and stdin input, and any rule that fires on every record fires one extra time with an empty `$0`, which for a counting or summing program is harmless and for a printing program is a spurious blank line. One caution on measuring this: `END { print NR }` is not a usable probe under `-j4`, which reports `0` there even for the default `RS` on ordinary line input. Count the records the rules actually see instead.

Three of the four `RS` modes agree between the families. The regex mode does not, in the one input shape that is most common. The correspondence Chapter 23 stated without qualification is a three-quarters correspondence, and the missing quarter is an unconditional `push` written twice in two modules.

What each mode costs

The eight cutters also differ sharply in asymptotic behaviour, and the split does not follow the streaming/splitting line — it follows the search primitive.

`split_lines_unix` walks the data with `memchr`, which is a vectorised byte scan; total work is linear in the input and each byte is examined once. `split_by_delimiter_mmap` builds a `memchr::memmem::Finder` once, outside the loop, and reuses it, so it gets a prepared substring searcher rather than rebuilding search state per call. Both are linear with a good constant.

The streaming newline reader is linear too, because `BufRead::read_until` is itself a `memchr`-driven scan over the reader’s buffer.

The multi-byte streaming literal reader is the outlier:

```

loop {
  if leftover.len() >= delim.len() {
    for start in 0..leftover.len() - delim.len() {
      if &leftover[start..start + delim.len()] == delim {
        out.extend_from_slice(&leftover[..start]);
        rt_sep.extend_from_slice(delim);
        leftover.drain(..start + delim.len());
        return Ok(true);
      }
    }
  }
  let n = reader.read(&mut chunk).map_err(Error::Io)?;
  ...
  leftover.extend_from_slice(&chunk[..n]);
}

```

The scan starts at index zero of `leftover` on every iteration. When a record is long relative to the chunk buffer — declared as `let mut chunk = [0u8; 4096];` — each refill rescans everything already scanned, so the byte-comparison count grows with the square of the record length rather than linearly. The comparison is also a naive slice equality in a hand-written loop, not the `memmem` finder its own memory-mapped counterpart uses.

How much this matters depends entirely on the reachability question from the first section. This loop runs only for a single multi-byte character `RS`, so the worked example cannot be `RS="\n\n"` — two characters, compiled to a regex, routed elsewhere. It has to be something like `RS="."`. For a bullet separating records of a few hundred bytes the quadratic term is invisible. For a bullet cutting a stream into multi-megabyte documents it is the difference between linear and quadratic, and the fix — hoist a `Finder`, remember how far the previous scan reached, and restart just short of that point — is entirely local to this function. It is also, given how few separators route here, a low-priority fix.

The regex streaming reader has the same rescan structure — `re.find(leftover)` runs over the whole accumulated buffer after every refill — but the constant is a regex engine rather than a byte compare, so the effect is larger for the same input size. This is the reader that carries the cost in practice, because every `RS` of two or more characters is routed to it. The quadratic rescan is worth fixing here first. Both readers also grow `leftover` without bound while searching, so a stream containing no separator at all is buffered entirely in memory before the end-of-input branch returns it as one record. That is the correct `awk` semantic — a file with no `RS` in it is one record — but it means `RS` is not a memory-safety valve.

The paragraph reader is the most allocation-heavy of the five. It allocates a `line` vector per call, allocates a second `peek` vector when it enters the blank-line drain, and in the pushback case allocates a third:

```

let mut new_leftover = peek.clone();
new_leftover.extend_from_slice(leftover);
*leftover = new_leftover;

```

That is a clone of the pushed-back line plus a copy of the entire existing leftover, per record boundary, because the buffer has no room at the front. A `VecDeque`, or a leftover represented as a buffer plus a start offset, would make the pushback a constant-time operation. Paragraph mode is the least common `RS` setting, which is presumably why it carries the least optimised implementation, and the code is readable in exchange — but the three allocations are there on every record.

RT, and which readers can tell you the truth

`gawk`'s `RT` holds the text that actually terminated the record, which for a regex `RS` is the only way to recover what matched. All five streaming strategies fill `rt_sep`, and the callers push it into the runtime with `set_rt_from_bytes`.

The values differ in how much information they carry. The newline reader always writes exactly the one byte `b"\n"`, which is a constant — `RT` under the default `RS` tells you only whether the record was terminated at

all, since the final record of a file with no trailing newline leaves `rt_sep` empty. The literal readers write the delimiter back, which is also a constant, and again the last record of an unterminated input gets an empty `RT`. Only the regex reader writes something you could not have predicted:

```
rt_sep.extend_from_slice(m.as_bytes());
```

`m.as_bytes()` is the matched text, so a program using a character-class `R5` can read `RT` to find out whether this record ended on a semicolon, a comma, or a run of both. That is the case the feature exists for.

Paragraph mode's `RT` is the most involved of the five, accumulating the trailing newlines stripped from the record and then the entire run of blank lines that separated it from the next one, so `RT` reports the full gap width. Chapter 23 covers that contract; the point here is only that it is the one mode whose `RT` construction is spread across several separate `extend_from_slice` calls in three branches, which is why it has the highest ratio of comment to code in the module.

None of the four *splitters* produce `RT` at all. They return borrowed record slices and nothing else, because the signature has no place to put a per-record terminator. That does not mean `RT` is broken on the slurp path, though, and it is worth being precise about why. The callers reconstruct it. `dispatch_slurp_record` takes the terminator bytes as a parameter and sets it:

```
rt.set_rt_from_bytes(rt_bytes);
```

The inline regex loop passes `m.as_bytes()`, the actual matched text; the literal loop passes `rs.as_bytes()`, or `b"\n"` in paragraph mode. So `RT` reads correctly on file input, and a program printing `RT` gets the same answer there as on stdin. The asymmetry is one of layering rather than behaviour: on the streaming side `record_io` owns the terminator, while on the slurp side the caller reconstructs it from what it already knows. The reconstruction is exact for the regex case and an approximation for the literal case — the trailing empty record produced by the unconditional push reports the separator as its `RT` under `-j4`, where the streaming reader would have reported nothing, because `rs.as_bytes()` is passed unconditionally rather than derived from a match.

Reading the module as a whole

`record_io.rs` is small, has no dependencies beyond `memchr`, the bytes regex, and the runtime's shared-reader type, and does one job. It is also the module where the the streaming and slurping input models have to agree, and the audit above says they agree in three of four modes, disagree by one empty record in the fourth, locate `RT` in different layers, and are implemented with five algorithms on one side against four on the other — two of which are barely reachable once `ensure_rs_regex_bytes` has routed every multi-character `R5` to the regex arm.

None of that makes the module wrong. It makes it the place to look first when a program produces a different answer reading a named file than reading the same bytes on standard input, and it says which of the four `R5` settings to suspect. If you are setting `R5` to a multi-character pattern and comparing record counts across those two input routes, the unconditional final push — in `split_by_regex_mmap` and again in `process_file_slurp`'s inline regex loop — is the first thing to read.

Chapter 57 – The Builtin Dispatch Arm, Read Exactly

Chapters 27 through 29 walk the builtin library by semantics: what `substr` does, what `gensub` does, what the filesystem builtins return. This chapter is about the code that decides *which* of them runs, and it is a different subject. Builtin dispatch in awkrs lives in `vm_builtins.rs`, and reading it answers questions the per-function chapters cannot: what happens when a user function and a builtin share a name, how POSIX mode removes functions from the language, why the arity errors come in two dialects, and where the bignum mode doubles the implementation.

It is not a module, exactly

The file opens with two lines that are not a doc comment:

```
// Builtin-call dispatch for the awkrs VM, split out of vm.rs to keep
// that file manageable.
// Child module of `vm` via `#[path]`; `use super::*` resolves to
// vm.rs's items.
```

`vm_builtins.rs` is not declared in `lib.rs` alongside the other modules. It is attached to `vm` with a path attribute and pulls its entire namespace in with `use super::*`. That single glob is why the file can call `truthy`, `value_to_float`, the `bignum` helpers, and the `VmCtx` type without importing any of them, and why its functions are `pub(super)` and `pub(crate)` rather than `pub`.

The practical effect is that the file is a continuation of `vm.rs` that happens to live in a separate file — the split is for editor and reviewer sanity, not for encapsulation. There is no interface between them to violate, and no way to reason about `vm_builtins.rs` without `vm.rs` open next to it. When you are tracing a builtin and the definition of something it calls is nowhere in the file, the glob is the reason, and `vm.rs` is where to look.

User functions win, at the dispatch level

The first thing `exec_call_builtin` does, before it touches the argument stack, is check the user function table:

```
pub(super) fn exec_call_builtin(
    ctx: &mut VmCtx<'_>, name: &str, argc: u16,
) -> Result<()> {
    let argc = argc as usize;

    // First, check if it's a user function
    if ctx.cp.functions.contains_key(name) {
        return exec_call_user(ctx, name, argc as u16);
    }

    let start = ctx.stack.len() - argc;
    let args: Vec<Value> = ctx.stack.drain(start..).collect();

    let result = exec_builtin_dispatch(ctx, name, args)?;
    ctx.push(result);
    Ok(())
}
```

The ordering is load-bearing. If a name reaches this opcode and a user function of that name exists in the compiled program, the user function is what runs — the builtin arm further down the file is never consulted. Whether the parser and compiler permit such a definition in the first place is decided upstream; what this file establishes is the resolution order once the call is executing.

Note also what the check costs. It is a hash lookup against the compiled program's function map, performed on every builtin call, before the arguments are even drained from the stack. For a tight loop calling `length($0)` a few million times that is a hash of the string on every iteration on top of the string match in the dispatch table below it.

There is a doc comment here that invites a wrong conclusion, and it is worth following to the end because the error it invites is the one Chapter 54 spends its length teaching readers to catch. `exec_builtin_dispatch` is introduced like this:

```
/// Core builtin implementation (also used by JIT `MIXED_BUILTIN_CALL`).
```

Read straight, that says the function has a second caller which bypasses the user-function check above it, which would make the shadowing rule a property of the interpreted opcode rather than of builtin calls in general — a behavioural seam between the interpreted and compiled paths. It is a tidy finding, and it is not true. `MIXED_BUILTIN_CALL` does not exist anywhere in the crate; grepping the whole source tree for it returns exactly one hit, the doc comment itself. `exec_builtin_dispatch` has one caller, `exec_call_builtin`, three lines above it. Every builtin call in `awkrs` goes through the user-function check.

Chapter 28 independently rules out the seam from the other direction: the warning-on-negative behaviour of `log` and `sqrt` is precisely why those functions stay on the interpreter and are not JIT-eligible, since a pure native op cannot emit a host stderr warning. The math builtins most likely to be compiled are the ones the comment would have to be describing, and they are not compiled.

So what the comment documents is not an architecture but a plan, or a removed experiment — an opcode name that either never shipped or was taken out while the comment naming it stayed behind. That is worth more to a reader than the seam would have been. A doc comment asserting a caller is exactly the kind of claim that ages invisibly: nothing compiles it, no test reads it, and it will keep telling every future reader that this function is a shared entry point long after it stopped being one. The correct posture toward a comment naming a symbol is to grep the symbol.

POSIX mode deletes thirty-eight functions

`exec_builtin_dispatch` opens with a compatibility gate:

```

if ctx.rt.posix || ctx.rt.traditional {
  const GAWK_ONLY_BUILTINS: &[&str] = &[ /* thirty-eight names */ ];
  if GAWK_ONLY_BUILTINS.contains(&name) {
    return Err(Error::Runtime(format!(
      "`{name}` is a gawk extension not available in \
      POSIX/traditional mode"
    )));
  }
}

```

The list has thirty-eight names. Grouped by what they are, they run: the bitwise set (`and`, `or`, `xor`, `compl`, `lshift`, `rshift`); the extended string and array functions (`gensub`, `patsplit`, `mkbool`); the time group (`mktime`, `strftime`, `systemtime`, `gettimeofday`, `getlocaltime`); the type predicates (`isarray`, `typeof`); the numeric conversion `strtonum`; the gettext three (`dcgettext`, `dcngettext`, `bindtextdomain`); the filesystem set (`chdir`, `stat`, `statvfs`, `fts`, `readfile`, `readdir`, `rename`); the character conversions (`chr`, `ord`); `sleep`; array persistence (`reada`, `writtea`); the in-place editing pair (`inplace_tmpfile`, `inplace_commit`); the reversal demos (`revoutput`, `revtwoaway`); and integer division (`intdiv`, `intdiv0`).

Three properties of this gate are worth naming. First, it is a *runtime* check, not a compile-time one: a program that calls `strftime` under `--posix` parses, compiles, and starts, and fails at the moment the call executes. A `strftime` call inside a branch that never runs never errors. Second, the check runs on every builtin call under POSIX mode — a linear scan of thirty-eight string comparisons before the real dispatch — though only when one of the two flags is set. Third, `posix` and `traditional` are ORed, so the two flags produce an identical builtin surface even though they differ elsewhere in the runtime.

What the list omits is as informative as what it contains. `asort` and `asorti` are gawk extensions and are not on it, and they do remain callable under `--posix` — but not because of the omission. They are not builtins at this layer at all. The compiler lowers them to dedicated opcodes, `Op::Asort` and `Op::Asorti`, handled in `vm.rs`, and neither name appears anywhere in `vm_builtins.rs`. A call to `asort` never enters this `match`, so the gate never runs against it. Adding both names to `GAWK_ONLY_BUILTINS` would change nothing; the check they would have to fail is on a code path they do not take.

That distinction matters for anyone extending the gate. The list is a hand-maintained set, not a derived one; nothing in the file cross-checks it against the dispatch table below, so a gawk-only builtin added to the `match` without being added here is silently POSIX-legal. And any gawk-only construct promoted to its own opcode leaves the gate's jurisdiction entirely, which is a quieter way for the same hole to open.

Two error dialects in one match

The dominant arity-error form in this file is a template:

```

return Err(Error::Runtime(format!(
  "{argc} is invalid as number of arguments for length"
)));

```

That phrasing is deliberate: it is gawk's message text, and the source says so — the `length` arm carries a comment noting that `length("a", "b")` fatals with “2 is invalid as number of arguments for length” and that awkrs previously ignored the extra arguments silently. `index`, `substr`, `tolower`, `toupper`, `int`, `sqr`, `sin`, `cos`, `atan2`, `exp`, and `log` all use the same template.

A second group does not. `intdiv` and `mkbool` are representative:

```

"intdiv" => {
  if argc != 2 {
    return Err(Error::Runtime(
      "`intdiv` expects two arguments".into(),
    ));
  }
  ...
}
"mkbool" => {
  if argc != 1 {
    return Err(Error::Runtime(
      "`mkbool` expects one argument".into(),
    ));
  }
  ...
}

```

Nor are they the only two. The `expects N argument` phrasing accounts for twenty-four arms in the file: `intdiv`, `mkbool`, `bindtextdomain`, `dcgettext`, `dcngettext`, `chdir`, `stat`, `statvfs`, `fts`, `gettimeofday`, `sleep`, `ord`, `chr`, `readfile`, `revoutput`, `revtwoway`, `rename`, `inplace_tmpfile`, `inplace_commit`, `wrotea`, `reada`, `intdiv0`, `readdir`, and `getlocaltime`. That is not a two-function exception; it is a second dialect covering most of the extension surface, sitting alongside the gawk-parity template used by the core builtins.

The tempting explanation — that the split tracks provenance, with gawk-parity phrasing on the functions gawk has always had and `awkrs`'s own idiom on the newer ones — does not survive the list. `chdir`, `stat`, `statvfs`, and `fts` are gawk's `filefuncs` extension; `ord` and `chr` are `ordchr`; `wrotea` and `reada` are `rwwarray`; `readdir`, `revoutput`, and `revtwoway` are gawk extensions with their own long histories, and `awkrs` documents them as such in `gawk_extensions.rs`. These are not new functions. Whatever produced the split, it was not the age of the builtin.

The practical consequence stands regardless of cause: both forms are diagnostics, both name the function and the requirement, but a test that greps `stderr` for the parity phrasing will pass for the core builtins and fail for twenty-four others, and that is a sharp edge worth knowing about before writing such a test.

The arity checks themselves are also inconsistent in *strictness* across arms in a way that matters. `length` rejects two or more arguments but accepts zero. `substr` uses a range check, `if !(2..=3).contains(&argc)`, so both too few and too many are caught. `index` demands exactly two. The pattern is per-function and correct per-function, but there is no shared arity table — every arm states its own rule inline, which is why adding a builtin means remembering to write the check by hand.

IGNORECASE reaches further than you would guess

The `index` arm implements case-insensitive search, which is not obvious from the function's specification:

```

let pos = if ctx.rt.ignore_case_flag() {
    let hay_lc = hay.to_lowercase();
    let needle_lc = needle.to_lowercase();
    hay_lc.find(needle_lc.as_str()).map(|b| {
        if ctx.rt.characters_as_bytes {
            b + 1
        } else {
            hay.char_indices()
                .take_while(|&(off, _)| off < b)
                .count() + 1
        }
    })
} else { /* ... */ };

```

The comment above it states the parity target: with `IGNORECASE` set, `index("ABC", "b")` returns 2. So `IGNORECASE` is not only a regex control — it changes the behaviour of a plain substring search too, which is the kind of thing that surprises a reader who has only ever seen it applied to `~`.

The implementation has an admitted weakness, and the source admits it in place. The byte offset comes from searching the *lowercased* haystack, and the code needs a position in the *original*. Its own comment says: “The byte offset in `hay_lc` matches the byte offset in `hay` for ASCII; for non-ASCII this can differ in length but the relative position is preserved for common cases.” That is honest and it is also a statement that the result can be wrong. Rust’s `str::to_lowercase` is not length-preserving in general, so a haystack containing a character whose lowercase mapping has a different UTF-8 length, positioned before the match, shifts the offset relative to the original string and the reported position is off. The comment does not claim otherwise; it claims the common cases work, which is a narrower and true claim.

The conversion from byte offset to character position is also the most expensive line in the arm:

```

hay.char_indices().take_while(|&(off, _)| off < b).count() + 1

```

That walks the haystack from the beginning, counting characters, once per `index` call. The case-sensitive branch is cheaper — `hay[..b].chars().count() + 1`, which slices first and then counts — but both are linear in the prefix length, so `index` is proportional to the match position rather than constant after the search whenever byte and character offsets can differ. Under `-b` both branches short-circuit to `b + 1` and the cost disappears entirely. That is a concrete reason `-b` is faster and not only different, and it is not visible from the semantics chapter.

Negative zero, negative NaN, and platform normalisation

A pattern repeats across the math arms that has nothing to do with awk and everything to do with the platform math library:

```

// Normalize NaN sign - Linux glibc sqrt(-1) sets the sign bit
// (would print as `-nan`); gawk emits `+nan` regardless of platform.
let r = x.sqrt();
Value::Num(if r.is_nan() { f64::NAN } else { r })

```

The same guard, with a comment naming the same reason, appears in `sin`, `cos`, `atan2`, `exp`, and `log` — six arms in all, at lines 256, 274, 291, 309, 326, and 359 of the file. The specific hazards named differ per function — `sqrt` of a negative, `sin` and `cos` of an infinity, `atan2` propagating a NaN input’s sign bit, `exp` of a negative NaN, `log` of a negative — but the fix is identical: if the result is NaN, replace it with the canonical `f64::NAN` rather than whatever the platform produced.

This exists because the sign bit of a NaN is not specified for these operations, and different C libraries make different choices. Without the normalisation, the same awkr program printing the square root of a negative number would emit `-nan` on one platform and `+nan` on another. The sign then flows all the way to output because `format_number` in `runtime.rs` reads it back out:

```
let sign = if n.is_sign_negative() { '-' } else { '+' };
let body = if n.is_nan() { "nan" } else { "inf" };
return format!("{sign}{body}");
```

so a platform-dependent bit in the arithmetic becomes a platform-dependent character in the output. Normalising at the point of production rather than at the point of printing is the right call — it keeps the invariant that a NaN in an awkrs program has one spelling — but the guard is applied per-arm by hand. Any future math builtin added to this file that can produce NaN needs the same three lines, and nothing enforces that.

The negative-zero counterpart lives in `format_number` rather than here:

```
if n == 0.0 {
    return "0".to_string();
}
```

The comparison is true for both zeros, so negative zero prints as `0`. Same class of problem — a signed bit pattern that the language should not expose — solved in the output path instead of the arithmetic path, because unlike NaN, negative zero has arithmetically meaningful uses that normalising away would break.

Every math arm is written twice

The `-M` bignum mode does not layer over the double-precision implementations; it forks each one. `sqr` is representative:

```
if ctx.rt.bignum {
    let prec = ctx.rt.mpfr_prec_bits();
    let round = ctx.rt.mpfr_round();
    let f = value_to_float(&args[0], prec, round);
    if matches!(f.cmp0(), Some(Ordering::Less)) {
        ctx.rt.warn_builtin_negative_arg("sqr", f.to_f64());
    }
    Value::Mpfr(Float::with_val_round(prec, f.sqr(), round).0)
} else {
    let x = args[0].as_number();
    if x < 0.0 {
        ctx.rt.warn_builtin_negative_arg("sqr", x);
    }
    let r = x.sqr();
    Value::Num(if r.is_nan() { f64::NAN } else { r })
}
```

`sin`, `cos`, `atan2`, `exp`, and `log` have the same two-armed shape — the same six functions that carry the NaN normalisation. The MPFR side reads precision and rounding mode from the runtime on every call, builds a `Float` from the argument, calls the arbitrary-precision operation, and wraps the result in `Value::Mpfr`. The other side does the direct thing.

The duplication is not accidental and it is not removable cheaply. The two branches differ in return type, in how they test for a negative argument (`cmp0()` against `Ordering::Less` versus a plain `< 0.0`), in whether NaN normalisation applies at all, and in whether precision has to be threaded. A generic abstraction over both would need to be generic over the numeric type, the comparison, and the result wrapper, and would obscure exactly the platform details the comments are there to record.

What it means in practice is that `-M` is a parallel implementation of the numeric tower rather than a mode switch, and that a change made in one arm is not made in the other. The NaN normalisation is the visible instance: it is present in the double-precision arms and absent from the MPFR arms. Whether that is correct depends on whether the arbitrary-precision library produces sign-varying NaNs across platforms, which is a different question than the one the C math library raises, and the source does not answer it either way.

substr and the shape of a lint at runtime

Not every bad argument is an error. Chapters 18 and 24 covered the numeric case — `log` and `sqrt` of a negative argument warn on stderr and return NaN, gated on lint. `substr` shows the same mechanism applied to a string index, and it is the more instructive instance because the clamping it performs changes the result rather than merely annotating it:

```
if m < 1 {
    ctx.rt.lint_warn(&format!(
        "substr: start index {start_raw} is less than 1, treated as 1"
    ));
    m = 1;
}
```

Notice that `start_raw` is the pre-truncation floating-point value, so the message reports what the program actually computed rather than what it was rounded to. That is a small piece of care with a real payoff when the index came from an expression.

The clamping itself follows gawk’s documented POSIX extension, and the arm’s comment says so: “gawk: start < 1 is treated as 1; length is not shortened.” The second half of that sentence is the subtle part. In the older reading, a start index below one conceptually begins before the string and the requested length is consumed by the out-of-range prefix, so fewer characters come back. Under the rule awkrs implements, the start moves to 1 and the length stays whole, so the full requested count comes back. The arm implements the second, and the fact that it is a deliberate divergence from one lineage rather than an oversight is recorded in the comment rather than in a test name.

The non-positive-length case short-circuits before any of that:

```
let l = v.as_number() as i64;
if l <= 0 {
    return Ok(Value::Str(String::new()));
}
```

Zero and negative lengths return the empty string immediately, without evaluating the start index or emitting the clamp warning. So a call with both an out-of-range start and a zero length is silent, while the same start with a length of one warns — the diagnostic depends on an argument the diagnostic is not about. Defensible, since the result is empty either way, but it means the warning is not a reliable signal that a program contains an out-of-range start.

Reading dispatch as a layer

The value of reading `vm_builtins.rs` as a unit rather than function by function is that it exposes the decisions made *around* every builtin rather than inside any one: user functions take precedence on every builtin call, whatever a stale doc comment implies; POSIX mode removes thirty-eight names at call time rather than parse time, and cannot reach the constructs that became their own opcodes; arity is checked per-arm with two message dialects; `IGNORECASE` extends past the regex engine into `index`; NaN signs are normalised in six arms of the double-precision path and none of the MPFR path; and `-M` forks every math arm. None of those are visible from the reference documentation for any individual function, and all of them are things you need when a program behaves differently than the per-function semantics predict.

Chapter 58 – `rwarray` Is a Local Format, Not `gawk`'s

`wrotea` and `reada` were introduced in Chapter 29 as `awk`'s closest thing to a persistent data store: write an array to disk, read it back on the next run, and carry a tally across invocations. That description is accurate and it leaves out the part that matters if you have ever used `gawk`'s `rwarray` extension. `awkrs`'s on-disk format is its own. The two are not interoperable in either direction, and the divergence is deliberate and visible in the first line of every file `awkrs` writes.

The format, in full

`wrotea` is short enough to read whole:

```
pub(crate) fn wrotea(
    rt: &mut Runtime, path: &str, arr_name: &str,
) -> Result<Value> {
    rt.require_unsandboxed_io()?;
    rt.clear_errno();
    let keys = rt.array_keys(arr_name);
    let mut f = match File::create(path) {
        Ok(f) => f,
        Err(e) => {
            rt.set_errno_io(&e);
            return Ok(Value::Num(-1.0));
        }
    };
    writeln!(f, "awkrs-rwarray-v1").map_err(Error::Io)?;
    for k in keys {
        let v = rt.array_get(arr_name, &k);
        let line = format!(
            "{}\t{}\n", escape_rw(&k), escape_rw(&v.as_str())
        );
        f.write_all(line.as_bytes()).map_err(Error::Io)?;
    }
    Ok(Value::Num(0.0))
}
```

A magic line, then one line per entry, key and value separated by a tab, both escaped. That is the entire specification. It is a text format you can inspect with `cat`, diff with `diff`, edit with an editor, and — because the escaping is minimal — read with `awk` itself. `gawk`'s `rwarray` writes a binary format with its own value typing, which you cannot do any of those things to.

The magic string carries a version suffix and `reada` enforces it:

```

let mut magic = String::new();
reader.read_line(&mut magic).map_err(Error::Io)?;
if magic.trim() != "awkrs-rwarray-v1" {
    rt.set_errno_str("reada: not an awkrs rwarray file");
    return Ok(Value::Num(-1.0));
}

```

Handing `reada` a file written by something else fails this check on the first line and returns -1 with `ERRNO` set, rather than misreading foreign bytes as text. That is the right failure mode — a wrong-format file is rejected, not misinterpreted — and it is the reason to have a magic line at all. The `v1` suffix means a future format change can be introduced without a flag: `reada` would accept both spellings and `writtea` would emit the newer one.

The escape grammar, and why it is exactly four characters

```

fn escape_rw(s: &str) -> String {
    let mut o = String::with_capacity(s.len());
    for c in s.chars() {
        match c {
            '\\' => o.push_str("\\\\"),
            '\n' => o.push_str("\\n"),
            '\r' => o.push_str("\\r"),
            '\t' => o.push_str("\\t"),
            _ => o.push(c),
        }
    }
    o
}

```

Four characters get escaped and everything else passes through as itself. The selection is exactly the set that would break the line-and-tab framing: a literal tab would create a false field boundary, a literal newline would create a false record boundary, a carriage return would be eaten by `reada`'s line trimming, and the backslash has to be escaped because it is the escape character. Nothing else needs to be, so nothing else is — which is what keeps the format readable. A file of plain string keys and numeric values comes out as plain text with no escapes at all.

The inverse is more permissive than the forward direction, and the asymmetry is intentional:

```

Some(x) => {
    o.push('\\');
    o.push(x);
}
None => o.push('\\'),

```

An unrecognised escape is passed through *with its backslash intact*, and a trailing lone backslash is preserved. That makes the pair round-trip-stable for any input: a string containing a backslash followed by some other letter escapes to a doubled backslash followed by that letter, which unescapes by consuming the doubled backslash to a single one and then copying the letter, recovering the original. And a hand-edited file containing a backslash sequence that `writtea` would never have produced loads as its two literal characters rather than being silently dropped. Both properties come from the same default arm.

The escaping is applied to key and value with the same function, so a key containing a tab survives the round trip. That matters because awk array keys are arbitrary strings and `SUBSEP`-joined multidimensional keys are exactly the kind of key that contains a control character.

Parsing is deliberately loose in one place

`reada`'s loop skips blank lines:

```
let s = line.trim_end_matches(['\r', '\n']);
if s.is_empty() {
    line.clear();
    continue;
}
let mut parts = s.splitn(2, '\t');
let key = parts.next().unwrap_or("");
let val = parts.next().unwrap_or("");
```

`writea` never produces a blank line — even an empty key with an empty value emits a line consisting of the tab alone, which after trimming is not empty — so the skip exists for files that have been touched by something else. A trailing newline added by an editor, a file concatenated from two dumps, a stray blank line: all tolerated.

`splitn(2, '\t')` splits on the *first* tab only, so a value containing a tab would survive even if it had somehow not been escaped. And both `unwrap_or("")` calls mean a line with no tab at all loads as a key with an empty value rather than being rejected. The parser accepts more than the writer produces, which is the correct direction for a checkpoint format that people will edit by hand.

What it does not do is detect truncation. A file cut off mid-write loads as however many complete lines it contains, with no error, because there is no entry count in the header to check against. `writea` does not force the data to disk before returning either, so a crash between the last write and process exit can leave exactly such a file. For a checkpoint that is usually fine — you lose the last run's tally and recompute it — but it is not a transactional store and the format has no room to become one without a v2.

Values reload as `Str`, and that is the whole typing story

The single line that determines how a reloaded array behaves is:

```
rt.array_set(
    arr_name, unescape_rw(key), Value::Str(unescape_rw(val))
);
```

`Value::Str`, not `Value::StrLit`. The distinction is documented on the enum in `runtime.rs`:

```
/// Dynamic string (fields, concat, I/O, etc.) – may be a POSIX
/// *numeric string* in comparisons.
Str(String),
/// String literal from program text (`"...`)" – not a numeric string
/// for relational ops (POSIX).
StrLit(String),
```

So the choice of variant is the choice of comparison semantics. A value that was a number when it was written comes back as a `Str` holding its digits, which is a POSIX numeric string and therefore compares numerically — a reloaded counter still compares as a number against a numeric threshold, exactly as it would if those digits had arrived as a field from input. Had `reada` constructed a `StrLit`, the same comparison would have gone string-wise and short numbers would have sorted above long ones.

This is the correct decision and it is not an obvious one. Data read back from a file is data from outside the program, which is the same category as a field or a `getline` result, so it should get the same numeric-string treatment. The one-word difference between two enum variants is what makes a checkpointed counter still a counter.

It does mean the round trip is lossy in the other direction: everything comes back as a string-shaped value regardless of what it was. A value that was a `Num` is now a `Str` that *behaves* numerically; a value that was a program string literal is now a numeric string and will compare numerically if it happens to look like a number.

The second case is the one that can bite — a version string assigned as a literal and reloaded now compares numerically against another version string and loses the lexicographic ordering the program expected. That is inherent to a text format with no type column, and it is the price of being inspectable.

Under `-M`, an arbitrary-precision value is stringified on the way out through the MPFR display path and reloaded as a plain `Str`, so extended precision does not survive a checkpoint either.

Precision, and the `CONVFMT` that is not consulted

`writeln` stringifies with `v.as_str()`, which for a number reaches `format_number`:

```
if n.fract() == 0.0 {
    if n == 0.0 { return "0".to_string(); }
    format!("{:.0}", n)
} else {
    format!("{n}")
}
```

The non-integral branch is Rust's `Display` for `f64`, which produces the shortest decimal string that round-trips to the same bit pattern. It is not `num_to_string_convfmt`, which is the runtime's separate function that reads the `CONVFMT` global and applies it.

The consequence is worth stating plainly, because it runs against expectation. `awk`'s own number-to-string coercion — the one that fires when you concatenate a number onto a string — goes through `CONVFMT`, whose default the runtime initialises to `%.6g`, and therefore loses precision. `writeln` does not. An array element holding a long repeating fraction is written as its full round-trip decimal expansion, not as six significant digits, and reloads to the identical value. So the checkpoint file preserves more precision than the language's own string conversion would, and a program that maintains a running average across many checkpointed runs does not accumulate truncation error from the persistence layer.

The integral branch uses a fixed-point format rather than a cast to a 64-bit integer, and the comment on `format_number` explains why: values past the signed 64-bit range still come out as decimal digits rather than saturating. A counter that has grown past that range still checkpoints as a readable number.

The one thing `writeln` cannot represent

`array_get` returns a `Value`, and an array value stringifies to nothing:

```
Value::Array(_) => String::new(),
```

So a nested array — an element that is itself an array, which `awkrs`'s value model permits — is written as an empty value with no error and no warning, and reloads as an empty string rather than as an array. The format has no nesting and the writer has no check for it.

That is a real gap and it is a quiet one: the return value is still 0, `ERRNO` is still clear, and the file looks fine. A program checkpointing a structure built with true nesting rather than `SUBSEP` keys loses it silently. The `SUBSEP` idiom, where a two-index reference is really one flat key containing a separator character, checkpoints perfectly — the key is a string, `escape_rw` handles the separator if it needs escaping, and the reload reconstructs the same flat map. So the format supports `awk`'s traditional multidimensional arrays completely and true nested arrays not at all.

Error handling has two modes, and the split is by failure point

The functions are careful about `ERRNO` at the open, and not careful after it:

```
let mut f = match File::create(path) {
    Ok(f) => f,
    Err(e) => {
        rt.set_errno_io(&e);
        return Ok(Value::Num(-1.0));
    }
};
writeln!(f, "awks-rwarray-v1").map_err(Error::Io)?;
```

An open failure — bad path, no permission, directory missing — sets `ERRNO`, returns -1, and lets the awk program decide what to do. That is the gawk convention for this family of builtins, and every function in `gawk_extensions.rs` that opens something follows it.

A failure *during* writing takes the `?` and propagates as an I/O error, which is fatal and terminates the program. So a disk that fills up between the header and the last entry does not give the awk program a -1 to check; it aborts. The same asymmetry appears in `reada`: the open failure is catchable, a mid-file read error is not.

Whether that is right depends on what you think a partially written checkpoint means. There is an argument that it is unrecoverable and aborting is honest — the file on disk is now a truncated dump, and returning -1 would invite the program to carry on as though nothing had happened. There is also an argument that a program that checks the return value of `writea` deserves to be told about every failure through the same channel. The code picks the first, consistently, and does not document the choice; you have to read the `?` to find it.

Both functions start with `rt.require_unsandboxed_io()`, which is the same gate the rest of the extension surface uses, so under the sandbox flag they fail before touching the filesystem at all — and that failure is a third kind, a fatal sandbox error rather than either an `ERRNO` return or an I/O abort. Three error channels in a short prologue.

Why a local format was the right call

Reimplementing gawk's binary format would have bought interoperability with gawk's extension and cost the ability to read a checkpoint with `cat`. The trade is not close for awks's purposes, and the version-tagged magic line means the decision is reversible: a v2 could be gawk-compatible, or a flag could select between them, without breaking any file already on disk.

What the current design gets is a checkpoint you can debug. When a persisted tally comes back wrong, the file is a two-column text dump you can sort, diff against the previous run, and grep — and the failure is usually visible on sight. Against a binary format the same investigation needs a decoder first. For a feature whose entire purpose is to carry state across runs that you are not watching, being able to see what got carried is worth more than reading someone else's files.

Chapter 59 — What the Bytecode Cache Does Not Do

Chapter 18 described the bytecode cache as `awkr`'s claimed first and walked its read path: memory-map the shard, validate once, look the entry up in the archived map, decode one blob. That chapter is about what the cache does. This one is about the parts of `script_cache.rs` it did not cover — the write path's cost model, the unsafe that makes the read path possible, the maintenance API that exists and is never called, and the several places where a failure is turned into a silent no-op.

put rewrites the entire shard

The read path is genuinely incremental: one entry is decoded out of a mapped image. The write path is not.

```
pub fn put(
    &self, path: &str, mtime_secs: i64, mtime_nsecs: i64,
    cp: &CompiledProgram,
) -> std::io::Result<()> {
    let cp_blob = bincode::serialize(cp)
        .map_err(|e| std::io::Error::other(e.to_string()))?;

    let _lock = match acquire_lock(&self.lock_path) {
        Some(l) => l,
        None => return Ok(()),
    };

    let mut shard = match read_owned_shard(&self.path) { /* ... */ };

    shard.entries.insert(path.to_string(), entry);
    shard.header.built_at_secs = now_secs() as u64;

    write_shard_atomic(&self.path, &shard)?;
    self.invalidate_mmap();
    Ok(())
}
```

`read_owned_shard` is not the memory-mapped path. It reads the whole file into a byte vector, validates it, and then fully deserialises it out of the archived form into owned Rust structures:

```
fn read_owned_shard(path: &Path) -> Option<ScriptShard> {
    let bytes = std::fs::read(path).ok()?;
    let archived =
        rkyv::check_archived_root::<ScriptShard>(&bytes[..]).ok()?;
    archived.deserialize(&mut rkyv::Infallible).ok()
}
```

Then `write_shard_atomic` re-serialises the whole thing and writes it back with an explicit `sync_all` before the rename.

So caching one script costs a full read, a full deserialise, a full re-serialise, and a full durable write of every entry in the shard. The cost of adding an entry is proportional to the total size of everything already cached.

Cold-caching a directory of scripts by running each one once grows superlinearly in the total compiled size, and every one of those writes is synced under an exclusive lock, so they serialise against each other.

The asymmetry is by design and it is the right one for this workload. Compilation happens once per script and lookups happen every run, so the read path is what deserves optimisation and it got it. But it means the cache is fastest when it is warm and slowest exactly when a user first tries it, and that the “second run is free” property has an unadvertised counterpart: the *first* run is slower than an uncached run, by the full serialise-and-sync of the shard.

The single-file shard design is what forces this. A per-script file, or a shard partitioned by a hash of the path, would let a write touch only its own file. The module header names the constraint as a deliberate match to strykelang’s layout, and it is the one structural decision in the file with a cost this visible.

The unsafe, and what it is actually buying

`MmappedShard` is self-referential and says so:

```
pub struct MmappedShard {
    _mmap: Mmap,
    archived: *const ArchivedScriptShard,
}

// SAFETY: the pointer aliases an immutable mmap that lives as long as
// Self. rkyv-validated reads are immutable.
unsafe impl Send for MmappedShard {}
unsafe impl Sync for MmappedShard {}
```

A raw pointer into the mapped bytes, kept next to the mapping that backs it. The `unsafe impls` are needed because a raw pointer is neither `Send` nor `Sync` automatically, and the struct has to be both to live inside the process-wide lazy static.

The reasoning in the comment is sound as far as it goes: the mapping is created read only, is never written through, and is dropped in the same struct as the pointer, so the pointer cannot outlive its target and no aliasing mutable borrow exists. What the comment does not mention is the hazard that mapped files actually have — the file underneath can be replaced or truncated by another process while it is mapped, and a truncation turns subsequent reads into a fault rather than a Rust error. The write path is careful to avoid creating that situation: `write_shard_atomic` writes to a uniquely named temporary and renames over the target, so an existing mapping keeps pointing at the old, now-unlinked inode rather than seeing new bytes appear underneath it. The safety of the mapping therefore depends on the *writer’s* discipline, in a different function, and the `SAFETY` comment does not say so. It is correct; it is under-documented, and the temporary-name construction that makes it correct reads like a uniqueness measure when it is also a correctness one.

There is a second thing the mapping does not buy, and Chapter 18’s framing can be read as promising it. `MmappedShard::open` calls:

```
let archived =
    rkyv::check_archived_root::<ScriptShard>(mmmap[..]).ok()?;
```

That is a *validating* entry point. It walks the whole archived image checking that every relative pointer stays in bounds, every discriminant is valid, and every collection’s layout is consistent — which is what the `#[archive(check_bytes)]` attribute on the three types generates. It is a full pass over the shard, proportional to the shard’s size, before any lookup happens.

So the read path is *zero-copy*, not *zero-work*. A large shard is validated in full on the first lookup in each process, and only then is the single entry located and its blob decoded. The validation is the price of safely reading corruption-or-attacker-shaped bytes as structured data, and paying it is the right call — the unchecked alternative would turn a corrupt cache file into undefined behaviour. But it means shard size has a per-process cost that does not go away no matter how few entries you look at, and it is another argument the single-file

design has to answer for.

The validation is at least paid once. `ScriptCache` keeps the mapping in a `Mutex<Option<MmappedShard>>` and `ensure_mmap` populates it lazily, so repeat lookups within a process reuse the validated view.

The maintenance API has no callers

`ScriptCache` exposes three functions that no part of `awkr`s invokes:

```
pub fn stats(&self) -> (i64, i64) { /* ... */ }
pub fn evict_stale(&self) -> usize { /* ... */ }
pub fn clear(&self) -> std::io::Result<()> { /* ... */ }
```

Searching the crate for uses of any of them outside `script_cache.rs` returns nothing. Within the file, `stats` and `clear` appear only inside the test module, and `evict_stale` appears only in its own definition and in tests named for it. The `impl` block carries `#[allow(dead_code)]`, which is the compiler being told about exactly this.

Only two functions in the module are reachable from the running program, and `lib.rs` names both — `try_load` at the cache-lookup site and `try_save` after a successful compile. `try_load` calls `get`; `try_save` calls `put`. Nothing calls `evict_stale`.

So the cache never prunes itself. An entry for a script that has been deleted stays in the shard indefinitely, and an entry for a script that has been edited stays too, taking up space and being re-validated on every process's first lookup, because `get` returns `None` on a modification-time mismatch without removing anything. The shard is append-and-overwrite-only in practice, and the only way to shrink it is to delete the file by hand.

Chapter 18 says the cache “self-maintains, with an eviction pass that drops entries whose source files have vanished or changed.” The eviction pass is written, is tested, and does exactly what that sentence describes — but it is not wired to anything, so the cache does not self-maintain today. The correct statement is that the machinery for self-maintenance exists and the trigger for it does not. Wiring it would be a few lines: a call from `try_save` on some interval, or a subcommand, or a threshold check against what `stats` already returns. Until then, the shard file grows monotonically with the number of distinct scripts ever run.

This is worth being precise about because it is adjacent to the failure mode the cache's own design guards against everywhere else. The invalidation logic is conservative to the point of paranoia — modification time to nanosecond precision, binary-mtime comparison, version and pointer-width and magic checks — because a stale hit would be worse than no cache. Growth is a different problem than staleness and it does not produce wrong answers, which is presumably why it went unwired. But a shard that grows without bound eventually makes the full-image validation on first lookup a measurable cost, and at that point the cache starts paying for entries no one will ever hit.

Failures that turn into no-ops

The module's failure handling is uniformly silent, and the choice is defensible per site and worth cataloguing as a set.

`acquire_lock` returns an `Option`, and `put` treats the absent case as success:

```
let _lock = match acquire_lock(&self.lock_path) {
    Some(l) => l,
    None => return Ok(()),
};
```

A lock that cannot be taken — the lock file cannot be created, the filesystem does not support the advisory lock, the call is interrupted — means the entry is simply not cached, and `put` reports success. The caller cannot

distinguish “cached” from “did not cache” by the return value. Since `try_save` returns nothing and discards `put`’s result, that distinction would have nowhere to go anyway.

`try_load` and `try_save` are chains of fallible steps over `Option`, so any failure at any step — the cache is disabled, the path does not canonicalise, the metadata call fails — falls out as a miss:

```
pub fn try_load(path: &Path) -> Option<CompiledProgram> {
    let cache = CACHE.as_ref()?;
    let canonical = path.canonicalize().ok()?;
    let path_str = canonical.to_string_lossy();
    let (mtime_s, mtime_ns) = file_mtime(&canonical)?;
    cache.get(&path_str, mtime_s, mtime_ns).map(|c| c.cp)
}
```

That is right for a cache. Every one of these failures has a correct fallback — compile the script — and none of them is something the user asked for or can act on. A cache that printed a warning when it could not take a lock would be noise on stderr in exactly the situation where the program is otherwise working fine.

The one place the silence has teeth is the blob decode in `get`:

```
let cp_bytes: &[u8] = entry.cp_blob.as_slice();
let cp: CompiledProgram = bincode::deserialize(cp_bytes).ok()?;
```

A blob that fails to decode is a miss, so the script recompiles — and then `put` overwrites the bad entry with a good one. The cache heals itself for that specific corruption without anyone noticing, which is the behaviour you want. But it is also the case that a `CompiledProgram` whose serialised representation changed shape without a corresponding version change would produce this every single run: a permanent, silent, invisible cache miss on every script, with the cost of the failed decode and the rewrite added on top. There is no counter, no log line, and no way to observe it short of instrumenting the process.

`SHARD_FORMAT_VERSION` is what would prevent that, and its comment states its trigger as “bumped on incompatible rkyv schema changes.” But the blob is bincode, not rkyv — the module header is explicit that the inner encoding is bincode and that moving it to rkyv is a future phase — so a `CompiledProgram` layout change is not an rkyv schema change and the version constant’s stated trigger does not cover it. The `awkrs_version` check catches it in practice for released builds, since any change to the compiled-program type ships with a version bump; it does not catch it on an unbumped working tree during development, which is the situation where a developer would be most confused by it.

The header check is not the same check twice

`get` validates through `header_ok`:

```
magic == SHARD_MAGIC
&& fv == SHARD_FORMAT_VERSION
&& pw as usize == std::mem::size_of::<usize>()
&& h.awkrs_version.as_str() == env!("CARGO_PKG_VERSION")
```

Four conditions. `put` validates a shard it is about to modify with three:

```
Some(s)
if s.header.awkrs_version == env!("CARGO_PKG_VERSION")
&& s.header.pointer_width as usize
    == std::mem::size_of::<usize>()
&& s.header.format_version == SHARD_FORMAT_VERSION =>
{
    s
}
_ => fresh_shard(),
```

The magic check is absent on the write side. In practice the gap is closed by the validating read inside `read_`

`owned_shard`, which rejects bytes that are not a valid archive of the right type before the header is ever examined, so the only shard that could reach `put`'s match with a wrong magic is one that is a structurally valid archive carrying a different magic value — which is to say, essentially nothing. The consequence is nil and the inconsistency is still worth noting, because the two predicates are the definition of “is this shard mine” and there are two of them, written out longhand in different orders, in the same file. A shared predicate function would make them one definition and would make it obvious that `header_ok` is that function applied to the archived form.

`SHARD_MAGIC` itself checks out against its comment, which is not something to take on faith:

```
pub const SHARD_MAGIC: u32 = 0x41574B52; // "AWKR"
```

`0x41` is A, `0x57` is W, `0x4B` is K, `0x52` is R. The comment is accurate and the constant is four bytes for four characters.

binary_mtime_at_cache and the direction of the comparison

The staleness rule that invalidates the whole cache on an awkr's rebuild is one comparison:

```
if let Some(bin_mtime) = current_binary_mtime_secs() {
    let cached_bin_mtime: i64 = entry.binary_mtime_at_cache.into();
    if cached_bin_mtime < bin_mtime {
        return None;
    }
}
```

The module header states the rule as the cached binary time being at least the running binary's, and the code implements exactly that — a strictly-less comparison rejects, so equal or greater is a hit. Comment and code agree, which is worth confirming rather than assuming.

Two properties follow that are not obvious from the rule's description. First, the comparison is one-directional: a binary *older* than the entry is a hit, not a miss. Reverting awkr's to a previous build with an older modification time therefore does not by itself invalidate entries the newer build wrote — but the `awkr's_version` string check in `header_ok` catches that case whenever the version differs, and if it does not differ then the two builds claim to be the same release.

Second, the resolution is whole seconds. `file_mtime` returns both seconds and nanoseconds, and the source-file comparison uses both:

```
if entry_mtime_s != mtime_secs || entry_mtime_ns != mtime_nsecs {
    return None;
}
```

but `current_binary_mtime_secs` discards the nanoseconds, keeping only the seconds field from the same helper. So a rebuild that lands in the same second as the cached entry's recorded binary time compares equal and the entries survive. Two rebuilds a second apart is not a realistic build cadence, and the version check is the real guard for release boundaries, so the practical exposure is narrow. The source-file check is nanosecond-precise precisely because editing a file twice within a second *is* realistic; the binary check did not need the same precision and does not have it. That is a defensible asymmetry, and reading the two comparisons next to each other is the only way to notice it exists.

`current_binary_mtime_secs` memoises through a `OnceLock`, so the executable-path and metadata calls happen once per process regardless of how many lookups occur — one of the few places in the module where a cost was explicitly avoided rather than accepted.

The shape of the module, summarised

Read as a whole rather than as a read path, `script_cache.rs` is a fast, carefully validated, conservatively invalidated lookup bolted to a write path that rewrites everything and a maintenance API that nothing calls. Every one of those is a reasonable engineering position for a cache whose reads outnumber its writes by orders of magnitude. What matters is knowing which is which: the second run of a script is fast because the read path earned it, the first run is slower than an uncached run because the write path pays for the whole shard, and the file on disk will grow indefinitely because the pruning is written but not wired.

This book has tried to do for awkrs what the reference cannot: explain not just what it does but why it is shaped the way it is — why the field model is bidirectional, why division by zero is fatal even in JIT-compiled code, why the parallel path can be both concurrent and correct, why the JIT is on by default yet deliberately not the headline, and why a forty-year-old programming model deserved a new engine rather than a replacement. awkrs's bet is that AWK was never the problem — the model is as sharp as it ever was — and that what it needed was a machine worthy of it: fast on the common case, faithful to the standard, modern in its tooling, honest about its experiments. Read the reference for the *what*. This was the *why*. The work is MenkeTechnologies'; the code is all there to check.